

dsai-2526-theory

June 11, 2026

1 Module 1. Samples

1.1 Learning goals

- You are able to use Jupyter Notebooks (locally or in Google Colab) and configure your environment to run the code examples in this course.
- You understand the basic terminology used in data science (indicated with $\propto$).
- You are able to load a dataset in Python and explore its structure and characteristics.

1.2 Getting Started with Jupyter Notebooks

Jupyter Notebooks (or iPython Notebooks) are files with a mixture of Markdown and Python code. It's a very convenient format so we'll be using this quite a lot during this course!

Read the instructions below carefully, as they do not necessarily apply to all environments!

1.2.1 Google Colab

To get started, it is not necessary to install software on your laptop. [Google Colab](#) is a free online iPython development environment, but it does require a Google account in order to use it. You can create a local clone of the Github repo with the lab assignments and upload the Notebook files you want to work with to Colab.

Note that the Google Colab environment has older versions installed of the Python libraries that we will be using throughout this course. Usually, that's not a problem, but sometimes you will run into errors, e.g. because some parameter in our Python code didn't exist yet in that version. The solution is to add a new code block at the beginning of your Notebook file with content:

```
[ ]: %pip install --upgrade matplotlib
      %pip install --upgrade scipy
      %pip install --upgrade statsmodels
```

You only have to do this in Notebooks where you encounter errors, and only for the libraries that cause these errors. The installation may take a while and you need to restart the Python runtime afterwards by clicking the button that appears.

1.2.2 Local environment

If you prefer to have a local development environment, you need to install Python and Visual Studio Code with the necessary extensions for working with Python (e.g. Pylance, Jupyter, Jupyter

Keymap, etc.) The first time you open a Notebook file, VSCode will offer you to install these extensions.

1.2.3 Windows

To install Python and VSCode on Windows, we recommend using `winget`. First, check the latest version of Python on the [Python website](#). For example, at the time of writing, the latest version was 3.14.2. We'll need to specify the major and minor version (e.g. 3.14) when installing. Then, open a PowerShell or command prompt as Administrator and run:

```
> winget install Microsoft.VisualStudioCode
> winget install Python.Python.3.14
```

Remark that `winget upgrade --all` will install maintenance releases of Python, but it will not upgrade to a new major or minor version.

1.2.4 Linux

Some Linux distributions no longer allow you to use `pip` to install Python packages globally. In that case, we recommend to work with a Python Virtual Environment. VS Code will help you to create one when you select the Python version to use for executing code cells. Alternatively, you can install and use `pipx` to install Python packages globally.

1.2.5 MacOS

To install Python on MacOS, it's best to use [Homebrew](#). After installing issue the following commands.

```
> brew install python
> export PATH="/opt/homebrew/opt/python/libexec/bin:$PATH"
```

You probably want to add the `export PATH` line to your `.zshrc` (or whichever shell you use)

You can check everything is correctly set through

```
> which python
/opt/homebrew/opt/python/libexec/bin/python
```

It's always a good idea to set up a `venv`

```
dsai-labs> python3 -m venv .venv
source .venv/bin/activate
```

You should now be able to use this environment as your kernel environment in VSCode. We provided a `requirements.txt` file in the root of the repository, so you can install all necessary libraries (after activating the virtual environment) with:

```
> pip install -r requirements.txt
```

1.3 Libraries

Most Python scripts (or Jupyter Notebooks) for analyzing data use the same program libraries. The most important ones are:

- `numpy` - multidimensional arrays, linear algebra, etc.

- `scipy` - mathematics, science, engineering etc.
- `pandas` - data-analysis and -manipulation
- `openpyxl` - reading and writing Excel files (most exercises in this course use CSV files)
- `matplotlib`, `seaborn` - data visualisation
- `scikit-learn` - regression and machine learning

You can install these using `pip`:

```
> pip install matplotlib numpy openpyxl pandas scipy seaborn scikit-learn statsmodels
```

Or, alternatively, by executing the following code block:

```
[ ]: %pip install matplotlib
      %pip install numpy
      %pip install openpyxl
      %pip install pandas
      %pip install scipy
      %pip install seaborn
      %pip install scikit-learn
      %pip install statsmodels
```

Since you usually need the same packages every time, it is best to put them at the top of every script or Notebook file you write. By convention, package names are often abbreviated in a consistent manner, e.g. `np` for `numpy`, `sns` for `seaborn`, etc.

```
[5]: # Importing the necessary packages
import numpy as np                    # "Scientific computing"
import scipy.stats as stats           # Statistical tests

import pandas as pd                  # Data Frame
from pandas.api.types import CategoricalDtype

import matplotlib.pyplot as plt      # Basic visualisation
from statsmodels.graphics.mosaicplot import mosaic # Mosaic diagram
import seaborn as sns                # Advanced data visualisation
↪ visualisation
```

Tip: you can define a code snippet in VSCode for this block of code, so you don't have to type it every time. See e.g. [this guide](#) on how to create code snippets in VSCode.

1.4 Basic Concepts in Data Science

1.4.1 Variables and Measurement Levels

In statistics and data science, a **variable** is any attribute of an object of study. Variables can have **values** that differ between objects. Usually, variables are classified into different types, or **measurement levels**, depending on the kind of values they can take and the mathematical operations that can be performed on them. The most common types are:

- **Qualitative** (or categorical) variables: variables that can take on a limited number of distinct categories or labels. These are not necessarily numerical.

- $\propto$ **Nominal** variables: categorical variables *without a specific order* or ranking among the categories (e.g., colors, types of animals).
- $\propto$ **Ordinal** variables: categorical variables *with a specific order* or ranking among the categories (e.g., education level, customer satisfaction ratings).
- $\propto$ **Quantitative** (or numerical) variables: variables that can take on numerical values, which can be discrete (countable values) or continuous (any value within a range). These are usually the result of measurements. Often, there are many different values of quantitative variables in a dataset, or they may even be unique.
 - $\propto$ **Interval** variables: numerical variables where the intervals between values are meaningful, but there is no true zero point (e.g., temperature in Celsius or Fahrenheit).
 - $\propto$ **Ratio** variables: numerical variables with a true zero point, allowing for meaningful ratios between values (e.g., height, weight, age).

It is important to correctly identify the type of variable you are working with, as this influences the choice of statistical methods and visualizations used for analysis. This is something that a computer is not very good at, so you often need to make these distinctions yourself when exploring and analyzing a dataset.

1.4.2 Populations, samples, observations

In data science and statistics, a $\propto$ **population** refers to the entire set of individuals or items that you are interested in studying. A $\propto$ **sample** is a subset of the population that is selected for analysis. A single individual or item within a population or sample is called an $\propto$ **observation**.

The goal of sampling is to gather information about the population without having to study every individual or item within it. Under certain conditions (which will be explored later in this course), you can use the information obtained from a sample to make inferences about the entire population. This is often necessary because studying the entire population can be impractical, time-consuming, or expensive.

When selecting a sample from a population, it is important to ensure that the sample has specific characteristics. Specifically, a sample should be:

- $\propto$ **Randomly selected**, i.e. every individual or item in the population should have an *equal chance of being included* in the sample in order to reduce bias, but randomness alone does not guarantee that a sample is perfectly representative - especially for small sample sizes..
- **Large enough** to provide reliable estimates of population parameters. We will explore this more in chapter 3.
- $\propto$ **Representative** of the population, meaning that it should accurately reflect the characteristics of the population as a whole.

Ideally, a researcher would do the following to draw a sample from a population:

- First and foremost, the population must be well defined
- Then you can compile a $\propto$ **sampling frame**, i.e. a list of all individuals in the population
- Then, the researcher randomly selects a number of individuals from this list.

It is often difficult or impossible to set up a sampling frame. In such cases, the researcher chooses a different sampling method and tries to approximate the properties listed above as closely as possible within the available time and budget.

1.4.3 Sampling errors

Even under the best of circumstances, the result within a sample is likely to deviate from the “true” value in the population, although we hope we can approach it sufficiently. So by taking a sample we inevitably introduce errors. A useful way to categorize errors is given below:

- $\propto$ **Sampling errors** are caused by the way we take a sample. We distinguish between $\propto$ *random* or $\propto$ *systematic* sampling errors:
 - **Random sampling errors** are only due to chance and cannot be avoided. Every time you take a random sample, you will get a different result within that sample.
 - **Systematic sampling errors** are worse, because they can cause the results to be biased and therefore no longer be *representative* of a population. Examples:
 - * Online survey: people without internet are excluded
 - * Street survey: only people who are currently walking there
 - * Voluntary survey: only interested parties participate
- $\propto$ **Non-sampling errors** are errors that occur after the sample has been taken, e.g. in the measurement and analysis. In this case, we speak of non-sampling errors and again we distinguish between *random* and *systematic*:
 - **Random non-sampling errors** For example, a respondent accidentally selects or fills out a wrong answer in a survey.
 - **Systematic non-sampling errors** For example:
 - * a measuring device that is not calibrated correctly and systematically gives a slightly higher value than the true value.
 - * respondents intentionally underestimate or overestimate reality (e.g. when asked about how much you smoke or drink alcohol on a daily basis)

1.4.4 Representative samples

A sample is said to be $\propto$ **representative** of a population when the characteristics of the sample closely match those of the population. This means that the sample accurately reflects the diversity and distribution of key variables within the population, allowing for valid inferences to be made about the entire population based on the sample data.

In a $\propto$ **stratified** sample, the population is divided into subgroups (or strata) based on certain characteristics, and samples are drawn from each stratum in proportion to their representation in the population. This helps ensure that the sample is representative of the population as a whole, especially when certain subgroups may be underrepresented in a simple random sample.

For example, if a population of 5810 individuals consists of the following age groups and genders:

	≤ 18	$]18, 25]$	$]25, 40]$	> 40	Total
Woman	500	1500	1000	250	3250
Man	400	1200	800	160	2560
Total	900	2700	1800	410	5810

A stratified sample of 10% would then consist of 581 individuals, with the following distribution:

	≤ 18	$]18, 25]$	$]25, 40]$	> 40	Total
Woman	50	150	100	25	325
Man	40	120	80	16	256
Total	90	270	180	41	581

In chapter 4, we will discuss a method that allows us to verify whether a sample is representative of a population.

1.4.5 Univariate vs multivariate analysis

In chapter 2, on **univariate analysis**, we will focus on analyzing and summarizing data for a single variable at a time. This involves techniques such as calculating measures of central tendency (*mean, median, mode*) and measures of dispersion (*range, variance, standard deviation*), as well as creating visualizations like histograms and box plots to understand the distribution of the variable.

In chapters 4 to 6, we will explore various methods to investigate whether two variables have some kind of relationship or association with each other, under the umbrella term of **bivariate analysis**.

The appropriate analysis or visualization technique always depends on the measurement levels of the variables involved. For example, we might use a scatter plot to visualize the relationship between two quantitative variables, or a contingency table to examine the association between two categorical variables.

We say that two variables have an **association** or a **relationship** when the value of one variable provides some information about the value of the other variable. For example, there may be an association between age and income, where older individuals tend to have higher incomes. The variable that provides information about the other variable is called the **independent variable**, while the variable being predicted or explained is called the **dependent variable**.

It is essential to understand that when you find an association between two variables, it does not necessarily imply a causal relationship. A phrase often used in this context is **correlation is not causation**. In other words, just because two variables are associated does not mean that one variable causes the other to change. There may be other factors at play, or the association could be entirely coincidental (*spurious correlation*). See some examples [here](#) and [here](#).

1.5 Opening a Dataset, General Information

Throughout this course, we will load **datasets** from various sources to practice our data analysis skills. A dataset usually corresponds to a *sample* taken from a *population* being studied, and is typically organized in a tabular format where rows represent individual *observations* and columns represent different *variables*.

To read a dataset into Python, you can use the **pandas** library, which provides powerful tools for data manipulation and analysis. Pandas has several functions to read data from various file formats, such as CSV (`read_csv()`), Excel (`read_excel()`), JSON (`read_json()`), and more.

You can specify a path to a local file or even a URL to download the dataset directly from the web:

```
[6]: # Importing the Titanic dataset. (Rajagopalan, 2021, p. 106)
titanic = pd.read_csv('https://raw.githubusercontent.com/DataRepo2019/
↳Data-files/master/titanic.csv')
# Show the first few records of the Data Frame
titanic.head()
```

```
[6]:
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

If you want to open a dataset that you stored on Google Drive (where Google Colab stores all your Notebooks), you can use the following code instead:

```
[ ]: # Ensure your Google Drive is accessible from within the notebook
from google.colab import drive
drive.mount('/content/drive')

# Load the dataset from your Google Drive (here, we stored it in a subfolder
# called "data")
titanic = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/titanic.csv')
titanic.head()
```

Let's take a look at the properties of the dataset we just loaded:

```
[8]: # How many rows does the DataFrame have?
print(f"Number of rows: {len(titanic)}")
# How many columns?
print(f"Number of columns: {len(titanic.columns)}")
# How many rows and columns, i.e. the shape
print(f"The shape of the Data Frame is: {titanic.shape}")
# General information about the DataFrame
```

```

print("*"*50)
titanic.info()

# Give the data type of each column.
print("*"*50)
print(titanic.dtypes)

# How many columns of each data type are there?
# Watch it! The book says to use get_dtype_counts(), but this method no
↳ longer exists
print("*"*50)
print(titanic.dtypes.value_counts())

```

```

Number of rows: 891
Number of columns: 12
The shape of the Data Frame is: (891, 12)
*****
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId      891 non-null   int64
1   Survived         891 non-null   int64
2   Pclass           891 non-null   int64
3   Name             891 non-null   str
4   Sex              891 non-null   str
5   Age              714 non-null   float64
6   SibSp            891 non-null   int64
7   Parch            891 non-null   int64
8   Ticket           891 non-null   str
9   Fare             891 non-null   float64
10  Cabin            204 non-null   str
11  Embarked         889 non-null   str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
*****
PassengerId      int64
Survived          int64
Pclass            int64
Name              str
Sex               str
Age               float64
SibSp             int64
Parch             int64
Ticket            str
Fare              float64
Cabin             str

```



```

Embarked      str
dtype: object
*****
int64         5
str           5
float64       2
Name: count, dtype: int64

```

1.5.1 Indices

The column “PassengerId” is not an actual variable, but contains a number to identify each observation. You can mark this column as an index:

```
[9]: titanic.set_index(['PassengerId'])
```

```
[9]:
```

	Survived	Pclass	\
PassengerId			
1	0	3	
2	1	1	
3	1	3	
4	1	1	
5	0	3	
...	...	...	
887	0	2	
888	1	1	
889	0	3	
890	1	1	
891	0	3	

	Name	Sex	Age	\
PassengerId				
1	Braund, Mr. Owen Harris	male	22.0	
2	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	
3	Heikkinen, Miss. Laina	female	26.0	
4	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	
5	Allen, Mr. William Henry	male	35.0	
...	...	...	...	
887	Montvila, Rev. Juozas	male	27.0	
888	Graham, Miss. Margaret Edith	female	19.0	
889	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	
890	Behr, Mr. Karl Howell	male	26.0	
891	Dooley, Mr. Patrick	male	32.0	

	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId						
1	1	0	A/5 21171	7.2500	NaN	S
2	1	0	PC 17599	71.2833	C85	C
3	0	0	STON/O2. 3101282	7.9250	NaN	S

4	1	0	113803	53.1000	C123	S
5	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...
887	0	0	211536	13.0000	NaN	S
888	0	0	112053	30.0000	B42	S
889	1	2	W./C. 6607	23.4500	NaN	S
890	0	0	111369	30.0000	C148	C
891	0	0	370376	7.7500	NaN	Q

[891 rows x 11 columns]

1.5.2 Qualitative variables

Earlier, we stated that the computer is not always able to correctly identify the measurement levels of variables. In this dataset, some of the variables, such as `Survived` and `Pclass`, are incorrectly considered to be quantitative. You can correct this by explicitly converting them to a **qualitative** (categorical) variable:

```
[10]: # Describe the variable Survived -> is treated as quantitative
print(titanic.Survived.describe())
# Convert to a categorical variable
titanic.Survived = titanic.Survived.astype('category')
# Call describe() again -> now it is treated as qualitative
print(titanic.Survived.describe())
```

```
count      891.000000
mean        0.383838
std         0.486592
min         0.000000
25%         0.000000
50%         0.000000
75%         1.000000
max         1.000000
Name: Survived, dtype: float64
count      891
unique      2
top         0
freq       549
Name: Survived, dtype: int64
```

You can also mark variables as **ordinal**, that is, with an ordering. We will do this as an example with the variable “Embarked” and order the ports by the order in which the Titanic stopped there. The Titanic departed at Southampton, and then picked up passengers first at Cherbourg and then at Queenstown.

For cases like this, define your own datatype specifying the order:

```
[11]: print(titanic.Embarked.unique())
```

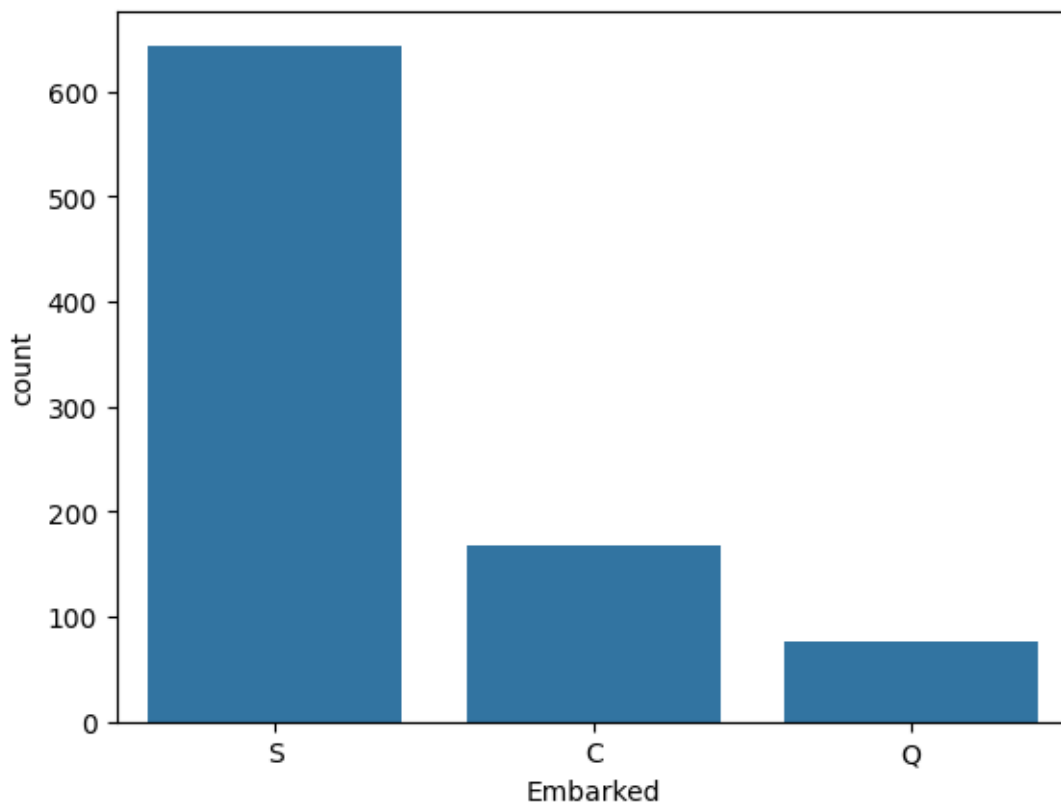
```
embarked_type = CategoricalDtype(categories=['S', 'C', 'Q'], ordered=True)
titanic.Embarked = titanic.Embarked.astype(embarked_type)
titanic.Embarked.describe()
```

```
<StringArray>
['S', 'C', 'Q', nan]
Length: 4, dtype: str
```

```
[11]: count      889
      unique      3
      top         S
      freq       644
      Name: Embarked, dtype: object
```

This order will then always be respected, e.g. in graphs

```
[12]: sns.countplot(data=titanic, x='Embarked');
```



1.5.3 Selecting Data

```
[13]: # Select all observations for a single variable (i.e. a DataFrame column)
      titanic.Age
      # This also works (and is preferable as it will also work when the column name
      #   ↪ has a space in it):
      # titanic['Age']
      # This also works, but isn't very nice
      # titanic.loc[:, 'Age']
```

```
[13]: 0      22.0
      1      38.0
      2      26.0
      3      35.0
      4      35.0
      ...
      886    27.0
      887    19.0
      888     NaN
      889    26.0
      890    32.0
      Name: Age, Length: 891, dtype: float64
```

```
[14]: # Select adjacent columns
      titanic.iloc[:, 2:4]
```

```
[14]:
```

	Pclass	Name
0	3	Braund, Mr. Owen Harris
1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...
2	3	Heikkinen, Miss. Laina
3	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)
4	3	Allen, Mr. William Henry
..	...	...
886	2	Montvila, Rev. Juozas
887	1	Graham, Miss. Margaret Edith
888	3	Johnston, Miss. Catherine Helen "Carrie"
889	1	Behr, Mr. Karl Howell
890	3	Dooley, Mr. Patrick

[891 rows x 2 columns]

You can also select multiple columns based on their names. This is often clearer than selecting based on position and the columns must not be adjacent.

```
[15]: titanic[['Name', 'Age', 'Cabin']] # Note: two sets of square brackets!
```

```
[15]:
```

	Name	Age	Cabin
0	Braund, Mr. Owen Harris	22.0	NaN

1	Cummings, Mrs. John Bradley (Florence Briggs Th...	38.0	C85
2	Heikkinen, Miss. Laina	26.0	NaN
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	C123
4	Allen, Mr. William Henry	35.0	NaN
..	...	...	...
886	Montvila, Rev. Juozas	27.0	NaN
887	Graham, Miss. Margaret Edith	19.0	B42
888	Johnston, Miss. Catherine Helen "Carrie"	NaN	NaN
889	Behr, Mr. Karl Howell	26.0	C148
890	Dooley, Mr. Patrick	32.0	NaN

[891 rows x 3 columns]

```
[16]: # Observation with row number 5 (counting from zero)
print(titanic.iloc[5])

# The first 4 observations
titanic.iloc[0:4]
```

PassengerId	6
Survived	0
Pclass	3
Name	Moran, Mr. James
Sex	male
Age	NaN
SibSp	0
Parch	0
Ticket	330877
Fare	8.4583
Cabin	NaN
Embarked	Q

Name: 5, dtype: object

```
[16]: PassengerId Survived Pclass \
0      1      0      3
1      2      1      1
2      3      1      3
3      4      1      1
```

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cummings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C

2	0	STON/O2.	3101282	7.9250	NaN	S
3	0		113803	53.1000	C123	S

```
[17]: # Select observations where the value of Age is less than 18
titanic[titanic.Age < 18]

# The same, but only keep the column 'Embarked'
titanic[titanic.Age < 18].Embarked

# The same, but keep columns 'Age' and 'Embarked'
titanic[titanic['Age'] < 18][['Age', 'Embarked']]
```

```
[17]:      Age Embarked
7      2.0        S
9     14.0        C
10     4.0        S
14     4.0        S
16     2.0        Q
..     ...      ...
850    4.0        S
852    9.0        C
853   16.0        S
869    4.0        S
875   15.0        C

[113 rows x 2 columns]
```

```
[18]: # Select all boys younger than 10
titanic.query("(Sex=='male') and (Age < 18)")
```

```
[18]:      PassengerId  Survived  Pclass  \
7              8         0         3
16             17         0         3
50             51         0         3
59             60         0         3
63             64         0         3
78             79         1         2
86             87         0         3
125            126         1         3
138            139         0         3
163            164         0         3
164            165         0         3
165            166         1         3
171            172         0         3
182            183         0         3
183            184         1         2
193            194         1         2
```

220	221	1	3
261	262	1	3
266	267	0	3
278	279	0	3
282	283	0	3
305	306	1	1
333	334	0	3
340	341	1	2
348	349	1	3
352	353	0	3
386	387	0	3
407	408	1	2
433	434	0	3
445	446	1	1
480	481	0	3
489	490	1	3
500	501	0	3
532	533	0	3
549	550	1	2
550	551	1	1
574	575	0	3
683	684	0	3
686	687	0	3
721	722	0	3
731	732	0	3
746	747	0	3
751	752	1	3
755	756	1	2
764	765	0	3
787	788	0	3
788	789	1	3
791	792	0	2
802	803	1	1
803	804	1	3
819	820	0	3
824	825	0	3
827	828	1	2
831	832	1	2
841	842	0	2
844	845	0	3
850	851	0	3
869	870	1	3

	Name	Sex	Age	SibSp	\
7	Palsson, Master. Gosta Leonard	male	2.00	3	
16	Rice, Master. Eugene	male	2.00	4	
50	Panula, Master. Juha Niilo	male	7.00	4	

59	Goodwin, Master. William Frederick	male	11.00	5
63	Skoog, Master. Harald	male	4.00	3
78	Caldwell, Master. Alden Gates	male	0.83	0
86	Ford, Mr. William Neal	male	16.00	1
125	Nicola-Yarred, Master. Elias	male	12.00	1
138	Osen, Mr. Olaf Elon	male	16.00	0
163	Calic, Mr. Jovo	male	17.00	0
164	Panula, Master. Eino Viljami	male	1.00	4
165	Goldsmith, Master. Frank John William "Frankie"	male	9.00	0
171	Rice, Master. Arthur	male	4.00	4
182	Asplund, Master. Clarence Gustaf Hugo	male	9.00	4
183	Becker, Master. Richard F	male	1.00	2
193	Navratil, Master. Michel M	male	3.00	1
220	Sunderland, Mr. Victor Francis	male	16.00	0
261	Asplund, Master. Edwin Rojj Felix	male	3.00	4
266	Panula, Mr. Ernesti Arvid	male	16.00	4
278	Rice, Master. Eric	male	7.00	4
282	de Pelsmaecker, Mr. Alfons	male	16.00	0
305	Allison, Master. Hudson Trevor	male	0.92	1
333	Vander Planke, Mr. Leo Edmondus	male	16.00	2
340	Navratil, Master. Edmond Roger	male	2.00	1
348	Coutts, Master. William Loch "William"	male	3.00	1
352	Elias, Mr. Tannous	male	15.00	1
386	Goodwin, Master. Sidney Leonard	male	1.00	5
407	Richards, Master. William Rowe	male	3.00	1
433	Kallio, Mr. Nikolai Erland	male	17.00	0
445	Dodge, Master. Washington	male	4.00	0
480	Goodwin, Master. Harold Victor	male	9.00	5
489	Coutts, Master. Eden Leslie "Neville"	male	9.00	1
500	Calic, Mr. Petar	male	17.00	0
532	Elias, Mr. Joseph Jr	male	17.00	1
549	Davies, Master. John Morgan Jr	male	8.00	1
550	Thayer, Mr. John Borland Jr	male	17.00	0
574	Rush, Mr. Alfred George John	male	16.00	0
683	Goodwin, Mr. Charles Edward	male	14.00	5
686	Panula, Mr. Jaako Arnold	male	14.00	4
721	Jensen, Mr. Svend Lauritz	male	17.00	1
731	Hassan, Mr. Houssein G N	male	11.00	0
746	Abbott, Mr. Rossmore Edward	male	16.00	1
751	Moor, Master. Meier	male	6.00	0
755	Hamalainen, Master. Viljo	male	0.67	1
764	Eklund, Mr. Hans Linus	male	16.00	0
787	Rice, Master. George Hugh	male	8.00	4
788	Dean, Master. Bertram Vere	male	1.00	1
791	Gaskell, Mr. Alfred	male	16.00	0
802	Carter, Master. William Thornton II	male	11.00	1
803	Thomas, Master. Assad Alexander	male	0.42	0

819		Skoog, Master. Karl Thorsten	male	10.00	3
824		Panula, Master. Urho Abraham	male	2.00	4
827		Mallet, Master. Andre	male	1.00	0
831		Richards, Master. George Sibley	male	0.83	1
841		Mudd, Mr. Thomas Charles	male	16.00	0
844		Culumovic, Mr. Jeso	male	17.00	0
850		Andersson, Master. Sigvard Harald Elias	male	4.00	4
869		Johnson, Master. Harold Theodor	male	4.00	1

	Parch		Ticket	Fare	Cabin	Embarked
7	1		349909	21.0750	NaN	S
16	1		382652	29.1250	NaN	Q
50	1		3101295	39.6875	NaN	S
59	2		CA 2144	46.9000	NaN	S
63	2		347088	27.9000	NaN	S
78	2		248738	29.0000	NaN	S
86	3		W./C. 6608	34.3750	NaN	S
125	0		2651	11.2417	NaN	C
138	0		7534	9.2167	NaN	S
163	0		315093	8.6625	NaN	S
164	1		3101295	39.6875	NaN	S
165	2		363291	20.5250	NaN	S
171	1		382652	29.1250	NaN	Q
182	2		347077	31.3875	NaN	S
183	1		230136	39.0000	F4	S
193	1		230080	26.0000	F2	S
220	0	SOTON/OQ	392089	8.0500	NaN	S
261	2		347077	31.3875	NaN	S
266	1		3101295	39.6875	NaN	S
278	1		382652	29.1250	NaN	Q
282	0		345778	9.5000	NaN	S
305	2		113781	151.5500	C22 C26	S
333	0		345764	18.0000	NaN	S
340	1		230080	26.0000	F2	S
348	1	C.A.	37671	15.9000	NaN	S
352	1		2695	7.2292	NaN	C
386	2		CA 2144	46.9000	NaN	S
407	1		29106	18.7500	NaN	S
433	0	STON/O 2.	3101274	7.1250	NaN	S
445	2		33638	81.8583	A34	S
480	2		CA 2144	46.9000	NaN	S
489	1	C.A.	37671	15.9000	NaN	S
500	0		315086	8.6625	NaN	S
532	1		2690	7.2292	NaN	C
549	1	C.A.	33112	36.7500	NaN	S
550	2		17421	110.8833	C70	C
574	0	A/4.	20589	8.0500	NaN	S

683	2	CA 2144	46.9000	NaN	S
686	1	3101295	39.6875	NaN	S
721	0	350048	7.0542	NaN	S
731	0	2699	18.7875	NaN	C
746	1	C.A. 2673	20.2500	NaN	S
751	1	392096	12.4750	E121	S
755	1	250649	14.5000	NaN	S
764	0	347074	7.7750	NaN	S
787	1	382652	29.1250	NaN	Q
788	2	C.A. 2315	20.5750	NaN	S
791	0	239865	26.0000	NaN	S
802	2	113760	120.0000	B96 B98	S
803	1	2625	8.5167	NaN	C
819	2	347088	27.9000	NaN	S
824	1	3101295	39.6875	NaN	S
827	2	S.C./PARIS 2079	37.0042	NaN	C
831	1	29106	18.7500	NaN	S
841	0	S.O./P.P. 3	10.5000	NaN	S
844	0	315090	8.6625	NaN	S
850	2	347082	31.2750	NaN	S
869	1	347742	11.1333	NaN	S

1.5.4 Dropping Data and Working with Missing Data

Pandas tries to make working with missing data as easy as possible. E.g., all of the descriptive statistics on pandas objects exclude missing data by default.

Let's start by reading the titanic data again.

```
[19]: titanic = pd.read_csv('https://raw.githubusercontent.com/DataRepo2019/
↳Data-files/master/titanic.csv')
titanic.head()
```

```
[19]: PassengerId  Survived  Pclass  \
0             1         0         3
1             2         1         1
2             3         1         3
3             4         1         1
4             5         0         3

                                     Name    Sex  Age  SibSp  \
0                        Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                        Heikkinen, Miss. Laina    female  26.0      0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)    female  35.0      1
4                        Allen, Mr. William Henry    male  35.0      0

Parch          Ticket          Fare Cabin Embarked
```

0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

The `PassengerId` column is identical to the index hence it doesn't provide any additional useful information. Let's drop this column (rather than setting it as the index as before). This can be done using `drop`.

```
[20]: titanic.drop("PassengerId", axis="columns")
```

```
[20]:
```

	Survived	Pclass	Name \
0	0	3	Braund, Mr. Owen Harris
1	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...
2	1	3	Heikkinen, Miss. Laina
3	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)
4	0	3	Allen, Mr. William Henry
..	...	...	...
886	0	2	Montvila, Rev. Juozas
887	1	1	Graham, Miss. Margaret Edith
888	0	3	Johnston, Miss. Catherine Helen "Carrie"
889	1	1	Behr, Mr. Karl Howell
890	0	3	Dooley, Mr. Patrick

	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	female	38.0	1	0	PC 17599	71.2833	C85	C
2	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	female	35.0	1	0	113803	53.1000	C123	S
4	male	35.0	0	0	373450	8.0500	NaN	S
..	...	...	...	...	...	...	...	...
886	male	27.0	0	0	211536	13.0000	NaN	S
887	female	19.0	0	0	112053	30.0000	B42	S
888	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	male	26.0	0	0	111369	30.0000	C148	C
890	male	32.0	0	0	370376	7.7500	NaN	Q

[891 rows x 11 columns]

Notice that we didn't actually change the `titanic` `DataFrame`. We can either assign the result of `drop` to a (new) `DataFrame` or we can use `inplace=True`.

```
[21]: titanic.head(3) # PassengerId is still here
```

```
[21]:
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	

```
2          3          1          3
```

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S

```
[22]: titanic = titanic.drop("PassengerId", axis="columns")
titanic.head(3) # PassengerId is now gone
```

```
[22]:
```

	Survived	Pclass	Name	\
0	0	3	Braund, Mr. Owen Harris	
1	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	
2	1	3	Heikkinen, Miss. Laina	

	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	female	38.0	1	0	PC 17599	71.2833	C85	C
2	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S

From the output of the `info` method we can infer that certain columns contain many missing values, e.g. `Cabin` only contains 204 non-missing values. For the purposes of illustration, let's drop any row that has a missing observation by using the default behaviour of the `dropna` method.

```
[23]: titanic.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Survived    891 non-null    int64
1   Pclass      891 non-null    int64
2   Name        891 non-null    str
3   Sex         891 non-null    str
4   Age         714 non-null    float64
5   SibSp       891 non-null    int64
6   Parch       891 non-null    int64
7   Ticket      891 non-null    str
8   Fare        891 non-null    float64
9   Cabin       204 non-null    str
10  Embarked    889 non-null    str
dtypes: float64(2), int64(4), str(5)
```

memory usage: 76.7 KB

```
[24]: cleaned = titanic.dropna() # Drop any row that has at least one missing value
      print(cleaned.info())
      cleaned.head(5)
```

```
<class 'pandas.DataFrame'>
Index: 183 entries, 1 to 889
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Survived    183 non-null     int64
1   Pclass      183 non-null     int64
2   Name        183 non-null     str
3   Sex         183 non-null     str
4   Age         183 non-null     float64
5   SibSp       183 non-null     int64
6   Parch       183 non-null     int64
7   Ticket      183 non-null     str
8   Fare        183 non-null     float64
9   Cabin       183 non-null     str
10  Embarked    183 non-null     str
dtypes: float64(2), int64(4), str(5)
memory usage: 17.2 KB
None
```

```
[24]:
```

	Survived	Pclass	Name \
1	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...
3	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)
6	0	1	McCarthy, Mr. Timothy J
10	1	3	Sandstrom, Miss. Marguerite Rut
11	1	1	Bonnell, Miss. Elizabeth

	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	female	38.0	1	0	PC 17599	71.2833	C85	C
3	female	35.0	1	0	113803	53.1000	C123	S
6	male	54.0	0	0	17463	51.8625	E46	S
10	female	4.0	1	1	PP 9549	16.7000	G6	S
11	female	58.0	0	0	113783	26.5500	C103	S

This would only keep 183 values, which isn't a lot. We could also choose to drop only the rows that consist of nothing but missing values (there are no such rows in the `titanic DataFrame`), but here you can see how this would be done.

```
[25]: cleaned = titanic.dropna(how="all")
      cleaned.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
```

Data columns (total 11 columns):

#	Column	Non-Null Count	Dtype
0	Survived	891 non-null	int64
1	Pclass	891 non-null	int64
2	Name	891 non-null	str
3	Sex	891 non-null	str
4	Age	714 non-null	float64
5	SibSp	891 non-null	int64
6	Parch	891 non-null	int64
7	Ticket	891 non-null	str
8	Fare	891 non-null	float64
9	Cabin	204 non-null	str
10	Embarked	889 non-null	str

dtypes: float64(2), int64(4), str(5)
memory usage: 76.7 KB

Instead of relying on `info` to compute the number of non-missing values we can also use the `isnull` and/or `notnull` methods. `notnull` computes a boolean `DataFrame` where an entry is `True` if and only if that entry is considered to be non-missing in the original `DataFrame`. We can then use `sum` to compute the sum of each column. This gives the number of non-missing values per column.

```
[26]: not_null_df = titanic.notnull()
print(not_null_df.tail(3))
print("Number of non null values in each column:")
print(not_null_df.sum())
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	\
888	True	True	True	True	False	True	True	True	True	False	
889	True	True	True	True	True	True	True	True	True	True	
890	True	True	True	True	True	True	True	True	True	False	

	Embarked
888	True
889	True
890	True

Number of non null values in each column:

Survived	891
Pclass	891
Name	891
Sex	891
Age	714
SibSp	891
Parch	891
Ticket	891
Fare	891
Cabin	204
Embarked	889

dtype: int64

An even easier alternative is to simply use `count` on the `DataFrame`.

```
[27]: print(titanic.count())
```

```
Survived    891
Pclass      891
Name        891
Sex         891
Age        714
SibSp       891
Parch       891
Ticket      891
Fare        891
Cabin       204
Embarked    889
dtype: int64
```

The `Cabin` column has too many missing values, so probably isn't useful. For the `Age` column, we can *impute* the missing values, e.g. with the average age of all passengers. (We will see later what this means exactly). Let's do this using `fillna`.

```
[28]: # First compute the average age
avg_age = titanic['Age'].mean()
print(f"(Rounded) Average age of passengers: {round(avg_age)}")
titanic = titanic.fillna(value={'Age' : avg_age})
print("We should now confirm that the 'Age' column no longer has missing_
      ↪values")
titanic.info()
```

```
(Rounded) Average age of passengers: 30
```

```
We should now confirm that the 'Age' column no longer has missing values
```

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 891 entries, 0 to 890
```

```
Data columns (total 11 columns):
```

#	Column	Non-Null Count	Dtype
0	Survived	891 non-null	int64
1	Pclass	891 non-null	int64
2	Name	891 non-null	str
3	Sex	891 non-null	str
4	Age	891 non-null	float64
5	SibSp	891 non-null	int64
6	Parch	891 non-null	int64
7	Ticket	891 non-null	str
8	Fare	891 non-null	float64
9	Cabin	204 non-null	str
10	Embarked	889 non-null	str

```
dtypes: float64(2), int64(4), str(5)
```

```
memory usage: 76.7 KB
```

1.5.5 Creating New Columns

We use a dataset containing NO_2 measurements (expressed as ppb - parts pro billion) at the stations of Paris, Antwerp and London.

Let's read the data and check whether it worked.

```
[29]: URL = "https://raw.githubusercontent.com/pandas-dev/pandas/master/doc/data/
      ↪air_quality_no2.csv"
      air_quality = pd.read_csv(URL)
      print(air_quality.info())
      air_quality.head()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1035 entries, 0 to 1034
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   datetime              1035 non-null  str
1   station_antwerp       95 non-null    float64
2   station_paris         1004 non-null  float64
3   station_london        969 non-null   float64
dtypes: float64(3), str(1)
memory usage: 32.5 KB
None
```

```
[29]:      datetime  station_antwerp  station_paris  station_london
0  2019-05-07 02:00:00           NaN           NaN           23.0
1  2019-05-07 03:00:00          50.5           25.0           19.0
2  2019-05-07 04:00:00          45.0           27.7           19.0
3  2019-05-07 05:00:00           NaN           50.4           16.0
4  2019-05-07 06:00:00           NaN           61.9           NaN
```

We see that - the dates are stored as strings (str dtype) - `datetime` is added as a column and we have a `RangeIndex`.

We would like to - parse the dates, so that they are available as `datetime` objects - index the `DataFrame` rows by this date.

This can be achieved by specifying the index column and by indicating that dates must be parsed on `read_csv`. Let's try again.

```
[30]: air_quality = pd.read_csv(URL, index_col=0, parse_dates=True)
      print(air_quality.info())
      air_quality.head()
```

```
<class 'pandas.DataFrame'>
DatetimeIndex: 1035 entries, 2019-05-07 02:00:00 to 2019-06-21 02:00:00
Data columns (total 3 columns):
#   Column                Non-Null Count  Dtype
---  -
1   station_antwerp       95 non-null    float64
2   station_paris         1004 non-null  float64
3   station_london        969 non-null   float64
```



```

0    station_antwerp    95 non-null    float64
1    station_paris     1004 non-null   float64
2    station_london    969 non-null    float64
dtypes: float64(3)
memory usage: 32.3 KB
None

```

```

[30]:          station_antwerp  station_paris  station_london
datetime
2019-05-07 02:00:00          NaN          NaN          23.0
2019-05-07 03:00:00         50.5         25.0          19.0
2019-05-07 04:00:00         45.0         27.7          19.0
2019-05-07 05:00:00          NaN         50.4          16.0
2019-05-07 06:00:00          NaN         61.9          NaN

```

We can clearly see the difference!

Working with the datetime datatype is especially interesting if you want to do date time calculations:

```

[42]: # calculate difference between each index and the previous one, expressed in
      ↪ hours
      # to_series() converts the index data type to a pandas Series
      # diff() calculates the difference between each value and the previous one
      # dt.total_seconds() converts the difference to seconds, and we divide by 3600
      ↪ to convert to hours
air_quality['timespan'] = air_quality.index.to_series().diff().dt.
      ↪ total_seconds() / 3600
air_quality.head()

```

```

[42]:          station_antwerp  station_paris  station_london  \
datetime
2019-05-07 02:00:00          NaN          NaN          23.0
2019-05-07 03:00:00         50.5         25.0          19.0
2019-05-07 04:00:00         45.0         27.7          19.0
2019-05-07 05:00:00          NaN         50.4          16.0
2019-05-07 06:00:00          NaN         61.9          NaN

          london_mg_per_cubic  ratio_paris_antwerp  timespan
datetime
2019-05-07 02:00:00         43.286                NaN        NaN
2019-05-07 03:00:00         35.758             0.495050         1.0
2019-05-07 04:00:00         35.758             0.615556         1.0
2019-05-07 05:00:00         30.112                NaN         1.0
2019-05-07 06:00:00          NaN                NaN         1.0

```

```

[43]: # now we can check if the timespan is consistent (i.e. always 1 hour)
air_quality.timespan.value_counts()

```

```
[43]: timespan
      1.0      1031
      11.0       2
      27.0       1
      Name: count, dtype: int64
```

There are clearly some gaps.

1.5.6 Creating a New Column Derived from (an) Existing Column(s)

Assume we would like to express the concentration of the station in London in $\mu\text{g}/\text{m}^3$ (If we assume temperature of 25 degrees Celsius and pressure of 1013 hPa, the conversion factor is 1.882)

```
[31]: air_quality["london_mg_per_cubic"] = air_quality["station_london"] * 1.882
      air_quality.head()
```

```
[31]:
```

	station_antwerp	station_paris	station_london \
datetime			
2019-05-07 02:00:00	NaN	NaN	23.0
2019-05-07 03:00:00	50.5	25.0	19.0
2019-05-07 04:00:00	45.0	27.7	19.0
2019-05-07 05:00:00	NaN	50.4	16.0
2019-05-07 06:00:00	NaN	61.9	NaN


```

      london_mg_per_cubic
      datetime
      2019-05-07 02:00:00      43.286
      2019-05-07 03:00:00      35.758
      2019-05-07 04:00:00      35.758
      2019-05-07 05:00:00      30.112
      2019-05-07 06:00:00         NaN
```

To create a new column, use the `[]` brackets with the new column name at the left side of the assignment. The computation is done **element wise**, hence there is no need for an explicit loop!

We can also use multiple columns to derive a new column.

Let's check the ratio of the values in Paris versus Antwerp and save the result in a new column.

```
[32]: air_quality["ratio_paris_antwerp"] = air_quality["station_paris"] /
      ↪air_quality["station_antwerp"]
      air_quality.head()
```

```
[32]:
```

	station_antwerp	station_paris	station_london \
datetime			
2019-05-07 02:00:00	NaN	NaN	23.0
2019-05-07 03:00:00	50.5	25.0	19.0
2019-05-07 04:00:00	45.0	27.7	19.0
2019-05-07 05:00:00	NaN	50.4	16.0
2019-05-07 06:00:00	NaN	61.9	NaN

	london_mg_per_cubic	ratio_paris_antwerp
datetime		
2019-05-07 02:00:00	43.286	NaN
2019-05-07 03:00:00	35.758	0.495050
2019-05-07 04:00:00	35.758	0.615556
2019-05-07 05:00:00	30.112	NaN
2019-05-07 06:00:00	NaN	NaN

We can also perform more general mappings on columns, either using a function or a dictionary.

Let's work with the Titanic data once more. Let's say we want to numerically encode the **Sex** column, this can be done as follows. First let's check the unique values in the **Sex** column.

```
[33]: titanic['Sex'].unique()
```

```
[33]: <StringArray>
      ['male', 'female']
      Length: 2, dtype: str
```

```
[34]: mapping_dict = {'male' : 0, 'female' : 1}
      titanic['Sex'] = titanic['Sex'].map(mapping_dict)
```

Let's check whether it worked.

```
[35]: titanic.head()
```

```
[35]:
```

	Survived	Pclass		Name	Sex	\
0	0	3		Braund, Mr. Owen Harris	0	
1	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...		1	
2	1	3		Heikkinen, Miss. Laina	1	
3	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)		1	
4	0	3		Allen, Mr. William Henry	0	

	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	22.0	1	0	A/5 21171	7.2500	NaN	S
1	38.0	1	0	PC 17599	71.2833	C85	C
2	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	35.0	1	0	113803	53.1000	C123	S
4	35.0	0	0	373450	8.0500	NaN	S

Let's add a new feature (i.e. a new column). When **Age** is less than 12 we will call this passenger a **child**, between 12 and 18 a **teen** and over 18 will be **adults**. First, we define a Python function implementing this mapping, and then we apply it to the **Age** column.

```
[36]: def age_to_category(age):
      if age < 12:
          return "child"
      if age < 18:
```

```

    return "teen"
    return "adult"

```

```

titanic['AgeCategory'] = titanic['Age'].map(age_to_category)
titanic.head()

```

```

[36]:
Survived  Pclass                                     Name  Sex  \
0         0         3                                Braund, Mr. Owen Harris    0
1         1         1  Cumings, Mrs. John Bradley (Florence Briggs Th...    1
2         1         3                                Heikkinen, Miss. Laina    1
3         1         1      Futrelle, Mrs. Jacques Heath (Lily May Peel)    1
4         0         3                                Allen, Mr. William Henry    0

Age  SibSp  Parch                Ticket    Fare Cabin Embarked AgeCategory
0  22.0    1     0          A/5 21171    7.2500   NaN        S      adult
1  38.0    1     0          PC 17599   71.2833   C85        C      adult
2  26.0    0     0  STON/O2. 3101282    7.9250   NaN        S      adult
3  35.0    1     0          113803   53.1000  C123        S      adult
4  35.0    0     0          373450    8.0500   NaN        S      adult

```

Here, we only scratched the surface of what is possible with pandas (and indeed, whole books have been written about pandas). In general the pandas getting started guide, the user guide and the API reference are very good resources to find additional information. You can find all of these [here](#).

2 Module 2. Univariate analysis

Univariate analysis is a form of descriptive statistics that focuses on analyzing and summarizing data for a single variable at a time. Specifically, we look for values that are representative for the variable (also called *measures of central tendency*) and values that indicate how much the variable varies (called *measures of dispersion*). We also create *visualizations* to understand the distribution of the variable.

2.1 Learning goals

- You understand the definitions (indicated with $\propto$) of the most common measures of central tendency and dispersion, and you are able to calculate them in Python
- You are able to create and interpret the most common visualizations for univariate analysis in Python
- You are able to select the most suitable analysis and visualization techniques for a given variable, based on its measurement level

```

[1]: # Importing the necessary packages
import numpy as np                                # "Scientific computing"
import scipy.stats as stats                       # Statistical tests

import pandas as pd                               # Data Frame
from pandas.api.types import CategoricalDtype

```

```
import matplotlib.pyplot as plt           # Basic visualisation
from statsmodels.graphics.mosaicplot import mosaic # Mosaic diagram
import seaborn as sns                     # Advanced data visualisation
↳ visualisation
```

To calculate the measures of central tendency and dispersion for these variables, and for visualization, we will use mainly use the Python libraries **pandas**, **numpy**, and **seaborn**.

Seaborn contains some datasets that we will use here as examples, e.g., the **tips** dataset.

Pandas defines several functions to calculate summary statistics for a variable. When you select a column (e.g. **myvar**) of a DataFrame (e.g. **df**), you get a Series object, which has these functions as methods. For example, to get the mode of **myvar**, you can use **df.myvar.mode()**, to calculate the mean of **myvar**, you can use **df.myvar.mean()**, for the standard deviation **df.myvar.std()**, etc.

The libraries **numpy** and **scipy.stats** contain similar functions, that you can use as e.g. **scipy.stats.mode(df.myvar)**, **np.mean(df.myvar)**, or **np.std(df.myvar)**.

Depending on the situation, you may want to use one or the other. Keep in mind, however, that the behavior of these functions may differ in some cases! See e.g. the section on the denominator in the formula for standard deviation below for an example of this. Use the API documentation of these libraries to check the behavior of the functions you want to use, and make sure that they do what you expect them to do!

```
[2]: # Example dataset `tips`, included with Seaborn, containing information on tips
↳ given at restaurants
tips = sns.load_dataset("tips")
tips.head() # Show the first 5 rows of the dataset, to get an idea of the
↳ data structure
```

```
[2]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

Variables **total_bill**, **tip** and **size** are quantitative variables, while **sex**, **smoker**, **day** and **time** are qualitative variables.

2.2 Measures of central tendency

A **measure of central tendency** (or *mean*) for a variable is a value that is considered “typical” or “representative” for that group. There are several ways to calculate a measure of central tendency. Only a few are listed below.

Depending on the measurement level, specific measures are ideally suited:

- Qualitative variables: mode (and sometimes median for ordinal variables)
- Quantitative variables: arithmetic mean, median, and (sometimes) mode

In the media, you will often see the mean being used as a representative value (e.g. average income, average house price, etc.). However, the mean is not always the best choice, especially when the data is skewed or contains outliers. **Remember that it is never sufficient to rely solely on the mean to describe a dataset.** You should at least report a measure of dispersion (e.g. standard deviation, interquartile range) and ideally also a visualization of the data distribution.

2.2.1 Mode

The **mode** of a variable is the value that occurs most frequently in the *sample*. In principle you can also calculate the mode of a quantitative variable, but that is usually not useful, since the mode may not be representative of the data distribution, or each observation may have a unique value.

```
[3]: # Pandas: mode for all the variables in the DataFrame
tips.mode()
```

```
[3]:   total_bill  tip  sex smoker  day    time  size
0      13.42  2.0  Male     No   Sat  Dinner     2
```

```
[4]: # Pandas: mode for the variable `size`
tips['size'].mode()
# Remark that tips.size is not possible here, as `size` is an attribute of the
↳ DataFrame `tips`
```

```
[4]: 0      2
      Name: size, dtype: int64
```

```
[5]: # scipy.stats: mode for the variable `size`
stats.mode(tips['size'])
# Remark that scipy.stats.mode() only works on numeric variables!
```

```
[5]: ModeResult(mode=np.int64(2), count=np.int64(156))
```

```
[6]: # Mode for a specific variable
tips.day.mode()
```

```
[6]: 0      Sat
      Name: day, dtype: category
      Categories (4, str): ['Thur', 'Fri', 'Sat', 'Sun']
```

```
[7]: # When applied to a categorical variable, the describe() function also shows
↳ the mode
tips.day.describe()
```

```
[7]: count      244
      unique       4
      top       Sat
      freq       87
      Name: day, dtype: object
```

2.2.2 Median, quartiles, and percentiles

The **median** of a sample or population is the value that separates the lower half of the values from the higher half. You can determine the median by sorting the values and taking the middle value (if the number is odd), or the average of the two middle values (if the number is even).

The median can be used with quantitative variables or ordinal variables.

The *measure of dispersion* that is often used in combination with the median is the *interquartile range*.

```
[8]: # Pandas: median for the variable `tip`  
tips.tip.median()
```

```
[8]: np.float64(2.9)
```

```
[9]: # Numpy: median for the variable `tip`  
np.median(tips.tip)
```

```
[9]: np.float64(2.9)
```

2.2.3 Arithmetic mean or average

The **arithmetic mean** (or **average**) is a measure of central tendency that is especially suited for quantitative variables.

The mean is often used in combination with the standard deviation to describe the characteristics of a sample.

- The **sample mean** $\bar{x}$ of a sample $X = \{x_1, \dots, x_n\}$ of size n is

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

- The **population mean** is notated with the Greek letter μ (mu). The calculation is equivalent with the sample mean.

```
[10]: tips.tip.mean()
```

```
[10]: np.float64(2.99827868852459)
```

2.3 Measures of dispersion

A **measure of dispersion** expresses the extent to which values in a sample or population differ from each other.

There are several ways to determine a measure of dispersion, we will use only a few of them in this course. The measures of dispersion discussed are only suitable for *quantitative variables*. Combined with a measure of central tendency, a measure of dispersion forms a way of expressing the characteristics of observations for a variable.

2.3.1 Quartiles and percentiles

The concept of the median can be generalized to the concept of *quantiles* and *percentiles*.

A **percentile** of an ordered series of observations is one of basically 99 values that divide the series into 100 equally sized parts. The k -th percentile, then, is a number where $k\%$ of the observations are smaller and $(100 - k)\%$ are greater than that number.

The **quartiles** are formed from five specific percentiles that divide the series into 4 equally sized parts:

- min, the minimum, or percentile 0
- Q_1 , the first quartile or percentile 25
- Q_2 , the median, or percentile 50
- Q_3 , the third quartile or percentile 75
- max, the maximum, or percentile 100

2.3.2 Range

The **range** of a sample or population X is calculated as:

$$\max(X) - \min(X)$$

With $\max(X)$ the largest value and $\min(X)$ the smallest value in X .

In many cases, the range is not a very useful measure of dispersion, since it is highly sensitive to outliers. For example, if you have a sample of 10 values, and one of them is an extreme outlier, the range will be very large, even if the other 9 values are close together.

2.3.3 Interquartile range

The **interquartile range (IQR)** of a sample or population is calculated as:

$$IQR = |Q_3 - Q_1|$$

With Q_3 the third *quartile* and Q_1 the first quartile. Between Q_1 and Q_3 , you will find 50% of the observations in the sample.

The interquartile range is less sensitive to outliers than the range or standard deviation and is therefore better suited than the standard deviation for variables that are not normally distributed.

```
[11]: # Calculating Median & related measures of dispersion:
print(f"Minimum:           {tips.tip.min()}")
print(f"Median:            {tips.tip.median()}")
print(f"Maximum:           {tips.tip.max()}")
percentiles = [0.0, 0.25, 0.5, 0.75, 1.0]
print(f"Percentiles       {percentiles}\n{tips.tip.quantile(percentiles)}" )
print(f"Inter Quartile Range: {stats.iqr(tips.tip)}")
print(f"Range :             {tips.tip.max() - tips.tip.min()}")
```



```

Minimum:          1.0
Median:           2.9
Maximum:          10.0
Percentiles       [0.0, 0.25, 0.5, 0.75, 1.0]
0.00      1.0000
0.25      2.0000
0.50      2.9000
0.75      3.5625
1.00     10.0000
Name: tip, dtype: float64
Inter Quartile Range: 1.5624999999999996
Range :          9.0

```

2.3.4 Variance and standard deviation

The **variance** is a measure of dispersion that is ideally suited for quantitative variables that are normally distributed.

- The *variance* s^2 of a sample $X = \{x_1, \dots, x_n\}$ of size n with mean $\bar{x}$ is calculated as follows:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

- The *variance* σ^2 (sigma squared) of a *population* of size N with mean μ is calculated as follows:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

The **standard deviation** is the square root of the variance:

- The *sample standard deviation*:

$$s = \sqrt{s^2} = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

- The *population standard deviation*:

$$\sigma = \sqrt{\sigma^2} = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$

The mean and standard deviation are often used together to describe the characteristics of a sample or population, especially if the data is normally distributed.

Remark: The *normal distribution* is discussed in more detail in chapter 3. For now, it is sufficient to know that the normal distribution is a bell-shaped distribution that is symmetric around the mean and that it occurs frequently in nature and in many scientific disciplines.

Ask yourself the following questions about the mean and standard deviation:

- Can the standard deviation be negative?
- What would be the smallest possible value for the standard deviation? What would that value indicate about the data?
- What effect do outliers have on the mean and standard deviation?
- What is the unit of measurement for the mean and standard deviation? How does that compare to the unit of measurement for the variance?

```
[12]: # Pandas: mean, standard deviation and variance
print(f"Mean: {tips.tip.mean()}")
print(f"Standard deviation: {tips.tip.std()}") # Pay attention: n-1 in the
↪denominator
print(f"Variance: {tips.tip.var()}") # Pay attention: n-1 in the
↪denominator
```

```
Mean: 2.99827868852459
Standard deviation: 1.3836381890011822
Variance: 1.914454638062471
```

```
[13]: # Numpy: mean, standard deviation and variance
print(f"Mean: {np.mean(tips.tip)}")
print(f"Standard deviation: {np.std(tips.tip)}") # Pay attention: n in the
↪denominator!!
print(f"Variance: {np.var(tips.tip)}") # Pay attention: n in the
↪denominator!!
```

```
Mean: 2.99827868852459
Standard deviation: 1.3807999538298954
Variance: 1.9066085124966412
```

```
[14]: # Pandas: applied to a numerical variable, the describe() function shows the
↪mean, standard deviation, and percentiles
tips.tip.describe()
```

```
[14]: count    244.000000
mean      2.998279
std       1.383638
min       1.000000
25%      2.000000
50%      2.900000
75%      3.562500
max      10.000000
Name: tip, dtype: float64
```

2.3.5 Skewness and kurtosis

The **skewness** of a distribution is a measure of the asymmetry of the distribution. A distribution is said to be *positively skewed* if it has a long tail on the right side, and *negatively skewed* if it has a long tail on the left side. A distribution that is symmetric has a skewness of 0.

The α **kurtosis** of a distribution reflects the presence of outliers in the distribution. A distribution with high kurtosis has more outliers than a distribution with lower kurtosis.

The formulas for calculating skewness and kurtosis (there are several methods) are beyond the scope of this course, but they can be easily calculated in Python. For a normal distribution, the results of these functions should be close to 0.

```
[15]: # Skewness and kurtosis:
print(f"Skewness:           {tips.tip.skew()}")
print(f"Kurtosis:          {tips.tip.kurtosis()}")
```

```
Skewness:           1.4654510370979401
Kurtosis:           3.648375873352852
```

The *skewness* of the variable `tip` is about 1.47, which indicates that the distribution of `tip` is positively skewed, meaning that there are more values on the right side of the distribution than on the left side. See the visualizations of the distribution of `tip` below to verify that this is indeed the case! The *kurtosis* is about 3.65, which indicates that the distribution of `tip` has more outliers than a normal distribution.

2.3.6 The denominator in the formula for Standard Deviation

Why do we use $n - 1$ in the denominator of the formula for the *standard deviation of a sample* and not n ? It was found that the former gives a better (i.e. “unbiased”) estimate for what the standard deviation in the population will be, based on our sample. We can prove this mathematically, but that is beyond the scope of this course. We will, however, investigate it empirically.

We will first define the functions for variance of the population and of a sample. These functions already exist in Python, but we will define them ourselves to clearly show the difference between the two formulas.

```
[16]: # Variance of the population
def pop_var(x):
    n = len(x)
    mean = sum(x) / n
    return 1/n * sum((x - mean) ** 2)

# Standard deviation of the population
def pop_sd(x):
    return np.sqrt(pop_var(x))

# Variance of a sample
def sample_var(x):
    n = len(x)
    mean = sum(x) / n
    return 1/(n-1) * sum((x - mean) ** 2)

# Standard deviation of the sample
def sample_sd(x):
    return np.sqrt(sample_var(x))
```

```
# If we calculate the variance in both ways, we see a difference:
a = np.array([4, 8, 6, 5, 3, 2, 8, 9, 2, 5])

print(f"Population variance: {pop_var(a)}")
print(f"Sample variance:      {sample_var(a)}")
print()
print(f"Population standard deviation: {pop_sd(a)}")
print(f"Sample standard deviation:      {sample_sd(a)}")
```

```
Population variance: 5.7600000000000001
Sample variance:      6.3999999999999995
```

```
Population standard deviation: 2.4000000000000004
Sample standard deviation:      2.5298221281347035
```

What is the best formula for the standard deviation of a sample? Let us assume that our population consists of all numbers from 0 to 100. We first calculate the mean and standard deviation of this population.

```
[17]: population = np.arange(0, 101)

population_mean = sum(population) / len(population)
print(f'Population mean: {population_mean}')

population_variance = pop_var(population)
print(f'Population variance: {population_variance}')

population_stdev = pop_sd(population)
print(f'Population standard deviation: {population_stdev}')
```

```
Population mean: 50.0
Population variance: 850.0
Population standard deviation: 29.154759474226502
```

Now for the actual experiment: we repeatedly take a random sample from this population and look at the result of both formulas for the variance. We will choose the formula that comes closest to the true variance of the population over a large number of iterations.

```
[18]: sample_size = 15           # the size of the sample
num_samples = 5000           # number of times we will take a sample

# We are going to calculate the average of the result with each formula.
# In order to do this, we keep track of the sum of these results and
# divide afterwards by the number of samples
sum_pop = 0
sum_smp = 0

for _ in range(num_samples):
```

```

sample = np.random.choice(population, size=sample_size)
sum_pop = sum_pop + pop_sd(sample)
sum_smp = sum_smp + sample_sd(sample)

print(f"Average result with formula population      : {sum_pop/num_samples}")
print(f"Average result with formula sample         : {sum_smp/num_samples}")
print(f"The true standard deviation of the population: {population_stdev}")

```

```

Average result with formula population      : 27.91945701449165
Average result with formula sample         : 28.899383581860118
The true standard deviation of the population: 29.154759474226502

```

We see that the formula with n as denominator underestimates the true value of the standard deviation for the population. The formula with $n - 1$ as denominator consistently gives a better estimate. Therefore, we will use this formula every time.

Note that we do not have to implement the formula for variance and standard deviation ourselves. Here we have done so to show how these formulas work. `numpy` already contains functions that do what we want:

```

[19]: a = np.array([1,2,3,4,5])

print(f"Population variance with numpy : {np.var(a)}")
print(f"Population variance own function: {pop_var(a)}")
print()
print(f"Sample variance with numpy : {np.var(a, ddof=1)}")
print(f"Sample variance own function: {sample_var(a)}")
print()
print(f"Standard deviation population: {np.std(a)}")
print(f"Standard deviation sample : {np.std(a, ddof=1)}")

```

```

Population variance with numpy : 2.0
Population variance own function: 2.0

```

```

Sample variance with numpy : 2.5
Sample variance own function: 2.5

```

```

Standard deviation population: 1.4142135623730951
Standard deviation sample : 1.5811388300841898

```

Note: `numpy` by default computes variance and standard deviations of the *population* (denominator n), while `pandas` by default computes variance and standard deviation of the *sample* (denominator $n - 1$), as can be seen from the example below.

Make sure you always use the appropriate function or parameters!

```

[20]: print(f"Numpy uses ddof=0 by default: {np.std(tips.tip)}")
      print(f"Pandas uses ddof=1 by default: {tips.tip.std()}")
      print(f"Numpy with ddof=1: {np.std(tips.tip, ddof=1)}")

```

```
Numpy uses ddof=0 by default: 1.3807999538298954
Pandas uses ddof=1 by default: 1.3836381890011822
Numpy with ddof=1: 1.3836381890011822
```

2.4 Visualization of univariate data

For the visualization of univariate data, we will use the **seaborn** library. The type of visualization that is most suitable depends on the measurement level of the variable. In this course, we will work with the following types of visualizations:

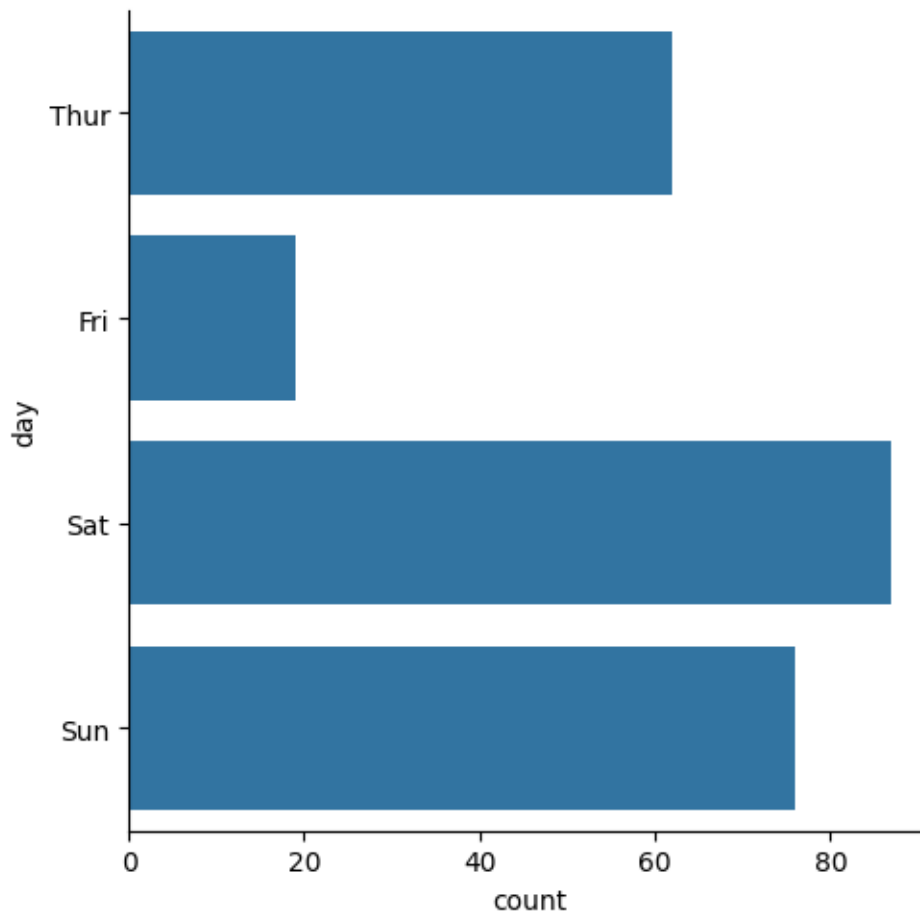
- Qualitative variables: bar chart
- Quantitative variables: histogram, boxplot, and density plot

Remark that we strongly recommend *against* using a pie chart for the visualization of qualitative variables, since it is difficult to compare the sizes of the slices. In a data set with a large number of categories, pie charts are practically unusable. A bar chart is much easier to read and interpret. Therefore, we will *not* explain how to generate a pie chart in Python.

2.4.1 Bar Chart

A **bar chart** is a graphical representation of the frequency of categories in a qualitative variable. Each category is represented by a bar, and the height of the bar corresponds to the frequency of that category in the data.

```
[21]: # Bar chart in Seaborn: catplot() with 'kind = "count"'
sns.catplot(data=tips, kind='count', y='day');
```

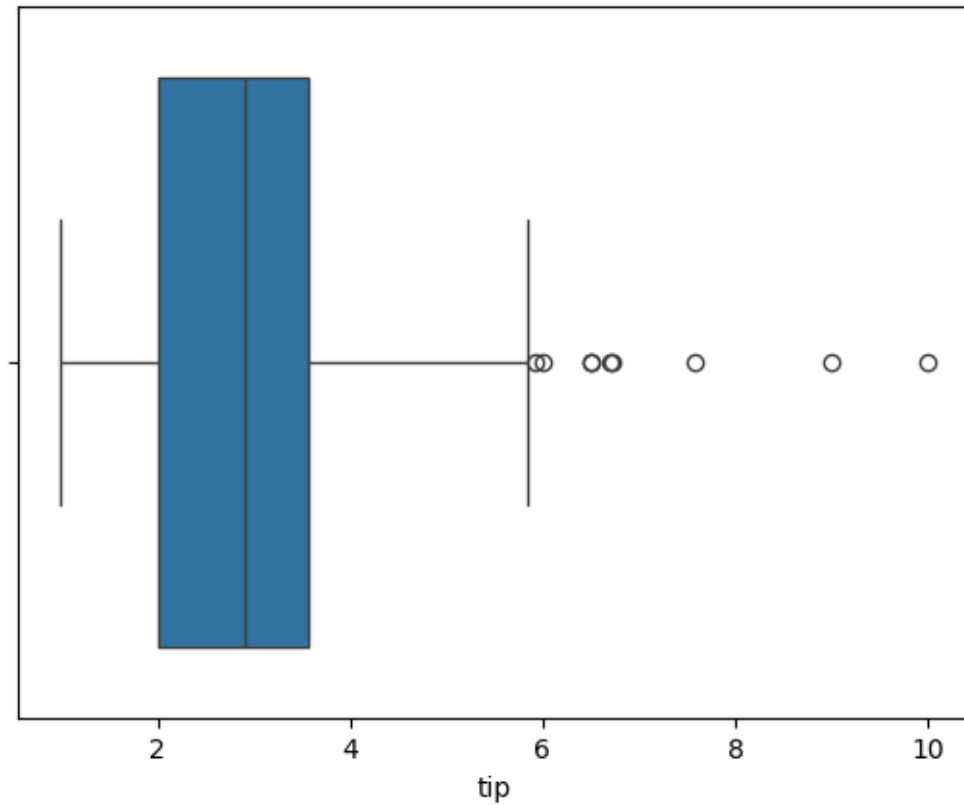


2.4.2 Boxplot (or box-and-whisker plot)

A **boxplot** (or **box-and-whisker plot**) is a graphical representation of the distribution of a quantitative variable. It displays the median, quartiles, and potential outliers in the data. Boxplots are particularly useful for comparing distributions across different groups.

Source: <https://spatiallychallenged.com/2019/05/31/box-and-whisker-plots/>

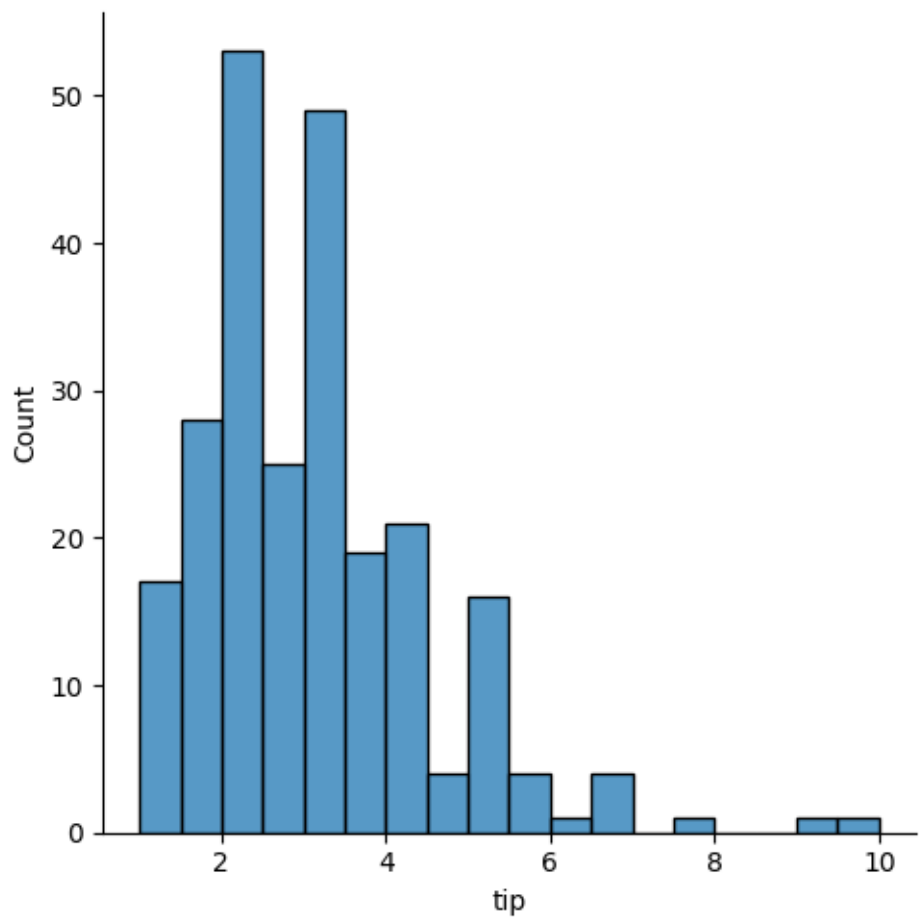
```
[22]: # Visualisation using a box plot (Seaborn)
sns.boxplot(data=tips, x='tip');
```



2.4.3 Histograms

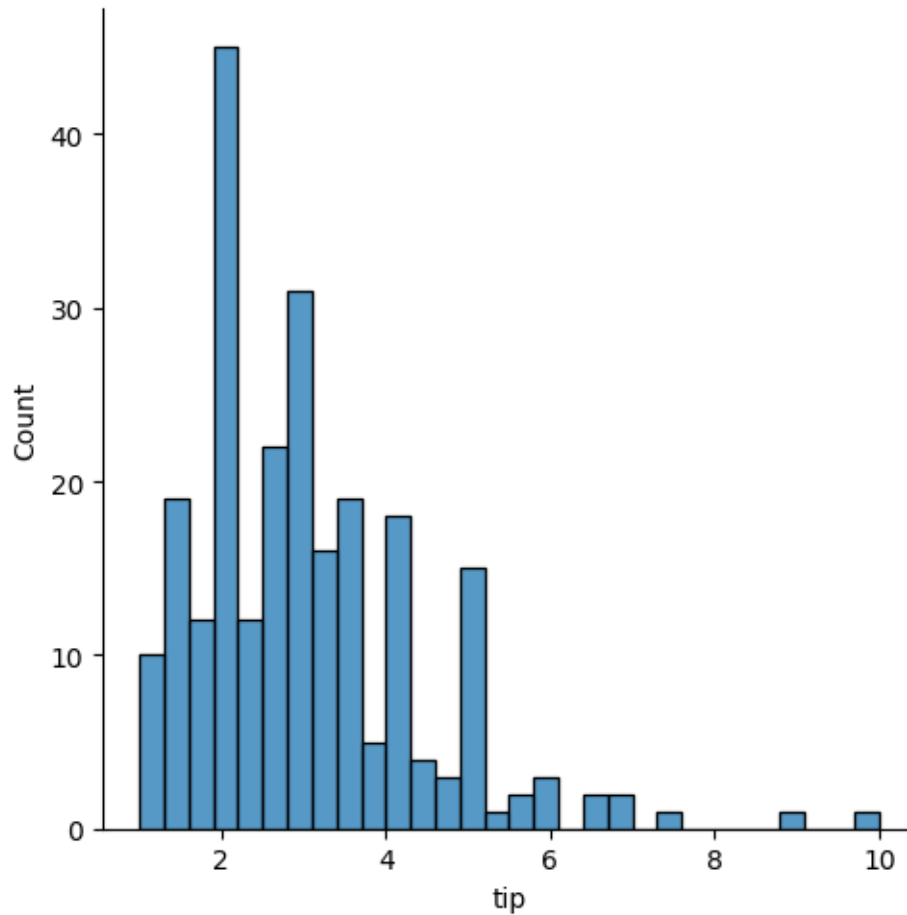
In a **histogram**, the domain of the variable is subdivided into intervals (also called *bins*), usually of equal size. The frequency of the variable in each interval is represented by the height of a bar.

```
[23]: sns.displot(data=tips, x='tip');
```

You can set the number of intervals with the parameter `bins`.

```
[24]: sns.displot(data=tips, x='tip', bins=30);
```

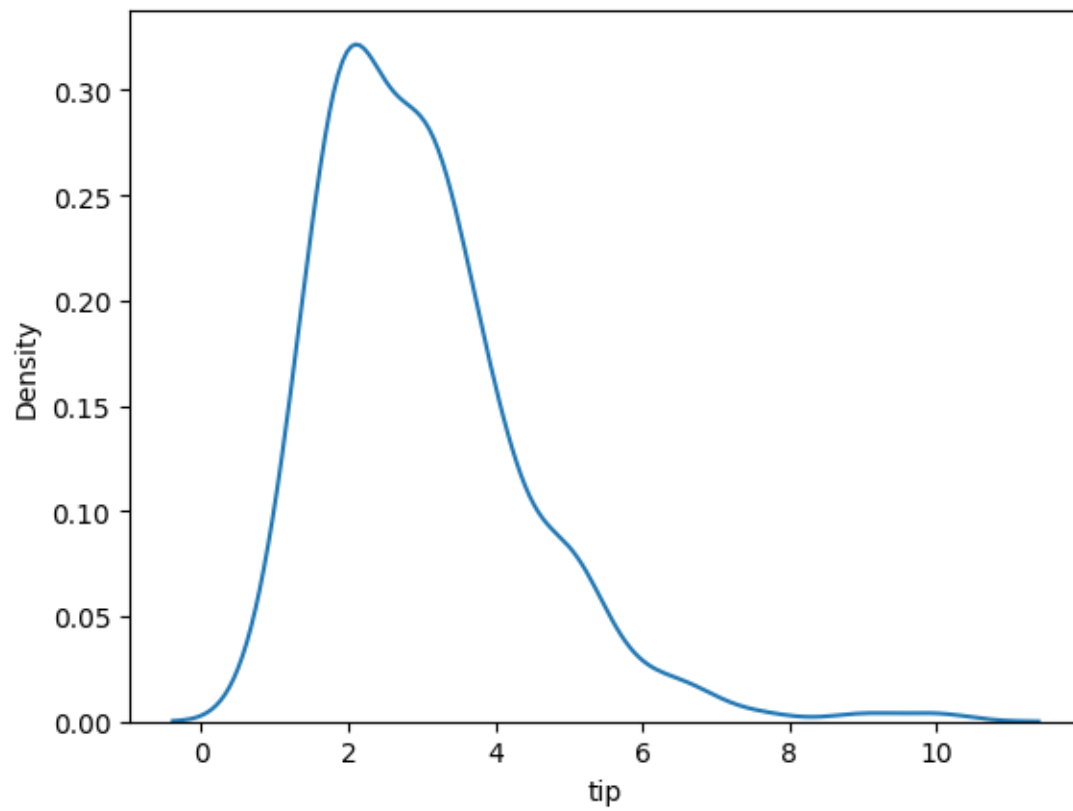


2.4.4 Density Plots

A **density plot** is a form of a graph that uses a single continuous curve to show the distribution of values in a dataset. A kernel density plot is similar to a histogram, but it is better at depicting the shape of distribution because the number of bins used in the histogram has no effect on it.

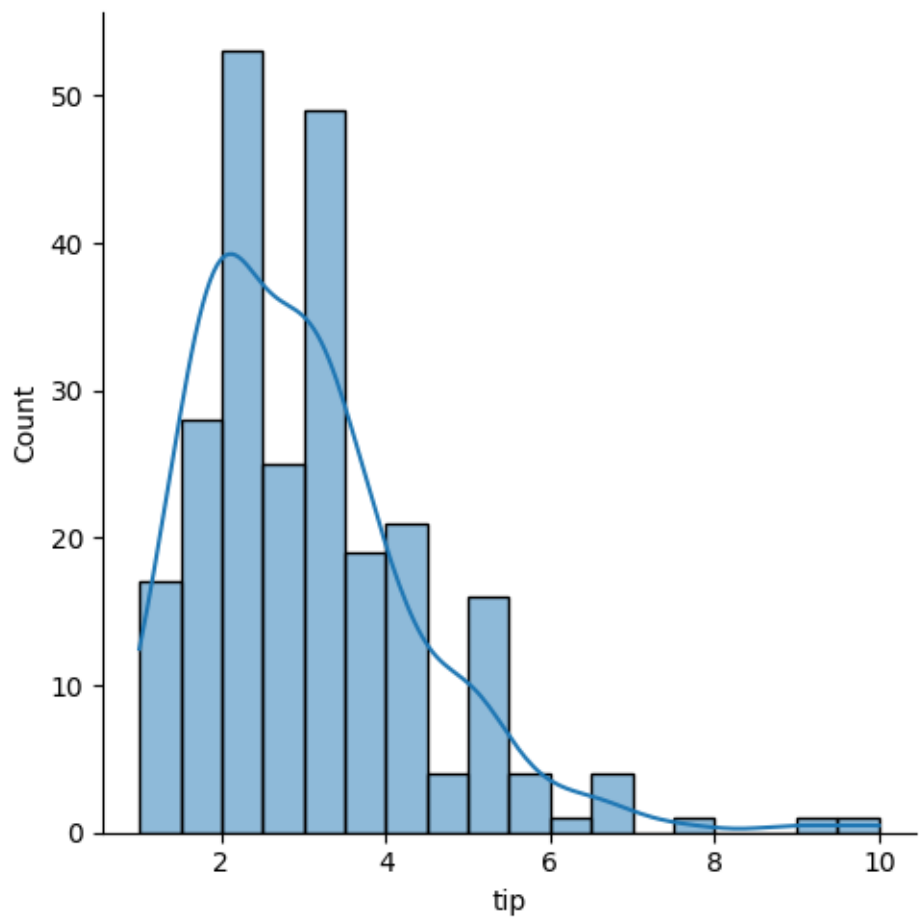
Below we show a plot of the probability density (kde or kernel density estimation).

```
[25]: sns.kdeplot(data=tips, x='tip');
```



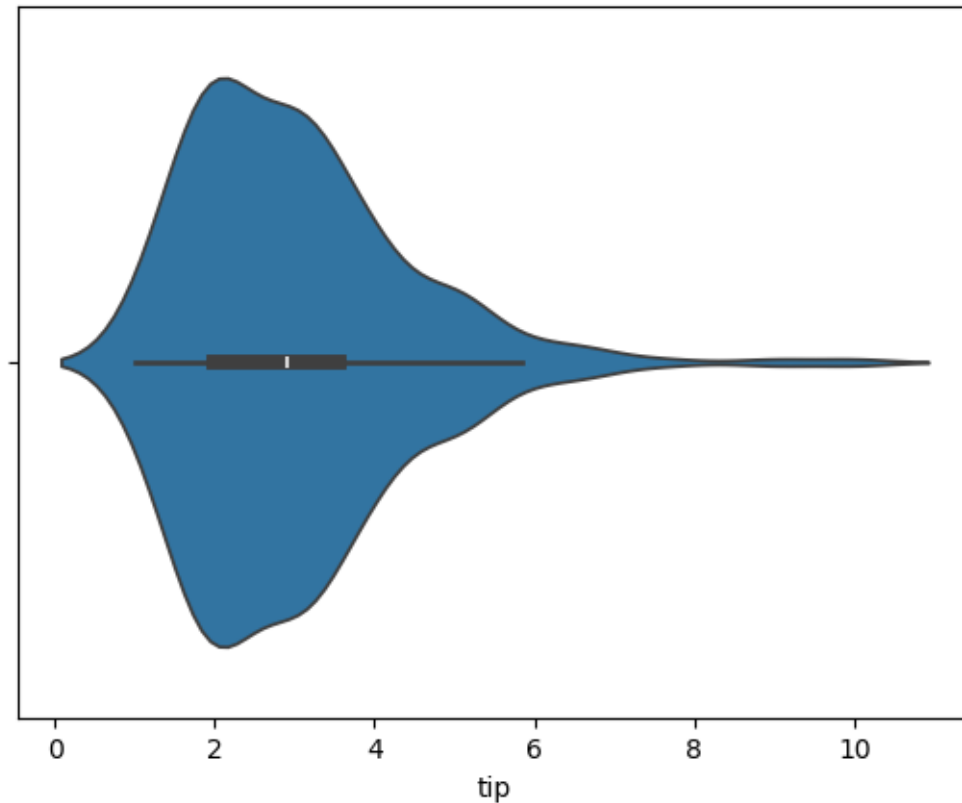
A histogram can be superimposed with a kernel density estimate:

```
[26]: sns.displot(data=tips, x='tip', kde=True);
```



A **violin plot** is a combination of a box plot and a density plot.

```
[27]: sns.violinplot(data=tips, x='tip');
```



2.4.5 The importance of data visualization

Beware that only calculating measures of central tendency and dispersion is not enough to understand the data. It is important to visualize the data as well, using histograms, density plots, box plots, etc. This will help you to see the shape of the distribution, identify any outliers, and understand the overall pattern of the data.

Anscombe's Quartet is a dataset that was constructed to illustrate this point. It consists of four different datasets that have the same mean, variance, correlation, and regression line (discussed in chapter 6), but they have very different distributions when visualized.

```
[ ]: # Load the dataset
anscombes_quartet = sns.load_dataset("anscombe")
# Show the mean and standard deviation for each of the four datasets
anscombes_quartet.groupby("dataset").agg(['mean', 'std']).round(2)
```

```
[ ]:
      x      y
      mean  std mean  std
dataset
I      9.0  3.32  7.5  2.03
II     9.0  3.32  7.5  2.03
III    9.0  3.32  7.5  2.03
```

IV 9.0 3.32 7.5 2.03

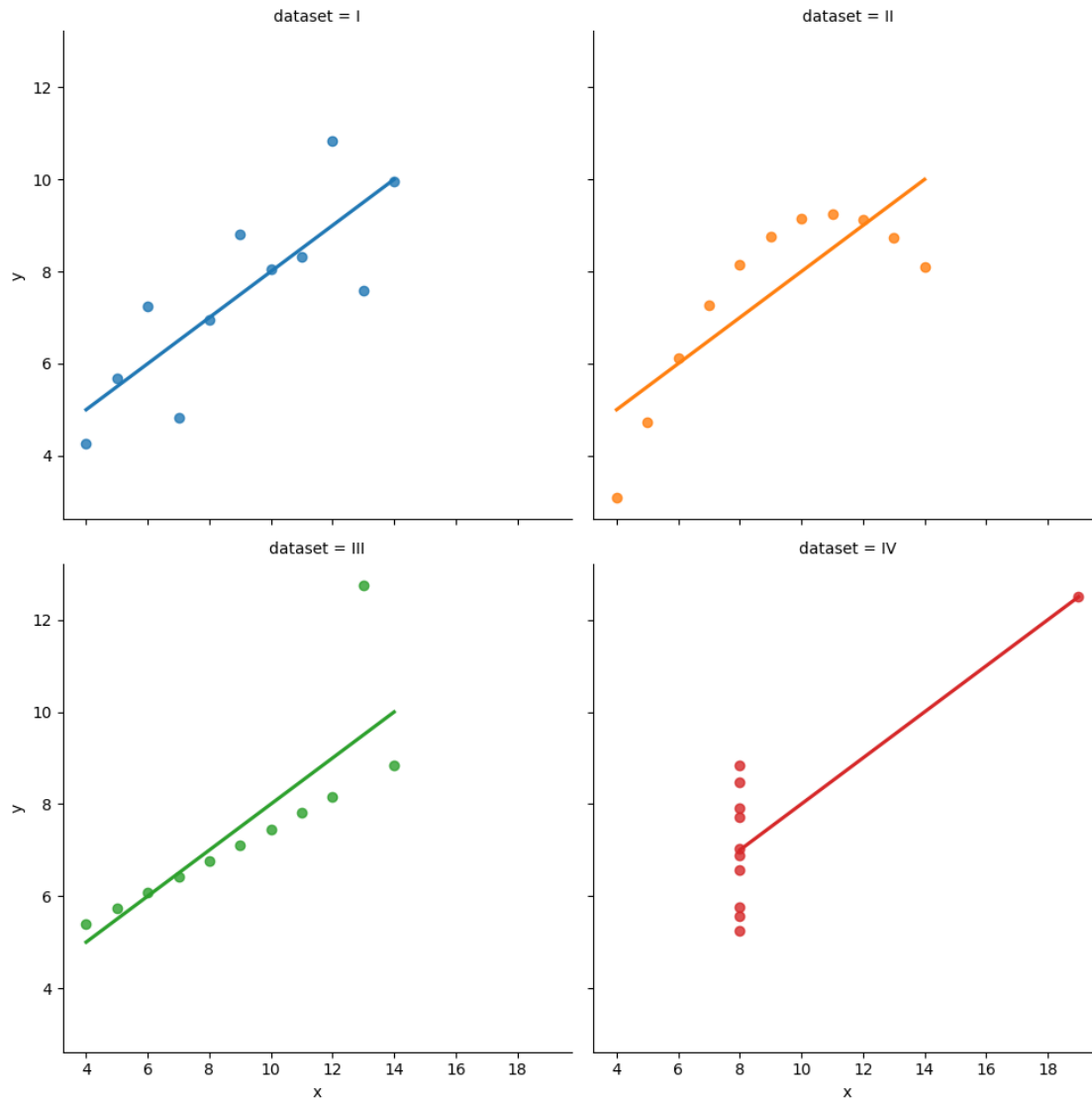
```
[ ]: # Show the correlation coefficient (See Chapter 6) for each of the four datasets
anscombes_quartet.groupby("dataset").corr().round(2)
```

```
[ ]:
      dataset      x      y
I      x  1.00  0.82
      y  0.82  1.00
II     x  1.00  0.82
      y  0.82  1.00
III    x  1.00  0.82
      y  0.82  1.00
IV     x  1.00  0.82
      y  0.82  1.00
```

```
[ ]: # Show trend and level of the linear regression line for each dataset
anscombes_quartet.groupby("dataset").apply(lambda df: np.polyfit(df.x, df.y,
    ↪deg=1).round(2)) # type: ignore
```

```
[ ]: dataset
I      [0.5, 3.0]
II     [0.5, 3.0]
III    [0.5, 3.0]
IV     [0.5, 3.0]
dtype: object
```

```
[31]: # Finally, visualize the datasets
sns.lmplot(data=anscombes_quartet, x="x", y="y", col="dataset", col_wrap=2,
    ↪hue="dataset", ci=None);
```



The [Datasaurus Dozen](#) is a more recent collection of constructed datasets with the same purpose. You can download the dataset from [this page](#) as a [CSV file](#). Try to calculate univariate statistics for each of the datasets and then visualize them to see how they differ!

2.4.6 Guidelines when visualizing data

When visualizing data, it is important to follow some guidelines to ensure that the visualization is effective and accurate:

- Always **label** your axes and provide a **title** for your plot. This will help others understand what the plot is showing.
- Name the **units** of measurement and the scale on the axes if applicable.
- Show the entire axis, including the **origin** (0,0) if it is relevant to the data. This will help to avoid misleading interpretations of the data.

- **Don't use 3D plots** unless they are necessary, as they distort the data and make it harder to interpret.
- In a report or thesis, always provide an **elaborate caption** for your plot, explaining what is shown exactly, and how the reader should interpret the data and what the main takeaways are.

2.5 Summary

The table below summarizes the most common measures of central tendency and dispersion for each measurement level, as well as the most suitable visualizations for univariate analysis.

Measurement Level	Measures of Central Tendency	Measures of Dispersion	Suitable Visualizations
Nominal	Mode	N/A	Bar chart
Ordinal	Mode, Median	N/A	Bar chart
Interval	Mean, Median, (Mode)	Variance, Standard Deviation, (Interquartile) Range	Histogram, Density plot, Box plot
Ratio	Mean, Median, (Mode)	Variance, Standard Deviation, (Interquartile) Range	Histogram, Density plot, Box plot

Remember that only reporting a measure of central tendency is never sufficient! Always include a measure of dispersion and a visualization to provide a complete picture of the data.

Source: Complete Probability & Statistics 1 for Cambridge International AS & A Level

3 Module 3.01 - Probability

The central topic in this chapter is the **central limit theorem** (CLT), an essential concept in statistics because it determines the conditions under which you can extrapolate measurement results from a *sample* to the *population* as a whole. We do not discuss the mathematical proof of the central limit theorem here, but we expect you to understand its meaning and applications. For this purpose, you need some knowledge of **probability theory** and stochastic distributions (see this notebook). One of the most important applications of the central limit theorem (see notebook 3.02-clt-confidence-intervals.ipynb) is the procedure of a **statistical test** (see notebook 3.03-hypothesis-testing.ipynb). In this chapter, we explain how to perform such a test and introduce the so-called *z*-test and *t*-test.

3.1 Learning goals

- You understand the definitions (indicated with $\propto$) in this notebook
- You understand the axioms (rules) and properties of probability
- You can use these properties to calculate probabilities of events

- You can calculate probabilities for a discrete/continuous random variable given its probability distribution and/or probability mass/density function
- You know the most important probability distributions (normal, t , exponential, uniform) and their properties, and can use Python to calculate probabilities for these distributions
- You understand the central limit theorem and its importance for statistical analysis
- You can calculate confidence intervals for the mean of a population

```
[80]: # Importing the necessary packages
import numpy as np                # "Scientific computing"
import scipy.stats as stats       # Statistical tests

import pandas as pd              # Data Frame
from pandas.api.types import CategoricalDtype

import matplotlib.pyplot as plt  # Basic visualisation
from statsmodels.graphics.mosaicplot import mosaic # Mosaic diagram
import seaborn as sns            # Advanced data visualisation
↪ visualisation
```

3.2 Probability

Informally, probabilities represent beliefs of how likely it is that a certain **event** will happen when performing a certain experiment. In the frequentist view, a **probability** represents the **relative frequency** of the occurrence of the event at hand (when performing a large number of independent experiments).

Example A fair six-sided die is thrown once.

- What is the probability of getting a “1”?
- What is the probability of throwing an even number?
- What is the probability of getting a number in the set $\{1, 2, 3, 4, 5, 6\}$?

3.2.1 Example

Two games of chance are considered where you bet on the following outcomes:

1. At least one six when throwing a fair die 4 consecutive times.
 - You will win in the long run when playing this game.
2. At least one “double six” when throwing two fair dice 24 times.
 - You will lose in the long run when playing this game, even though in some sense the two bets seem equivalent. (Probability divided by six but number of throws multiplied by six.)

The code below simulates both games and shows the win percentage over a large number of trials. Set the variable `number_of_games` to different values to see how the win percentage changes and which game is more favorable in the long run.

```
[99]: import random

number_of_games = 100

# Simulate game 1
number_of_times_won = 0
l_game_1 = []

for index in range(1, number_of_games + 1):
    number_of_times_6 = 0
    for _ in range(4):
        number_of_times_6 += 1 if random.randint(1,6) == 6 else 0
    number_of_times_won += 1 if number_of_times_6 >= 1 else 0
    l_game_1.append(number_of_times_won / index)

# Simulate game 2
number_of_times_won = 0
l_game_2 = []

for index in range(1, number_of_games + 1):
    number_of_times_double_6 = 0
    for _ in range(24):
        number_of_times_double_6 += 1 if (random.randint(1,6) == 6 and random.
↪ randint(1,6) == 6) else 0
    number_of_times_won += 1 if number_of_times_double_6 >= 1 else 0
    l_game_2.append(number_of_times_won / index)

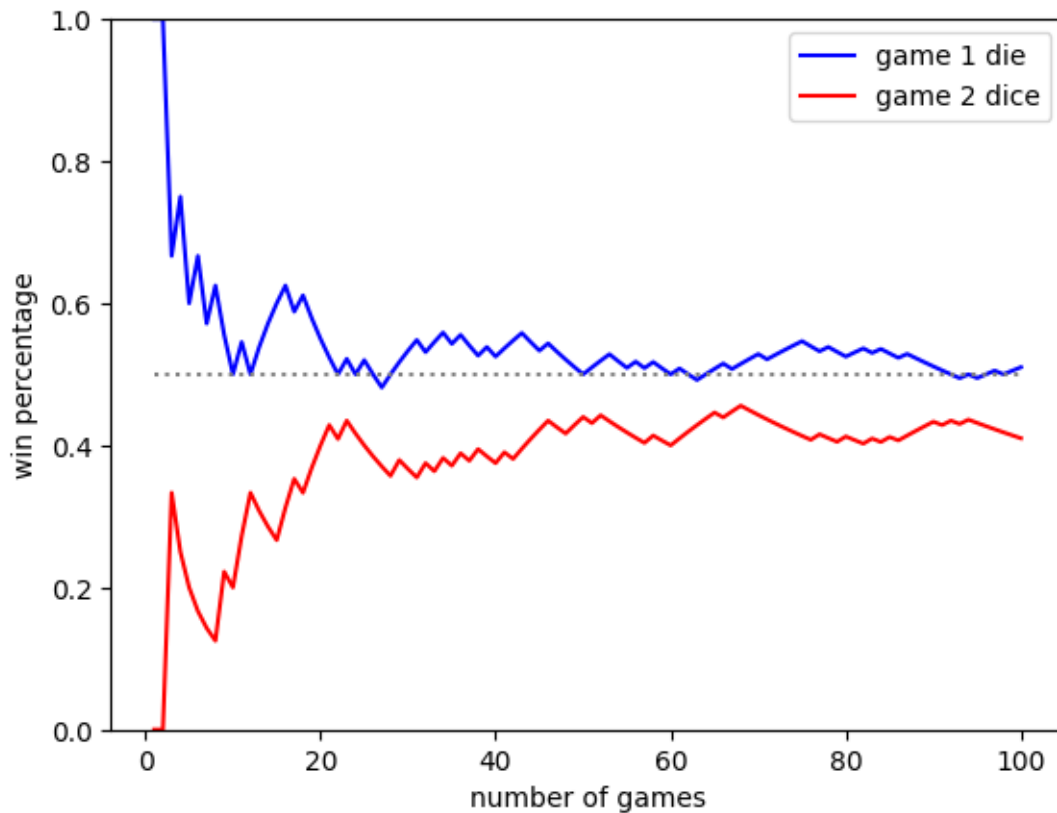
# Prepare a data structure for visualisation
data = {
    'number of games': range(1, number_of_games + 1),
    'game 1 die': l_game_1,
    'game 2 dice': l_game_2}

f, ax = plt.subplots(1, 1);

sns.lineplot(ax=ax, x='number of games', y='game 1 die', data=data, color =_
↪ "blue", label="game 1 die");
sns.lineplot(ax=ax, x='number of games', y='game 2 dice', data=data, color =_
↪ "red", label="game 2 dice");
sns.lineplot(ax=ax, x='number of games', y=0.5, data=data, color = "grey",_
↪ linestyle='dotted');

plt.xlabel('number of games');
```

```
plt.ylabel('win percentage');
plt.ylim((0,1));
plt.legend();
```



3.2.2 Two (or more) events

There are many contexts in which you are interested in the outcome of more than one thing at a time - that is, of a **compound event**. In simple cases it may be possible to make a list of all the possible outcomes. In order to make it easier to keep track, it is common to write the outcomes as ordered pairs inside brackets, $()$, and to put the whole list inside curly brackets, $\{\}$. This list is called the **sample space** or **universe** of the experiment, typically denoted by the symbol Ω (the Greek capital letter omega). An event, then, can be modeled by a subset (usually denoted with a capital letter like A , B , etc.) of the universe, e.g. $A \subset \Omega$

It helps to write a sample space list in a logical, methodical order.

- For tossing a coin two times the sample space can be written as:

$$=\{(H, H), (H, T), (T, H), (T, T)\}$$

- For tossing a coin three times the sample space can be written as:

$$=\{(H, H, H), (H, T, H), (T, H, H), (T, T, H), (H, H, T), (H, T, T), (T, H, T), (T, T, T)\}$$

Within the sample space, the list for tossing a coin twice appears two times. This list of eight outcomes for tossing a coin three times can then form the basis of the sixteen outcomes for tossing a coin four times. This is created by putting an extra ‘H’ at the end and then an extra ‘T’ at the end. It can in turn be used for the list of tossing a coin five times, and so on.

Example

What is the probability of at least one six when throwing a fair die 4 consecutive times.

- What is the universe for the first game?

$$= \{(1, 1, 1, 1), (1, 1, 1, 2), (1, 1, 1, 3), \dots, (6, 6, 6, 4), (6, 6, 6, 5), (6, 6, 6, 6)\}$$

- What set corresponds to the event “at least one 6”?

$$= \{(6, 1, 1, 1), (6, 1, 1, 2), \dots, (5, 5, 5, 6), (6, 6, 1, 1), \dots, (6, 6, 5, 5), (1, 6, 6, 1), \dots, (6, 6, 6, 6)\}$$

- What is the probability of event A?

The “brute force” approach is to count the number of outcomes in A and in Ω and divide by the former with the latter. For this case, you would count 671 outcomes in A and 1296 outcomes in Ω , so the probability is $671/1296 \approx 0.518$. Of course, for many cases this approach is infeasible, so we need some rules to calculate probabilities more efficiently.

3.2.3 Axioms of Probability

The probability of an event A is denoted by $P(A)$. The following three axioms are the basis of probability theory:

1. Probabilities are **non-negative**: For each A :

$$P(A) \geq 0$$

2. The **probability of the universe** is 1:

$$P(\Omega) = 1$$

3. **The sum rule**: When A and B are disjoint events (i.e. $A \cap B = \emptyset$) then it holds that

$$P(A \cup B) = P(A) + P(B)$$

3.2.4 Properties of Probabilities

From the three axioms of probability, we can derive all properties of probabilities. Some important ones are listed below:

1. **Complement rule**: for each A it holds that

$$P(\overline{A}) = 1 - P(A)$$

where $\overline{A}$ represents the event “A does not occur”.

2. The **impossible event** has probability zero: $P(\emptyset) = 0$.

3. The **general sum rule**:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

The general sum rule says that the probability of A or B can be found by adding the probabilities of A and B and then subtracting what has been counted twice (i.e., the intersection of A and B). You can check this works in simple examples like events based on throwing a die, but logic says it is always going to be true, even in situations where events are not equally likely or when you cannot list all possible outcomes.

Example If A is the event a fair die shows a factor of 6 and B is the event that the fair die shows a square number, then 1, 2, 3 and 6 satisfy A , while 1 and 4 satisfy B . A and B is satisfied by 1, and A or B is satisfied by 1, 2, 3, 4 and 6.

- $P(A \cup B) = \frac{5}{6}$
- $P(A) = \frac{4}{6}$
- $P(B) = \frac{2}{6}$
- $P(A \cap B) = \frac{1}{6}$
- $\frac{5}{6} = \frac{4}{6} + \frac{2}{6} - \frac{1}{6}$

Exercise A coin is tampered with such that $(\text{head}) = 0.6$. What is (tail) ?

Exercise From the 250 students enrolled in this course, 200 own a PlayStation game console, while 100 own a Nintendo Switch. There are 75 students who own both game consoles. What is the probability, when you pick a random student, that

1. this student does not own a PlayStation
2. this student owns a PlayStation or a Nintendo Switch (or both)
3. this student owns neither a PlayStation nor a Nintendo Switch.

(Answers: (1) $1 - 200/250 = 1/5$, (2) $(200 + 100 - 75)/250 = 225/250 = 9/10$, (3) $1 - 9/10 = 1/10$)

3.2.5 Independent Events

Example Two events are **independent** if the occurrence of one event (or knowing that the event occurred) does not change the probability of the other event occurring. As an example: draw a card from a standard deck of cards and consider the following events:

- $A =$ a was drawn
- $B =$ an ace was drawn

When I tell you that B occurred, this will not change your opinion about the probability of occurring, this will still be $13/52 = 1/4$.

(Note: this also works the other way around!)

Mathematically Two events A and B are **independent** if

$$P(A \cap B) = P(A)P(B)$$

3.2.6 Dependent events

Example Consider the events from before where students own either a PlayStation, a Nintendo Switch or both. Let A be the event that a student owns a PlayStation, while B represents the fact that a student owns a Nintendo Switch. From the numbers above we see that

- $P(A) = \frac{200}{250} = 0.8$ and
- $P(B) = \frac{100}{250} = 0.4$

when we multiply these two probabilities we get

$$P(A)P(B) = 0.8 * 0.4 = 0.32$$

However, the probability that a student owns both a PlayStation and a Nintendo Switch, i.e. is given by

$$P(A \cap B) = \frac{75}{250} = 0.3 \neq 0.32$$

Hence, $P(A \cap B) \neq P(A)P(B)$ and the two events are **dependent**.

In practical terms this means that e.g. knowing that a student owns a PlayStation gives you additional information about their ownership of a Nintendo Switch. Before you know that the student owns a PlayStation the probability that this student owns a Nintendo Switch is

$$P(B) = \frac{100}{250} = 0.4$$

However, once you know that that the student owns a PlayStation you know that this student belongs to the group of 200 PlayStation owners, of which 75 also own a Nintendo Switch. With fancy notation we write this as

$$P(B|A) = \frac{75}{200} = 0.375$$

So the probability now is slightly lower.

On the other hand, when you know that a student owns a Nintendo Switch, what is the probability of this student owning a PlayStation?

$$P(A|B) = \frac{75}{100} = 0.75$$

Formally, the probability $P(B|A)$ is called the **conditional probability** of B given A , i.e. the probability that B occurs if you know that A occurred.

Exercise The results of a survey of colours and types of cars are shown in the table.

	Saloon	Hatchback
Silver	65	59
Black	27	22
Other	16	19

One car is selected from the group at random. Find the probability that the selected car is

- a silver hatchback
- a hatchback
- a hatchback, given that it is silver

Exercise A golfer enters two tournaments. He estimates the probability that he wins the first tournament is 0.6, that he wins the second tournament is 0.4 and that he wins both tournaments is 0.35.

- Find the probability that he does not win either tournament
- Show, by calculation, that winning the first tournament and winning the second tournament are not independent events.
- The tournaments are played in successive weeks. Explain why it would be surprising if these were independent events.

Exercise Of the students who took English in a certain school one year, 60% of them took History, 30% of them took Biology, and 10% took History and Biology. One of the students taking English is chosen at random.

- Find the probability that the student took neither History nor Biology
- Given that the student took exactly one of History and Biology, find the probability it was History

Exercise Data about age of males and females in a small rural area are shown in the table.

	Under 35	35 and over
Male	345	380
Female	362	472

A person from this area is chosen at random. Let M be the event that the person is male and let Y be the event that the person is under 35.

- Find $P(M)$
- Find $P(M \text{ and } Y)$
- Are M and Y independent events? Justify your answer.
- Given that the person chosen is under 35, find the probability that the person is female.

[]:

3.3 Discrete random variable

A **random variable** is a quantity that has its value determined by the outcome of a random event. Random variables can arise from **probability experiments**. For example, when you throw two dice, the sum of the scores X is a random variable. Similarly, the product of the scores Y and the larger of the scores Z are also random variables.

Random variables can arise from real-life observation. For example the number of telephone calls X arriving at a switchboard between 10 and 10.30 am.

When values of a variable have a probability attached, they form a **probability distribution**.

X is a **discrete random variable** if X takes values $x_1, x_2, x_3, \dots$ and $P(X = x_i) = p_i$ where all $p_i \geq 0$ and $\sum p_i = 1$

Note that this means X can only take distinct values, all probabilities of X must be non-negative and their sum is 1.

3.3.1 Example

A player draws a card from a standard deck of cards. They get 1 € when drawing a jack, 2 € when drawing a queen and 3 € when drawing a king. In all other cases they receive nothing.

We can formalise this by defining a mapping X from Ω to $\mathbb{R}$:

\$

$$X : \Omega \rightarrow \mathbb{R} : \omega \mapsto X(\omega) = \begin{cases} 1 & \text{when drawing a jack} \\ 2 & \text{when drawing a queen} \\ 3 & \text{when drawing a king} \\ 0 & \text{in all other cases} \end{cases} \quad (1)$$

\$

We have done nothing else but associate a number with different events.

3.3.2 Probability mass function

Using the rules of probability we can assign probabilities to the different numerical outcomes

$$\begin{aligned} f_X(3) &= P(X = 3) = P(king) = \frac{4}{52} = \frac{1}{13} \\ f_X(2) &= P(X = 2) = P(queen) = \frac{4}{52} = \frac{1}{13} \\ f_X(1) &= P(X = 1) = P(jack) = \frac{4}{52} = \frac{1}{13} \\ f_X(0) &= P(X = 0) = P(any other card) = \frac{40}{52} = \frac{10}{13} \end{aligned}$$

This function f_X is called the **probability mass function** of the random variable X . It gives the probability that X results in a specific outcome.

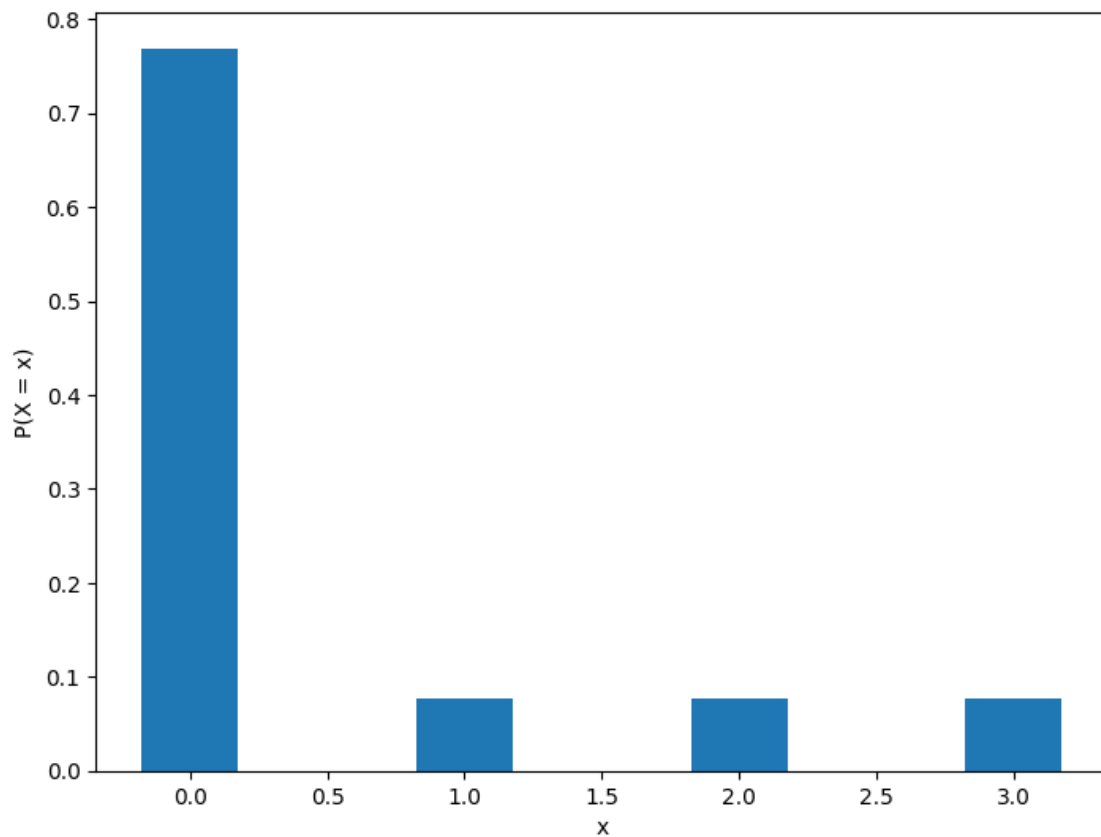
Note: the probability mass function follows the axioms of probability:

- It holds that $f_X(0) + f_X(1) + f_X(2) + f_X(3) = 1$
- And also: $0 \leq f_X(x_i) \leq 1$

To make things more explicit we can summarize the probabilities of all possible events into a table:

x	0	1	2	3
$f_X(x) = P(X = x)$	$\frac{10}{13}$	$\frac{1}{13}$	$\frac{1}{13}$	$\frac{1}{13}$

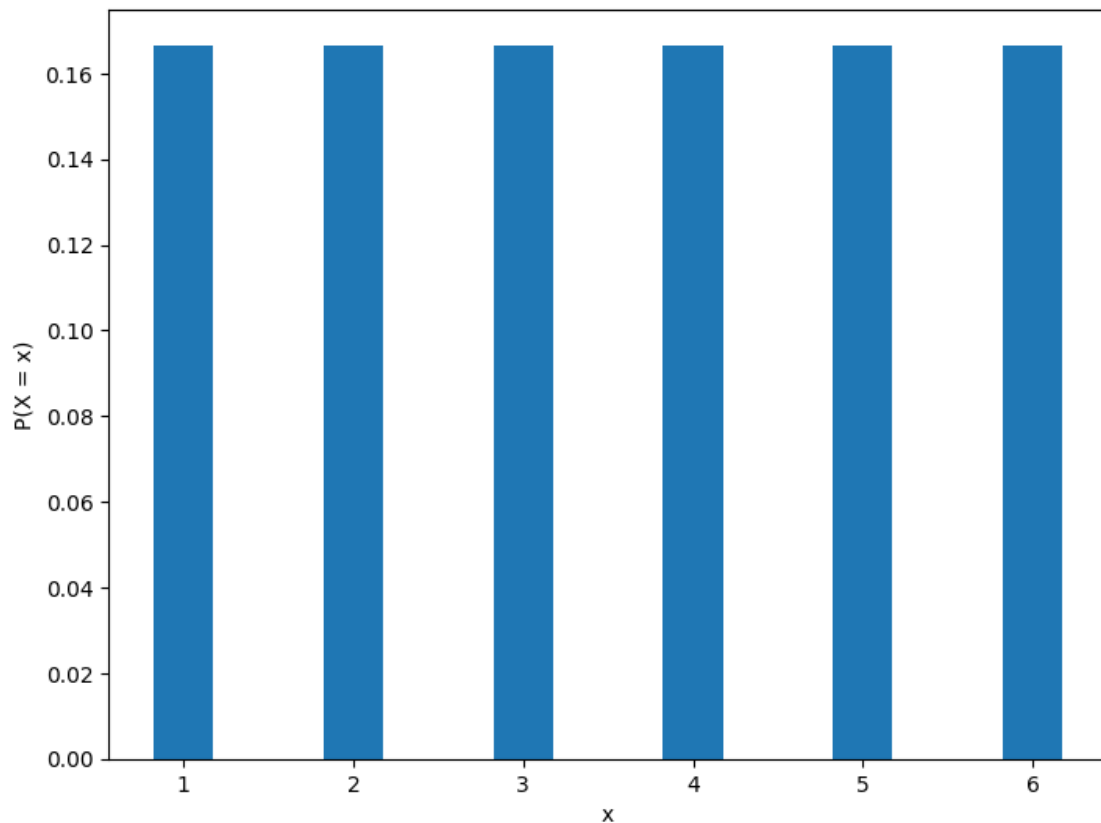
```
[82]: # Plot the probability distribution of the card drawing game described above
x = np.arange(4)
y = [10/13, 1/13, 1/13, 1/13]
fig = plt.figure()
ax = fig.add_axes(rect=(0,0,1,1))
ax.bar(x, y, 0.35)
ax.set_xlabel('x')
ax.set_ylabel('P(X = x)')
plt.show()
```



Exercise - probability distribution of rolling 1 die What is the probability of each outcome when rolling a single die?

x	1	2	3	4	5	6
$f_x(x) =$ $P(X = x)$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$

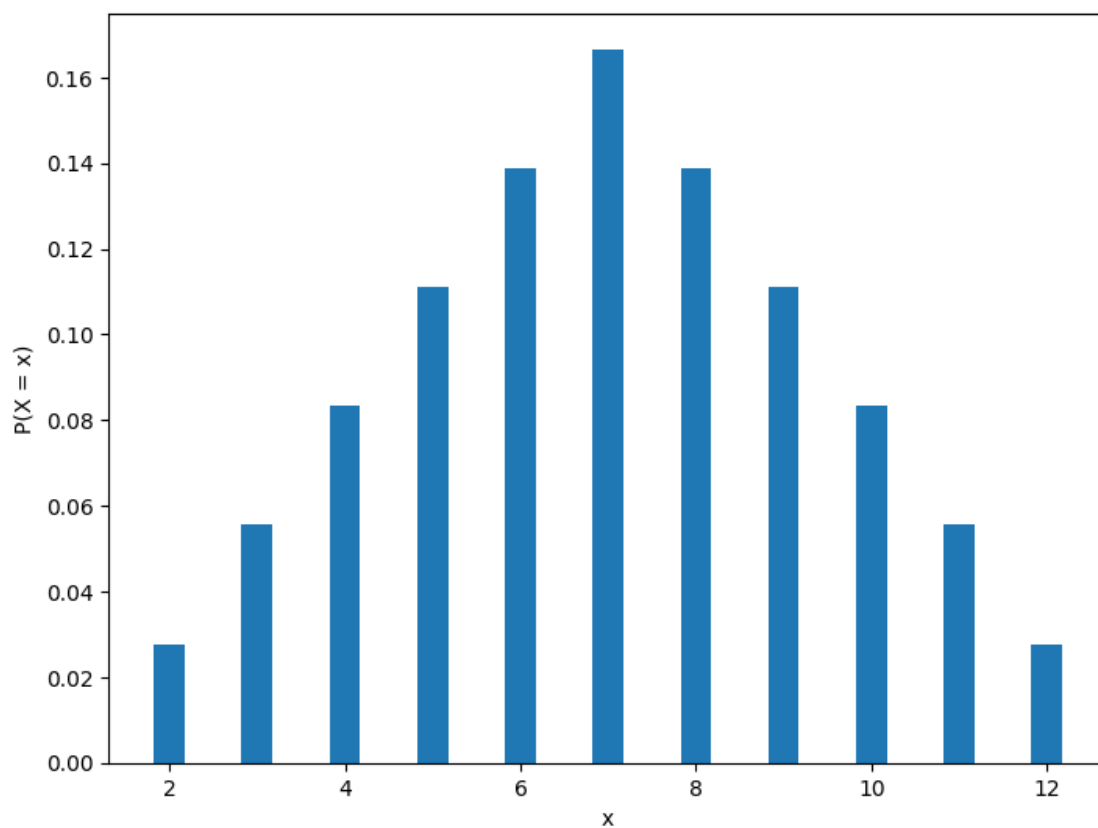
```
[83]: # Plot the probability distribution of rolling a die
x = np.arange(1, 7)
y = np.full(6, 1/6)
fig = plt.figure()
ax = fig.add_axes(rect=(0,0,1,1))
ax.bar(x, y, 0.35)
ax.set_xlabel('x')
ax.set_ylabel('P(X = x)')
plt.show()
```



Exercise - probability distribution of rolling 2 dice What is the probability of each outcome when rolling 2 dice?

x	2	3	4	5	6	7	8	9	10	11	12
$f_x(x)$	$\frac{1}{36}$	$\frac{2}{36}$	$\frac{3}{36}$	$\frac{4}{36}$	$\frac{5}{36}$	$\frac{6}{36}$	$\frac{5}{36}$	$\frac{4}{36}$	$\frac{3}{36}$	$\frac{2}{36}$	$\frac{1}{36}$
$=$ $P(X$ $= x)$											

```
[84]: # Plot the probability distribution of the sum of two dice
x = np.arange(2, 13)
y = [1/36, 2/36, 3/36, 4/36, 5/36, 6/36, 5/36, 4/36, 3/36, 2/36, 1/36]
fig = plt.figure()
ax = fig.add_axes(rect=(0,0,1,1))
ax.bar(x, y, 0.35)
ax.set_xlabel('x')
ax.set_ylabel('P(X = x)')
plt.show()
```



Exercise Find which of the following are discrete random variables, giving a reason for any which are not.

1. Variable X

x	1	2	3	4	5
P($X = x$)	$\frac{2}{10}$	$\frac{3}{10}$	$\frac{4}{10}$	$\frac{3}{10}$	$\frac{2}{10}$

2. Variable Y :

y	-2	-1	0	1	2
P($Y = y$)	$\frac{2}{10}$	$\frac{3}{10}$	$\frac{1}{10}$	$\frac{3}{10}$	$\frac{1}{10}$

3. Variable Z :

z	5	4	10	15	20
P($Z = z$)	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{8}$	$\frac{1}{16}$	$\frac{1}{16}$

4. Variable W :

w	1	2	3	4	5
P($W = w$)	$\frac{2}{10}$	$\frac{3}{10}$	$\frac{4}{10}$	$\frac{3}{10}$	$\frac{-2}{10}$

Exercise A fair die is thrown. List the probability distribution for the following random variables:

1. X = score on the die
2. $Y = 2 \cdot$ score on the die
3. Z = square of the score on the die
4. $W = \begin{cases} 0 & \text{if the score on the die is a factor of 6} \\ 1 & \text{otherwise} \end{cases}$

3.3.3 Using the Probability Mass Function

Once you have a probability mass function, you can compute other probabilities. With the card game, what is the probability that you gain at most 2 EUR?

$$P(X \leq 2) = P(X = 0) + P(X = 1) + P(X = 2) = \frac{10}{13} + \frac{1}{13} + \frac{1}{13} = \frac{12}{13}$$

We could also have used the complement rule in this case:

$$P(X \leq 2) = 1 - P(X > 2) = 1 - P(X = 3) = 1 - \frac{1}{13} = \frac{12}{13}.$$

Exercise When throwing two dice, what is the probability that the sum of the results is between 4 and 7, boundaries included?

$$P(4 \leq X \leq 7) = P(X = 4) + P(X = 5) + P(X = 6) + P(X = 7) = \frac{18}{36} = \frac{1}{2}$$

We could also compute this using left-tail probabilities only:

$$P(4 \leq X \leq 7) = P(X \leq 7) - P(X < 4) = P(X \leq 7) - P(X \leq 3) = \frac{21}{36} - \frac{3}{36} = \frac{1}{2}$$

Exercise

1. Variable X :

x	1	2	3	4	5
P(X = x)	$\frac{2}{10}$	$\frac{1}{10}$	$\frac{3}{10}$	a	$\frac{1}{10}$

- Find a
- Find $P(X \geq 2)$

2. Variable Y :

y	-2	-1	0	1
P(Y = y)	k	$2k$	$2k$	k

- Find k
- Find $P(Y \leq 0)$

3. Variable Z :

z	5	6	8	9	12
P(Z = z)	$\frac{4}{10}$	$\frac{2}{10}$	$\frac{3}{10}$	a	$\frac{1}{10}$

- Find a .
- Find $P(Z \geq 9)$

3.3.4 Expectation of a discrete random variable

Consider the random variable associated with playing the card game with the following probability mass function

$$f_X(3) = P(X = 3) = \frac{1}{13}$$

$$f_X(2) = P(X = 2) = \frac{1}{13}$$

$$f_X(1) = P(X = 1) = \frac{1}{13}$$

$$f_X(0) = P(X = 0) = \frac{10}{13}$$

If somebody played this game a very large number of times, what would their average earnings be? I.e. what is the **expected value** of this random variable?

The **expected value** (also: **mean** or **expectation**) of a discrete random variable is denoted by μ_X or $E(X)$ and is given by

$$\mu_X = \sum_i x_i P(X = x_i) = \sum_i x_i f_X(x_i)$$

Notes:

- The meaning of the formula is: we multiply each value by its probability and sum over all possible values of the random variable.
- there is a large similarity between this formula and the way you calculate the sample mean when a frequency table is given.
- we often write μ instead of μ_X .

Example The expected value of the random variable associated with the card game is given by:

$$\begin{aligned} \mu_X &= \sum_i x_i f_X(x_i) \\ &= 0 \cdot f_X(0) + 1 \cdot f_X(1) + 2 \cdot f_X(2) + 3 \cdot f_X(3) \\ &= 0 \cdot \frac{10}{13} + 1 \cdot \frac{1}{13} + 2 \cdot \frac{1}{13} + 3 \cdot \frac{1}{13} \\ &= \frac{6}{13} \end{aligned}$$

Exercise Members of a public library may borrow up to five books at any one time. The number of books borrowed by a member on each visit is a random variable, X , with the following probability distribution:

X	0	1	2	3	4	5
$P(X = x)$	0.24	0.12	0.14	0.3	0.05	0.15

Find the expected value of X .

3.3.5 Variance of a discrete random variable

The variance of a random variable is a measure of dispersion that resembles the sample variance very closely.

The **variance of a random variable** is defined by:

$$\sigma_X^2 = \sum_i (x_i - \mu_X)^2 P(X = x_i) = \sum_i (x_i - \mu_X)^2 f_X(x_i).$$

The **standard deviation** is the positive square root of the variance.

$$\sigma_X = \sqrt{\sigma_X^2}$$

Exercise The random variable X represents the number of received emails per day for a first office worker. This person receives either 48 or 52 emails per day, each with a 50 percent probability.

The random variable Y represents the number of received emails of a second office worker. This second person receives no emails at all with 50 percent probability and with 50 percent chance receives 100 emails.

- Determine the probability mass function of X and of Y .
- Compute the expected value of X and Y . What do you notice?
- Who is experiencing a larger variability w.r.t. the number of emails received?
- Compute the variance of X and Y .

3.3.6 Exercise

1. Y is the larger score showing when two dice are thrown.

Calculate $E(Y)$.

2. Y is the smaller score showing when two dice are thrown.

Calculate $E(Y)$ and $Var(Y)$.

3. A regular customer in a shop observes that the number of customers, X , in a shop when she enters has the following probability distribution

No. of customers	0	1	2	3	4
$P(X = x)$	0.1	0.25	0.3	0.25	0.1

- Find the mean and standard deviation of X .

The same customer also observes that the average waiting time, Y , before being served is as follows:

No. of customers	0	1	2	3	4
Average wait (minutes)	0	2	5	8	11

- Find the mean waiting time.

3.4 Continuous Random Variable

A **continuous random variable** takes on an uncountably infinite number of possible values.

- In this case it doesn't make a lot of sense to consider the probability that X equals some number a exactly, because this probability is always zero.

- What does make sense is to consider the probability that X takes on a value in some interval $[a, b]$.
- This probability can be found by integrating (i.e. “summing up”) the **probability density function** of the random variable.

3.4.1 Example

The times taken by competitors in the swimming section of a triathlon are shown in the following histograms, with smaller and smaller intervals:

As the number of intervals increases (in the first three graphs), the data show jagged edges. This is a representation of a finite number of observations - and if another set of observations was made, you would expect to see something broadly similar but the detail is unlikely to be exactly the same. The fourth graph looks like a plausible description of the distribution of times for the swim section of the triathlon in that competition.

On a different day, if the weather is different, with different competitors, or on a different course the distribution might be different.

These histograms are drawn as **frequency density** diagrams, so the total area of all the bars is always 1. The limit of the cases where smaller and smaller intervals are taken, but the total area stays at 1, is what is meant by a **continuous random variable**. The probabilities are defined as the area under the curve between two values of x . Measurements of length, time, ... are examples of continuous random variables.

- A continuous random variable takes on an uncountably infinite number of possible values.
- In this case it doesn't make a lot of sense to consider the probability that equals some number exactly, because this probability is always zero.
- What does make sense is to consider the probability that takes on a value in some interval $[,]$.
- This probability can be found by integrating (i.e. “summing up”) the probability density function of the random variable.

3.4.2 Expectation and Variance of a Continuous Random Variable

As there are formulas for expectation and variance for discrete random variables, there are also formulas for expectation and variance for continuous random variables.

The **expectation of a continuous random variable** is given by

$$\mu_X = \int_{-\infty}^{+\infty} x f_X(x) dx,$$

while the **variance** σ_X^2 is given by

$$\sigma_X^2 = \int_{-\infty}^{+\infty} (x - \mu_X)^2 f_X(x) dx$$

These are essentially the same formulas as for discrete random variables but with integrals instead of summations.

[You are **not** expected to know how to calculate integrals!]

3.5 The normal distribution

Many random variables that occur in the real world have very similar probability distributions that look like a bell-shaped curve. The most famous of these is the **normal distribution** (also known as the Gaussian distribution). For example, the heights, weights, of people often follow an approximate normal distribution.

The dimensions of manufactured articles will often follow a normal distribution as well.

The normal distribution is a type of **continuous probability distribution**. This means that:

- It relates to a continuous variable (height, weight, etc.).
- It describes the **probability** of this variable taking a particular range of values.

Generally, the normal distribution looks like this:

The idealised normal distribution has the following properties:

- It is symmetric around its expected value (mean) μ
- It is infinite in both directions
- It has a single peak at the centre, which is the mean
- It is continuous
- 95% of values lie within approximately 2 **standard deviations** of the **mean**
- 99% lie within approximately 3 standard deviations of the mean
- The total area under the curve is 1, which represents the total probability of all outcomes
- A normal distribution is fully described by its mean μ and standard deviation σ

The normal distribution is written As $X \sim Nor(\mu, \sigma)$.

This means “ X is distributed as a normal random variable with mean μ and standard deviation σ ”. Since all normal distributions are the same basic shape, we only need to have probabilities for one particular case to allow us to calculate probabilities for all cases.

In practice, real-life variables usually do not match all these conditions perfectly and the situations will only approximate the normal distribution. For example, height and weight cannot take negative values, however a zero value may occur a long way out towards the tail of the distribution, as in the previous diagram, where the left hand end of the graph is already invisible at 20. If the value

were not towards the tail, the the normal distribution would not be a reasonable approximation to use in that situation.

3.5.1 Python functions

Python functions related to the normal distribution are available in the `scipy.stats` module, which is part of the SciPy library. You can import it using:

```
import scipy.stats
```

For a normal distribution with mean `m` and standard deviation `s`:

Function	Purpose
<code>norm.pdf(x, loc=m, scale=s)</code>	Probability density at x
<code>norm.cdf(x, loc=m, scale=s)</code>	Left-tail probability ($< x$)
<code>norm.sf(x, loc=m, scale=s)</code>	Right-tail probability ($> x$)
<code>norm.isf(1-p, loc=m, scale=s)</code>	$p\%$ of observations are expected to be lower than result

Since the normal distribution is continuous the probability of particular values is zero, so it does not make any difference whether the inequality is strict or not.

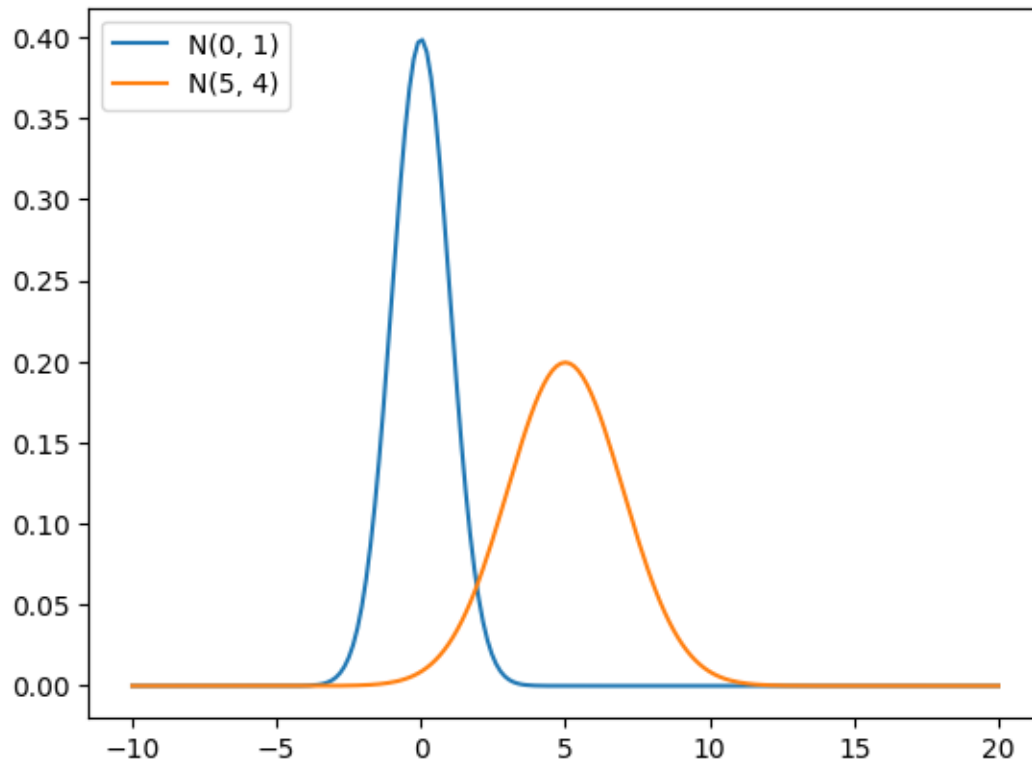
See the [SciPy API documentation](#) for more details.

3.5.2 Parameters of a normal distribution

As stated before, all normal distributions are essentially the same shape. They may have a different centre, or be more peaked.

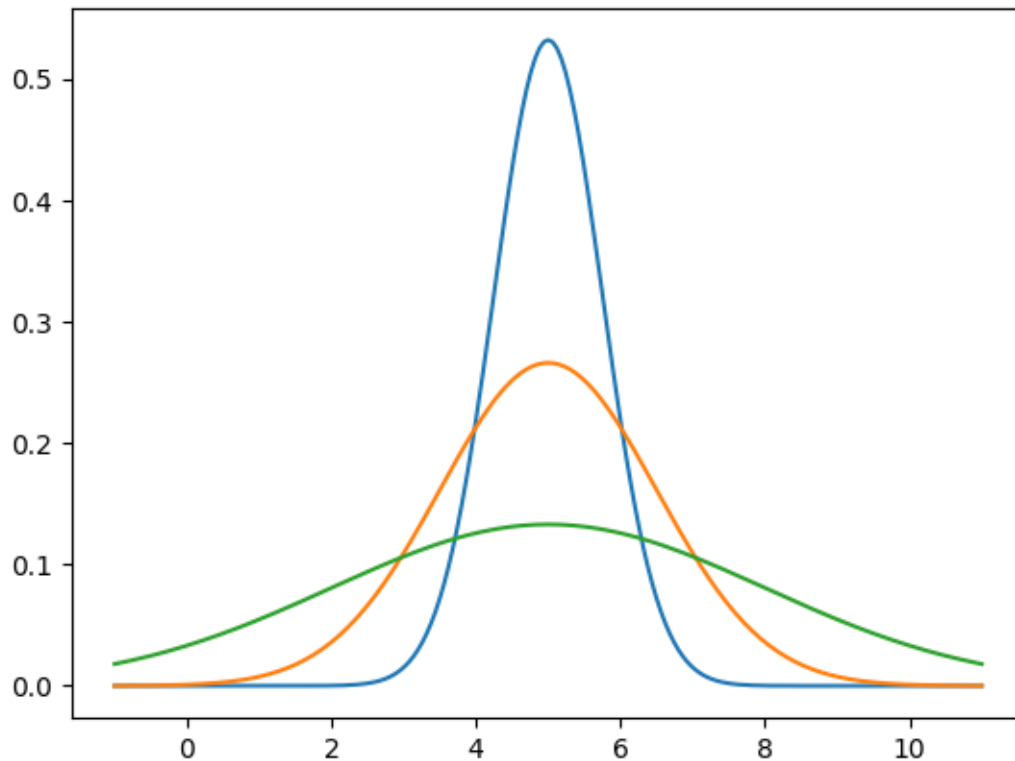
```
[85]: # Example of two normal distributions with different means and standard_
      ↪ deviations
x = np.linspace(-10, 20, num=201)

plt.plot(x, stats.norm.pdf(x, 0, 1), label="N(0, 1)"); # Standard normal_
      ↪ distribution
plt.plot(x, stats.norm.pdf(x, 5, 2), label="N(5, 4)"); # N(5, 4)
plt.legend(loc="upper left");
```



```
[86]: # Probability density of a normal distribution with different standard
      ↪ deviations
m = 5    # Mean
x = np.linspace(-1, 11, num=201)

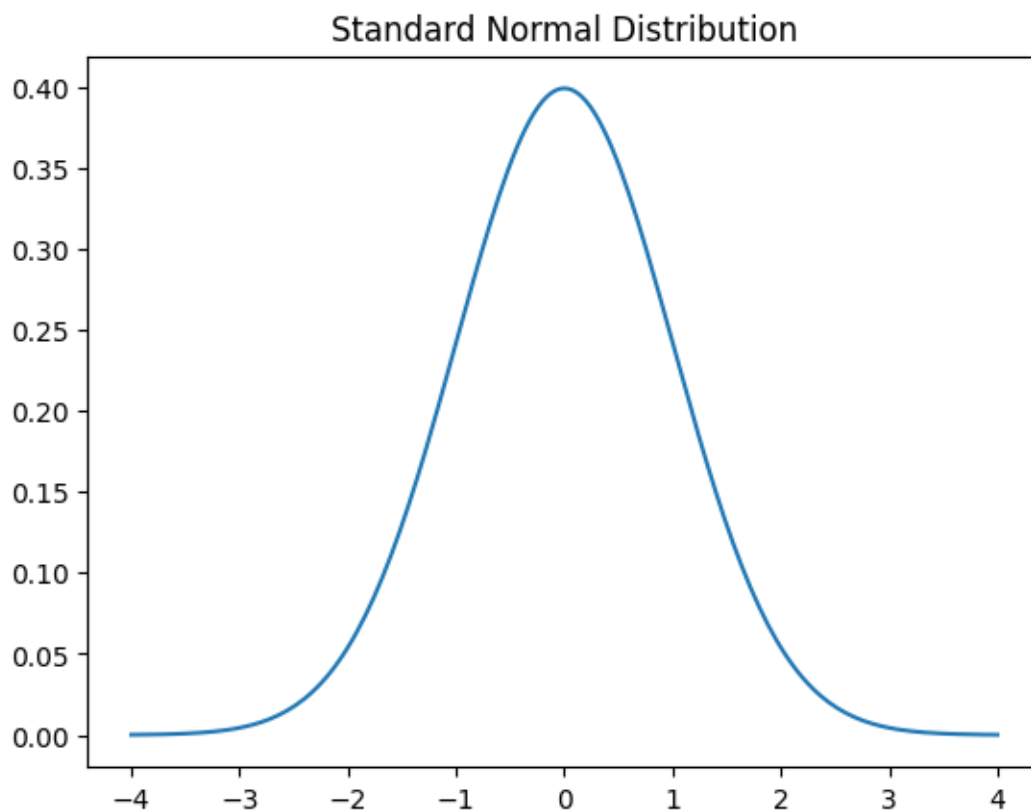
plt.plot(x, stats.norm.pdf(x, loc=m, scale=.75));
plt.plot(x, stats.norm.pdf(x, loc=m, scale=1.5));
plt.plot(x, stats.norm.pdf(x, loc=m, scale=3));
```



3.5.3 The standard normal distribution

The **standard normal distribution** is a normal distribution with mean 0 and variance 1. The variable Z is often used for the standard normal distribution: $Z \sim N(0, 1)$

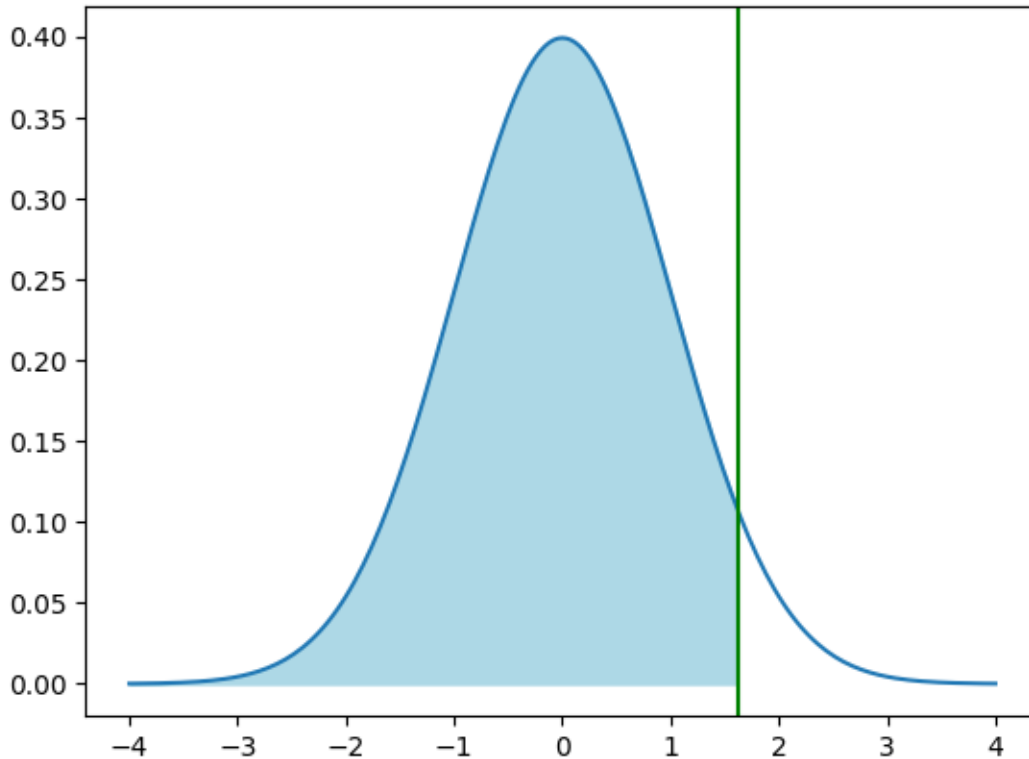
```
[87]: mu = 0      # Mean
sigma = 1      # Standard deviation
x = np.linspace(mu - 4 * sigma, mu + 4 * sigma, num=201)
y = stats.norm.pdf(x, mu, sigma)
plt.plot(x, y)
plt.title("Standard Normal Distribution");
```



Exercise If $Z \sim \text{Nor}(0, 1)$ find

1. $P(Z < 1.62)$
2. $P(Z > 0.76)$
3. $P(Z < -1.32)$
4. $P(-1.2 < Z < 1.7)$

```
[88]: # Exercise 1. Plot for  $P(Z < 1.62)$ 
mu = 0      # Mean
sigma = 1   # Standard deviation
x = 1.62
dist_x = np.linspace(mu - 4 * sigma, mu + 4 * sigma, num=201)
dist_y = stats.norm.pdf(dist_x, mu, sigma)
plt.plot(dist_x, dist_y)
# Fill the area left of x
plt.fill_between(dist_x, 0, dist_y, where=dist_x <= x, color='lightblue')
plt.axvline(x, color="green");
```



```
[89]: print("1. P(Z < 1.62) ~= %.4f" % stats.norm.cdf(1.62, loc=0, scale=1))
```

1. P(Z < 1.62) ~= 0.9474

3.5.4 Standardised scores

The normal distribution allows us to make comparisons between individuals in a particular normal population. However, it becomes harder with different normal populations, for example we use different criteria to judge a ‘tall man’ than we use for a ‘tall woman’.

To compare individuals across different normal populations, we can use **standardised scores** or **z-scores**. A standardised score is the number of standard deviations a particular value is from the mean. The formula for calculating a standardised score is:

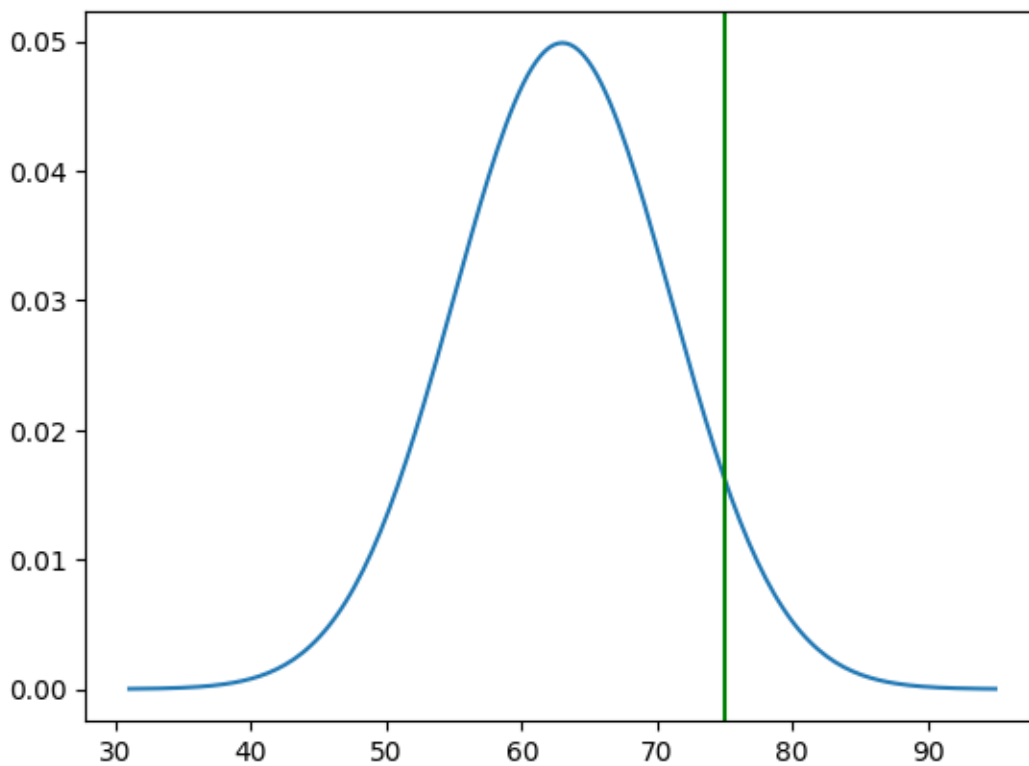
$$z = \frac{x - \mu}{\sigma}$$

where x is the value, μ is the mean and σ is the standard deviation.

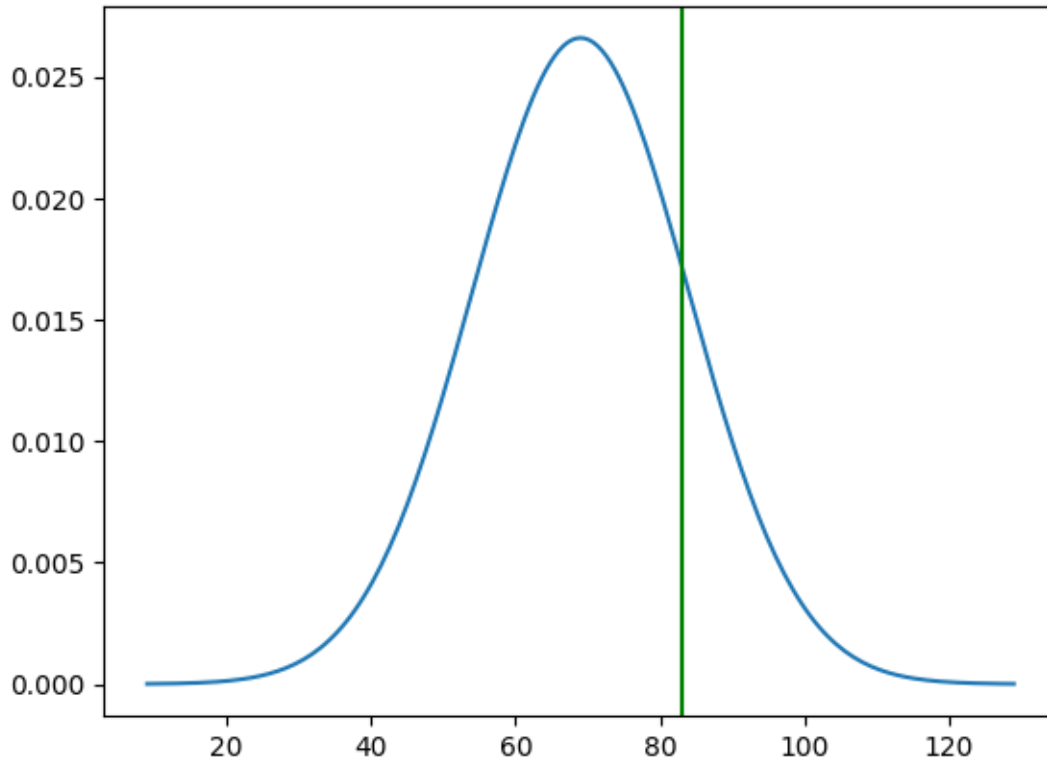
By converting values to the standard normal distribution, you can calculate probabilities for any normal distribution. I.e. one way to compare different normal populations is to standardise the scores - by looking at their distance from the mean, then dividing by the size of the standard deviation.

Example In her exams, Alexandra scores 75 in History and 83 in Maths. For the year group as a whole, History has a mean score of 63 in the examination with a standard deviation of 8, while Maths has a mean of 69 with a standard deviation of 15. Compare Alexandra's performance in these two subjects.

```
[90]: mu = 63      # Mean score History
sigma = 8      # Standard deviation History
x = np.linspace(mu - 4 * sigma, mu + 4 * sigma, num=201)
y = stats.norm.pdf(x, mu, sigma)
plt.plot(x, y)
plt.axvline(75, color="green"); # Show a green line for Alexandra's score for History
```



```
[91]: mu = 69      # Mean score Maths
sigma = 15      # Standard deviation Maths
x = np.linspace(mu - 4 * sigma, mu + 4 * sigma, num=201)
y = stats.norm.pdf(x, mu, sigma)
plt.plot(x, y)
plt.axvline(83, color="green"); # Show a green line for Alexandra's score for Maths
```



There is reason to say that Alexandra's performance is better in History than in Maths.

3.5.5 Probabilities in the normal distribution

Exercise The lengths of steel girders produced in a factory are normally distributed with a mean length of 12.5 m and a variance of 0.0004 m^2 . Girders need to be between 12.47 and 12.53 metres to be used in construction.

1. Find the proportion of girders which cannot be used for construction. (Answer: approx. 13.36%)
2. Girders are extremely expensive to produce, and the company is not happy with this level of wastage. A new machine is installed which reduces the variance of the production to 0.0002 m^2 while maintaining a normal distribution with the same mean as before. Find the proportion of girders produced by the new machine which cannot be used for construction. (Answer: approx. 3.39%)

[]:

Exercise In an examination the marks are normally distributed with mean 60.7 and a standard deviation of 12.3.

1. A candidate needs a mark of 40 to pass. Approximately what percentage of candidates fail? (Answer: approx. 4.62%)
2. The board awards distinctions to the best 10% of the candidates. What is the least mark a candidate will to get a distinction? (Answer: approx. 76.46)

[]:

Exercise A machine pours melted chocolate into moulds. The standard deviation of the amount is pours is 0.7 grams, and the mean amount can be set on the machine. The amount poured may be assumed to follow a normal distribution.

The machine is to produce bars whose labels says 50g of chocolate.

The company's lawyers want to have no more than 0.5% of bars containing less than the advertised weight.

What should the mean be set at? (Answer: approx. 51.803g)

[]:

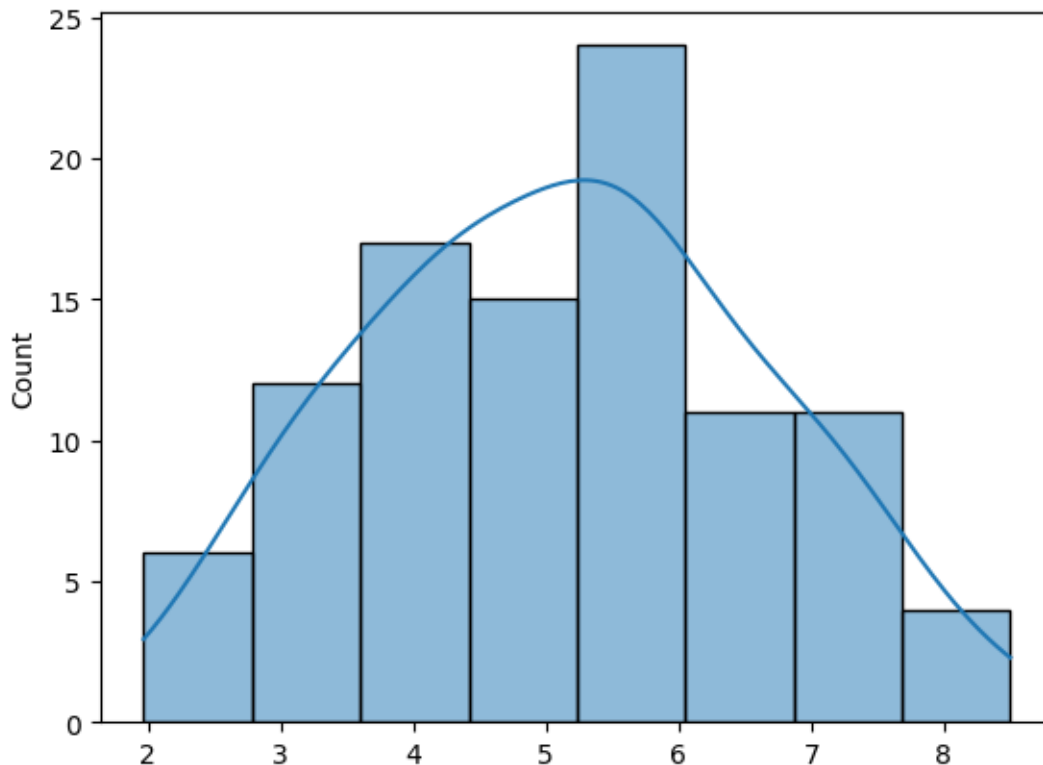
3.5.6 Histogram with theoretical probability density

The next example shows how to plot a histogram of normally distributed data, with the theoretical probability density function superimposed as a line graph.

```
[97]: # Generate n random numbers, following a normal distribution
m = 5      # Mean
s = 1.5    # Standard deviation

# Increase the value for n and see how the histogram changes!
n = 100
observations = np.random.normal(loc=m, scale=s, size=n)

# Plot histogram, without probability density
obs_hist = sns.histplot(observations, kde=True)
```



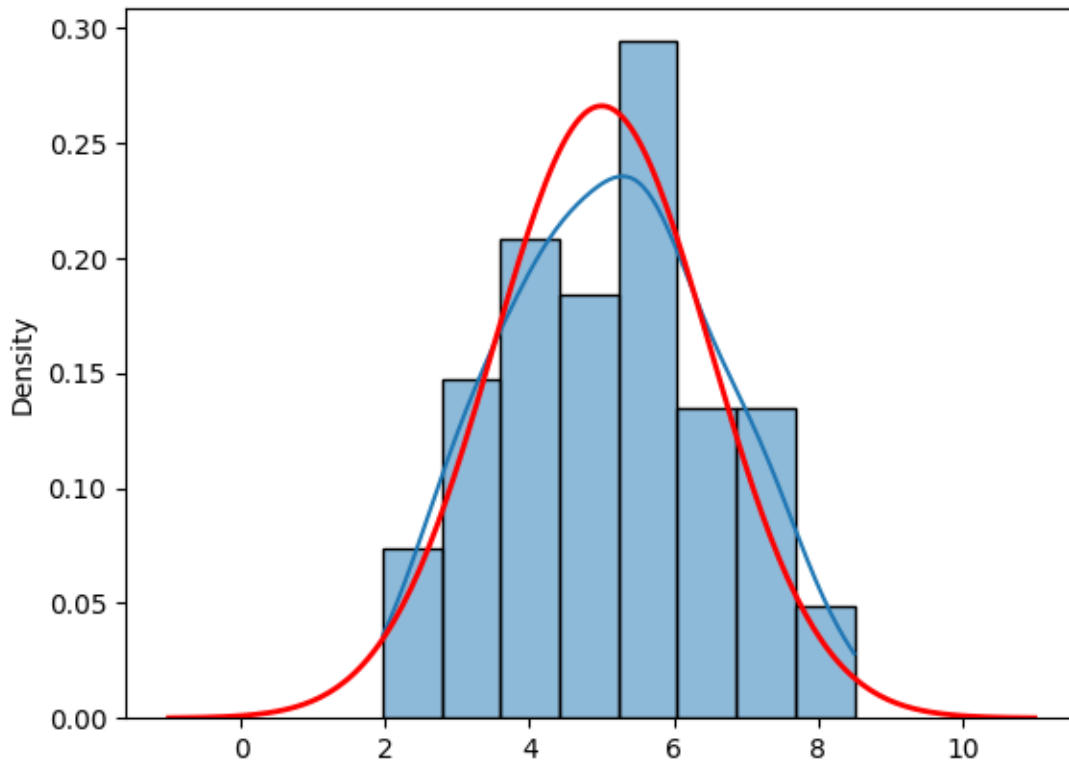
If you plot a Gauss curve on the histogram above, you won't see much (try this for yourself!). After all, the scale on the y-axis for the Gauss curve is much smaller than that of the histogram (see the plot of a normal distribution above). The maximum value of a density function is always a number between 0 and 1 (since the area under the curve must be equal to 1), while a histogram counts how many times each observation occurs and consequently is usually larger than 1.

The solution is to draw a histogram showing *densities*. In this case, the *area* of a member is equal to the *relative* frequency of that class. (The *density* or *density* is the *relative* frequency divided by the *width* of the class.) The sum of all areas of all bars is one. This corresponds to the area under the Gauss curve which is also equal to one. The histogram and the Gauss curve can then use the same y-axis.

```
[98]: # histogram with observations
obs_hist = sns.histplot(observations, stat = "density", kde=True)

# X-axis values
dist_x = np.linspace(m - 4 * s, m + 4 * s, num=201)
# Probability density for the X-values
dist_y = stats.norm.pdf(dist_x, m, s)

# Plot the Gauss curve superimposed on the histogram
obs_hist.plot(dist_x, dist_y, 'red', lw=2);
```



3.6 Other continuous distributions

There exist numerous other continuous distributions, each with its own characteristics and applications. In this section, we will briefly discuss the exponential and uniform distributions, which are commonly used in various fields. Further along in this course, we will explore more distributions in detail, a.o. the Student- t and χ^2 (chi-squared) distributions.

3.6.1 Exponential Distribution

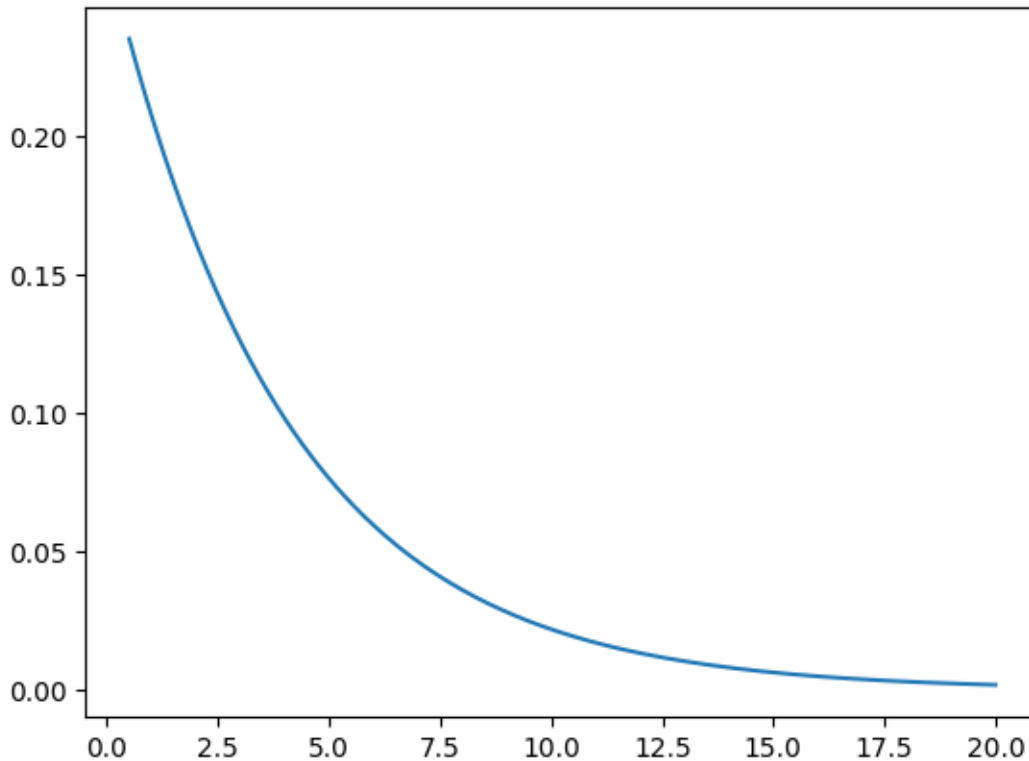
Besides the Normal Distribution, there are other often used continuous distributions, e.g. the **Exponential Distribution**. Values for an exponential random variable occur when there are fewer large values and more small values. For example, the amount of money customers spend in a trip to the supermarket follows an exponential distribution. There are more people who spend small amounts of money and fewer people who spend large amounts of money.

Another example is the length, in minutes, of long distance business telephone calls or the amount of time (in minutes) a postal clerk spends with his or her customers. In the image below, the time is known to have an exponential distribution with an average amount of time equal to four minutes.

Python functions for the exponential distribution are available in the `scipy.stats.expon` module. See the [SciPy API documentation](#) for more details.

```
[94]: # Example of an exponential distribution
      # Change the values of loc and scale to see how the distribution changes

      dist_x = np.linspace(0.5, 20, num=201)
      dist_y = stats.expon.pdf(dist_x, loc = 0.25, scale = 4)
      plt.plot(dist_x, dist_y);
```



3.6.2 Continuous Uniform Distribution

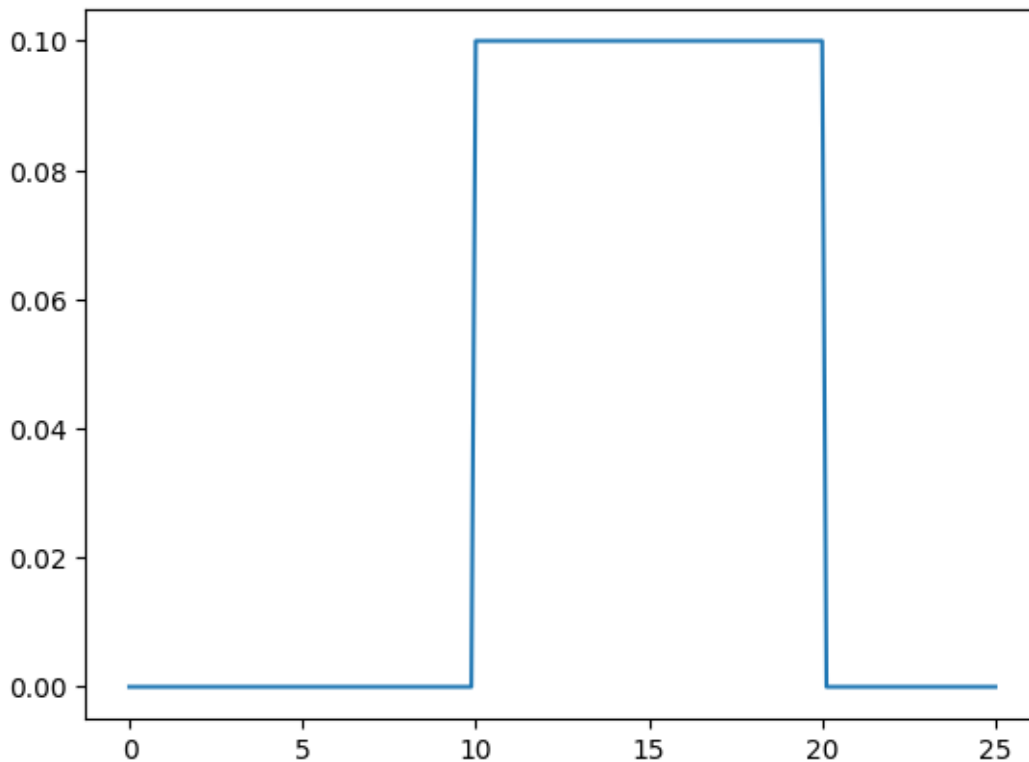
The continuous **Uniform Distribution** describes an experiment where there is an arbitrary outcome that lies between certain bounds. It is the simplest of all continuous probability distributions. The density function is constant where every value has an equal chance of occurring.

Python functions for the uniform distribution are available in the `scipy.stats.uniform` module. See the [SciPy API documentation](#) for more details.

Imagine you live in a building that has an elevator that will take you to your floor. From experience, once you push the button to call the elevator, it takes between ten and twenty seconds to arrive at your floor. This means the elevator arrival time is uniformly distributed between 10 and 20 seconds once you hit the button

```
[95]: # Example of a uniform distribution
      # Change the values of loc and scale to see how the distribution changes
```

```
dist_x = np.linspace(0, 25, num=201)
dist_y = stats.uniform.pdf(dist_x, loc=10, scale=10)
plt.plot(dist_x, dist_y);
```



The total area under the probability density function is always equal to 1.
So it holds that that $P(10 \leq X \leq 20) = 1$

$P(13 < X < 18) = \text{area under } f_X(x) \text{ between } 13 \text{ and } 18 = 5 \times 0.1 = 0.5$

Note:

$P(13 < X < 18) = P(13 < X \leq 18) = P(13 \leq X < 18) = P(13 \leq X \leq 18)$

[]:

3.7 Further reading

Nicholson, J. (2018). *Complete Probability & Statistics 1 for Cambridge International AS & A Level*. Oxford University Press.

Nicholson, J. (2018). *Complete Probability & Statistics 2 for Cambridge International AS & A Level*. Oxford University Press.

4 Module 3.02 - Central Limit Theorem and Confidence Intervals

The *central limit theorem* (CLT) is one of the most fundamental results in statistics. Basically, it determines the conditions under which it is allowed (to a certain extent) to extrapolate results obtained from a sample to the entire population, a process referred to as *statistical inference*. Previously, we stated that a sample should be “large enough” and “randomly selected”. The central limit theorem gives us a more precise definition of what “large enough” means.

The central limit theorem has led to several important applications in statistical inference, including the construction of *confidence intervals* (CI) and *hypothesis testing*.

4.1 Learning goals

By the end of this module, you should be able to:

- Understand the definitions and terminology (indicated with $\propto$) used in this notebook
- Understand the central limit theorem and explain its significance in statistics.
- Describe the conditions under which the central limit theorem applies.
- Construct confidence intervals for the population mean of both large and small samples.

```
[19]: # Importing the necessary packages
import numpy as np                    # "Scientific computing"
import scipy.stats as stats           # Statistical tests

import pandas as pd                  # Data Frame
from pandas.api.types import CategoricalDtype

import matplotlib.pyplot as plt      # Basic visualisation
from statsmodels.graphics.mosaicplot import mosaic # Mosaic diagram
import seaborn as sns                # Advanced data visualisation
↪ visualisation
```

4.2 Central Limit Theorem (CLT)

The $\propto$ **Central Limit Theorem** can be formulated as follows:

If the size of a *random sample* is *sufficiently large*, the probability distribution of the sample mean will approximate a normal distribution, regardless of the probability distribution of the underlying population.

In other words:

Consider a random sample of n observations drawn from a population with expected value μ and standard deviation σ . If n is sufficiently large, the probability distribution of the sample mean $\bar{x}$ will approximate a normal distribution with mean $\mu_{\bar{x}} = \mu$ and standard deviation $\sigma_{\bar{x}} = \frac{\sigma}{\sqrt{n}}$, or $\bar{X} \sim \text{Nor}\left(\mu, \frac{\sigma}{\sqrt{n}}\right)$.

The rule of thumb is that a sample size of $n = 30$ is sufficient to apply the CLT. No matter how strange the underlying probability distribution is, by the time the sample size gets to 30 the distribution will be passably close to the normal distribution. The larger the sample, the better the probability distribution of $\bar{x}$ will approximate the expected value of the population, μ .

A very nice visual demonstration of the effect of the CLT can be found at <https://www.zoology.ubc.ca/~whitlock/Kingfisher/CLT.htm>

4.2.1 Exercise

On a flight there are 43 bags checked in. Historical records show that on that route the mean weight of checked bags is 12.1kg with a standard deviation of 3.8kg.

What is the probability that the mean contents of the checked bags on this flight will be under 11 kg? (You may treat the bags on this flight as though they are a random sample of bags checked in on that route.)

Even though you know nothing about the distribution of the weights of bags themselves, the Central Limit Theorem allows you to solve this. (Answer: 0.0288)

[]:

4.2.2 Exercise

The times for a particular journey during rush hour in a large city have mean 23.3 minutes and standard deviation 8.9 minutes.

Find the probability that the mean time taken for a random sample of 60 of the times for this journey is under 25 minutes. (Answer: 0.9305)

[]:

4.3 Point estimate

A **Point Estimate** for a population parameter is a formula or equation that allows us to calculate a value to estimate that parameter.

Example: sample variance/standard deviation, sample mean.

A point estimate is a single numeric value, a ‘best guess’ of a population parameter, based on the data in a random sample.

4.4 Confidence Interval

A **confidence interval** is an equation or formula that allows us to construct an interval that will contain the parameter to be estimated with a certain level of confidence.

A confidence interval is an interval around the point estimate, calculated from the sample data, that is very likely to contain the true value of the population parameter.

The typical form of a Confidence Interval is **Point Estimate $\pm$ Factor $\times$ Standard Error**.

As will be explained in the following paragraphs, when estimating a population mean, the point estimate is the sample mean $\bar{x}$, the **standard error** is the standard deviation σ divided by the square root of the sample size n (i.e. the standard error is $\frac{\sigma}{\sqrt{n}}$), and the factor is approximately equal to 2.

We can calculate the Confidence Interval for

- a mean

- a standard deviation
- the difference between means
- a total
- ...

In this course, only the *confidence interval of the mean* is discussed

4.5 Confidence Interval for the mean of a large sample from any distribution

Given a sample with mean $\bar{x}$.

We are looking for an interval $[\bar{x} - b, \bar{x} + b]$ for which we can say with a **level of confidence** ($1 - \alpha$) of e.g. 95% that μ is inside this interval.

$$P(\bar{x} - b < \mu < \bar{x} + b) = 1 - \alpha = 0.95$$

Because of the central limit theorem we know that

$$\bar{x} \in \bar{X} \sim \mathcal{N}or\left(\mu, \frac{\sigma}{\sqrt{n}}\right)$$

And because of the symmetry we can say:

$$P(\bar{x} - b < \mu < \bar{x} + b) = P(\mu - b < \bar{x} < \mu + b)$$

We now calculate the z -score for $\bar{x} : z = \frac{\bar{x} - \mu}{\frac{\sigma}{\sqrt{n}}}$

We calculate the value $z_{\frac{\alpha}{2}}$ for which:

$$P(-z_{\frac{\alpha}{2}} < z < z_{\frac{\alpha}{2}}) = 1 - \alpha = 0.95$$

$$P(z < z_{\frac{\alpha}{2}}) = 1 - \frac{\alpha}{2} = 0.975$$

$$z_{\frac{\alpha}{2}} = \text{stats.norm.isf}(1 - 0.975) \approx 1.96$$

The result is:

$$P\left(-1.96 < \frac{\bar{x} - \mu}{\frac{\sigma}{\sqrt{n}}} < 1.96\right) = 0.95$$

$$P\left(\mu - 1.96 \frac{\sigma}{\sqrt{n}} < \bar{x} < \mu + 1.96 \frac{\sigma}{\sqrt{n}}\right) = 0.95$$

Because of symmetry we can swap μ and $\bar{x}$:

$$P\left(\bar{x} - 1.96 \frac{\sigma}{\sqrt{n}} < \mu < \bar{x} + 1.96 \frac{\sigma}{\sqrt{n}}\right) = 0.95$$

Now we can say with 95% confidence that:

$$\mu \in \left[\bar{x} - 1.96 \frac{\sigma}{\sqrt{n}}, \bar{x} + 1.96 \frac{\sigma}{\sqrt{n}} \right]$$

In practice we will use s_{sample} as a point estimate for $\sigma_{population}$

To determine a confidence interval, we proceed as follows:

1. We take the *sample mean* as an initial estimate and choose a *confidence level*, eg $1-\alpha=0.95$ (or 95%).
2. We then find the *z-score* between which 95% of all values lie with a standard normal distribution.
3. We use this to determine the values to the left and right of the sample mean between which we expect 95% of the values to fall for the probability distribution that we get from the central limit theorem.

4.5.1 Example

A random sample of 45 cans of mixed beans gave a sample mean of 324.6 grams and we know from historical data that the standard deviation of the population is 2.5. Calculate a 95% confidence interval for the population mean weight.

```
[20]: # Step 1.
m = 324.6      # Sample mean
s = 2.5        # Population standard deviation
n = 45         # Sample size
alpha = .05    # 1 - alpha is the confidence level

# Step 2.
z = stats.norm.isf(alpha/2)
print("z-score: %.5f" % z)

# Step 3.
lo = m - z * s / np.sqrt(n)
hi = m + z * s / np.sqrt(n)
print("Confidence interval: [%.4f, %.4f]" % (lo, hi))
```

z-score: 1.95996

Confidence interval: [323.8696, 325.3304]

```
[21]: # Or, with the interval() function:
stats.norm.interval(.95, loc=m, scale=s/np.sqrt(n))
```

```
[21]: (np.float64(323.86956454951957), np.float64(325.3304354504805))
```

4.5.2 True meaning of 95% Confidence Interval

Given the data in a particular sample, a *95% confidence interval* for the mean will either include the (unknown) population mean or it won't. The true meaning of a 95% confidence interval is that if the same procedure is used on many different random samples, about 95% of the resulting confidence intervals will include the population mean, and only about 5% won't. Therefore, you can be *95% confident* that any particular confidence interval is a "good" one.

Remark that it is *not correct* to say that there is a 95% *probability* that the population mean is in the confidence interval. The population mean is a fixed value, and it either already is or isn't in the confidence interval. You just don't know which is the case.

The diagram below tries to illuminate this visually. Alternatively, you can take a look at the animated simulation on this webpage: <https://www.zoology.ubc.ca/~whitlock/Kingfisher/CIMean.htm>

The bottom of the diagram shows $\mu - 1.96 \frac{\sigma}{\sqrt{n}}$, μ , and $\mu + 1.96 \frac{\sigma}{\sqrt{n}}$ as the interval centered on the true population mean μ . There are then four intervals shown, of the same width, centered on four possible observed values of $\bar{x}$. For the bottom two, the value of $\bar{x}$ lies inside the interval centered on μ but for the top two the value of $\bar{x}$ lies outside. The bold black line shows where the value of μ is - and it cuts the two bottom intervals but not the top two intervals.

So 95% of the time the true population mean will be captured by a confidence interval constructed in this way. You should talk about the true population mean lying in the confidence interval on 95% of occasions a confidence interval is constructed rather than talking about a probability.

One thing to note is that in a lot of production processes the variability of the output is quite stable - it is a function of the quality of the process. The mean can usually be altered by a setting on the machinery and can often move while the process is underway. So, where you usually need to calculate the standard deviation (or variance) in data, in a production process you will often know the standard deviation and can use this in constructing an interval estimate for the mean.

The usual size of confidence interval is 95%, but it is not the only possible one.

Technically, you can construct any size of confidence interval you choose to for a continuous distribution but in practical terms there are three others which are the most common alternatives: 90%, 98% and 99%.

4.5.3 Exercise

For the previous exercise about the *mixed beans*:

1. Give the 90% confidence interval (Answer: [323.9870,325.2130])
2. Give the 98% confidence interval (Answer: [323.7330,325.4670])
3. Give the 99% confidence interval (Answer: [323.6400,325.5600])

[]:

4.6 Confidence Interval for the mean of a small sample

When we have a small sample, the central limit theorem does not apply. However, there is still a way to construct a confidence interval for the mean, depending on the fact if the standard deviation of the population is known or not known.

4.6.1 Confidence Interval for the mean of a small sample from another (i.e. not normal) distribution

You need to look for an appropriate test!

4.6.2 Confidence Interval for the mean of a small sample from a normal distribution if the standard deviation of the population is known

The above formulas are retained if the sample comes from a normal distribution and the standard deviation of the population is known.

4.6.3 Confidence Interval for the mean of a small sample from a normal distribution if the standard deviation of the population is unknown

When we have a **small sample** ($n < 30$) and the standard deviation of the population is unknown, we cannot use the normal distribution.

However, there is still a way to construct a confidence interval via the so-called Student- t distribution and we will use the standard deviation of the sample. This Student- t distribution resembles the normal distribution in that the probability density function looks like a Gauss curve. However, the Student- t distribution also takes the sample size n into account, so you must specify it. The density function gets an extra parameter called the number of *degrees of freedom* (abbreviated **df** or **dof**) and is equal to $n - 1$.

So we can say:

If a population X has a normal distribution ($X \sim \text{Nor}(\mu, \sigma)$) and you take a *small* sample with mean $\bar{x}$ and standard deviation s , then

$$t = \frac{\bar{x} - \mu}{\frac{s}{\sqrt{n}}}$$

will behave as a t -distribution with $n - 1$ degrees of freedom.

4.6.4 Student- t distribution

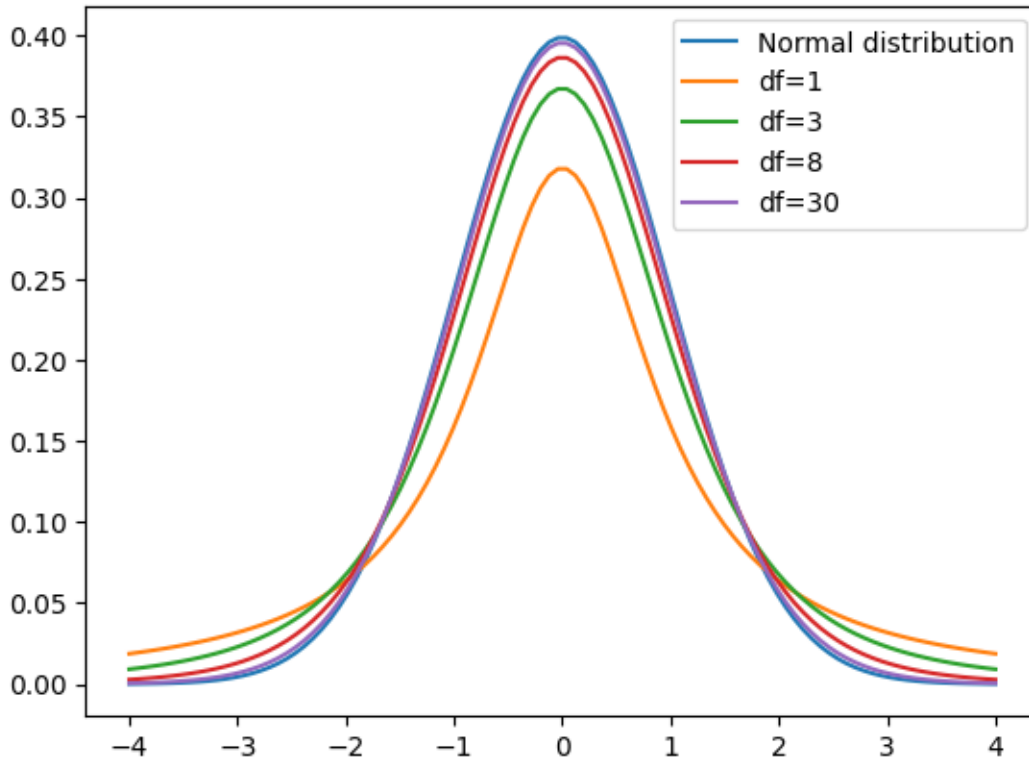
The **Student- t distribution** was developed by William Sealy Gosset, who worked for the Guinness brewery in Dublin, Ireland. He published his work in 1908 under the pseudonym “Student” because Guinness did not allow its employees to publish research under their own names. The Student- t distribution was developed to solve the problem of estimating the mean of a normally distributed population when the sample size is small and the population standard deviation is unknown.

The smaller the degrees of freedom, the “flatter” the curve and the wider the confidence intervals obtained will be. This represents the greater uncertainty we get because of the small sample size. The larger sample size n is, the closer the curve will approach that of the normal distribution.

In the graph below you will find the density function for the Student- t distribution for different degrees of freedom:

```
[16]: x = np.linspace(-4, +4, num=100)
fig, tplot = plt.subplots(1, 1)
tplot.plot(x, stats.norm.pdf(x, 0, 1), label="Normal distribution")
tplot.plot(x, stats.t.pdf(x, 1), label="df=1")
tplot.plot(x, stats.t.pdf(x, 3), label="df=3")
tplot.plot(x, stats.t.pdf(x, 8), label="df=8")
```

```
tplot.plot(x, stats.t.pdf(x, 30), label="df=30")
tplot.legend(loc='best');
```



4.6.5 Student t -distribution in Python

```
import scipy.stats
```

For a t -distribution with df degrees of freedom: (df = degrees of freedom)

Function	Purpose
<code>stats.t.pdf(x, df=d)</code>	Probability density for x
<code>stats.t.cdf(x, df=d)</code>	Left-tail probability ($< x$)
<code>stats.t.sf(x, df=d)</code>	Right-tail probability ($> x$)
<code>stats.t.isf(1-p, df=d)</code>	$p\%$ of observations are expected to be lower than this value

4.6.6 Calculation of a confidence Interval for a small sample from a normal distribution for which the standard deviation of the population is not known

To determine the confidence interval for the mean μ of a population, based on a small sample, we first calculate $t_{\frac{\alpha}{2}, df}$.

With a confidence level of 95% we have $\frac{\alpha}{2} = 0.025$

Assume for example $n = 5$ (so $df=4$), then we have

$$t_{\frac{\alpha}{2}, df} = \text{stats.t.isf}(1-0.975, df=4) = 2.776$$

We can say with a confidence of 95% that:

$$\mu \in \left[\bar{x} - t_{\frac{\alpha}{2}, df} \cdot \frac{s}{\sqrt{n}}, \bar{x} + t_{\frac{\alpha}{2}, df} \cdot \frac{s}{\sqrt{n}} \right]$$

4.6.7 Example

A random sample of 15 cans of mixed beans gave a sample mean of 324.6 grams and the standard deviation of the sample is 2.5. The weights of the samples are normally distributed. Calculate a 95% confidence interval for the population mean weight.

```
[22]: # Step 1.
m = 324.6      # Sample mean
s = 2.5        # Standard deviation of the sample
n = 15         # Sample size
alpha = .05    # 1 - alpha is the confidence level

# Step 2.
t = stats.t.isf(alpha/2, df = n - 1)
print("t-score: %.5f" % t)

# Step 3.
lo = m - t * s / np.sqrt(n)
hi = m + t * s / np.sqrt(n)
print("Confidence interval: [%.4f, %.4f]" % (lo, hi))
```

t-score: 2.14479

Confidence interval: [323.2155, 325.9845]

```
[23]: # Or, with the interval() function:
stats.t.interval(1-alpha, df = n - 1, loc=m, scale=s/np.sqrt(n))
```

```
[23]: (np.float64(323.21554614608783), np.float64(325.9844538539122))
```

This interval is wider than what we obtained with a larger sample. So we are less certain of the position of the population mean.

4.6.8 Exercise

The weights, in grams, of bags of cocoa beans are normally distributed and are known to have a standard deviation of 6 grams. A random sample of bags is weighed with the following results: [758, 748, 749, 752, 757, 760, 751, 745, 759, 761]

Calculate a 95% confidence interval for the mean weight of a bag of cocoa beans (Answer: [749.9393, 758.0607]).

```
[ ]:
```

[]: (749.9393036196667, 758.0606963803333)

Source

Complete Probability & Statistics 1 & 2 for Cambridge International AS & A Level

5 Module 3.03 - Statistical Hypothesis testing

Another important application of the *central limit theorem* is the concept of **statistical hypothesis testing**. This is a method used to check assumptions about a population based on a sample of observations. In this module, we introduce the concept of a hypothesis test, and we will learn how to perform a one-sample z -test and a one-sample t -test for the mean.

5.1 Learning goals

By the end of this module, you should be able to:

- Understand the definitions and terminology (indicated with α) used in this notebook
- Know the consecutive steps in the procedure of a hypothesis test
- Perform a one-sample z -test for the mean
- Perform a one-sample t -test for the mean
- Determine the appropriate test to use based on the data and assumptions

```
[30]: # Importing the necessary packages
import numpy as np                # "Scientific computing"
import scipy.stats as stats       # Statistical tests

import random
import math

import pandas as pd              # Data Frame
from pandas.api.types import CategoricalDtype

import matplotlib.pyplot as plt  # Basic visualisation
from statsmodels.graphics.mosaicplot import mosaic # Mosaic diagram
import seaborn as sns            # Advanced data visualisation
↪ visualisation
```

5.2 Statistical Hypothesis Testing

5.2.1 Definitions

- α **Hypothesis Test** = A procedure for verifying a hypothesis about the value of a population parameter.
- α **Hypothesis** = A statement or conjecture regarding the numeric value of a population parameter.
- The α **null Hypothesis** H_0 = an assumption about a population parameter that we want to test. The procedure of the test is based on the assumption that the null hypothesis is true. If we then encounter a contradiction, the null hypothesis is rejected and the alternative hypothesis is accepted.

- The $\propto$ **alternative Hypothesis** (H_1) = the negation of the null hypothesis. It represents the conclusion of the test if the null hypothesis is unlikely to be true.
- $\propto$ **Critical region or rejection region** = The set of all possible values for the test statistic for which we will reject the null hypothesis.
- $\propto$ **Acceptance region** = The set of all possible values for the test statistic that support the null hypothesis.
- $\propto$ **Critical value** g = Boundary of the critical region
- $\propto$ **Significance Level** α = The probability of rejecting the null hypothesis H_0 when it is actually true.
- $\propto$ **Probability value** or p -value = The probability of obtaining a test statistic as extreme as, or more extreme than, the observed value, assuming the null hypothesis is true.

The principle of **hypothesis testing** is to calculate how likely it is to observe the **test statistic** (the statistic calculated from the observations in the sample) if you assume that the null hypothesis H_0 is true. If the probability of observing the test statistic is very low, i.e. if the test statistic lies in the most extreme part of the probability distribution (called the **critical region**) you conclude that the null hypothesis is unlikely to be true and you reject it in favour of the alternative hypothesis H_1 .

All other values lie in the **acceptance region** of the test. The probability of this happening when H_0 is actually true is called the size of the test. 5% is often used as the common default size.

Take care in the language that you use!

- If you accept H_0 , never say that this is because the evidence supports it. You can only conclude there is no significant evidence in the sample *against* H_0 .
- If you reject H_0 , say that there is evidence in the sample that the null hypothesis is not true.

All statistical hypothesis tests follow the same $\propto$ **testing procedure**:

1. Formulate both *hypotheses* (H_0 and H_1)
2. Determine the *significance level* (α)
3. Calculate the *test statistic* from the sample
4. Determine the *critical region* or the *probability value*
5. Draw *conclusions* (reject or don't reject H_0)

5.3 The z-test

The z -test is used to confirm or refute an assumption about the (unknown) *population mean*, based on a sufficiently large random sample.

5.3.1 Example: the right-tailed z -test

Jars of jam are filled by a machine. It has been found that the quantity of jam in a jar is normally distributed and has mean $\mu = 351.2$ g, with standard deviation $\sigma = 4.1$ g. It is believed that the settings of the mean amount on the machine might have been altered accidentally and that the machine is now filling jars with *more* jam. Assume that the standard deviation has not been altered.

To test this, a random sample of $n = 40$ jars is taken and the mean quantity per jar is calculated. The mean quantity per jar is found to be $\bar{x} = 352.8$ g.

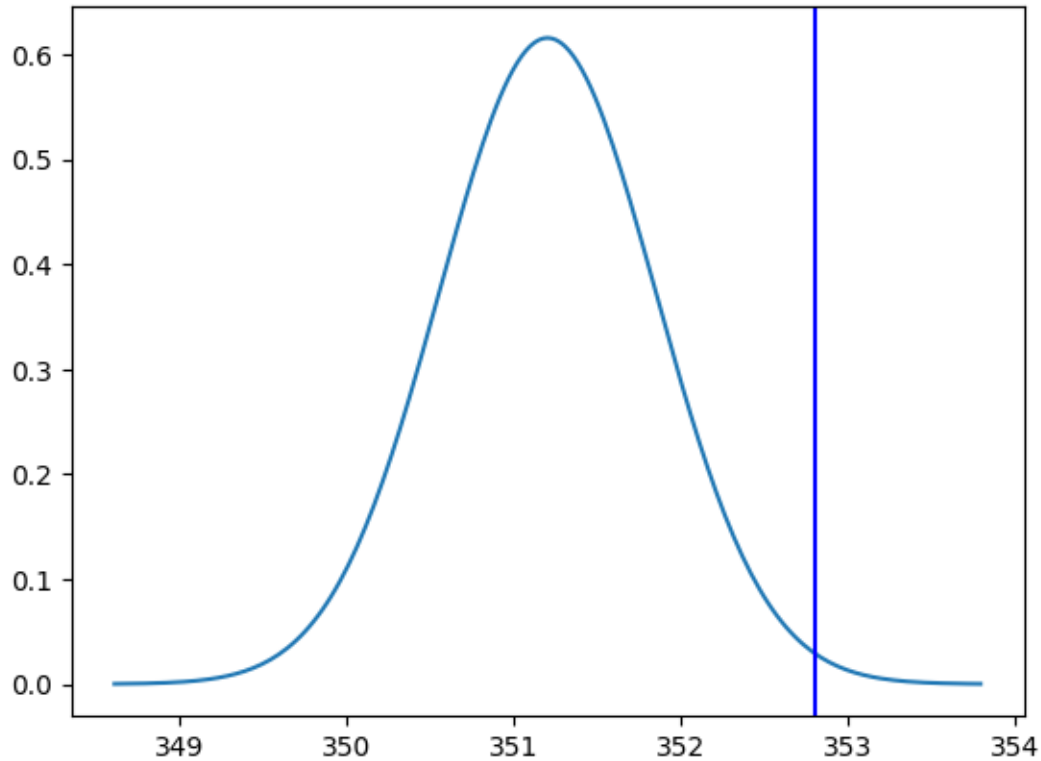
This sample meets the conditions where the *central limit theorem* applies, so we know that the probability distribution of the sample mean is a normal distribution with mean μ and standard deviation $\sigma/\sqrt{n}$, where n is the sample size (or $M \sim \text{Nor}(\mu, \sigma/\sqrt{n})$).

Let's first plot the situation:

```
[31]: # Properties of the sample:
n = 40          # Sample size
mu = 351.2      # (Hypothetical) population mean
sigma = 4.1     # Population standard deviation (assumed to be known)
m_sample = 352.8 # Sample mean
stderr = sigma / math.sqrt(n)

# X-values
dist_x = np.linspace(mu - 4 * stderr, mu + 4 * stderr, num=201)
# Y-values for drawing the Gauss curve
dist_y = stats.norm.pdf(dist_x, mu, stderr)
# Plot the Gauss-curve
plt.plot(dist_x, dist_y)

# Fill the area left of x
# plt.fill_between(dist_x, 0, dist_y, where=(dist_x <= x), color='lightblue')
# Show the mean with an orange line
plt.axvline(m_sample, color="blue");
```

The testing procedure then runs as follows:

- **Step 1.** Formulate the hypotheses:
 - $H_0 : \mu \leq 351.2$ (the amount of jam has not increased)
 - $H_1 : \mu > 351.2$ (the amount of jam has increased)
- **Step 2.** Choose a significance level, e.g. $\alpha = 0.05$
- **Step 3.** Determine the value of the test statistic, in this case $\bar{x} = 352.8$

Remark: in some textbooks, the test statistic is defined as the z -score of the sample mean, which is calculated as $z = (\bar{x} - \mu)/(\sigma/\sqrt{n})$

- **Step 4.** Determine the p -value and reject H_0 if $p < \alpha$.

```
[32]: alpha = 0.05 # Significance level chosen by the researcher
p = stats.norm.sf(m_sample, loc=mu, scale=sigma/np.sqrt(n))

print("p-value: %.5f" % p)
if(p < alpha):
    print("p < alpha: reject H0")
else:
    print("p > alpha: do not reject H0")
```

```
p-value: 0.00679
p < alpha: reject H0
```

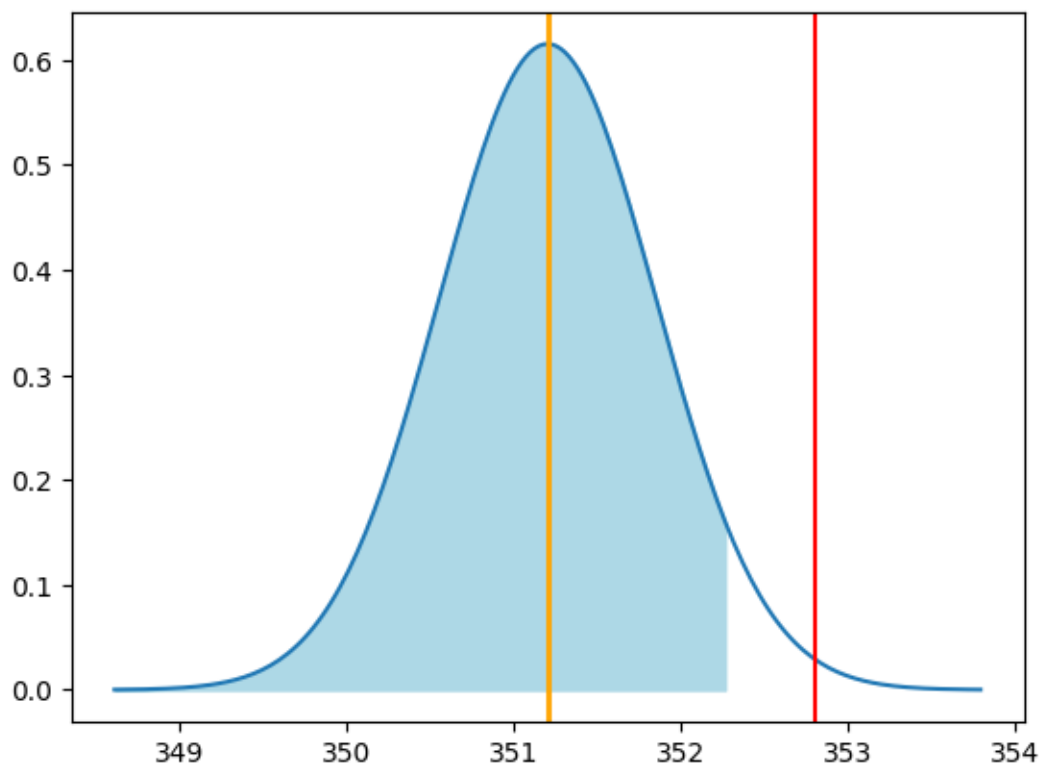
An alternative method is to determine the *critical region*, i.e. the set of all values for the sample mean where H_0 may be rejected. The boundary of that area is called the *critical value* g . To the left of it you can't reject H_0 (acceptance region), to the right you can (critical region). The area of the acceptance region is $1 - \alpha$, the area of the critical region is α .

```
[33]: # Critical value according to the formula:
#      g = mu + stats.norm.isf(alpha) * sigma / np.sqrt(n)
# Making use of the loc/scale parameters of isf:
g = stats.norm.isf(alpha, loc=mu, scale=sigma / np.sqrt(n))
print("Critical value g   %.3f" % g)
if (m_sample < g):
    print("sample mean = %.3f < g = %.3f: do not reject H0" % (m_sample, g))
else:
    print("sample mean = %.3f > g = %.3f: reject H0" % (m_sample, g))
```

```
Critical value g   352.266
sample mean = 352.800 > g = 352.266: reject H0
```

A plot of this case:

```
[34]: # Gauss-curve plot:
# X-values
dist_x = np.linspace(mu - 4 * sigma/np.sqrt(n), mu + 4 * sigma/np.sqrt(n),
    ↪ num=201)
# Y-values for the Gauss curve
dist_y = stats.norm.pdf(dist_x, mu, sigma/np.sqrt(n))
fig, dplot = plt.subplots(1, 1)
# Plot the Gauss-curve
dplot.plot(dist_x, dist_y)
# Show the hypothetical population mean with an orange line
dplot.axvline(mu, color="orange", lw=2)
# Show the sample mean with a red line
dplot.axvline(m_sample, color="red")
# Fill the acceptance area in light blue
dplot.fill_between(dist_x, 0, dist_y, where=dist_x <= g, color='lightblue');
```



- **Step 5** Draw conclusions.

The p -value is smaller than the significance level α , and the sample mean is in the critical region, so we reject the null hypothesis. This sample provides sufficient evidence to conclude that the machine fills the jars with more than 351.2 grams of jam.

5.3.2 Example: the left-tailed z -test

Let's assume the mean quantity per jar is found to be 350.4g. In this case we have reasons to assume that the population mean is *lower* than the value we are testing against ($\mu = 351.2$ g). We can verify this with a *left-tailed* z -test.

Step 1. Formulate the hypotheses:

- $H_0 : \mu \geq 351.2$ (the amount of jam has not decreased)
- $H_1 : \mu < 351.2$ (the amount of jam has decreased)

Step 2. Choose significance level, e.g. $\alpha = 0.05$

Step 3. Calculate the test statistic: $\bar{x} = 350.4$

Step 4. Determine the p -value and reject H_0 if $p < \alpha$.

```
[35]: # Properties of the sample:
n = 40          # Sample size
mu = 351.2      # (Hypothetical) population mean
```

```
sigma = 4.1          # Population standard deviation (assumed to be known)
m_sample = 350.4     # Sample mean
alpha = 0.05        # Significance level (chosen by the researcher)
```

```
[36]: # Pay attention! Use cdf() instead of sf()! Why?
p = stats.norm.cdf(m_sample, loc=mu, scale=sigma/np.sqrt(n))

print("p-value: %.5f" % p)
if(p < alpha):
    print("p < a, reject H0")
else:
    print("p > a, do not reject H0")
```

p-value: 0.10859

p > a, do not reject H0

In this case, the critical value g is left of the mean. On the right of g , you may not reject H_0 (acceptance region), on the left, you can (critical region).

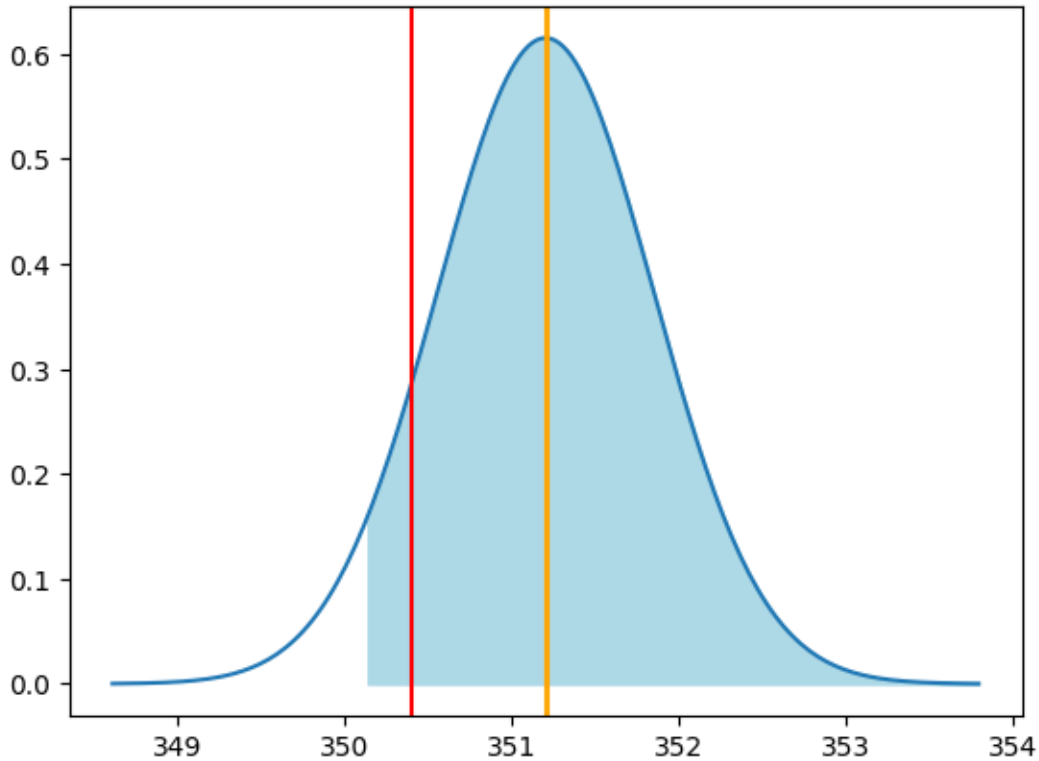
```
[37]: # Critical value according to the formula in the lecture slides
# g = m0 - stats.norm.isf(a) * s / np.sqrt(n)
# Making use of the loc/scale parameters of isf:
g = stats.norm.isf(1-alpha, loc=mu, scale=(sigma / np.sqrt(n)))
print("Critical value g   %.3f" % g)
if (m_sample > g):
    print("sample mean = %.3f > g = %.3f: do not reject H0" % (m_sample, g))
else:
    print("sample mean = %.3f < g = %.3f: reject H0" % (m_sample, g))
```

Critical value g 350.134

sample mean = 350.400 > g = 350.134: do not reject H0

A plot of this case:

```
[38]: # Gauss-curve:
# X-values
dist_x = np.linspace(mu - 4 * sigma/np.sqrt(n), mu + 4 * sigma/np.sqrt(n),
    ↪ num=201)
# Y-values
dist_y = stats.norm.pdf(dist_x, mu, sigma/np.sqrt(n))
fig, dplot = plt.subplots(1, 1)
# Plot
dplot.plot(dist_x, dist_y)
# Hypothetical population mean in orange
dplot.axvline(mu, color="orange", lw=2)
# Sample mean in red
dplot.axvline(m_sample, color="red")
# Fill the acceptance region in light blue
dplot.fill_between(dist_x, 0, dist_y, where=dist_x >= g, color='lightblue');
```



- **Step 5** Draw conclusions.

In this case, we see that the sample mean (the red line) still falls within the acceptance region (the area in light blue), so we cannot reject the null hypothesis. This is also confirmed by the p -value, which is greater than α . With this result, there is no reason to assume the machine is not working correctly. In other words, the sample does not provide sufficient evidence to conclude that the machine is filling the jars with less than 351.2 grams of jam.

5.4 The two-tailed z -test

When we don't want to test whether the population mean is either greater or lower than a hypothetical value, but only want to know if the sample mean is "close enough", we can use a two-tailed z -test.

Step 1. Formulate the hypotheses:

- $H_0 : \mu = 351.2$ (the amount of jam has not changed)
- $H_1 : \mu \neq 351.2$ (the amount of jam has changed)

Step 2. Choose significance level, e.g. $\alpha = 0.05$

Step 3. Determine the test statistic. Assume the mean quantity per jar is found to be 351.4g.

$$\bar{x} = 351.4$$

Step 4. Calculate the p -value and reject H_0 if $p < \alpha/2$ (why do we divide by 2?).

```
[39]: # Properties of the sample:
n = 40          # Sample size
mu = 351.2      # (Hypothetical) population mean
sigma = 4.1     # Population standard deviation (assumed to be known)
m_sample = 351.4 # Sample mean
alpha = 0.05    # Significance level (chosen by the researcher)
```

```
[40]: p = stats.norm.sf(m_sample, loc=mu, scale=sigma/np.sqrt(n))
print("p-waarde: %.5f" % p)
if(p < alpha/2):
    print("p < alpha/2, reject H0")
else:
    print("p > alpha/2, do not reject H0")
```

p-waarde: 0.37885

p > alpha/2, do not reject H0

In this case, we have two critical values: g_1 on the left of the mean and g_2 on the right. The acceptance region still has area $1 - \alpha$ and the critical region has area α .

```
[41]: g1 = mu - stats.norm.isf(alpha/2) * sigma / np.sqrt(n)
g2 = mu + stats.norm.isf(alpha/2) * sigma / np.sqrt(n)

print("Acceptance region [g1, g2]   [%.3f, %.3f]" % (g1,g2))
if (g1 < m_sample and m_sample < g2):
    print("Sample mean = %.3f is inside acceptance region: do not reject H0" % \
        ↪m_sample)
else:
    print("Sample mean = %.3f is outside acceptance region: reject H0" % \
        ↪m_sample)
```

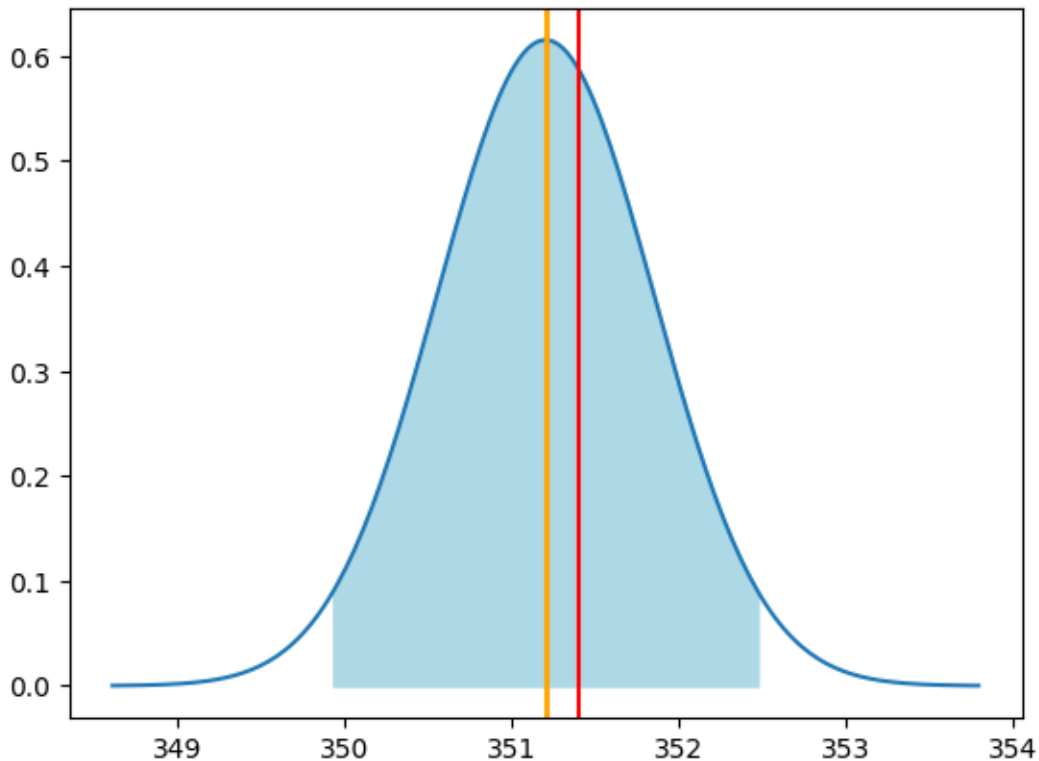
Acceptance region [g1, g2] [349.929, 352.471]

Sample mean = 351.400 is inside acceptance region: do not reject H0

A plot of this case:

```
[42]: # Gauss-curve
# X-values
dist_x = np.linspace(mu - 4 * sigma/np.sqrt(n), mu + 4 * sigma/np.sqrt(n), \
    ↪num=201)
# Y-values
dist_y = stats.norm.pdf(dist_x, loc=mu, scale=sigma/np.sqrt(n))
# Plot
plt.plot(dist_x, dist_y)
# Hypothetical population mean in orange
plt.axvline(mu, color="orange", lw=2)
# Sample mean in red
```

```
plt.axvline(m_sample, color="red")
acc_x = np.linspace(g1, g2, num=101)
acc_y = stats.norm.pdf(acc_x, loc=mu, scale=sigma/np.sqrt(n))
# Fill the acceptance region in light blue
plt.fill_between(acc_x, 0, acc_y, color='lightblue');
```



Step 5 Draw conclusions

The obtained sample mean turns out to be sufficiently probable. We cannot rule out a random sampling error. Or, in other words, we *cannot* reject the null hypothesis here. The sample does not provide sufficient evidence to conclude that the machine is not working correctly.

5.5 Student's t -test

The z -test can only be used under specific conditions:

- The sample needs to be random
- The sample size needs to be sufficiently large ($n \geq 30$)
- The test statistic needs to have a normal distribution
- The standard deviation of the population, σ , is known

Sometimes these assumptions will not hold and in this case we can not use the z -test! If the variable

The test was named after [William Sealy Gosset](#), who published results of his research under the pseudonym “Student”. Gosset worked for the Guinness Brewery in Dublin, Ireland, and was interested in the problems of small samples - for example, the chemical properties of barley. He developed the t -test to work with small samples. Guinness did not want to reveal its trade secrets, so Gosset had to publish his work under a pseudonym.

In one course, a new textbook was used this school year. The old textbook received an average score of 5.2 out of 10. To know if the new textbook is more appreciated than the old one, the opinion of the 41 students who are redoing the course this school year is asked. Below are the scores of these 41 students:

Can we say that the score of the new textbook is significantly higher?

Step 1. Formulate the hypotheses:

- Step 2.** Choose significance level, e.g. $\alpha = 0.05$

Sample mean: 6.146341463414634
Standard error: 0.29747099799147486

96


```
else:
    print("p > alpha: do not reject H0")
```

p-value: 0.0014163864505844037

p < alpha: reject H0

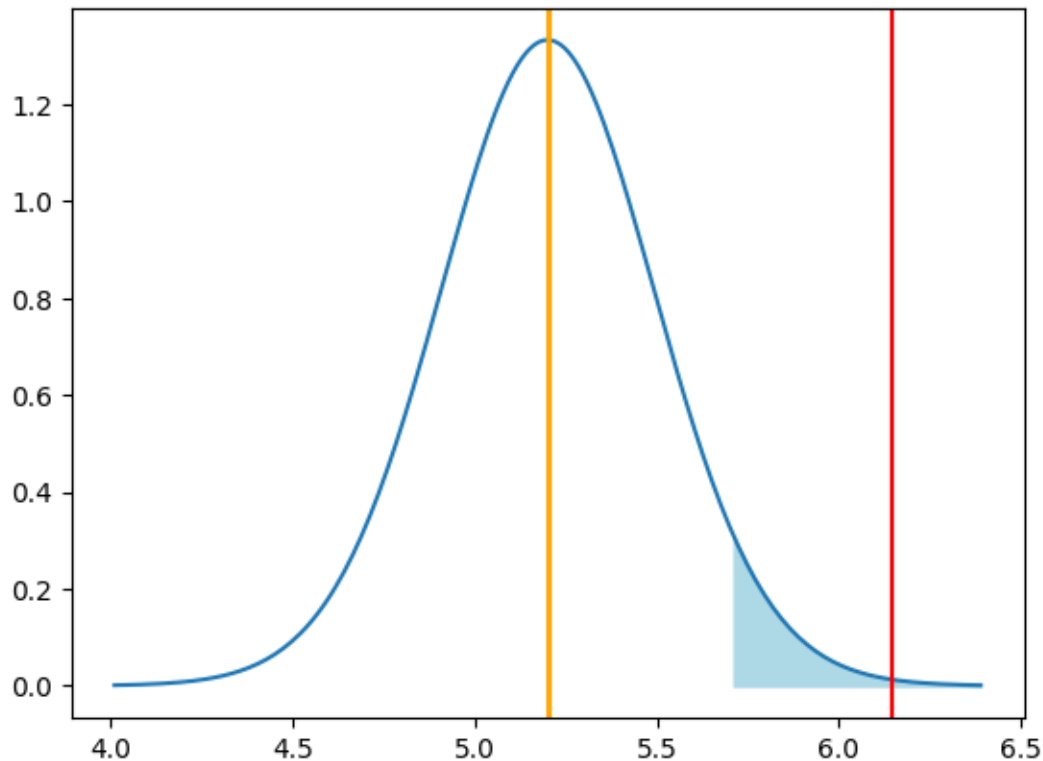
Or calculate the critical value g and do not reject H_0 if $\bar{x} > g$.

```
[27]: g = stats.t.isf(alpha, loc = mu, scale = stderr, df = n - 1)
print("Critical value g   %.3f" % g)
if (m_sample < g):
    print("sample mean = %.3f < g = %.3f: do not reject H0" % (m_sample, g))
else:
    print("sample mean = %.3f > g = %.3f: reject H0" % (m_sample, g))
```

Critical value g 5.701

sample mean = 6.146 > g = 5.701: reject H0

```
[28]: # Gauss-curve
# X-values
dist_x = np.linspace(mu - 4 * stderr, mu + 4 * stderr, num=201)
# Y-values
dist_y = stats.t.pdf(dist_x, loc=mu, scale=stderr, df=n-1)
# Plot
plt.plot(dist_x, dist_y)
# Hypothetical population mean in orange
plt.axvline(mu, color="orange", lw=2)
# Sample mean in red
plt.axvline(m_sample, color="red")
# Fill the acceptance region in light blue
plt.fill_between(dist_x, 0, dist_y, where=dist_x >= g, color='lightblue');
```



Step 5 Draw conclusions.

There is sufficient evidence to reject H_0 . Thus, there is sufficient evidence to conclude that the new textbook is more appreciated by the students.

5.5.2 Use of the function `ttest_1samp`

SciPy provides a function `ttest_1samp()` that allows you to perform a *two-tailed* t-test on a collection of observations. It is used to compare the mean of a single sample to a given population mean. It helps determine whether the sample mean is significantly different from the population mean.

When to use `ttest_1samp`? - You have one sample of data. - You want to test if its mean significantly differs from a known or hypothesized population mean. - The data is assumed to be normally distributed (or approximately normal for large samples). - The standard deviation of the population is not known

5.5.3 Example

A university claims that students spend an average of 20 hours per week studying. A researcher collects data from 15 students to test whether their average study time is different from 20 hours.

Step 1. Formulate the hypotheses:

- $H_0 : \mu = 20$ (The average study time is 20 hours.)

- $H_1 : \mu \neq 20$ (The average study time is not 20 hours.)

Step 2. Choose significance level, e.g. $\alpha = 0.05$

Step 3. Calculate the test statistic $\bar{x} = 19.93$

Step 4. Determine the p -value and do not reject H_0 if $p > \alpha$.

```
[43]: study_times = [18,22,19,21,23,20,17,22,19,18,21,20,16,24,19]
alpha = 0.05
mu = 20

t_stat, p_val = stats.ttest_1samp(a=study_times, popmean=mu, alternative='less')
print("Sample mean      : %.3f" % np.mean(study_times))
print("t-score          : %.3f" % t_stat)
print(f"p-value         : {p_val}")
```

```
Sample mean      : 19.933
t-score          : -0.115
p-value          : 0.4551527949196003
```

Step 5 Draw conclusions.

There is insufficient evidence to reject H_0 . Thus, there is no reason to believe that the average study time is significantly different from 20.

5.6 When to use the z -test and t -test?

There is no similar function as `ttest_1samp()` for the z -test. In practice, the z -test is rarely used because the conditions for its use are rarely met. In particular, the assumption that the population variance is known is not realistic. The t -test is more versatile and can be used in a wider range of situations.

As a conclusion:

When to use the z -test?

- in case of a large sample (no matter which distribution) or a small sample if the underlying distribution is the normal distribution

and

- if the variance or standard deviation of the population is known

When to use the t -test?

- if the underlying distribution is the normal distribution

and

- if the variance or standard deviation of the population is unknown

5.7 Errors in Hypothesis Tests

- **Type I error** (false positive)

The null hypothesis is actually true and you reject it.

Example: A COVID test wrongly indicates a person is infected when they are actually healthy.

- **Type II error** (false negative)

You accept the null hypothesis and it was not true.

Example: A COVID test wrongly indicates a person is healthy when they are actually infected.

- $P(\text{type I error}) = \alpha$ (= significance level)
- $P(\text{type II error}) = \beta$

Calculating β is not feasible (as you would need to know the true population parameter), but when you lower the value of α then β will increase, and vice versa.

This is because when you lower α , you are making it harder to reject the null hypothesis, which increases the chances of failing to reject a false null hypothesis (type II error). Conversely, when you increase α , you are making it easier to reject the null hypothesis, which decreases the chances of failing to reject a false null hypothesis.

5.8 Summary

5.8.1 z -test for population mean

Prerequisites:

- The samples are independent and randomly selected.
- The distribution of the population is normal, or the sample size is large enough ($n > 30$).
- The population standard deviation (σ) is known.

As *test statistic*, the sample mean ($\bar{x}$), (or the z -score of the sample mean) is used.

Formulation of the null and alternative hypotheses:

	Left-tailed	Right-tailed	Two-tailed
H_0	$\mu \geq \mu_0$	$\mu \leq \mu_0$	$\mu = \mu_0$
H_1	$\mu < \mu_0$	$\mu > \mu_0$	$\mu \neq \mu_0$

5.8.2 t -test for population mean

Prerequisites:

- The samples are independent and randomly selected.
- The underlying population is normal.
- The population standard deviation (σ) is unknown.

As *test statistic*, the sample mean ($\bar{x}$), (or the t -score of the sample mean) is used.

The formulation of the null and alternative hypotheses is the same as in the z -test.

6 Module 4: Bivariate analysis - 2 qualitative variables

In this module and the two next ones, we will be exploring the *relationship between two variables*. Just like we saw in module 2 (analysis of one variable), the measurement level of the variables we are analyzing will determine the techniques we will use to explore their relationship. In this module, we will be looking at the relationship between two *qualitative* variables.

To demonstrate the concepts in this module, we import dataset `rlanders.csv`. It contains synthetic data that was generated randomly (see [the description](#)).

6.1 Learning goals

By the end of this module, you should be able to:

- Understand the definitions and terminology (indicated with $\bowtie$) used in this notebook
- Select suitable techniques to analyze or visualize the relationship between two variables, depending on their measurement level
- Create and interpret a *contingency table* of two qualitative variables
- Create and interpret a *plot* of two qualitative variables (e.g. a stacked or grouped bar chart, or a mosaic plot)
- Calculate and interpret *Cramér's V* to measure the strength of association between two qualitative variables
- Perform a *chi-squared test of independence* to determine if there is a significant association between two qualitative variables
- Perform a *chi-squared goodness-of-fit test* to determine if the distribution of a qualitative variable differs from a specified distribution

6.2 Bivariate analysis: introduction

In *bivariate analysis*, we want to determine whether there is an $\bowtie$ **association** (also: **relationship**) between two stochastic variables X and Y . We say that there is an association between X and Y if the value of one variable provides information about the value of the other variable. For example, if we know that a person is a smoker, this might provide information about their risk of developing lung cancer. In contrast, if there is no association between X and Y , then knowing the value of one variable does not provide any information about the value of the other variable.

If we use the value of X to predict the value of Y , then we say that X is the $\bowtie$ **independent variable** and that Y is the $\bowtie$ **dependent variable**.

Note that the choice of which variable is the (in)dependent variable is not always clear-cut, and it can depend on the context of the analysis. In some cases, there may be a bidirectional relationship between the two variables, where both variables can influence each other.

Also note that finding an association between two variables does not necessarily imply that there is a causal relationship between them ("*correlation is not causation*"). The direction of causation may not be clear from the data, or there may be another variable that is influencing both X and Y , creating the appearance of an association between them when in reality there is no direct relationship.

This course is *not* an exhaustive discussion on the subject of bivariate analysis. Specifically, we will only focus on the techniques summarized below.

Independent	Dependent	Test/Metric
Qualitative	Qualitative	χ^2 -test/Cramér's V
Qualitative	Quantitative	two-sample t -test/Cohen's d
Quantitative	Quantitative	-/Regression, correlation

For visualizing the relationship between two variables, we will use the following plot types:

Independent	Dependent	Plot
Qualitative	Qualitative	Grouped/stacked bar chart, mosaic plot
Qualitative	Quantitative	Grouped boxplot, bar chart with error bars
Quantitative	Quantitative	Scatter plot, regression line

```
[1]: # Importing the necessary packages
import numpy as np                    # "Scientific computing"
import scipy.stats as stats           # Statistical tests

import pandas as pd                  # Data Frame
from pandas.api.types import CategoricalDtype

import matplotlib.pyplot as plt      # Basic visualisation
from statsmodels.graphics.mosaicplot import mosaic # Mosaic diagram
import seaborn as sns                # Advanced data visualisation
↪visualisation
```

```
[2]: # Read dataset + data preparation
rlanders = pd.read_csv('https://raw.githubusercontent.com/HoGentTIN/
↪dsai-en-labs/main/data/rlanders.csv').set_index(['ID'])
rlanders.Gender = rlanders.Gender.astype('category')
likert_scale = CategoricalDtype(categories=[1,2,3,4,5], ordered=True)
rlanders.Survey = rlanders.Survey.astype(likert_scale)
# rlanders.info()
# rlanders.Survey.dtype
```

6.3 Contingency tables and visualisation techniques

Qualitative variables have a limited number of possible values (categories). We usually summarize the distribution of a qualitative variable using a *frequency table*. When we want to analyze the relationship between two qualitative variables, we can use a **contingency table** (also: *cross-tabulation* or **crosstab**). A contingency table is a tabular representation of the joint distribution of two categorical variables. It shows the frequency (or proportion) of observations for each combination of categories of the two variables.

Let's say we want to know whether women and men (variable **Gender**) within the sample have responded differently on a survey question with a Likert scale (variable **Survey**).

If the proportions of the answers (1 to 5) for women and men are about the same, we say that there is no association between the two variables. If there is a significant difference, then there is an association. Now, of course, the question is when is the difference large enough to be considered *significant*...

A first step is to calculate a contingency table. The *independent* variable **Gender** is used to create the columns (second parameter), and the *dependent* variable **Survey** is used to create the rows (first parameter). We immediately add the **marginal totals**, i.e., the sums of the rows and columns:

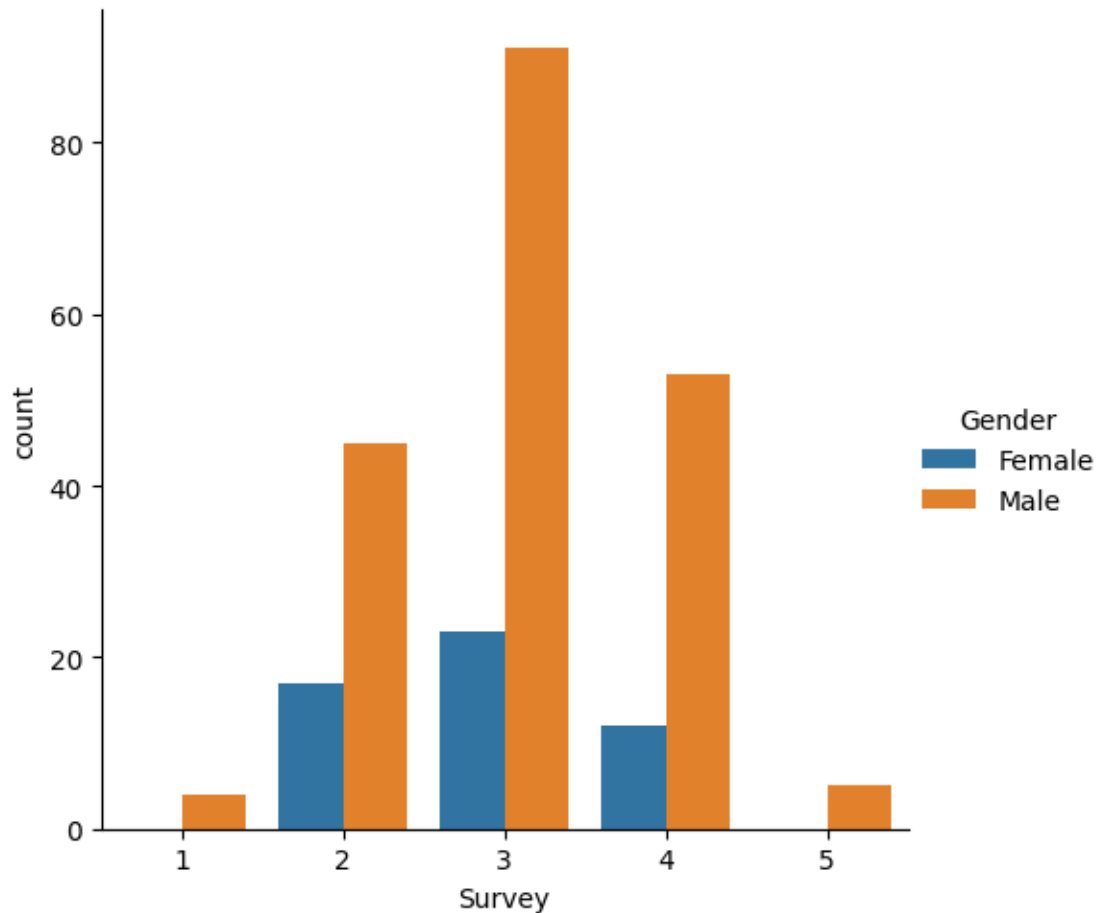
```
[3]: pd.crosstab(rlanders.Survey, rlanders.Gender, margins=True)
```

```
[3]: Gender  Female  Male  All
      Survey
      1          0     4    4
      2         17    45   62
      3         23    91  114
      4         12    53   65
      5          0     5    5
      All        52   198  250
```

6.3.1 Clustered bar chart

Of course, a table with numbers does not say much at first glance. Perhaps we should visualise the data, e.g. with a clustered bar chart.

```
[4]: sns.catplot(data=rlanders, x='Survey', hue='Gender', kind='count');
```



Remark that we mapped the *independent* variable **Gender** to the `hue` parameter and the *dependent* variable **Survey** to the `x` parameter.

If the “shape” of the bars for both categories is similar, we can conclude that both categories have a similar distribution, or that there is no association between the two variables. If the shape were different (e.g. highest peak for men at 2 and for women at 4), that would indicate that there actually is an association, or that men and women give significantly different answers to the survey question.

In this case, what is striking is that the women did not give the extreme answers (1 or 5). However, there are considerably fewer women in the sample, so that may be just a coincidence, not necessarily a significant pattern.

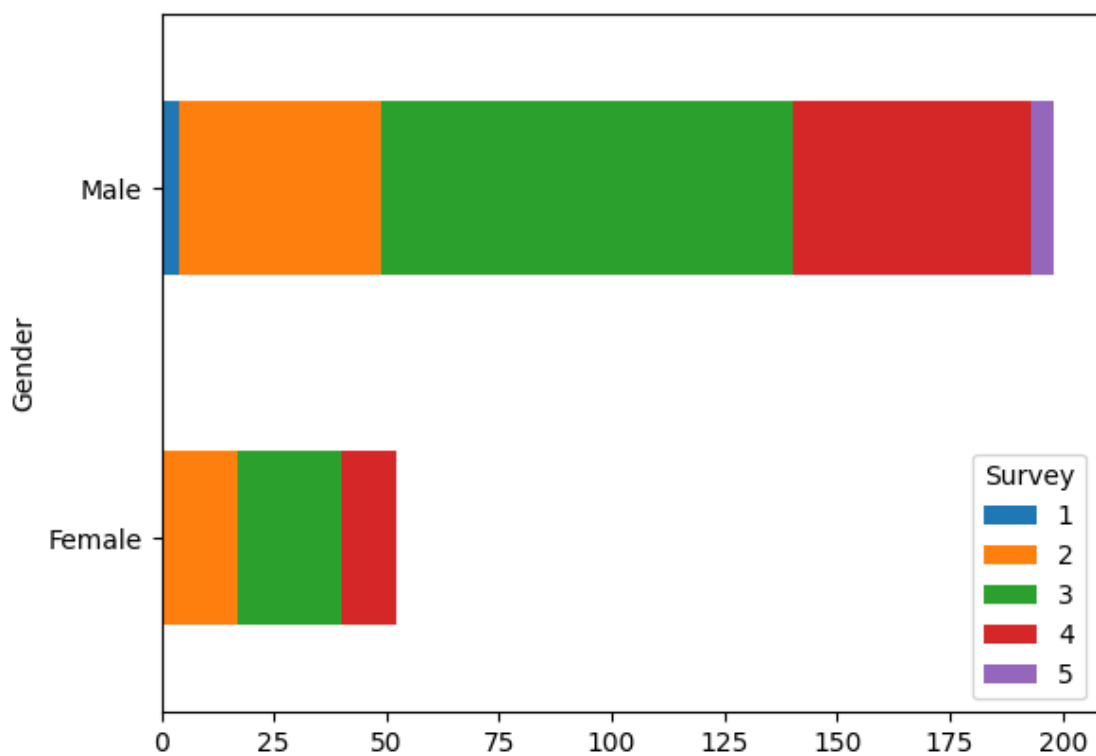
6.3.2 Stacked bar chart

Another way to visualize the data is by stacking the bars instead of clustering them. Unfortunately, Seaborn doesn’t provide us with a convenient method to do so. However, a crosstab object (actually a Pandas DataFrame) has a plot method that we can use. Remark that we swapped the variables when calling the crosstab function, so the *independent* variable **Gender** is now the row and the *dependent* variable **Survey** is the column.


```
[5]: pd.crosstab(rlanders.Gender, rlanders.Survey)
```

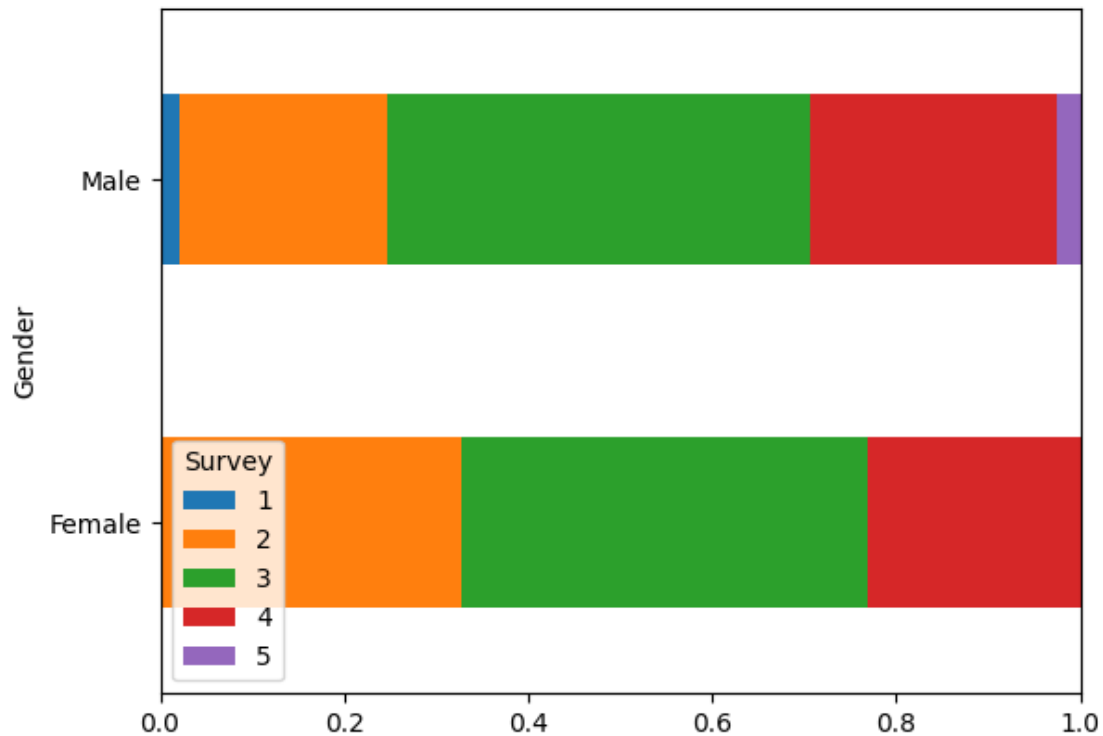
```
[5]: Survey  1   2   3   4   5  
Gender  
Female  0  17  23  12   0  
Male    4  45  91  53   5
```

```
[6]: # Contingency table without the margins  
observed = pd.crosstab(rlanders.Gender, rlanders.Survey)  
  
# Horizontally oriented stacked bar chart  
observed.plot(kind='barh', stacked=True);
```



We want to compare the proportions within each category (Male, Female) that replied to the survey question. In data sets with large differences in the number of observations for each category, this representation is still difficult to interpret. By looking at the *relative* frequencies (i.e. percentages), we may get a better result:

```
[7]: # Contingency table with proportions (normalized by row)  
observed_p = pd.crosstab(rlanders.Gender, rlanders.Survey, normalize='index')  
  
# Horizontally oriented stacked bar chart  
observed_p.plot(kind='barh', stacked=True);
```



We can see that there are some differences between the proportions, e.g. the percentage of women answering “2” (orange) seems larger than the men. Whether this is a significant difference remains to be seen (see below for further analysis of this case).

In this representation we lose the information about the number of men and women in the sample. Specifically, we don’t see that there are much less women. A mosaic diagram solves this problem.

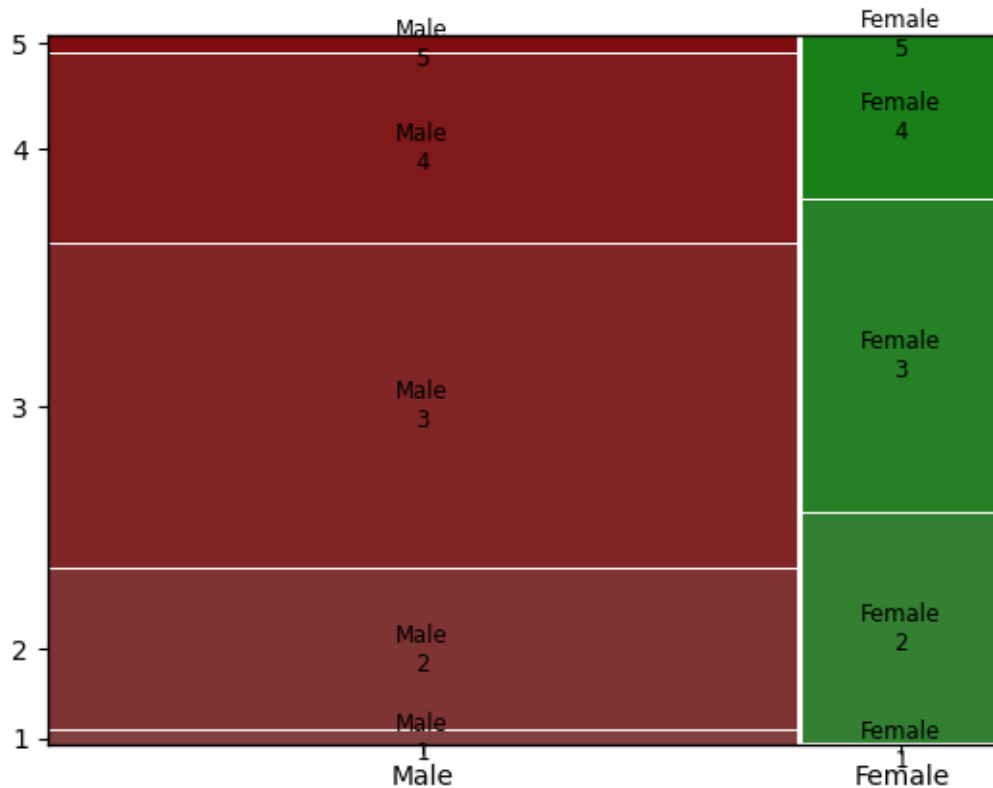
6.3.3 Mosaic diagram

A mosaic diagram is a graphical representation of the crosstab where each cell is represented by a tile whose area is proportional to the frequency of that cell with respect to the total number of observations.

You can compare this a bit with how [WinDirStat](#) visualises the size of files on your hard disk.

For this case:

```
[8]: # Mosaic diagram with default settings
mosaic(
  data=rlanders.sort_values(by=['Survey']),
  index=['Gender', 'Survey']);
```



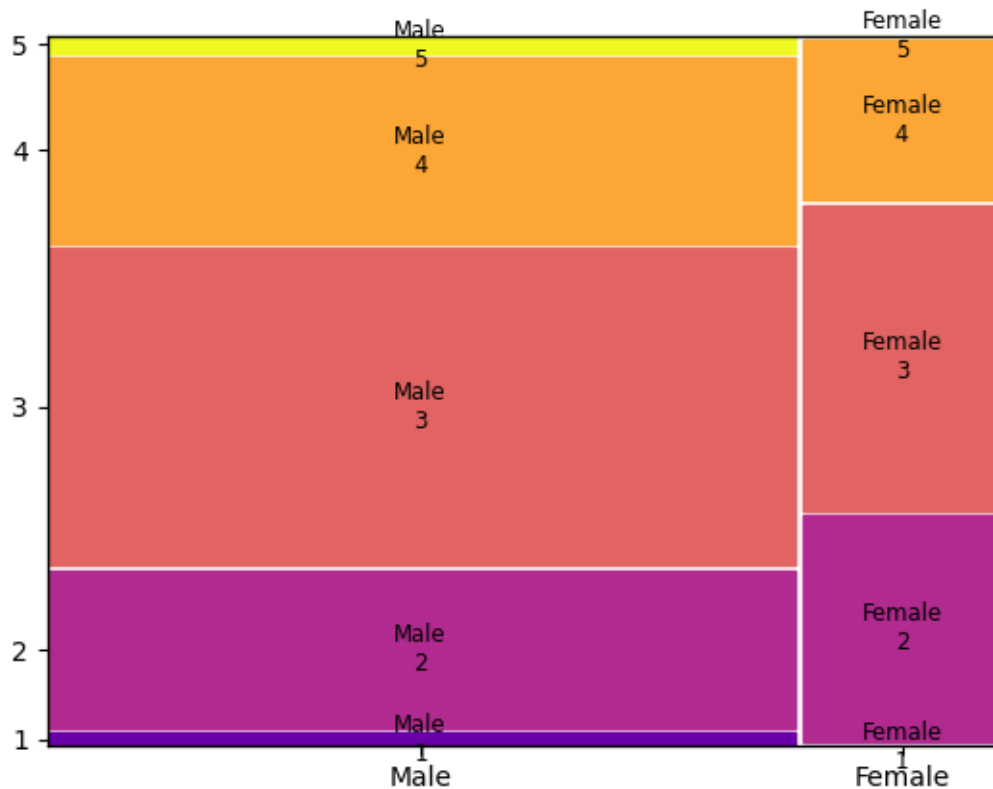
The colors in the mosaic diagram are not very informative, so we give another example where the colors are based on the `Survey` values instead. Unfortunately, this involves creating a function that maps the `Survey` values to colors, which is a bit more work. The result is shown below:

```
[9]: import matplotlib as mpl

# By default, mosaic() will color all columns the same. Here,
# we're going to give a separate color to each response (1-5).
# The response is kept in the label of each cell, a tuple of
# the two values that are represented in that cell (e.g.
# ('Female', '2')). We will be using the "plasma" color map
# in this example.
plasma_colors = mpl.colormaps['plasma']
# Create a function that maps the cell key to the color we
# want to give it. We use the colormap function created above
# and give it a number between 0 and 1. We convert the second
# part of the key (values '1' to '5') to a number and divide
# by 5.
props = lambda key: {'color': plasma_colors(int(key[1])/5)}

mos_gender_survey = mosaic(
```

```
data=rlanders.sort_values(by=['Survey'],
index=['Gender', 'Survey'],
gap=0.01, properties=props);
```



We can clearly see here that there are more men in the sample (wider tiles).

You can also compare the distributions of the answers given to the survey question. Except for the fact that the women did not give extreme answers (1 and 5), at first sight, the distributions still seem to match quite well.

6.4 Chi-squared and Cramér's V

Chi-squared (χ^2) and **Cramér's V** are statistics that can help us to determine whether there is an association between two qualitative (categorical) variables.

The reasoning goes as follows: if there is *no* association between **Gender** and **Survey**, then we expect the proportions of the values of **Survey** to be the same for all values of **Gender**. In other words, for both women and men, the same percentage of respondents will give the same answer to the question.

```
[10]: # As a reminder, here is the contingency table with marginal totals again:
pd.crosstab(rlanders.Survey, rlanders.Gender, margins=True)
```

```
[10]: Gender  Female  Male  All
      Survey
      1         0     4    4
      2        17    45   62
      3        23    91  114
      4        12    53   65
      5         0     5    5
      All        52   198  250
```

```
[11]: observed = pd.crosstab(rlanders.Survey, rlanders.Gender)
      row_sums = observed.sum(axis=1)
      col_sums = observed.sum()
      n = row_sums.sum()
```

Now we can calculate how often we can expect each combination of **Gender** and **Survey** to occur in the sample. For example, 114 out of 250 respondents answered “3”, i.e. $114/250 = 0.456$ (or 46.6%). Then we expect that both 46.6% of the women and 46.6% of the men would have answered “3”. Since there were 52 female respondents, we expect that $52 \times 0.456 = 23.712$ have answered “3”. In reality, there were 23. For the men, we expect $198 \times 0.456 = 90.288$ (compared to 91 in reality).

Applying the same principle to all cells in the table, we can calculate the expected frequencies using the formula: $\frac{\text{rowtotal} \times \text{columntotal}}{n}$ (with n the total number of observations).

The **numpy** function `outer()` can be leveraged to calculate the expected frequencies for all cells in the table at once like this:

```
[12]: expected = np.outer(row_sums, col_sums) / n
      expected
```

```
[12]: array([[ 0.832,  3.168],
             [12.896, 49.104],
             [23.712, 90.288],
             [13.52 , 51.48 ],
             [ 1.04 ,  3.96 ]])
```

In this table, the marginal totals should still sum to the same values as before. Let’s check if that is the case:

```
[13]: exp_row_sums = np.sum(expected, axis=1)
      exp_col_sums = np.sum(expected, axis=0)

      print(f'Row totals    : {exp_row_sums}')
      print(f'Column totals: {exp_col_sums}')
      print(f'Observations : {exp_col_sums.sum()}')
```

```
Row totals    : [ 4.  62. 114.  65.   5.]
Column totals: [ 52. 198.]
Observations : 250.0
```

Indeed, they do. Now, what’s the difference between the expected and observed values?

```
[14]: expected - observed
```

```
[14]: Gender  Female    Male
Survey
1         0.832 -0.832
2        -4.104  4.104
3         0.712 -0.712
4         1.520 -1.520
5         1.040 -1.040
```

Some observed values seem to be close to the expected ones (e.g. for “3”), others are further away (e.g. “2”). However, we must also see the differences in proportion. So just calculating the differences between observed and expected values is not very informative.

A measure for determining the overall deviation in a frequency table from the expected values is to square the differences between expected and observed values (just as one does with variance/standard deviation) and divide by the expected value:

```
[15]: diffs = (expected - observed)**2 / expected
print(diffs)
```

```
Gender    Female      Male
Survey
1         0.832000  0.218505
2         1.306050  0.343003
3         0.021379  0.005615
4         0.170888  0.044880
5         1.040000  0.273131
```

The sum of all these values is notated $\propto \chi^2$ (“**chi-squared**”, using the Greek letter *chi*):

$$\chi^2 = \sum_i \frac{(o_i - e_i)^2}{e_i}$$

with o_i the number of observations in the i 'th cell of the contingency table and e_i the expected frequency. Compare the formula with the code above to find similarities!

```
[16]: chi_squared = diffs.values.sum()
print(' ²   %.3f' %chi_squared)
```

```
²   4.255
```

The closer χ^2 is to 0, the closer the observed values are to the expected values, and thus the weaker the association between the two variables. The larger χ^2 , the stronger the association.

Now, this value in itself *still* does not say very much. Under what conditions do we say that there is or isn't an association between the two variables? That will also depend on the size of the table and the total number of observations. In a contingency table with more rows/columns, you will need a larger χ^2 before you can conclude that there is an association.

$\propto$ **Cramér's V** is a formula that normalises χ^2 to a value between 0 and 1 that is independent of the table size.

$$V = \sqrt{\frac{\chi^2}{n(k-1)}}$$

with n the total number of observations and k the number of rows or columns (whichever is smaller).

```
[17]: cramers_v = np.sqrt(chi_squared / ((min(observed.shape) - 1) * n))
      cramers_v
```

```
[17]: np.float64(0.13046762087242633)
```

The value of Cramér's V can also be calculated with the following Python function (from the `scipy.stats` module):

```
[18]: stats.contingency.association(observed, method='cramer')
```

```
[18]: 0.13046762087242636
```

To draw a conclusion from this figure, compare it with the values in the table below:

Cramér's V	Interpretation
0	No association
0.1	Weak association
0.25	Moderate association
0.50	Strong association
0.75	Very strong association
1	Complete association

So our result for Cramér's V indicates a fairly weak association. Note that in certain cases, Cramér's V is known to be too optimistic about the association between two variables. There is a modified formula for eliminating this bias, but it is beyond the scope of this course.

There is, however, another way to determine whether the chi-square is large enough to conclude that there is a relationship, namely by means of a statistical test.

6.5 The chi-squared test for independence

To answer the question of when the value of chi-square is sufficient to assume an association between two variables, we can use the *chi-square independence test*.

6.5.1 The chi-squared distribution

The value of χ^2 follows a specific stochastic distribution that is called the χ^2 -**distribution**. Just like with the Student-t distribution, the shape of the probability density function depends on the degrees of freedom, which in this context is $df = (r - 1) \times (k - 1)$ (with r the number of rows in the contingency table and k the number of columns).

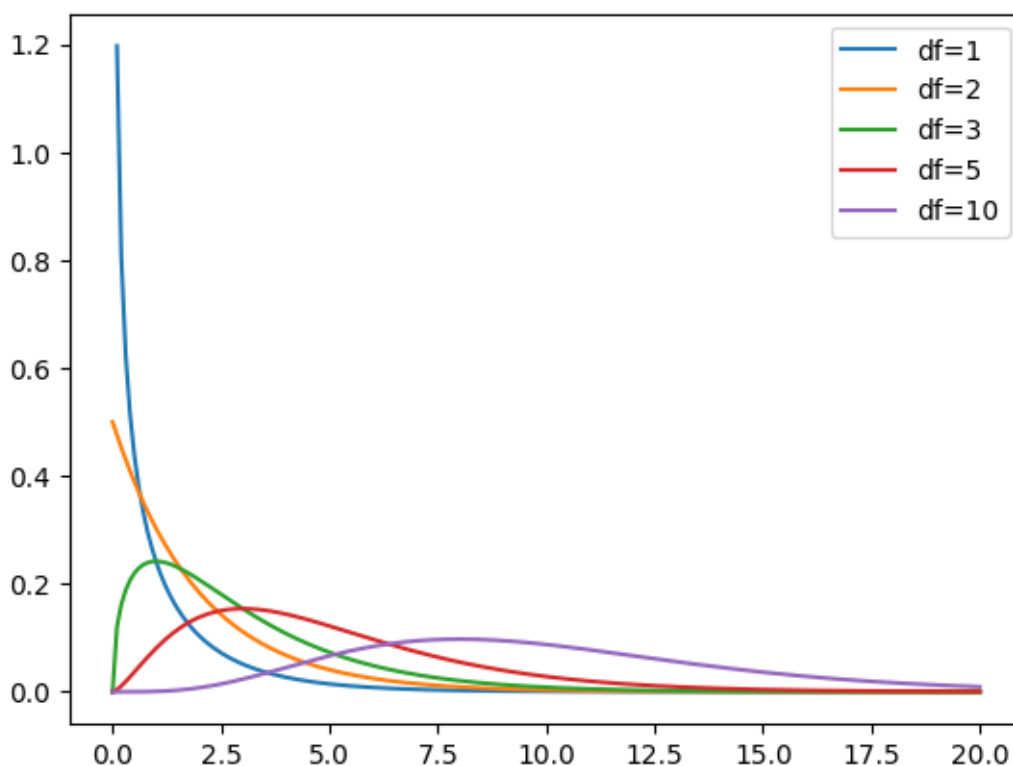
Some useful Python functions:

Function	Description
<code>stats.chi2.pdf(x, df)</code>	Probability density function
<code>stats.chi2.cdf(x, df)</code>	Left-tail probability $P(X < x)$
<code>stats.chi2.sf(x, df)</code>	Right-tail probability $P(X > x)$

Function	Description
<code>stats.chi2.isf(q, df)</code>	q percent of observations exceed this value

The code below plots the probability density function of the χ^2 -distribution for some values of degrees of freedom:

```
[19]: # Plot of the chi-squared distribution for different degrees of freedom
x = np.linspace(0, 20, num=200)
plt.plot(x, stats.chi2.pdf(x, 1), label="df=1");
plt.plot(x, stats.chi2.pdf(x, 2), label="df=2");
plt.plot(x, stats.chi2.pdf(x, 3), label="df=3");
plt.plot(x, stats.chi2.pdf(x, 5), label="df=5");
plt.plot(x, stats.chi2.pdf(x, 10), label="df=10");
plt.legend(loc='best');
```



Using this figure, you can follow a reasoning similar to a right-sided z-test. You can calculate a **critical value** g for which the area under the curve to the right of g is equal to our chosen significance level α . If the **test statistic** χ^2 is greater than g , we say that the deviations from the expected values in the contingency table are too large to be coincidental and that we have reason to believe that there is an association between the two variables.

You can also calculate the **excess probability** p , i.e. the probability that the deviations you see

in the sample could be due to random sampling errors. To do this, you calculate the area under the density curve to the right of p . If this area is smaller than α , we also say that the deviations are too large to be caused by random sampling errors and that there is therefore an association between the variables.

In Python, we can use the following Scipy functions for this purpose:

- `stats.chi2.sf(x)` - the right tail probability of x (or survival function)
- `stats.chi2.isf(q)` - the inverse function of `sf(x)`, i.e. calculate a number x for which the right tail probability is exactly q .

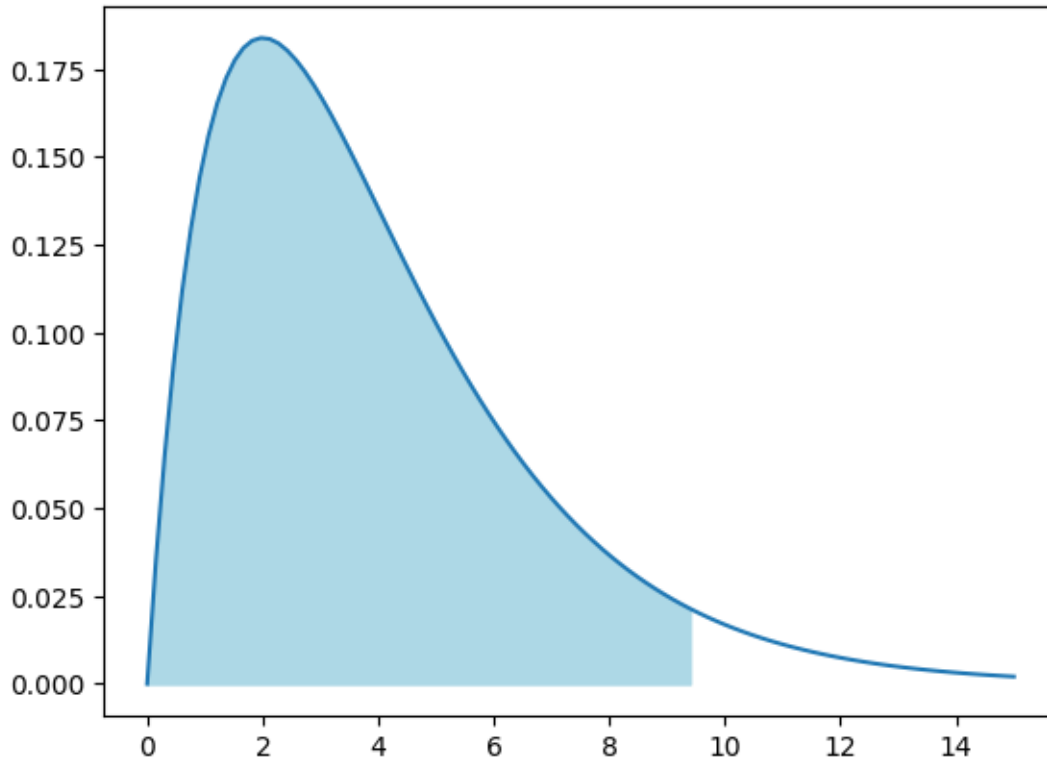
A plot for the case where $\alpha = 0.05$ (an often chosen value for the significance level) and 4 degrees of freedom:

```
[20]: # Right tail probability in the chi-squared distribution

# x-values:
x = np.linspace(0, 15, num=100)
# probability density of the chi-squared distribution with 4 degrees of freedom
y = stats.chi2.pdf(x, df=4)
# the critical value g for which the right tail probability is exactly 5%:
g = stats.chi2.isf(.05, df=4)
print(f'Critical value   {g:.3f}');

plt.plot(x, y)                                # probability density
plt.fill_between(x, y, where=x<=g,           # region of acceptance
                 color='lightblue');
```

Critical value 9.488



6.5.2 Test procedure

Formally, the procedure of the χ^2 **test for independence** runs as follows:

1. Formulate the hypotheses:
 - H_0 : There is no association between the variables (the differences between observed and expected values are small)
 - H_1 : There is an association between the variables (the differences are large)
2. Choose significance level α
3. Calculate the value of the test statistic in the sample (here: χ^2).
4. Use one of the following methods (based on the degrees of freedom $df = (r - 1) \times (k - 1)$):
 1. Determine critical value g so $P(\chi^2 > g) = \alpha$
 2. Calculate the p -value
5. Draw a conclusion based on the outcome:
 1. $\chi^2 < g$: do not reject H_0 ; $\chi^2 > g$: reject H_0
 2. $p > \alpha$: do not reject H_0 ; $p < \alpha$: reject H_0

In Python, we can calculate this as follows:

```
[21]: alpha = .05
      dimensions = observed.shape
      dof = (dimensions[0]-1) * (dimensions[1]-1)
```

```

print("Chi-squared      : %.4f" % chi_squared)
print("Degrees of freedom : %d" % dof)

# Calculate critical value
g = stats.chi2.isf(alpha, df = dof)
print("Critical value    : %.4f" % g)

# Calculate p-value
p = stats.chi2.sf(chi_squared, df=dof)
print("p-value          : %.4f" % p)

```

```

Chi-squared      : 4.2555
Degrees of freedom : 4
Critical value    : 9.4877
p-value          : 0.3725

```

Actually, we do not need to memorise these formulas to determine chi-square, the critical value and the p-value. SciPy already has a function that allows us to calculate the chi-square and p-value directly from a contingency table:

```

[22]: # Chi-squared test for independence based on a contingency table
observed = pd.crosstab(rlanders.Survey, rlanders.Gender)
chi2, p, df, expected = stats.chi2_contingency(observed)

print("Chi-squared      : %.4f" % chi2)
print("Degrees of freedom: %d" % df)
print("P-value          : %.4f" % p)

```

```

Chi-squared      : 4.2555
Degrees of freedom: 4
P-value          : 0.3725

```

As you can see, the outcome for the p -value is the same as in our worked out calculations!

A graphical representation of our case:

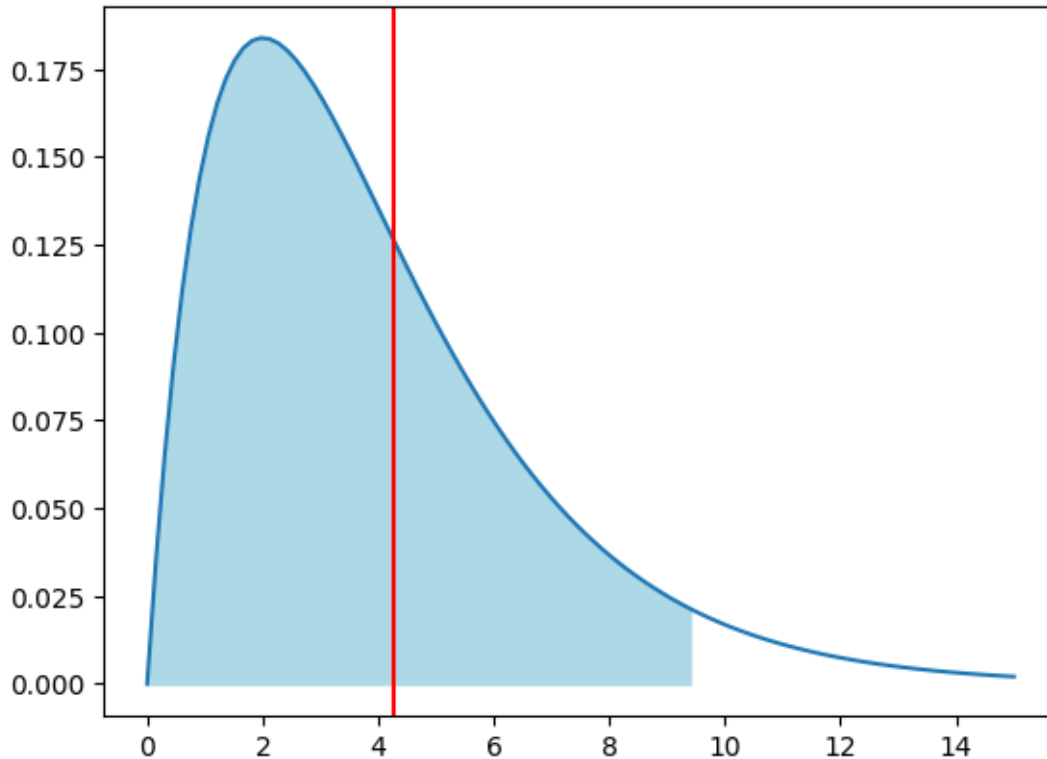
```

[23]: # Is there an association between Gender and Survey?

# x-values:
x = np.linspace(0, 15, num=100)
# probability density of the chi-squared distribution with 4 degrees of freedom
y = stats.chi2.pdf(x, df=dof)
# the critical value g for which the right tail probability is exactly 5%:
g = stats.chi2.isf(alpha, df=dof)

plt.plot(x, y); # probability density
plt.fill_between(x, y, where=x<=g, # critical area
                color='lightblue');
plt.axvline(chi2, color='red'); # chi-squared

```



We see that χ^2 is well within the region of acceptance. The p -value is also greater than α . Consequently, we cannot reject the null hypothesis and conclude that, based on this sample, there is no reason to believe that there is a significant difference between women's and men's answers to the survey question.

6.6 Goodness-of-fit test

A variant of the chi-squared test is the **goodness-of-fit** test. This test is used to determine whether the distribution of a single categorical variable in a sample matches a specified distribution (e.g., a theoretical distribution or the distribution in the population). Consequently, this test can be used to check if a sample of data is representative of the population from which it was drawn, at least with respect to the distribution of a particular categorical variable.

Suppose that in a sample consisting of respondents to a survey, we keep track of what age group they belong to and that we also know how often each age group occurs in the entire population (as a percentage). Given below are the absolute frequencies in the sample o_i and the expected relative frequencies π_i in the population:

```
[24]: # Age groups=      18-25  26-35  36-45  46-55  56+
      observed = np.array([ 75,   98,  127,   73,  27])
      expected_p = np.array([ .17,  .23,  .35,  .17, .08])
```

The question now is: is this sample representative of the population? Does each age group occur in the sample in proportion to the expected percentage in the population as a whole?

6.6.1 Test procedure

To answer these kinds of questions, a goodness-of-fit test is suitable. The procedure is as follows:

1. Formulate the hypotheses:
 - H_0 : The sample is representative of the population, i.e. the frequency of each class within the sample corresponds well to that in the population.
 - H_1 : The sample is *not* representative of the population, i.e. the differences with the expected frequencies are too large.
2. Choose significance level α
3. Calculate the value of the test statistic in the sample (here: χ^2).
4. Use one of the following methods (based on the degrees of freedom $df = (k - 1)$ with k the number of categories in the sample):
 1. Determine critical value g so $P(\chi^2 > g) = \alpha$
 2. Calculate the p -value
5. Draw a conclusion based on the outcome:
 1. $\chi^2 < g$: do not reject H_0 ; $\chi^2 > g$: reject H_0
 2. $p > \alpha$: do not reject H_0 ; $p < \alpha$: reject H_0

A plot of this case, including the calculation of p and g :

```
[25]: alpha = 0.05           # Significance level
n = sum(observed)           # Sample size
k = len(observed)           # Number of categories
dof = k - 1                 # Degrees of freedom
expected = expected_p * n   # Expected absolute frequencies in the sample
g = stats.chi2.isf(alpha, df=dof) # Critical value

# Goodness-of-fit-test in Python:
chi2, p = stats.chisquare(f_obs=observed, f_exp=expected)

print("Significance level    = %.2f" % alpha)
print("Sample size          n = %d" % n)
print("k = %d; df = %d" % (k, dof))
print("Chi-squared          ^2 = %.4f" % chi2)
print("Critical value       g = %.4f" % g)
print("p-value              p = %.4f" % p)
```

```
Significance level    = 0.05
Sample size          n = 400
k = 5; df = 4
Chi-squared          ^2 = 3.4679
Critical value       g = 9.4877
p-value              p = 0.4828
```

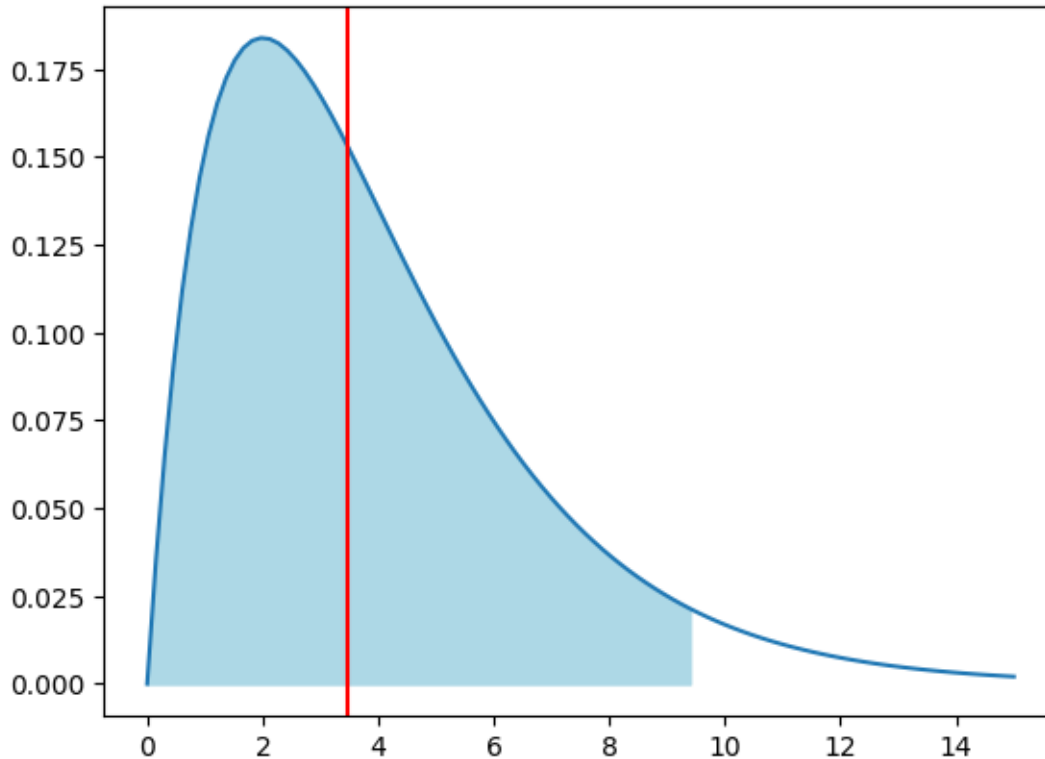
```
[26]: # Plot of the case:
# x-values:
x = np.linspace(0, 15, num=100)
# probability density of the chi-squared distribution with 4 degrees of freedom
```

```

y = stats.chi2.pdf(x, df=dof)
# the critical value g for which the right tail probability is exactly 5%:
g = stats.chi2.isf(alpha, df=dof)

plt.plot(x, y)                                # probability density
plt.fill_between(x, y, where=x<=g, # critical area
                 color='lightblue')
plt.axvline(chi2, color='red');               # chi-squared

```



We can see that χ^2 in the sample is left of the critical value, so within the area of acceptance. Therefore, we cannot reject the null hypothesis and can conclude that the sample is representative for the population, at least w.r.t. the age groups.

6.7 Standardised residuals

After performing a chi-squared test, it is often also interesting to know in which categories the largest deviations occur. You could look at the value $\frac{(o-e)^2}{e}$ for each cell in the contingency table (or vector), which is used in the calculation of χ^2 . However, from this you cannot tell to what extent the deviations can really be considered “extreme”. For this we use so-called **standardised residuals**.

We introduce this concept by means of an example:

Suppose that a certain study looks at families with 5 children. A sample of 1022 families is taken,

and they are divided into categories according to the number of boys in the family. The frequencies are given below in a Pandas DataFrame that we will create manually and systematically extend.

```
[27]: # Data frame with 2 columns:
# - number of boys in the family (index)
# - number of families in the sample with that number of boys
families = pd.DataFrame(
    np.array(
        [[0, 58],
         [1, 149],
         [2, 305],
         [3, 303],
         [4, 162],
         [5, 45]]),
    columns=['num_boys', 'observed'])
families.set_index(['num_boys'])
n = families.observed.sum() # sample size
```

Now what is the expected number of families with the given number of boys? Let's assume that the probability of conceiving boys or girls is equally large, i.e. 0.5. In that case, we can calculate the expected relative frequencies with this formula:

$$\pi_i = (0.5)^i (1 - 0.5)^{5-i} \binom{5}{i}$$

Why that is the case is not relevant for the subject at hand and is beyond the scope of this course.

The expected absolute frequencies in the sample can then be calculated as $e_i = n\pi_i$.

We apply this to our case:

```
[28]: from scipy.special import binom # binomial-function

# probability for a boy
prob_boy = .5
# Add new colum to the data frame for the expected percentages
families['expected_p'] = binom(5, families.num_boys) * prob_boy**families.
    ↪ num_boys * prob_boy**(5-families.num_boys)
# Expected absolute frequencies in the sample:
families['expected'] = families['expected_p'] * n
families
```

```
[28]:
```

	num_boys	observed	expected_p	expected
0	0	58	0.03125	31.9375
1	1	149	0.15625	159.6875
2	2	305	0.31250	319.3750
3	3	303	0.31250	319.3750
4	4	162	0.15625	159.6875
5	5	45	0.03125	31.9375

We perform the chi-squared goodness-of-fit test with significance level $\alpha = 0.01$.

```
[29]: alpha=0.01                                # significance level
      dof=len(families)-1                       # degrees of freedom
      g = stats.chi2.isf(alpha, df=dof)         # Critical value
      # Perform Chi-squared test, calculate  $\chi^2$  and p
      chi2, p = stats.chisquare(f_obs=families.observed, f_exp=families.expected)

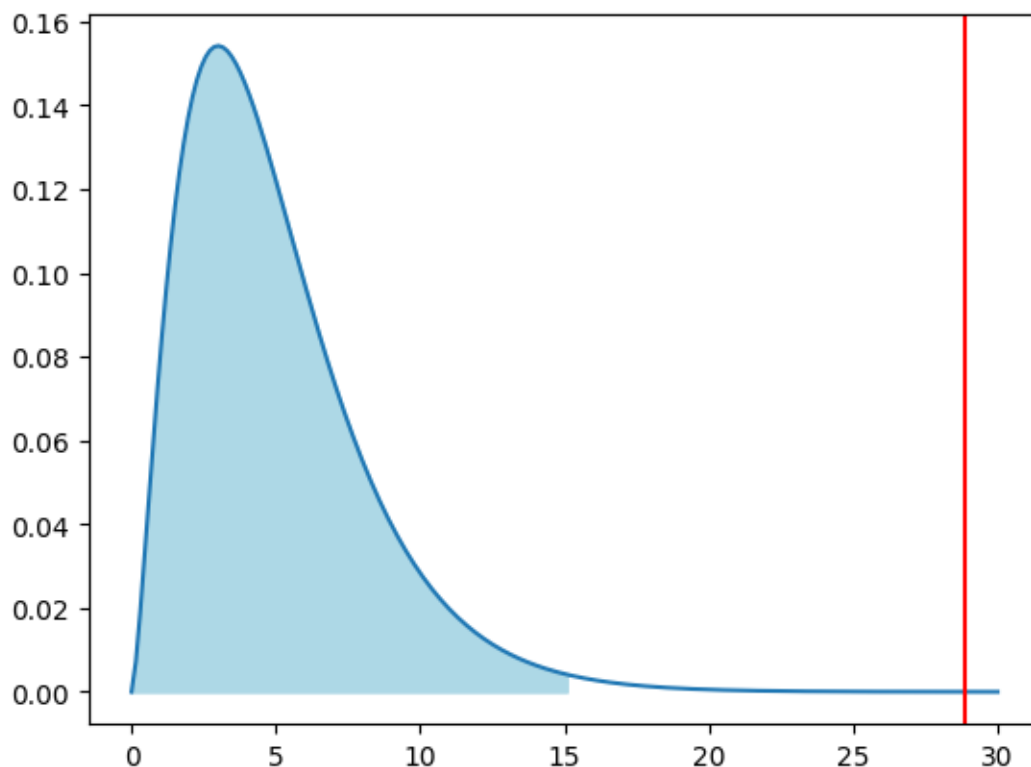
      print("Chi-squared  $\chi^2$  = %.4f" % chi2)
      print("Critical value g = %.4f" % g)
      print("p-value p = %f" % p)
```

```
Chi-squared  $\chi^2$  = 28.8462
Critical value g = 15.0863
p-value p = 0.000025
```

This is a very small p-value. The chi-squared is also to the right of the critical value. This means that we can reject the null hypothesis. A plot:

```
[30]: # x-values:
      x = np.linspace(0, 30, num=200)
      # probability density of the chi-squared distribution
      y = stats.chi2.pdf(x, df=dof)
      # the critical value g for which the right tail probability is exactly 1%:
      g = stats.chi2.isf(alpha, df=dof)

      plt.plot(x, y)                                # probability density
      plt.fill_between(x, y, where=x<=g, # critical area
                      color='lightblue')
      plt.axvline(chi2, color='red');               # chi-squared
```

We conclude that this sample is NOT representative for the population!

The formula for calculating **standardised residuals** is:

$$r_i = \frac{o_i - e_i}{\sqrt{e_i(1 - \pi_i)}}$$

We will add these to our data frame in a new column:

```
[31]: families['stdres'] = (families.observed - families.expected) / np.sqrt(families.
    ↪ expected * (1 - families.expected_p))
families
```

```
[31]:   num_boys  observed  expected_p  expected  stdres
0         0         58      0.03125    31.9375  4.685541
1         1        149      0.15625   159.6875 -0.920733
2         2       305      0.31250   319.3750 -0.970110
3         3       303      0.31250   319.3750 -1.105082
4         4       162      0.15625   159.6875  0.199223
5         5        45      0.03125    31.9375  2.348389
```

Standardised residuals are a measure of the extent to which a certain category is over- or under-represented in the sample. A value of 0 is given when the observed frequency is equal to the expected frequency. A negative value is given when there are fewer observations than expected and a positive value when there are more. As long as $r_i \in [-2, 2]$, we consider the differences to be

random sampling errors. A value $r_i < -2$ indicates underrepresentation of this category, $r_i > 2$ indicates overrepresentation.

Thus, families with only girls and only boys are over-represented in the sample. In practice, researchers could choose to randomly remove a number of observations in these categories from the study, so that the sample becomes representative of the population.

Remark that there is a way to calculate standardized residuals using `statsmodels.stats.contingency_tables.Table`. However, that only works with `contingency tables for two variables`, not for this case with one variable.

6.8 Cochran's rule

A chi-squared test can only give good results if you have enough observations in each category. The statistician Cochran (1954) formulated a rule of thumb to determine what exactly *enough* is on contingency tables larger than 2x2:

1. All expected values must be at least 1
2. At most 20% of the expected values may be smaller than 5

Consequently, if a contingency table does *not* meet these criteria, the results of the chi-squared test may not be reliable!

7 Module 5 - Bivariate analysis: qualitative vs quantitative

In this module, we discuss the case where the independent variable is qualitative and the dependent is quantitative. Some typical examples of research question in this case:

- Within a certain species of animals, are male individuals significantly larger than females?
- Is there a gender gap in salaries?
- Does a new vaccine protect against the disease like it's supposed to?
- Does a certain study method like "retrieval practice" actually improve learning outcomes (i.e. student's grades)?

In this course, we only treat the case where we compare two groups. There are methods to compare more than two groups (e.g. the ANOVA test), but those are beyond the scope of this course.

7.1 Learning goals

By the end of this module, you should be able to:

- Understand the definitions and terminology (indicated with $\propto$) used in this notebook
- Visualize data for these types of research questions with appropriate plots: grouped boxplot, grouped violin plot, density plot, or (when appropriate) bar plot with error bars
- Perform a two-sample *t*-test to determine if the difference between the two groups is statistically significant
- Calculate the effect size (Cohen's *d*) to understand the practical significance of the results

```
[1]: # Package imports
import numpy as np
import scipy.stats as stats

# "Scientific computing"
# Statistical tests
```

```
import pandas as pd                # Dataframe
import matplotlib.pyplot as plt    # Basic visualisation
from statsmodels.graphics.mosaicplot import mosaic # Mosaic plot
import seaborn as sns              # Advanced dataviz
```

7.2 Data visualisation

For data visualisation, we can use the same graph types as for the case of one quantitative variable, but we need to group the data by the categories of the qualitative variable. This allows us to visually compare the distributions of the quantitative variable across the different groups.

The following chart types may be appropriate for this type of data:

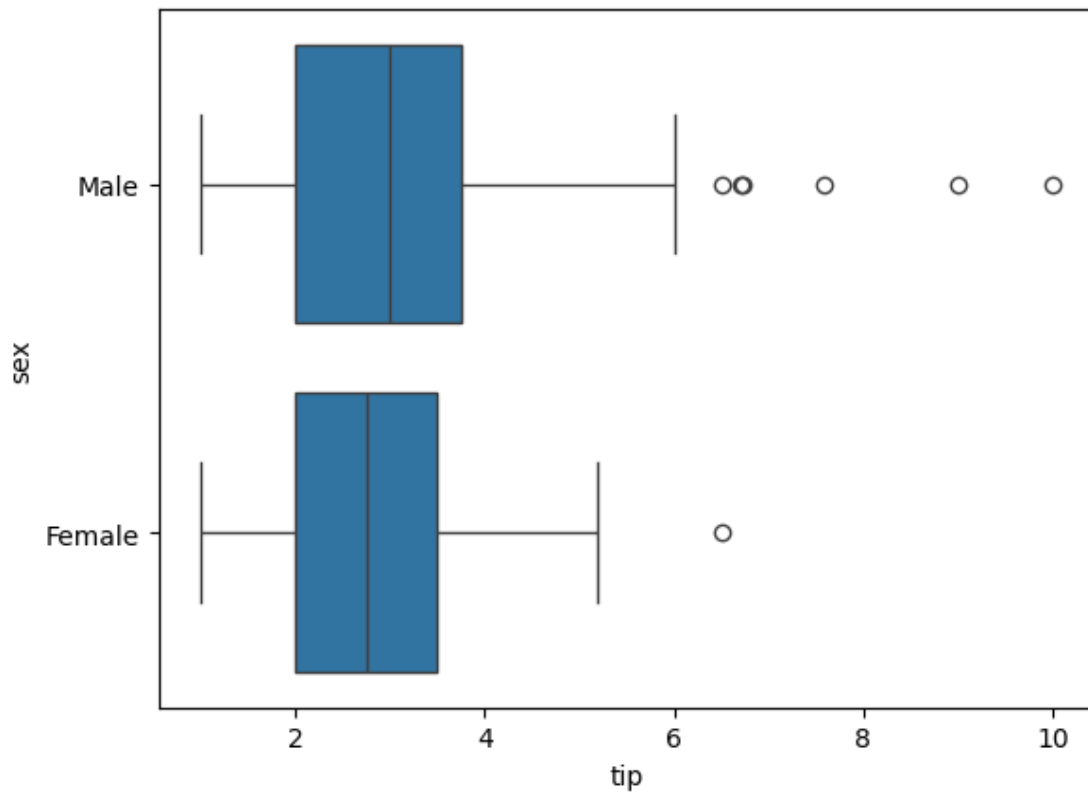
- grouped boxplot or violin plot
- grouped density (kde-)plot
- bar chart with error bars (e.g. standard deviation, *only for data that is normally distributed*)

We'll use the dataset `tips`, included in the Seaborn package to illustrate these.

7.2.1 Boxplot or violin plot

For a horizontal boxplot, map the independent (qualitative) variable on the Y-axis and the dependent (quantitative) variable on the X-axis. Swap for a vertical plot.

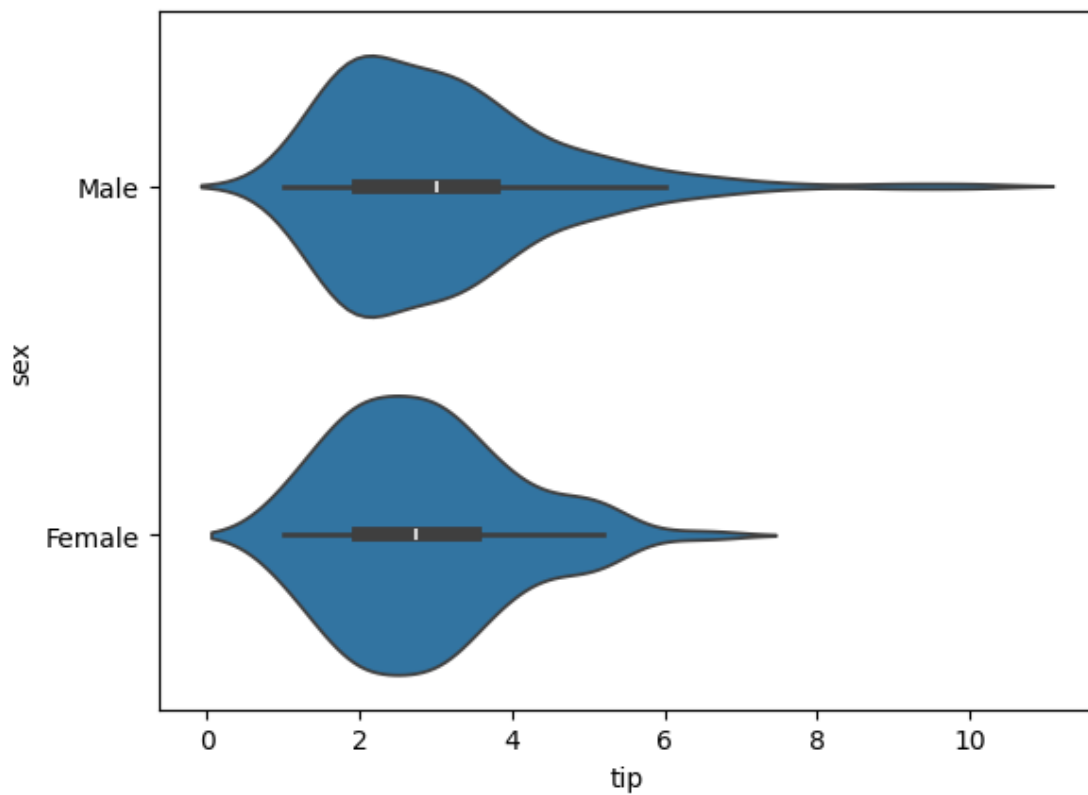
```
[ ]: tips = sns.load_dataset("tips")
      sns.boxplot(data=tips, x='tip', y='sex');
```



If there's a lot of overlap between the (boxes of the) different groups, like in this example, this indicates that the relation between both variables is weak or non-existent.

The violin plot works similarly:

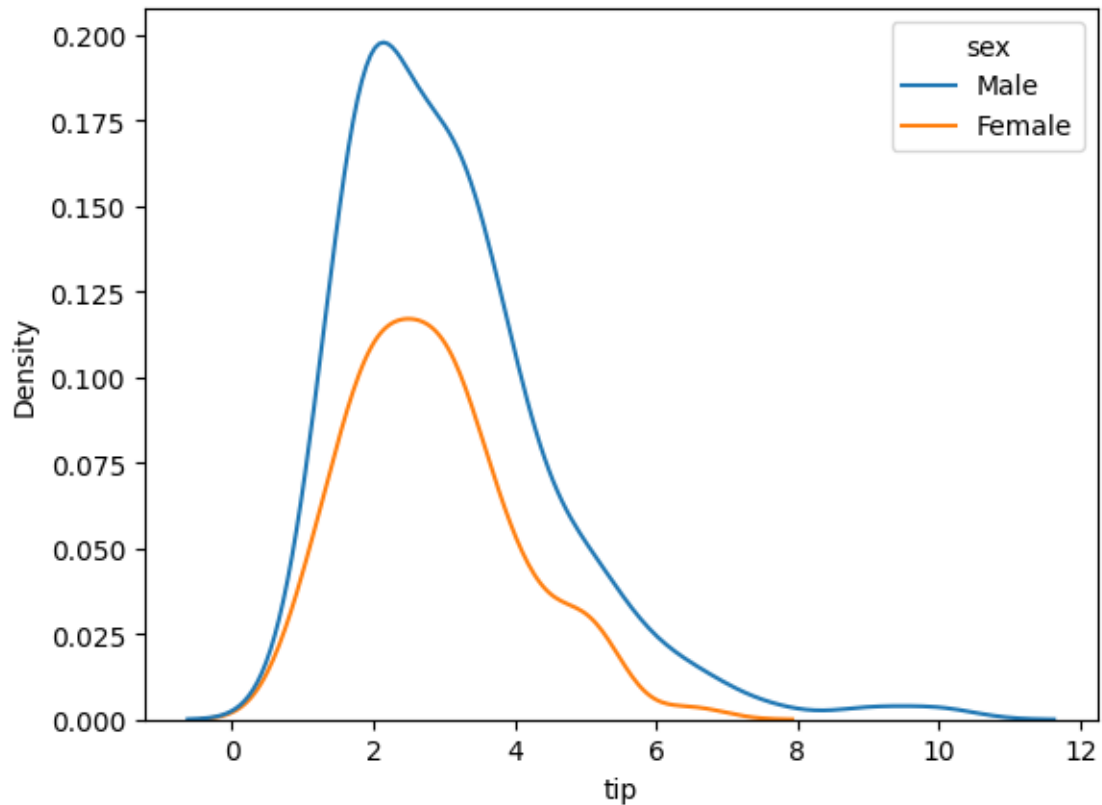
```
[ ]: sns.violinplot(data=tips, x='tip', y='sex');
```



7.2.2 Density plot

For a density (kernel density estimate) plot, map the independent (qualitative) variable on the `hue` attribute and the dependent variable on the X-axis.

```
[ ]: sns.kdeplot(data=tips, hue='sex', x='tip');
```

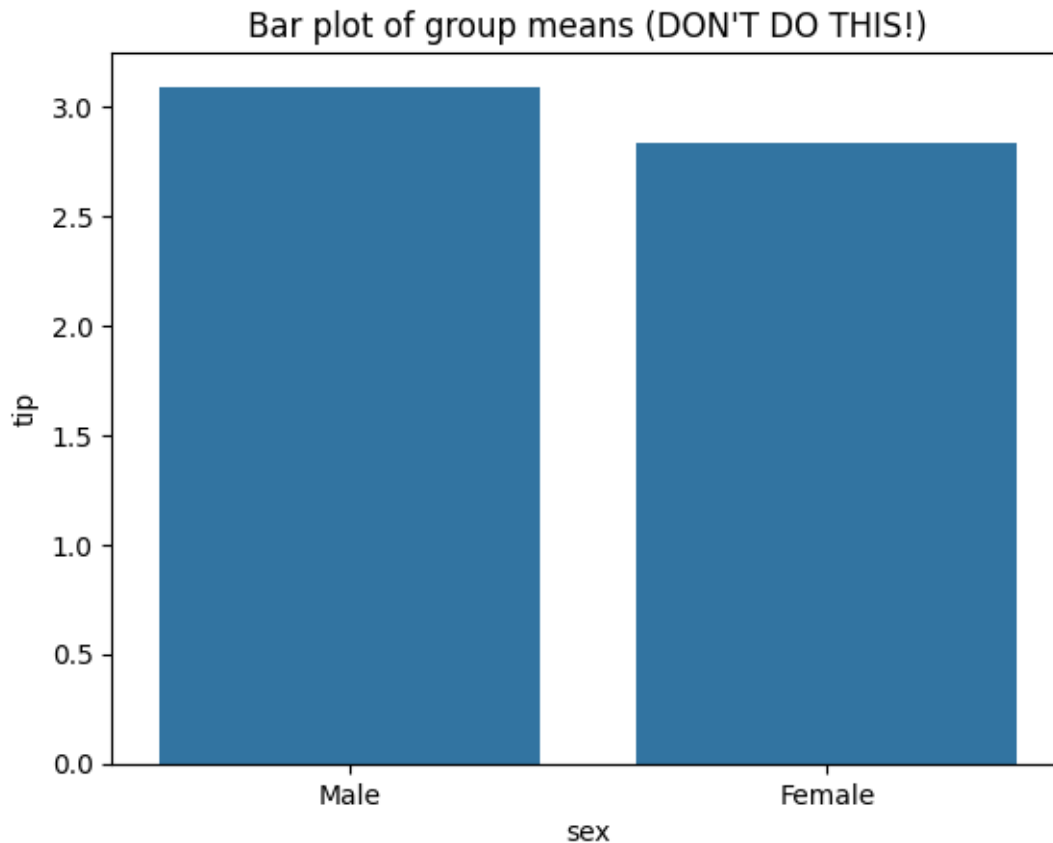


The density curves of the two groups in this example are completely overlapping, which again indicates that there is no relation between both variables.

7.2.3 Bar chart with error bars

Another chart type that is often used for this type of data is a *bar plot of the sample mean of each group*. However, watch out with this. Only showing the sample mean in a chart is insufficient to determine whether any difference between groups is significant.

```
[ ]: sns.barplot(data=tips, x='sex', y='tip', errorbar=None).set_title('Bar plot of_
    ↳group means (DON\'T DO THIS!)');
```

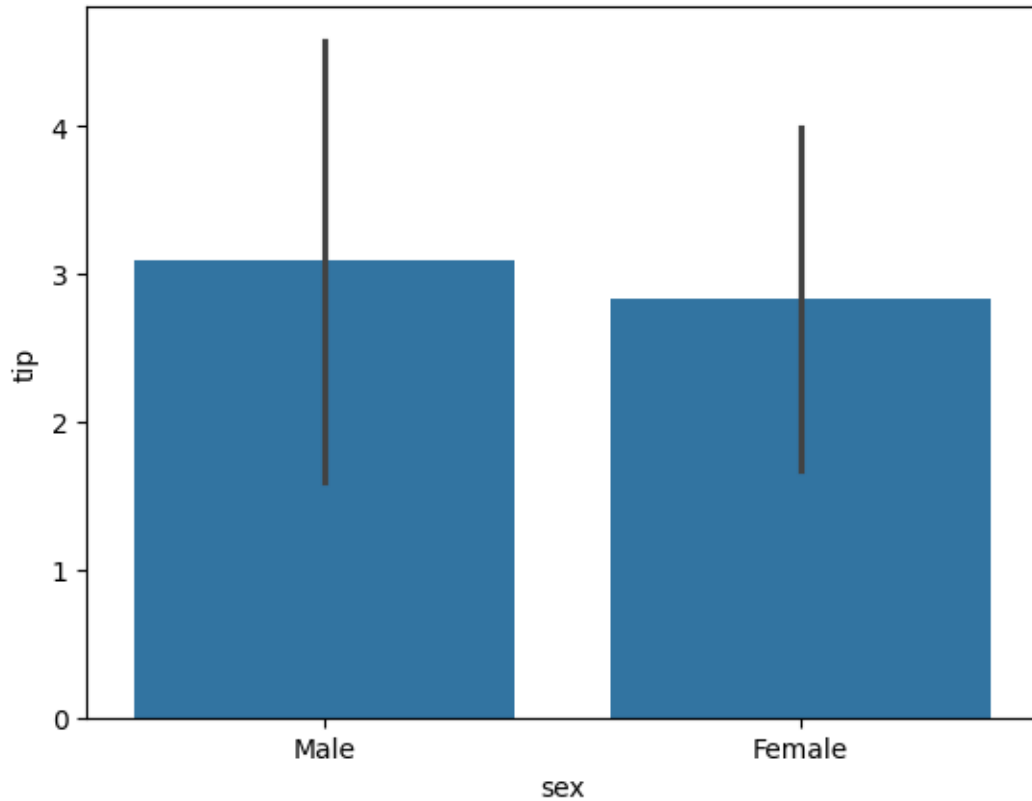


The example above suggests that the average tip amount is higher for the male group than for the female group. With the other chart types, we already saw that there is a lot of overlap, but that information is lost in this bar chart.

A chart should also show the *dispersion* in the data set. This can be visualized using error bars.

In the example below, the parameter `errorbar='sd'` (in older versions of Seaborn: `ci='sd'`) will show the standard deviation:

```
[11]: sns.barplot(data=tips, x='sex', y='tip', errorbar='sd');  
plt.savefig("../slides/img/5/barplot-bars.png", dpi=600)
```



Again, if there's a lot of overlap between the error bars, this indicates that there is no (or only a small) association between the variables.

Remark that this type of chart is only useful if the variable is *normally distributed*. In this dataset with a lot of outliers (see the boxplot from earlier), the symmetrical error bars *do not correctly represent* how the data is distributed. Usually a boxplot or density plot is more appropriate.

7.3 Introduction

Earlier (Chapter 3), we saw how to apply the t -test to verify a statement about the sample mean of a small sample, or when the population standard deviation is unknown. The t -test can also be used to compare two samples. So:

- * We use the one sample t -test when we want to compare the mean of a sample with a known reference mean.
- * We use the t -test for independent samples when we want to compare the means of two independent groups or samples. We want to know if there is a significant difference between these means.
- * We use the Paired Samples t -Test is to compare the means of two dependent groups. In a dependent sample (paired sample), the measured values are available in pairs. The pairs are created, for example, by repeated measurements on the same persons.

First, we will look at the case of **independent samples** that were taken separately.

7.4 The t-test for two independent samples

Earlier, we saw how to apply the t -test to verify a statement about the sample mean of a small sample, or when the population standard deviation is unknown. The t -test can also be used to compare two samples. First, we will look at the case of α **independent samples** that were taken separately.

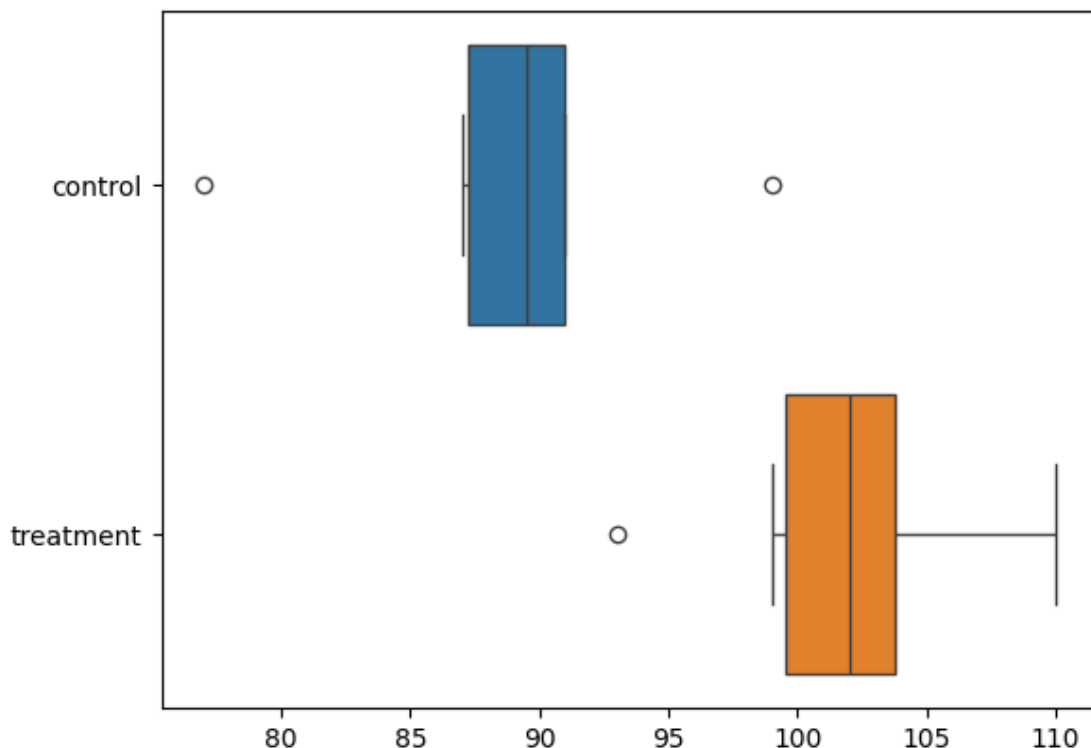
7.4.1 Example: clinical trial

In a clinical trial, researchers want to determine whether some new drug has a reduced reaction speed, i.e. a higher reaction time, as a side effect.

In our example six participants received the drug (treatment group), while six others received a placebo (control group). Their response time to a stimulus was then measured (in ms). We want to investigate whether there are significant differences between the treatment and control groups.

The response time measurements are given below:

```
[22]: # Sample:
control = np.array([91, 87, 99, 77, 88, 91])
treatment = np.array([101, 110, 103, 93, 99, 104])
# Visualization:
sns.boxplot(
    data=pd.DataFrame({'control': control, 'treatment': treatment}),
    orient='h');
```



The population mean for patients in the control group is notated μ_1 , for those in the treatment group μ_2 .

The test procedure runs as follows:

1. Formulate the hypotheses:
 - H_0 : $\mu_1 - \mu_2 = 0$ (reaction time is the same in both groups)
 - H_1 : $\mu_1 - \mu_2 < 0$ (reaction time is significantly higher in the treatment group compared to the control group)
2. Choose a significance level α (e.g. 0.05)
3. Calculate the test statistic t :

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}}.$$

with $\bar{x}_1$ and $\bar{x}_2$ being the sample means of the two groups, s_1 and s_2 the sample standard deviations, and n_1 and n_2 the sample sizes.

```
[30]: x1 = control.mean()
      x2 = treatment.mean()
      s1 = control.std(ddof=1)
      s2 = treatment.std(ddof=1)
      n1 = len(control)
      n2 = len(treatment)
      t = (x1 - x2) / np.sqrt(s1**2/n1 + s2**2/n2)
      print(f't = {t:.4f}');
```

t = -3.4456

In Python, we can use the `ttest_ind()` function from `scipy.stats`. Parameters `a` and `b` are the two groups to be compared. `alternative='less'` indicates that we want to test for the alternative hypothesis that the mean of the control group is less than the mean of the treatment group. Finally, by setting `equal_var=False`, it is *not* assumed that both groups have the same standard deviation.

```
[23]: stats.ttest_ind(a=control, b=treatment,
                    alternative='less', equal_var=False)
```

```
[23]: TtestResult(statistic=np.float64(-3.445612673536487),
                pvalue=np.float64(0.0033912300792068996), df=np.float64(9.479682492670932))
```

4. Calculate the p -value

The p -value is also emitted by the `ttest_ind()` function (we won't go into how it was calculated exactly). It represents the probability of observing a test statistic as extreme as, or more extreme than, the one calculated from the sample data, assuming that the null hypothesis is true. For this case, $p \approx 0.0034$.

5. Draw a conclusion based on the p -value and the significance level α .

The p -value is very small ($p < \alpha$), which means that, if the null hypothesis would be correct, it is highly unlikely to observe the differences between the two groups. Consequently, we reject the null hypothesis. Based on the sample, there is reason to conclude that the drug does indeed have a significant impact on reaction speed.

7.4.2 Example: tips dataset

In the `tips` dataset, do male and female staff get the same tips or is there a significant difference? Take a look at the plot above to make a guess first.

In this case, we perform a *two-sided test*. I.e. we don't have a specific preconception about which group would get higher tips.

```
[25]: stats.ttest_ind(alternative='two-sided',
    a=tips.tip[tips.sex == 'Male'],
    b=tips.tip[tips.sex == 'Female'])

[25]: TtestResult(statistic=np.float64(1.387859705421269),
    pvalue=np.float64(0.16645623503456755), df=np.float64(242.0))
```

The p value of 0.166 is quite large ($p > \alpha$), so we do not reject the null hypothesis. On a significance level of e.g. 0.05, the data set does *not* contain evidence that male and female serving staff receive different tips.

7.5 The t -test for paired samples

In this variant of the two-sample t -test, a measurement is taken on each element of the sample, one time *before* and one time *after* an intervention. The aim is to determine whether the intervention had a significant effect on the measurement.

7.5.1 Example: petrol additives

A study examined whether cars that run on petrol with additives also have a lower consumption. Ten cars were first filled with either regular gasoline or gasoline with additives (determined by tossing a coin), after which consumption was measured (expressed in miles per gallon). The cars were then refueled with the other type of petrol and consumption was measured again.

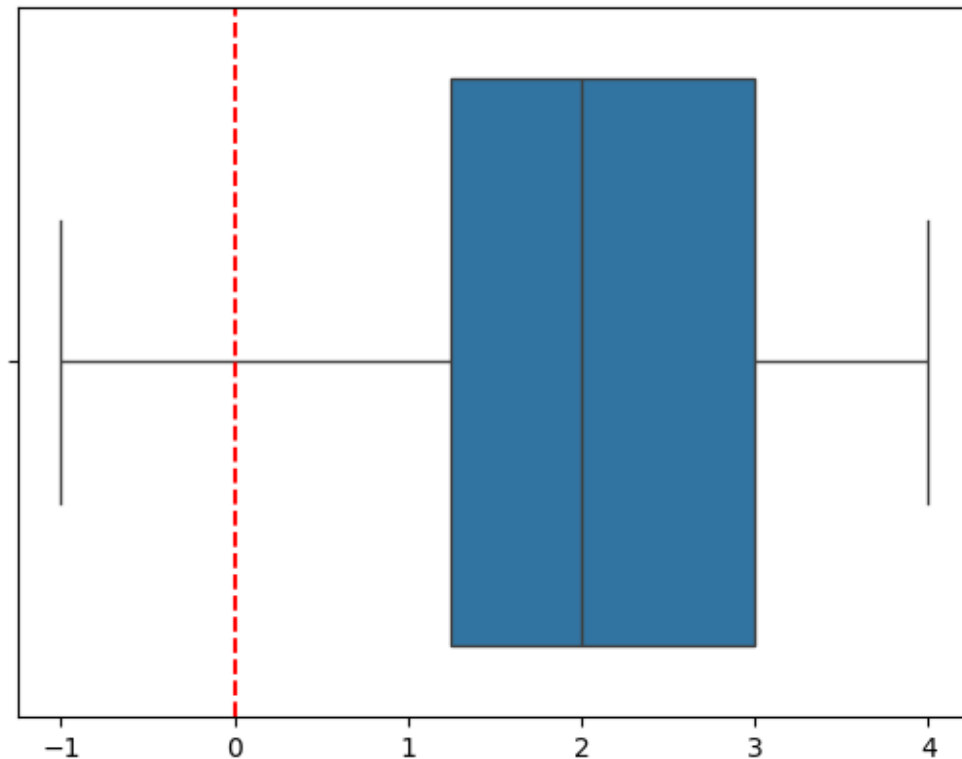
We will use a paired t test to determine whether cars are significantly more economical on gasoline with additives. In a paired t -test, we first calculate the difference between each pair of measurements (additives - regular), and then we test whether the mean of these differences is significantly less than zero.

The boxplot below shows the distribution of the differences. The red dashed line indicates a difference of zero (i.e. no effect of the additives). The mean difference is positive, which suggests that the additives do indeed have a positive effect on fuel economy.

```
[33]: # Measurements
regular = np.array([16, 20, 21, 22, 23, 22, 27, 25, 27, 28])
additives = np.array([19, 22, 24, 24, 25, 25, 26, 26, 28, 32])

differences = additives - regular
print(f'Mean difference: {differences.mean():.2f}')
# Visualization
sns.boxplot(differences, orient='h');
plt.axvline(0, color='red', linestyle='--');
```

Mean difference: 2.00



In the case of a (left-tailed) paired t -test, the hypotheses are formulated as follows:

- $H_0: \mu_{X-Y} = 0$
- $H_1: \mu_{X-Y} < 0$

With μ_{X-Y} being the population mean of the differences between the paired measurements.

The test statistic can be calculated with Python function `ttest_rel()` from `scipy.stats`:

```
[ ]: # Paired t-test with ttest_rel()
stats.ttest_rel(regular, additives, alternative='less')
```

```
[ ]: TtestResult(statistic=-4.47213595499958, pvalue=0.00077494295585091, df=9)
```

We use `alternative='less'` because we want to check whether the mileage per gallon on regular fuel is smaller than on fuel with additives.

The p -value, 0.0007749, is very small. Even with an $\alpha = 0.01$, we can reject the null hypothesis. In conclusion, according to this sample, we have reasons to assume that the fuel with additives indeed leads to significantly lower fuel consumption.

7.6 Effect size

If we want to know whether two groups are significantly different, we can use a statistical test like the two sample t -test. The result of a statistical test is generally either “true” or “false”, depending on the p -value and the chosen significance level.

⌘ **Effect size** is another metric to express the *magnitude* of the difference between two groups. The effect size can be used in addition to a hypothesis test.

Several definitions of effect size exist, but one of the most commonly used is ⌘ **Cohen’s d** :

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s}$$
$$s = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

With: $\bar{x}_1$ and $\bar{x}_2$ the mean of each group, n_1 and n_2 the respective sample sizes and s_1 and s_2 the respective sample standard deviations.

Cohen’s d is used in several settings where you want to measure the difference between two groups, e.g. a *control group* vs. an *intervention group*. Particularly, in education research, Cohen’s d is used to evaluate what factors influence learning outcomes for students. Factors include learning and teaching strategies, use of technology, classroom management, student and teacher attributes, etc.

Research papers in this domain always report Cohen’s d , which allows us to compare the results of different studies. For example, Hattie (2012) performed a massive meta-analysis that synthesises findings from 80,000 studies into what works best in education (see e.g. <https://visible-learning.org/backup-hattie-ranking-256-effects-2017/>). As a rule of thumb, an influence with d of at least 0.4 is considered to potentially accelerate student achievement. As a rule of thumb, a value for d of 1 means that students can acquire competencies in about half the time they normally would.

The table below gives a general interpretation of Cohen’s d :

d	Interpretation
0.01	very small effect
0.2	small effect
0.5	medium effect
0.8	large effect
1.2	very large effect

d	Interpretation
2.0	huge effect

7.6.1 Python function for Cohen's d

Cohen's d is defined as the difference between the means of both groups, divided by a pooled standard deviation. There's no Python function for calculating Cohen's d readily available, so we define it here, according to the formula given earlier:

```
[35]: def cohen_d(a, b):
      na = len(a)
      nb = len(b)
      pooled_sd = np.sqrt( ((na-1) * np.var(a, ddof=1) +
                           (nb-1) * np.var(b, ddof=1)) / (na + nb - 2) )
      return (np.mean(b) - np.mean(a)) / pooled_sd
```

The effect sizes for the previously given examples are as follows:

```
[37]: # Effect size for the clinical trial example
      cohen_d(control, treatment)
```

```
[37]: np.float64(1.989325404589477)
```

```
[38]: # Effect size of additives in gasoline:
      cohen_d(regular, additives)
```

```
[38]: np.float64(0.5573262931770191)
```

8 Module 6. Bivariate analysis of 2 quantitative variables

In this module, we discuss the case where *both variables are quantitative*. Specifically, we'll explore the concept of **linear regression**. Other analysis methods do exist, but those are beyond the scope of this course.

Regression analysis is an extremely flexible tool that can aid decision making in many areas. [Kimes and Fitzsimmons \(1990\)](#) describe how it has been used by La Quinta Motor Inns, a moderately priced hotel chain oriented toward serving the business traveler, to help make site location decisions.

There are two potential objectives of regression analysis.

The first objective is to *understand* how the world operates, for example, one might try to discover whether there is any gender discrimination in salaries, taking into account differences in work experience and education level.

The second objective is to *make predictions*. A good example of this is when the variable in question is company sales, and variables such as advertising and past sales levels are used as explanatory variables.

8.1 Learning goals

By the end of this module, you should be able to:

- Understand the definitions and terminology (indicated with $\propto$) used in this notebook
- Visualize data for these types of research questions with a scatter plot and regression line
- Calculate the parameters of a regression line for a given dataset (i.e. the intercept/level and slope/trend)
- Calculate Covariance Cov , Pearson's Product-moment Correlation Coefficient R , Determination Coefficient R^2
- Interpret the values of Cov , R and R^2 using the correct terminology

```
[2]: # Package imports
import numpy as np;                # "Scientific computing"
import scipy.stats as stats;       # Statistical tests

import pandas as pd;              # Dataframe
import matplotlib.pyplot as plt;  # Basic visualisation
from statsmodels.graphics.mosaicplot import mosaic; # Mosaic plot
import seaborn as sns;            # Advanced dataviz
```

8.2 Visualization

A good way to begin any regression analysis is to draw one or more $\propto$ **scatterplots** (also: *X-Y plot* or *graph*). Each observation in the data set will be represented as a point in a Cartesian coordinate system with the value of the independent variable as the X coordinate and the value of the dependent variable as the Y coordinate.

If there is any (linear) relationship between the two variables, it is usually immediately apparent from the scatterplot.

In the examples below we will use the [Palmer Penguins](#) dataset.

```
[3]: penguins = sns.load_dataset('penguins') # Load the Palmer Penguins demo dataset
penguins.head()
```

```
[3]: species      island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen      39.1             18.7             181.0
1  Adelie  Torgersen      39.5             17.4             186.0
2  Adelie  Torgersen      40.3             18.0             195.0
3  Adelie  Torgersen      NaN             NaN             NaN
4  Adelie  Torgersen      36.7             19.3             193.0

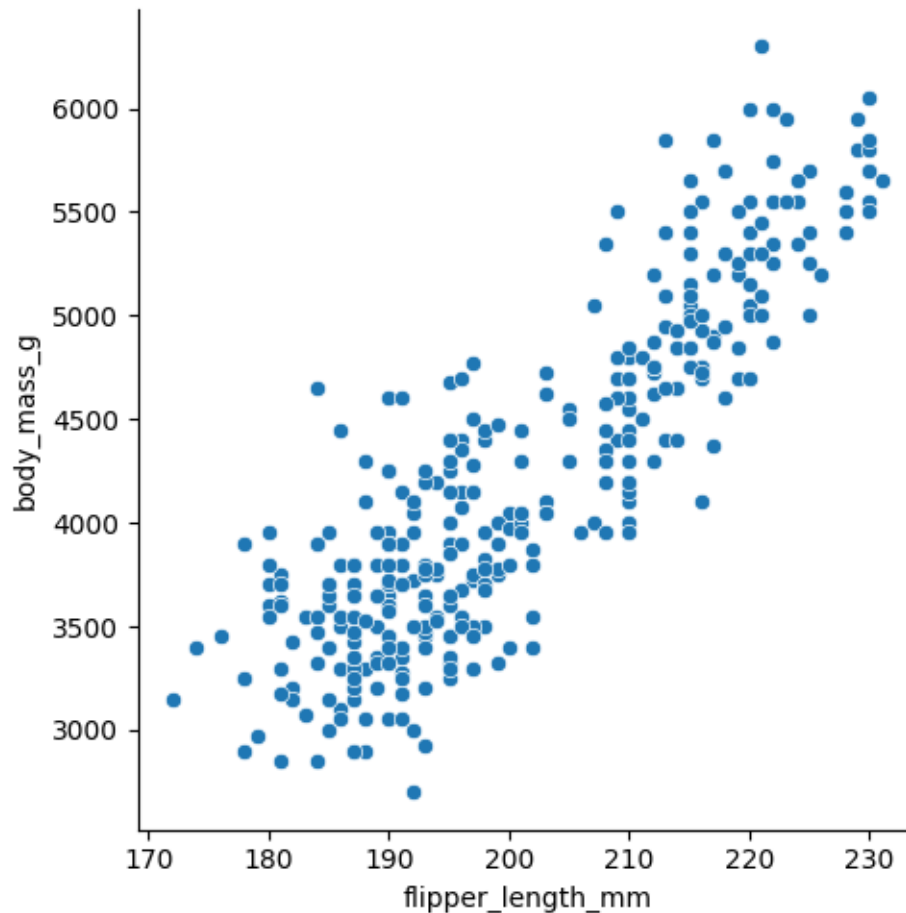
   body_mass_g  sex
0      3750.0  Male
1      3800.0 Female
2      3250.0 Female
3         NaN   NaN
```

4 3450.0 Female

In Seaborn, you can use the `relplot()` function to show the relation between two qualitative variables. The independent variable should be mapped to the `x` attribute and the dependent to the `y` attribute.

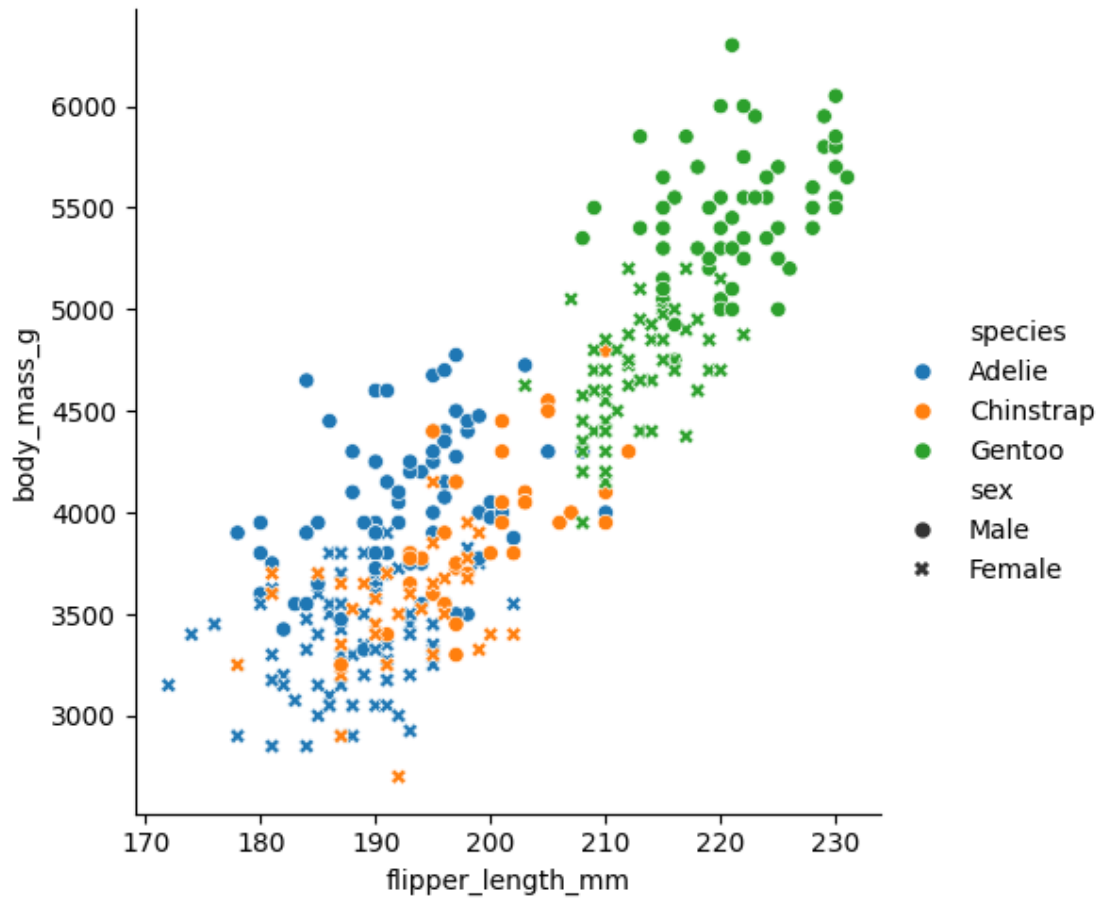
```
[4]: sns.relplot(data=penguins, x='flipper_length_mm', y='body_mass_g')
```

```
[4]: <seaborn.axisgrid.FacetGrid at 0x1f05aab86e0>
```



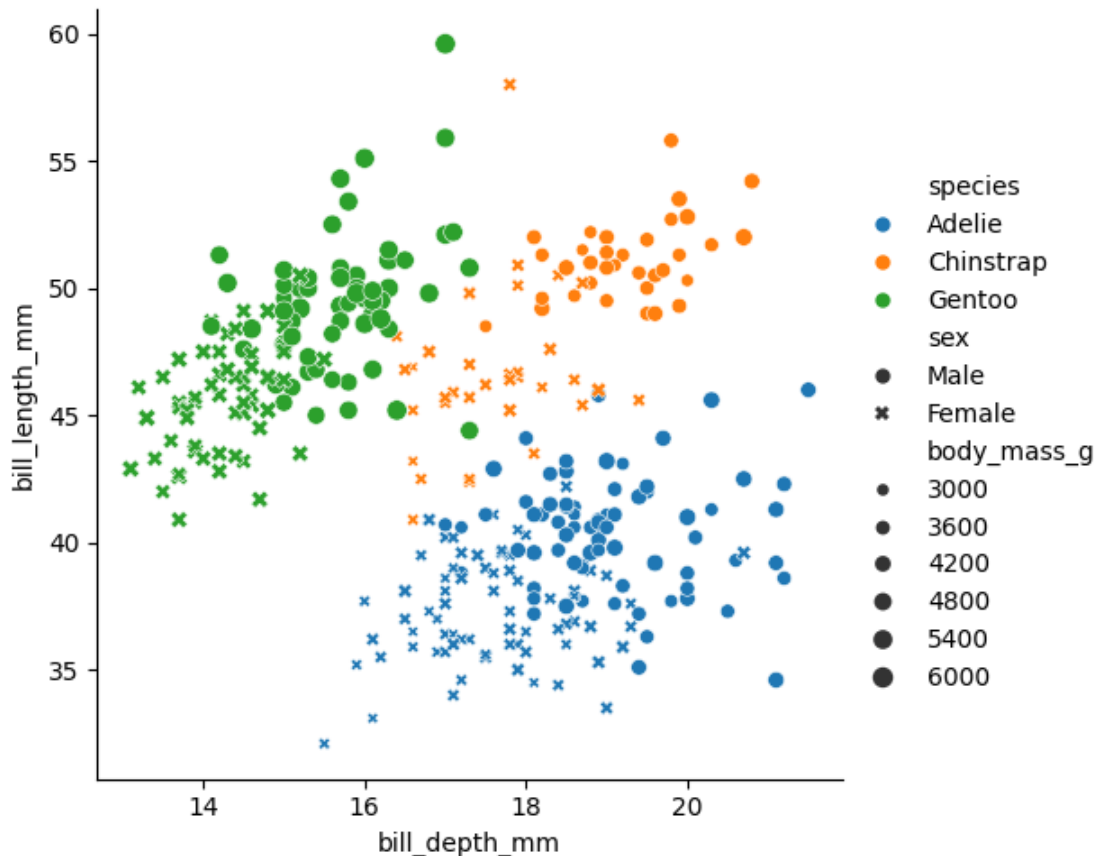
Using other chart attributes like `hue`, `style`, etc. you can also visualize categorical variables in the data set, e.g.:

```
[5]: sns.relplot(data=penguins,
                  x='flipper_length_mm', y='body_mass_g',
                  hue='species', style='sex');
```

Another example:

```
[6]: sns.relplot(data=penguins,  
               x='bill_depth_mm', y='bill_length_mm',  
               size='body_mass_g',  
               hue='species', style='sex');
```



8.3 Linear regression

From the scatterplot showing the relation between `body_mass_g` and `flipper_length_mm`, it is immediately apparent that there is a systematic relationship between them. A lower flipper length corresponds with a lower body mass and a higher flipper length with a higher body mass. And although there is quite a bit of dispersion on the datapoints, the relationship seems to be linear in nature, i.e. we could approximate the relationship by drawing a line that follows the general direction of the datapoints.

8.3.1 Introduction: the regression line

Scatterplots are very useful for indicating linear relationships and the strengths of these relationships, but they do not actually *quantify* the relationships. The scatterplot does not specify *exactly* what this relationship is. For example, if the `flipper_length_mm` is 190, what would you predict `body_mass_g` to be?

To answer these questions, the relationship between the dependent variable `body_mass_g` and the independent variable `flipper_length_mm` must be quantified.

To reduce the number of points in the scatterplot and make the plots as clear as possible, we are going to take only the male, Chinstrap penguins into account.

```
[7]: male_chinstrap = penguins[(penguins['species'] == 'Chinstrap') &
    ↪(penguins['sex'] == 'Male')]
male_chinstrap.head()
```

```
[7]:      species island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
153  Chinstrap  Dream         50.0         19.5         196.0
154  Chinstrap  Dream         51.3         19.2         193.0
156  Chinstrap  Dream         52.7         19.8         197.0
159  Chinstrap  Dream         51.3         18.2         197.0
161  Chinstrap  Dream         51.3         19.9         198.0

      body_mass_g  sex
153         3900.0  Male
154         3650.0  Male
156         3725.0  Male
159         3750.0  Male
161         3700.0  Male
```

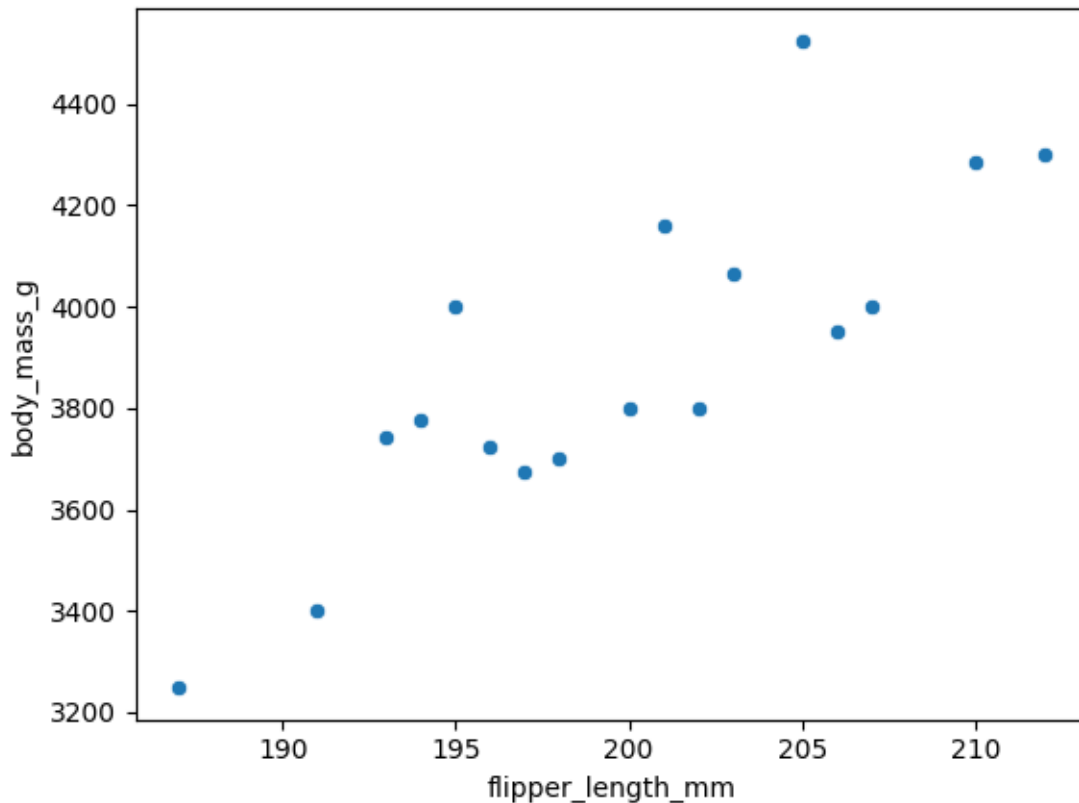
Some male, Chinstrap penguins have the same `flipper_length_mm`, but a different `body_mass_g`. To simplify things, we calculate the average value for `body_mass_g` per `flipper_length_mm`.

```
[8]: male_chinstrap = male_chinstrap.groupby('flipper_length_mm')['body_mass_g'].
    ↪mean().reset_index()
male_chinstrap.head(10)
```

```
[8]:      flipper_length_mm  body_mass_g
0          187.0  3250.000000
1          191.0  3400.000000
2          193.0  3741.666667
3          194.0  3775.000000
4          195.0  4000.000000
5          196.0  3725.000000
6          197.0  3675.000000
7          198.0  3700.000000
8          200.0  3800.000000
9          201.0  4160.000000
```

The data is plotted on this scatter plot:

```
[9]: sns.scatterplot(data=male_chinstrap, x='flipper_length_mm', y='body_mass_g');
```



The scatterplot above hints at a linear relationship between these two variables. It would not be difficult to draw a straight line through these points to produce a reasonably good fit. But which is the best line to use?

A line can be defined by

$$y = ax + b$$

in which a is the $\propto$ **slope** (also: **trend**) of the line and b is the $\propto$ **intercept** (also: **level**) of the line (that's the location on the Y axis that the line crosses when X equals 0).

We will try the following values for a and b and see how well the line fits the data:

- $a = 37$ and $b = -3385$
- $a = 23$ and $b = -820$
- $a = 23$ and $b = -715$
- $a = 20.5$ and $b = -92$
- $a = 29$ and $b = -1635$

We need a way to 'calculate' if one line is a better fit than another line. For that we will use the sum of squared residuals.

8.3.2 Squared residuals

Consider the following image. Three points in the scatterplot are shown, along with a straight line. The vertical distance from the horizontal axis to any point, can be decomposed into two parts: the vertical distance from the horizontal axis to the line, and the vertical distance from the line to the point. The first of these is called the $\propto$ **fitted value**, and the second is called the $\propto$ **residual**. So a fitted value is the predicted value of the dependent variable. The corresponding residual is the difference between the actual and fitted values of the dependent variable.

In the following code blocks, a small animation is created to try to find the straight line that fits the points the best.

The dotted green lines are the residuals.

```
[10]: import math
import time

from matplotlib import rc
from matplotlib.animation import FuncAnimation
from matplotlib.collections import LineCollection

# set the minimum and maximum value of the x- and y-axis
xmin = math.floor(male_chinstrap['flipper_length_mm'].min() / 10) * 10
xmax = math.ceil(male_chinstrap['flipper_length_mm'].max() / 10) * 10

ymin = math.floor(male_chinstrap['body_mass_g'].min() / 10) * 10
ymax = math.ceil(male_chinstrap['body_mass_g'].max() / 10) * 10

a_values = [37, 23, 23, 20.5, 29]
b_values = [-3385, -820, -715, -92, -1635]

def animate(i):
    len_male_chinstrap_plus_2 = len(male_chinstrap) + 2

    j = i // len_male_chinstrap_plus_2
    k = i % len_male_chinstrap_plus_2

    # alles leegmaken
    axs.clear();

    # axs instellen
    axs.set_xlim((xmin, xmax));
    axs.set_ylim((ymin, ymax));

    # punten tekenen
```

```

for l in range(len(male_chinstrap)):
    axs.plot(male_chinstrap.iloc[l, 0], male_chinstrap.iloc[l, 1], marker="o",
    ↪ markersize=10, markeredgecolor="red", markerfacecolor="green");

if (k == 0):
    # these are matplotlib.patch.Patch properties
    props = dict(boxstyle='round', facecolor='wheat', alpha=0.5)
    axs.text(0.05, 0.95, f'Attempt n° {j + 1}', transform=axs.transAxes,
    ↪ fontsize=14, verticalalignment='top', bbox=props);
    print()

if k > 0:
    # rechte tekenen
    x_values = [xmin + 2, xmax - 2]
    y_values = [a_values[j] * x_values[0] + b_values[j], a_values[j] *
    ↪ x_values[1] + b_values[j]]
    axs.plot(x_values, y_values, lw = 3);

if k > 1:
    # lijn segmenten toevoegen
    segs = []
    sum_squared_residuals = 0
    for q in range(k - 1):
        sum_squared_residuals += ((a_values[j] * male_chinstrap.iloc[q, 0] +
    ↪ b_values[j]) - male_chinstrap.iloc[q, 1]) ** 2
        segs.append([(male_chinstrap.iloc[q, 0], a_values[j] * male_chinstrap.
    ↪ iloc[q, 0] + b_values[j]), (male_chinstrap.iloc[q, 0], male_chinstrap.
    ↪ iloc[q, 1])]);
    lc = LineCollection(segs, linewidths=1, colors='g', linestyle='dashed')
    axs.add_collection(lc);

    # these are matplotlib.patch.Patch properties
    props = dict(boxstyle='round', facecolor='wheat', alpha=0.5)
    # place a text box in upper left in axes coords
    axs.text(0.05, 0.95, f'Sum Squared Residuals = {sum_squared_residuals:5.
    ↪ 2e}', transform=axs.transAxes, fontsize=14, verticalalignment='top',
    ↪ bbox=props);

# calculating the frames needed
len_male_chinstrap_plus_2 = len(male_chinstrap) + 2

frames = []
for i in range(len(a_values)):
    frames.append(i * len_male_chinstrap_plus_2);
    frames.append(i * len_male_chinstrap_plus_2);

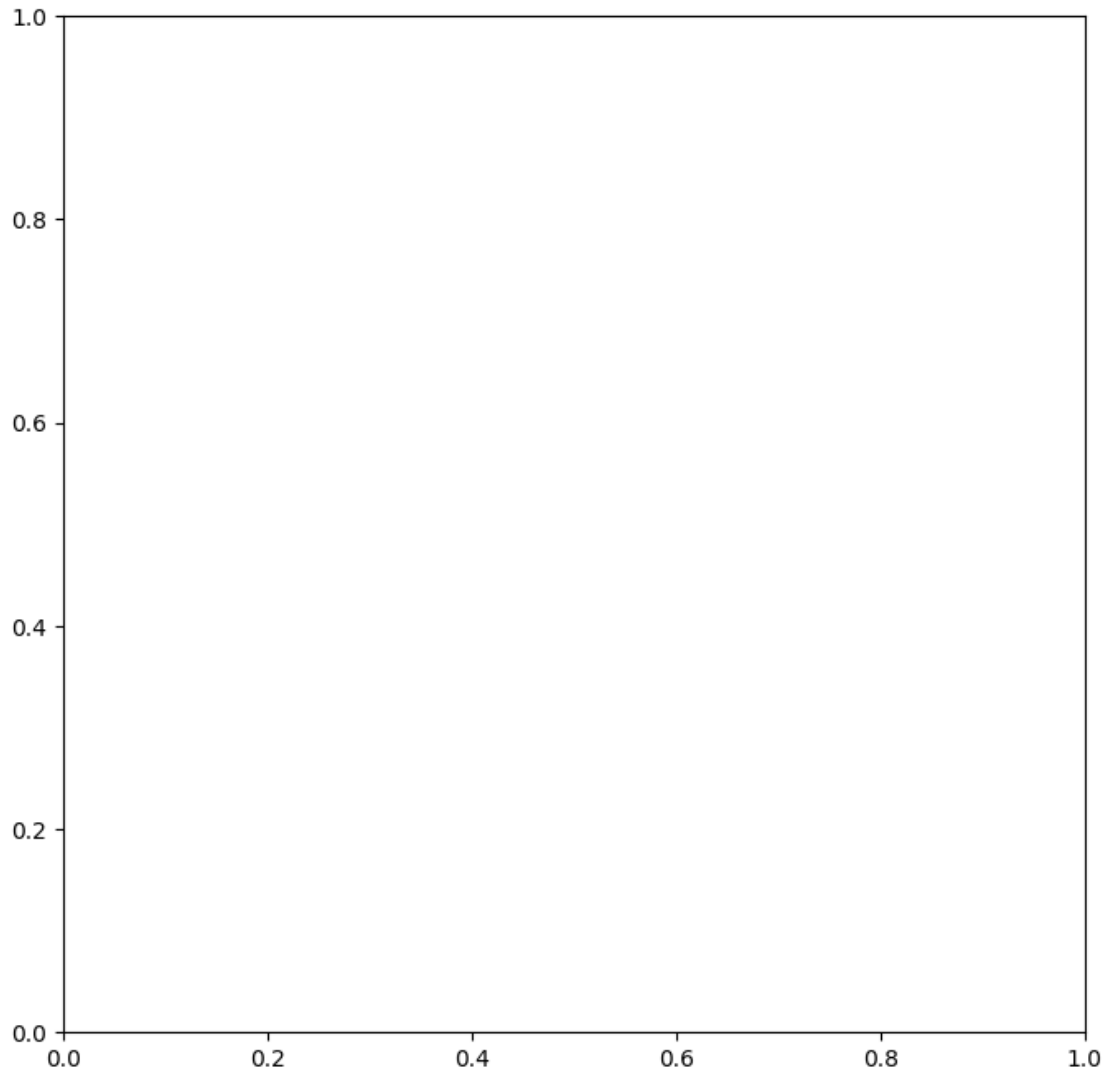
```

```

frames.append(i * len_male_chinstrap_plus_2 + 1);
frames.append(i * len_male_chinstrap_plus_2 + 1);
for j in range(len(male_chinstrap)):
    frames.append(i * len_male_chinstrap_plus_2 + 2 + j);
frames.append(i * len_male_chinstrap_plus_2 + 2 + j);
frames.append(i * len_male_chinstrap_plus_2 + 2 + j);

fig, axs = plt.subplots(figsize=(8, 8));

```



```

[11]: rc('animation', html='jshtml');
anim = FuncAnimation(fig, animate, frames=frames, interval=500, repeat=False);
anim

```

[11]: <matplotlib.animation.FuncAnimation at 0x1f05ce67b60>

We can measure how well a line fits the data by seeing how close it is to the data points. We'll start with the point in the lower left-hand corner of the graph. We can draw a dashed line from this point up to the gray line. The **residual** is the difference between the actual and fitted values of the dependent variable. The term is squared to ensure that each term is positive. Finally we take the sum of all the squared residuals between the line and the data points. This is called the **sum of squared residuals**.

Note: If we would simply use the sum of the residuals, the positive residuals would cancel the negative residuals. We could also minimize the sum of the *absolute values* of the residuals and this would be a perfectly reasonable procedure. However, for technical and historical reasons, it is not the procedure usually chosen. By the way, this technique is also used in the calculation of the variance, which also contains a sum of squared deviations, but from the mean!

8.3.3 Method of least squares

The *least squares line* is the line that minimizes the sum of the squared residuals.

The minimization problem itself is a calculus problem, but in essence we want to approximate a collection of points (x_i, y_i) (with $i : 1, \dots, n$) as closely as possible with a line

$$\hat{y} = \beta_0 + \beta_1 x.$$

The notation $\hat{y}$ stands for **an estimation for y** . Parameters β_0 (intercept with the Y axis or level) and β_1 (slope or trend) are calculated as follows:

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

These values for β_0 and β_1 can be proven to minimize the sum of the squared residuals between the observed and the fitted values (hence the name of the “method of least squares”). However, the proof to obtain these values is beyond the scope of this course.

In the code block below, we calculate the components of the formula for β_1


```
[12]: least_squares = pd.DataFrame({
        'x': male_chinstrap.flipper_length_mm,
        'y': male_chinstrap.body_mass_g
    })
mx = least_squares.x.mean()
my = least_squares.y.mean()

least_squares['(x- $\bar{x}$  )'] = least_squares['x'] - mx
least_squares['(y- $\bar{y}$  )'] = least_squares['y'] - my

least_squares['(x- $\bar{x}$  )(y- $\bar{y}$  )'] = least_squares['(x- $\bar{x}$  )'] * least_squares['(y- $\bar{y}$  )']
least_squares['(x- $\bar{x}$  )2'] = least_squares['(x- $\bar{x}$  )'] ** 2
least_squares
```

```
[12]:
```

	x	y	(x- $\bar{x}$)	(y- $\bar{y}$)	(x- $\bar{x}$)(y- $\bar{y}$)	(x- $\bar{x}$) ²
0	187.0	3250.000000	-12.823529	-641.274510	8223.402537	164.442907
1	191.0	3400.000000	-8.823529	-491.274510	4334.775087	77.854671
2	193.0	3741.666667	-6.823529	-149.607843	1020.853518	46.560554
3	194.0	3775.000000	-5.823529	-116.274510	677.128028	33.913495
4	195.0	4000.000000	-4.823529	108.725490	-524.440600	23.266436
5	196.0	3725.000000	-3.823529	-166.274510	635.755479	14.619377
6	197.0	3675.000000	-2.823529	-216.274510	610.657439	7.972318
7	198.0	3700.000000	-1.823529	-191.274510	348.794694	3.325260
8	200.0	3800.000000	0.176471	-91.274510	-16.107266	0.031142
9	201.0	4160.000000	1.176471	268.725490	316.147636	1.384083
10	202.0	3800.000000	2.176471	-91.274510	-198.656286	4.737024
11	203.0	4066.666667	3.176471	175.392157	557.128028	10.089965
12	205.0	4525.000000	5.176471	633.725490	3280.461361	26.795848
13	206.0	3950.000000	6.176471	58.725490	362.716263	38.148789
14	207.0	4000.000000	7.176471	108.725490	780.265283	51.501730
15	210.0	4283.333333	10.176471	392.058824	3989.775087	103.560554
16	212.0	4300.000000	12.176471	408.725490	4976.833910	148.266436

```
[13]: # Numerator and denominator of the formula for  $b_0$ :
num = sum(least_squares['(x- $\bar{x}$  )(y- $\bar{y}$  )'])
denom = sum(least_squares['(x- $\bar{x}$  )2'])
beta1 = num/denom
beta0 = my - beta1 * mx

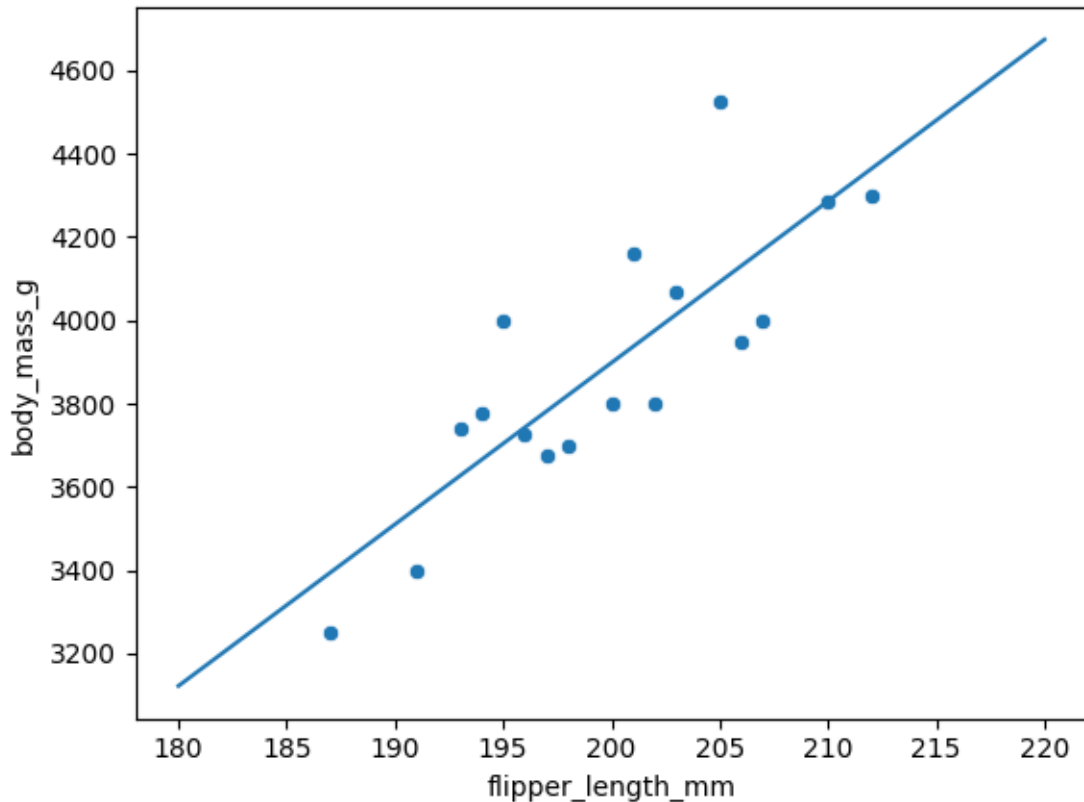
print(f"beta_1 = {num:.4f} / {denom:.4f} = {beta1:.4f}")
print(f"beta_0 = {my:.4f} - {beta1:.4f} * {mx:.4f} = {beta0:.4f}")
print(f" $\hat{y}$  = {beta0:.4f} + {beta1:.4f} x")
```

```
beta_1 = 29375.4902 / 756.4706 = 38.8323
beta_0 = 3891.2745 - 38.8323 * 199.8235 = -3868.3320
 $\hat{y}$  = -3868.3320 + 38.8323 x
```

Let's check if the values for β_0 and β_1 provide a good approximation of the observed data:

```
[14]: x_values = [xmin, xmax]
y_values = [beta1 * x_values[0] + beta0, beta1 * x_values[1] + beta0]

sns.lineplot(x=x_values, y=y_values);
sns.scatterplot(x=male_chinstrap.flipper_length_mm, y=male_chinstrap.
↪body_mass_g);
```

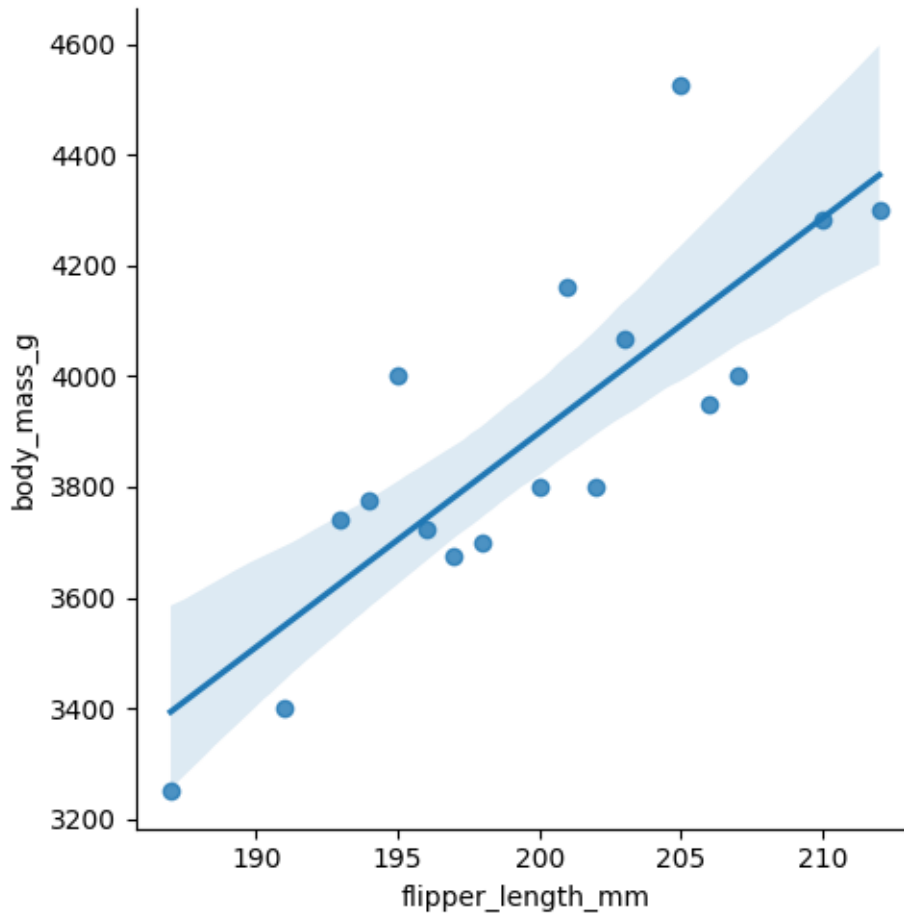


This seems to be a good fit! The line is close to the data points and the residuals are small.

You can plot a data set **with its regression line** using the Seaborn functions `regplot()` or `lmplot()`:

```
[15]: sns.lmplot(data=male_chinstrap, x='flipper_length_mm', y='body_mass_g')
```

```
[15]: <seaborn.axisgrid.FacetGrid at 0x1f05e9d65d0>
```



The regression line corresponds with the one we drew using the calculated parameters!

The formulas for the slope and intercept in simple linear regression given above are mostly for conceptual purposes, i.e. we don't expect you to know them by heart or to calculate them manually. Virtually all statistical software packages perform this minimization automatically.

8.3.4 Calculating the parameters of a linear regression model in Python

In Python, several libraries have functions to calculate the parameters of a linear regression model.

First, there's `numpy.polyfit()`, which fits a polynomial of a specified degree to the data. For linear regression, we would specify a degree of 1.

```
[16]: np.polyfit(male_chinstrap.flipper_length_mm, male_chinstrap.body_mass_g, deg=1)
```

```
[16]: array([ 38.83229653, -3868.33203733])
```

A second way to calculate the regression line is to use the `linregress()` function from the `scipy.stats` module. This function returns the slope and intercept of the regression line, as well as other statistics such as the correlation coefficient (see below).

```
[17]: stats.linregress(male_chinstrap.flipper_length_mm, male_chinstrap.body_mass_g)
```

```
[17]: LinregressResult(slope=np.float64(38.83229652669775),
  intercept=np.float64(-3868.332037325036), rvalue=np.float64(0.8268244668793293),
  pvalue=np.float64(4.2602852335973965e-05), stderr=np.float64(6.820650601261842),
  intercept_stderr=np.float64(1363.6857025333725))
```

The library **scikit-learn** also has code to calculate a linear regression model from a set of observations.

You can create an instance of the class `LinearRegression` and pass the X- and Y-values to be fitted. The array with X-values must be first converted in a 2D array.

```
[18]: from sklearn.linear_model import LinearRegression

male_chinstrap_x = male_chinstrap.flipper_length_mm.values.reshape(-1,1)
male_chinstrap_y = male_chinstrap.body_mass_g

weight_model = LinearRegression().fit(male_chinstrap_x, male_chinstrap_y)

print(f"Regression line:  $\hat{y} = \{weight\_model.intercept_:.4f\} + \{weight\_model.coef\_ [0] :.4f\} x$ ")
```

Regression line: $\hat{y} = -3868.3320 + 38.8323 x$

We can now predict the `body_mass_g` for a new penguin with flipper length 220

```
[ ]: # using the formula
body_mass_g = weight_model.intercept_ + weight_model.coef_[0] * 220
print(body_mass_g)
```

4674.77319854847

```
[25]: # using the predict function of LinearRegression
body_mass_g = weight_model.predict([[220]]) # predict() needs a matrix
print(body_mass_g)
```

[4674.77319855]

Interprete the result. Does this look ok?

This code snippet seems more complicated compared to the previous functions, but it fits in the concept of machine learning, where we have a model (the class `LinearRegression`) that can be trained on data (the X- and Y-values) to learn the parameters (the slope and intercept). The `fit()` method is used to train the model, and the `coef_` and `intercept_` attributes are used to access the learned parameters. In the chapter on time series analysis, we will write code based on a similar approach.

8.4 Expressing the strength of a linear relationship: correlation

8.4.1 Covariance

✧ **Covariance** is a measure that indicates whether a (linear) relationship between two variables is increasing or decreasing.

The **covariance of a sample** is defined as:

$$\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

Note: The **covariance of a population** is similar, but the denominator is n instead of $n-1$.

Covariance is essentially an average of products of deviations from means. If X and Y vary in the same direction, then when X is above its mean, Y will tend to be above its mean, and when X is below its mean, Y will tend to be below its mean. In either case, the product of deviations will be positive (a positive times a positive or a negative times a negative) so the covariance will be positive. The opposite is true when X and Y vary in opposite directions. Then the covariance will be negative.

Covariance measures the direction of a linear relationship between two numerical variables. If this straight line rises from left to right, the relationship is increasing and the covariance will be a positive number. If it falls from left to right, the relationship is decreasing and the covariance will be a negative number.

- $\text{Cov}(X, Y) > 0$: increasing
- $\text{Cov}(X, Y) \approx 0$: no relationship
- $\text{Cov}(X, Y) < 0$: decreasing

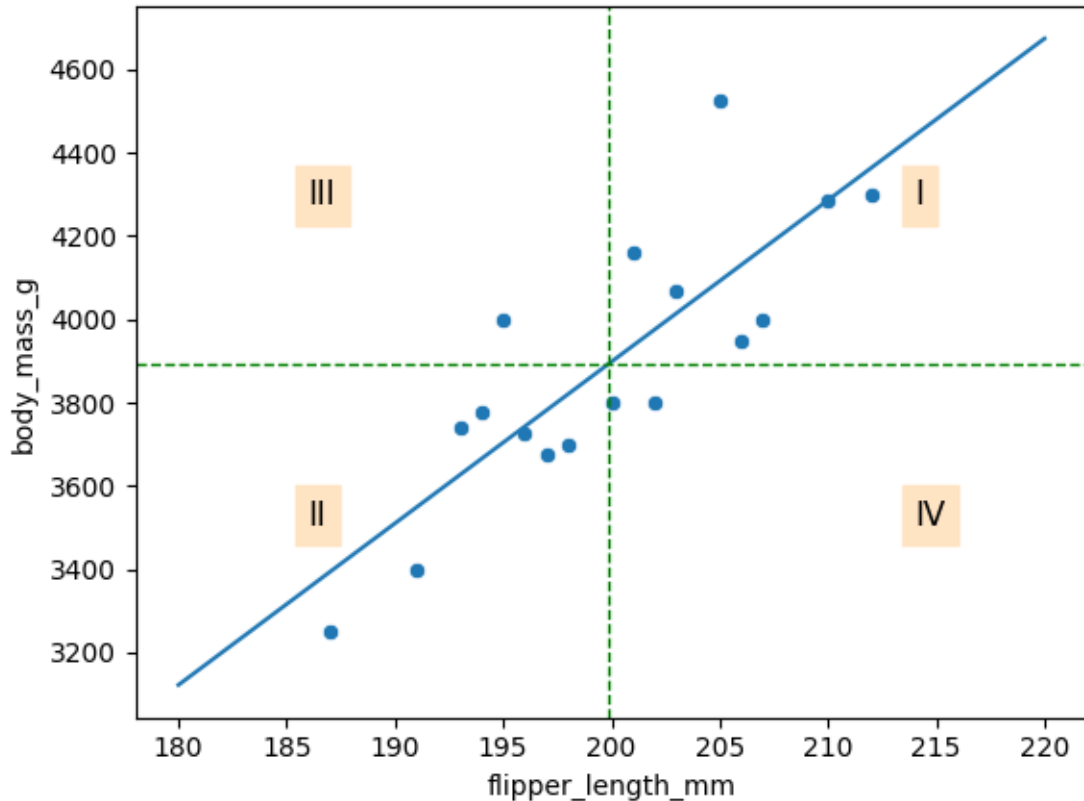
To understand why this is the case, take a look at the plot of the chinstrap flipper length vs body mass. The dotted green lines are the means of each variable.

```
[23]: # Plot the observations
sns.scatterplot(x=male_chinstrap.flipper_length_mm, y=male_chinstrap.
    ↪body_mass_g);
# Draw the regression line through two points
sns.lineplot(x=[xmin, xmax], y=[beta1 * x_values[0] + beta0, beta1 *
    ↪x_values[1] + beta0]);

# Plot the means of x and y
plt.axvline(male_chinstrap.flipper_length_mm.mean(), lw=1, color='g',
    ↪linestyle='dashed')
plt.axhline(male_chinstrap.body_mass_g.mean(), lw=1, color='g',
    ↪linestyle='dashed')

# Add quadrant labels
```

```
plt.text(xmin + (xmax-xmin)*.85, ymin + (ymax-ymin)*.8, "I",
        ↪backgroundcolor="bisque", fontsize="large");
plt.text(xmin + (xmax-xmin)*.15, ymin + (ymax-ymin)*.2, "II",
        ↪backgroundcolor="bisque", fontsize="large");
plt.text(xmin + (xmax-xmin)*.15, ymin + (ymax-ymin)*.8, "III",
        ↪backgroundcolor="bisque", fontsize="large");
plt.text(xmin + (xmax-xmin)*.85, ymin + (ymax-ymin)*.2, "IV",
        ↪backgroundcolor="bisque", fontsize="large");
```



Each observation will contribute to the sum that results in the value of the covariance:

- In quadrant I, both x and y are *larger* than their respective means, so $(x - \bar{x})$ and $(y - \bar{y})$ are both *positive*, resulting in a **positive** term
- In quadrant II, both x and y are *smaller* than their respective means, so $(x - \bar{x})$ and $(y - \bar{y})$ are both *negative*, resulting in a **positive** term
- In quadrant III, however, $(x - \bar{x})$ is *negative* while $(y - \bar{y})$ is *positive*, resulting in a **negative** term
- In quadrant IV, $(x - \bar{x})$ is *positive* while $(y - \bar{y})$ is *negative*, also resulting in a **negative** term

So, all observations in quadrants I and II will make the covariance larger, and observations in quadrants III and IV will make it smaller. If there are a lot of observations in I and II, and fewer in III and IV (which is the case in datasets with an increasing relation), the overall result will be

a positive covariance. A lot of observations in III and IV and fewer in I and II (i.e. a decreasing relation) will result in a negative covariance.

If we calculate the covariance for this dataset, we expect a positive result:

```
[24]: # Calculate covar using the formula
covar = sum((male_chinstrap.flipper_length_mm - male_chinstrap.
    ↪ flipper_length_mm.mean()) *
            (male_chinstrap.body_mass_g - male_chinstrap.body_mass_g.mean())) / ↪
    ↪ (len(male_chinstrap) - 1)

print(f"Cov(x,y) = {covar}")
```

Cov(x,y) = 1835.9681372549019

To calculate covariance, you can use the Numpy function `cov`. It's a bit complicated to use, because it returns not a single value, but a covariance matrix:

```
[[cov(a,a), cov(a,b)],
 [cov(b,a)  cov(b,b)]]
```

Remark that the definition of `cov(a,a)` reduces to the variance of **a**! Can you explain why?

We can extract the value we need with:

```
[25]: np.cov(
    male_chinstrap.flipper_length_mm,
    male_chinstrap.body_mass_g,
    ddof=1)[0][1]
```

```
[25]: np.float64(1835.9681372549019)
```

Remark that we use the parameter `ddof=1` (delta Degrees of Freedom) for the *sample* covariance, which produces an unbiased estimate!

Covariance has a serious limitation as a descriptive measure because it is very sensitive to the units in which X and Y are measured. For example, if we had expressed the body mass of the chinstrap penguins in kg instead of g, the covariance would be ≈ 1.836 instead of ≈ 1836 . This limits the usefulness of covariance as a descriptive measure in itself.

8.4.2 Pearson's product-moment correlation coefficient

The **correlation coefficient** (or, specifically, *Pearson's product-moment correlation coefficient*) is defined as:

$$R = \frac{\text{Cov}(X, Y)}{\sigma_x \sigma_y} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2} \sqrt{\sum (y_i - \bar{y})^2}}$$

R is a measure for the strength of a linear correlation between x and y .

Correlation is a *unitless* quantity that is unaffected by the measurement scale. For example, the correlation is the same regardless of whether the variable X (e.g. height of a person) is measured in millimeters, centimeters, decimeters or meters.

The correlation is not only unaffected by the units of measurement of the two variables, but it is always between -1 and $+1$. The closer it is to either of these two extremes, the closer the points in a scatterplot are to a straight line, either in the negative or positive direction.

On the other hand, if the correlation is close to 0, the scatterplot is typically a ‘cloud’ of points with no apparent relationship. However, although it is not common, it is also *possible* that the points are close to a curve and have a correlation close to 0. This is because correlation is relevant only for measuring **linear** relationships.

```
[26]: # Correlation calculated from covariance
stdx = male_chinstrap.flipper_length_mm.std()
stdy = male_chinstrap.body_mass_g.std()

R1 = covar / (stdx * stdy)
print(f"R {covar:.4f} / ( {stdx:.4f} * {stdy:.4f} ) {R1:.4f}")
```

```
R 1835.9681 / ( 6.8760 * 322.9355 ) 0.8268
```

```
[27]: # Correlation from elaborated formula
xx = male_chinstrap.flipper_length_mm - male_chinstrap.flipper_length_mm.mean()
yy = male_chinstrap.body_mass_g - male_chinstrap.body_mass_g.mean()
R2 = sum(xx * yy) / (np.sqrt(sum(xx ** 2) * sum(yy ** 2)))
print(f"R {R2:.4f}")
```

```
R 0.8268
```

```
[29]: np.corrcoef(
    male_chinstrap.flipper_length_mm,
    male_chinstrap.body_mass_g)
```

```
[29]: array([[1.          , 0.82682447],
            [0.82682447, 1.          ]])
```

```
[28]: # Python function numpy.corrcoef() - returns a matrix, like numpy.cov()
cor = np.corrcoef(
    male_chinstrap.flipper_length_mm,
    male_chinstrap.body_mass_g)[0][1]
print(f"R {cor:.4f}")
```

```
R 0.8268
```

The value of R indicates an increasing and strong linear relation between `flipper_length_mm` and `body_mass_g`.

Remark: It is incorrect to say that a correlation coefficient of .82 indicates a “strong increasing relationship”. The correlation coefficient says nothing about the steepness of the relation ship, but

only how closely the points are located near a straight line. A correlation coefficient of .82 indicates a strong linear relationship, but it could be a very steep or a very flat line.

8.4.3 Coefficient of determination

The **coefficient of determination** R^2 gives the percentage of the variance of the observed values relative to the regression line, i.e. the interpretation of R^2 is ‘the fraction of variation of the dependent variable explained by the regression line’. The closer to 1, the more the variance is explained by the regression line, and the stronger the linear relation between the variables: when the residuals are small, R^2 will be close to 1, but when they are large, R^2 will be close to 0. Graphically, the higher the coefficient of determination, the closer the observations are located near the regression line.

The determination coefficient can be calculated as the square of Pearson’s correlation coefficient:

```
[30]: cor ** 2
```

```
[30]: np.float64(0.6836386990302872)
```

So for this case, about 68% of the variance in the observations is explained by the regression line, which indicates a strong linear relation (see below).

The `LinearModel` class also has a method to calculate the coefficient of determination. We first need to call the `fit()` method and then we can use the method `score()` that returns R^2

```
[31]: chinstrap_x = male_chinstrap.flipper_length_mm.values.reshape(-1,1)
chinstrap_y = male_chinstrap.body_mass_g

families_model = LinearRegression().fit(chinstrap_x, chinstrap_y)
families_model.score(chinstrap_x, chinstrap_y)
```

```
[31]: 0.6836386990302872
```

8.4.4 Interpretation of R and R^2

Depending on the value of $|R|$ (or R^2), you can draw a conclusion about the strength of the linear relation between the two variables:

$abs(R)$	R^2	Explained variance	Linear relation
$< .3$	$< .1$	$< 10\%$	very weak
$.3 - .5$	$.1 - .25$	$10\% - 25\%$	weak
$.5 - .7$	$.25 - .5$	$25\% - 50\%$	moderate
$.7 - .85$	$.5 - .75$	$50\% - 75\%$	strong
$.85 - .95$	$.75 - .9$	$75\% - 90\%$	very strong
$> .95$	$> .9$	$> 90\%$	exceptionally strong

Remark that the value of R doesn’t say anything about the steepness of the regression line! It only indicates how close the observations are to the regression line. Therefore, it is **wrong** to say

that a value of e.g. $R = 0.8$ indicates a strongly increasing linear relation! Instead, you should say it indicates an *increasing and strong linear relation*.

The image below with the correlation coefficient for several data sets illustrates this.

8.4.5 Considerations

- The correlation coefficient only looks at the relationship between *two* variables. Interactions with other variables are not taken into account.
- The correlation coefficient explicitly does **not** assume or prove a **causal** relationship!
- Pearson's correlation coefficient only measures the strength of a **linear** relationship. It is possible that two variables have a strong non-linear relationship, but a correlation close to 0.
- R^2 is easier to interpret than R . The interpretation of R^2 as the square of a correlation helps to clarify the issue of when a correlation is “large”.
 - For example: how much better is $R = 0.8$ than $R = 0.7$ or $R = 0.9$?
 - If the correlation between two variables Y and X is ± 0.8 , the regression of Y on X will have an R^2 of 0.64; that is, the regression with X as the only explanatory variable will explain 64% of the variation in Y .
 - If the correlation drops to ± 0.7 , this percentage drops to 49%;
 - If the correlation increases to ± 0.9 , the percentage increases to 81%.
- The point is that before a single variable X can explain a large percentage of the variation in some other variable Y , the two variables must be highly correlated, in either a positive or negative direction.

Another example: How much better is $R = 0.7$ than $R = 0.5$?

- $R^2 = 0.7 * 0.7 \sim 0.5 \Rightarrow 50\%$ of the original variation in explained
- $R^2 = 0.5 * 0.5 = 0.25 \Rightarrow 25\%$ of the original variation in explained

With R^2 , it is easy to see that the first correlation is twice as good as the second.

```
[ ]: # If you're using Google Colab, you can run this cell to upgrade the required
      ↪dependencies
      !pip install matplotlib==3.4.3
      !pip install --upgrade statsmodels
```

9 Module 7. Time series

9.1 Introduction

Many decision-making processes rely on forecasts of future values, such as sales, demand, or economic indicators. However, forecasting is inherently challenging, both in the short term and in the long term. Most forecasts are based on historical data, where analysts attempt to identify patterns or relationships and then project these into the future.

This approach comes with two major limitations.

First, it is often difficult to distinguish meaningful patterns from noise—that is, random fluctuations in the data. As a result, analysts may sometimes overinterpret randomness and assign significance to patterns that are unlikely to persist.

Second, even when genuine patterns are identified, there is no guarantee that they will continue in the future. External factors—such as economic crises, geopolitical events, technological changes, or shifts in consumer behavior—can disrupt historical trends and render forecasts inaccurate.

9.1.1 Extrapolation Models

Extrapolation models are quantitative methods that use historical time series data to predict future values. The underlying assumption is that past behavior of a variable—such as company sales or exports—contains useful information about its future evolution.

There are several widely used extrapolation techniques, including:

- Moving averages
- Exponential smoothing
- Trend projection models

These models aim to identify patterns in historical data and extend them into the future. Some models focus on capturing long-term upward or downward trends, while others emphasize recurring seasonal patterns. For example, a retailer may observe higher sales during the holiday season and incorporate this pattern into future forecasts.

Although extrapolation models are relatively simple and often effective in stable environments, their performance can deteriorate when structural changes occur in the underlying system.

9.1.2 Components of Time Series Data

A time series typically consists of four main components:

1. Trend Component The trend represents the long-term direction of the data. If observations consistently increase or decrease over time, the series is said to exhibit a trend. Trends can be linear or nonlinear and are often driven by underlying factors such as economic growth, population changes, or technological progress.

2. Seasonal Component The seasonal component captures regular patterns that repeat over a fixed period, such as days, months, or quarters. For example, a company selling swimming pool equipment will typically see increased sales in spring and summer, followed by a decline in autumn and winter. Seasonal patterns are generally predictable and recur consistently over time.

3. Cyclical Component Cyclical patterns refer to fluctuations that occur over longer, irregular periods and are not tied to a fixed calendar. These are often associated with economic cycles, such as expansions and recessions. For instance, during an economic downturn, housing sales may decrease and unemployment may rise, while the opposite occurs during periods of growth.

The key distinction between seasonal and cyclical components is that seasonal patterns have a fixed and known period, whereas cyclical fluctuations vary in length and intensity.

4. Random (Noise) Component The random component, also known as noise, represents unpredictable variations in the data. These fluctuations arise from unforeseen events, measurement errors, or inherently random behavior. This component gives time series data its irregular, jagged appearance and cannot be reliably predicted.

9.1.3 Conclusion

In practice, effective forecasting involves identifying and modeling the trend, seasonal, and cyclical components while acknowledging the presence of randomness. Although no model can perfectly predict the future, understanding these components helps analysts make more informed and robust forecasts.

The figure below shows the decomposition of the pregnancy number time series (top panel) into three components: trend, seasonality, and residuals. [Published in Annual review of public health 2006 On time series analysis of public health and biomedical data. S. Zeger, R. Irizarry, R. Peng]

```
[1]: # Package imports
import numpy as np                # "Scientific computing"
import pandas as pd               # Dataframe

import scipy.stats as stats       # Statistical tests
from sklearn.linear_model import LinearRegression # Linear regression model

import matplotlib.pyplot as plt   # Basic visualisation
from statsmodels.graphics.mosaicplot import mosaic # Mosaic plot
import seaborn as sns             # Advanced dataviz
```

In this notebook, we use the figures on the number of seriously injured victims of road traffic accidents in Belgium. These figures are sourced from [StatBel](#).

Remark that when creating the index for the data frame, we set the frequency to **MS**, i.e. Month Start, since each observation is taken at the first day of the month. A list of valid frequencies ([offset aliases](#)) can be found [in the Pandas documentation](#). Frequently used values are:

Alias	Description
D	calendar day frequency
W	weekly frequency
MS	month start frequency
ME	month end frequency
QS	quarter start frequency
QE	quarter end frequency
YS	year start frequency
YE	year end frequency

```
[ ]: # Read the data from CSV, parse dates and set the index
wounded_data = pd.read_csv(
    'https://raw.githubusercontent.com/HoGentTIN/dsai-labs/refs/heads/main/data/
↳number_of_heavily_wounded_car_accidents_BE.txt',
    delimiter = "\t",
    parse_dates=['date']).set_index(['date'])

# Add frequency info to the index
wounded_data.index = pd.DatetimeIndex(wounded_data.index, freq='MS')

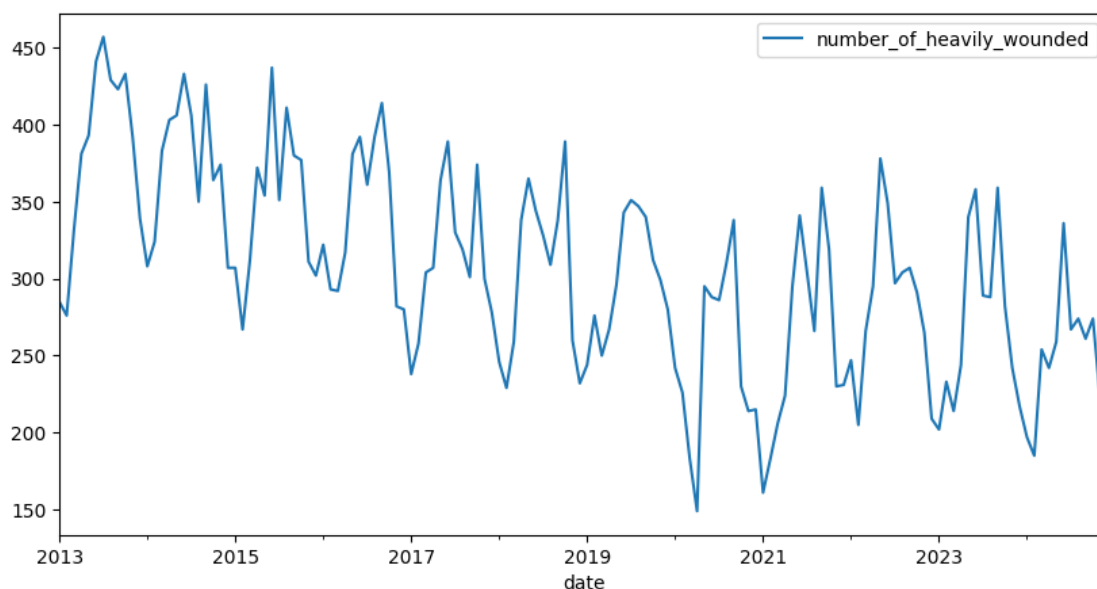
wounded_data.head()
```

```
[ ]:      number_of_heavily_wounded
date
2013-01-01      285
2013-02-01      276
2013-03-01      332
2013-04-01      381
2013-05-01      393
```

In Python, a time series can also be represented by an instance of the Pandas [Series](#) class with an index of type `DatetimeIndex`.

As always, a graph of the time series is a good place to start analyzing the data. It indicates whether a linear trend is likely to provide a good fit. Generally, the graph should rise or fall at approximately a constant rate through time, without too much random (unpredictable) variation.

```
[ ]: wounded_data.plot(y='number_of_heavily_wounded', figsize=(10,5));
```



9.1.4 Interludium

- What is the trend?
- What would you do to ‘calculate’ the exact trend?
- What is the seasonal component?
- Try to predict the number of heavily wounded for january 2025, based on the number of december 2024.
- Try to predict the number of heavily wounded for february 2025, based on the number of december 2024.
- Try to predict the number of heavily wounded for march 2025, based on the number of december 2024.

9.1.5 Overview of the notebook

In this notebook we will study six methods to predict future values based on time series data.

- (1) Horizontal line / Constant model
- (2) Simple linear regression
- (3) Simple moving average
- (4) Simple exponential smoothing
- (5) Double exponential smoothing
- (6) Triple exponential smoothing

Note that numerous other models are available for use in time series analysis!

9.2 Mean absolute error and mean squared error

We need a metric to evaluate how accurate a forecasting model is. To do so, Mean Absolute Error (MAE) and Mean Squared Error (MSE) are often used by measuring the difference between predicted values and actual outcomes.

MAE and MSE help assess model performance and compare different forecasting methods, allowing analysts to choose the most appropriate model for a given problem.

Mean Absolute Error (MAE)

The mean absolute error measures the average of the absolute differences between predicted values and actual values. It indicates how far, on average, the predictions are from the true values, regardless of direction. The MAE is easy to interpret (same units as the data) and is less sensitive to large errors (outliers).

$$MAE = \frac{1}{m} \sum_{i=t+1}^{t+m} |x_i - F_i|$$

Mean Squared Error (MSE)

The mean squared error measures the average of the squared differences between predicted values

and actual values. By squaring the errors, larger mistakes are penalized more heavily. Therefore it is more sensitive to large errors (outliers).

$$MSE = \frac{1}{m} \sum_{i=t+1}^{t+m} (x_i - F_i)^2$$

9.3 (1) Constant Model (Horizontal Line)

We start with the simplest model. In this approach, future values are predicted using nothing more than the average of a selected number of past observations.

In the chart below, we attempt to model the observed values using a constant function:

$$X_t = b + \varepsilon_t,$$

where ε_t represents random noise with constant variance.

As an estimate for b , we take the average of the first observations. The number of observations to include can be chosen based on the analyst's judgment.

9.3.1 Why is this useful?

Despite its simplicity, this model has several advantages:

- It is easy to understand and implement.
- It provides a **baseline model** against which more complex models can be compared.
- It works reasonably well when the data fluctuates around a stable mean (i.e., no clear trend or seasonality).
- It reduces the impact of random fluctuations by averaging past observations.

9.3.2 Why is this often too simple?

In most real-world applications, this model is too limited:

- It **ignores trends** (e.g., increasing sales over time).
- It **does not capture seasonality** (e.g., higher sales in certain months).
- It assumes the future will behave like a constant average, which is rarely realistic.
- It reacts slowly (or not at all) to structural changes in the data.
- The choice of how many past observations to include is subjective and can strongly affect the results.

9.3.3 Conclusion

The constant model is a useful starting point and benchmark, but it is generally insufficient for capturing the complexity of real-world time series data. More advanced models are typically required to account for trends, seasonal patterns, and other dynamics.

9.3.4 Example

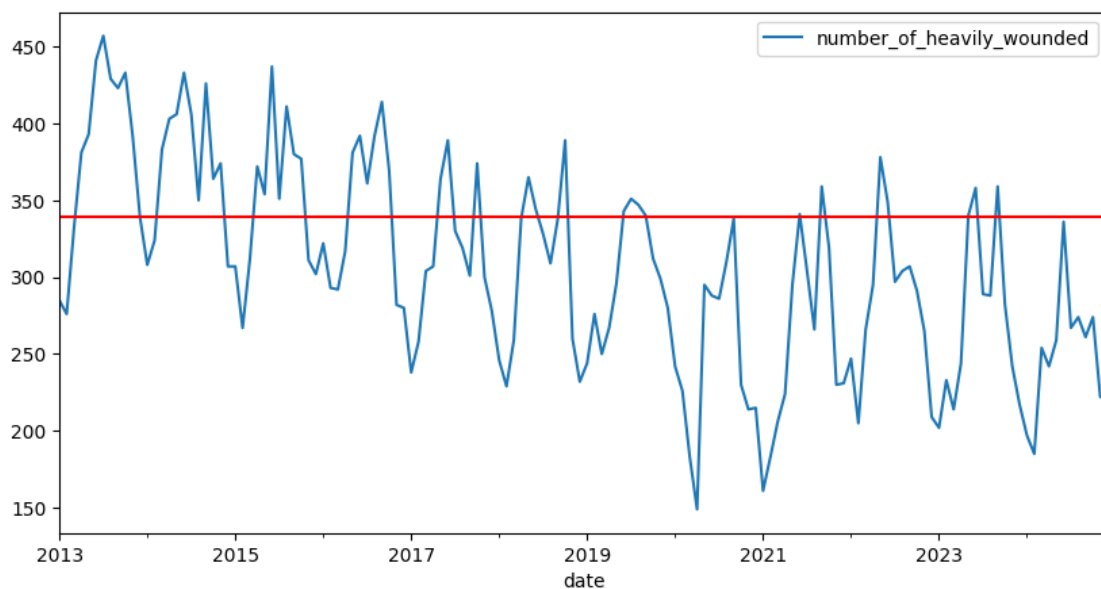
In the example below the first 80 values are used as past observations. Although this model has clear limitations, it remains useful as it provides a rough estimate of the monthly number of people

seriously injured in traffic accidents in Belgium. It indicates that this number is approximately 340 per month, rather than, for example, 1000 or 100.

```
[ ]: # first 80 values are used as a past observations
b_est_80 = wounded_data['number_of_heavily_wounded'][:80].mean()
print(f'Approximation of the number of heavily wounded in Belgian traffic per_
      month = {b_est_80}')

wounded_data.plot(y='number_of_heavily_wounded', figsize=(10,5))
plt.axhline(y=b_est_80, color='red'); # Mean of first 80 observations
```

Approximation of the number of heavily wounded in Belgian traffic per month = 338.925



9.4 Train and test data

As already mentioned, two commonly used metrics for evaluating different forecasting methods are MAE and MSE. In this notebook we will compute the MAE for each of the six methods discussed. To be able to calculate the MAE, we need to split the available data into **training data** and **test data**. When building a machine learning model, this is a very common approach.

The main reason for this split is to evaluate how well the model generalizes to new, unseen data. The model learns patterns from the training set, while the test set is used to provide an unbiased estimate of its performance. Without this separation, a model might simply memorize the data (overfitting) and appear to perform well, even though it fails on new inputs.

In many standard machine learning problems, the data is randomly divided into training and test sets. This works because the observations are assumed to be independent and identically distributed.

However, in the case of time series data, this approach is not appropriate. Time series data has a natural temporal order, and future values should not influence the past. Randomly shuffling the data would break this structure and lead to overly optimistic performance estimates.

Instead, the split is done chronologically. The training set consists of earlier observations, and the test set contains more recent data. The model is trained on historical data and then evaluated on its ability to predict future values. This setup reflects real-world scenarios, where only past information is available at prediction time.

9.4.1 Exercise

1. Create training data, i.e. the all data up to and including 2023
2. Create test data, i.e all data from 2024
3. Calculate the MAE

```
[ ]: from sklearn.metrics import mean_absolute_error, mean_squared_error

train = wounded_data[:'2023-12-01']
test = wounded_data['2024-01-01':]
```

9.5 (2) Linear Trend Model (Simple Linear Regression)

We now consider a slightly more advanced model that allows for a trend over time. Instead of assuming a constant value, we assume that the data evolves linearly.

In this approach, the observed values are modeled using a linear function:

$$X_t = b_0 + b_1 t + \varepsilon_t$$

where:

- b_0 is the intercept, this is less important. It literally represents the expected value of the series at time $t = 0$.
- b_1 represents the slope (trend), it represents the expected change in the series from one period to the next. If b_1 is positive, the trend is upward; if b_1 is negative, the trend is downward.
- ε_t is random noise with constant variance.

The parameters b_0 and b_1 are typically estimated using ordinary least squares, which fits the best straight line through the observed data.

9.5.1 Why is this useful?

This model improves upon the constant model in several ways:

- It captures **trends over time** (increasing or decreasing patterns).
- It is still relatively **simple and easy to interpret**.
- It provides a better approximation when the data shows a clear upward or downward movement.
- It can serve as a **next-step benchmark model** after the constant model.

9.5.2 Why is this still limited?

Despite being more flexible than the constant model, this approach also has important limitations:

- It assumes the trend is **linear**, which is often unrealistic in practice.
- It does not account for **seasonality** (e.g., recurring yearly patterns).
- It ignores **cyclical behavior** and structural changes.
- It may perform poorly if the trend changes over time (e.g., accelerates or slows down).
- It is sensitive to outliers, which can distort the estimated line.

9.5.3 Conclusion

The linear trend model is a natural extension of the constant model and is useful when the data exhibits a clear trend. However, like the constant model, it remains a simplification of reality. In many real-world time series, more advanced models are needed to capture seasonal effects, nonlinear trends, and other complex dynamics.

9.5.4 Example

In the example below, all observations are used to estimate the line of best fit. We first use `numpy.polyfit` to calculate the slope and intercept, afterwards `sklearn` is used. Since `numpy.polyfit` requires numerical x-values and our dataframe `wounded_data` uses dates as an index, we need to convert the time variable into numbers first.

```
[ ]: # Build a linear regression model using np.polyfit
y = wounded_data["number_of_heavily_wounded"].values
x = np.arange(len(y)) # 0,1,2,...

b1, b0 = np.polyfit(x=x, y=y, deg=1)
print(f'Equation of the regression line: y = {b0} + {b1} * t')
```

Equation of the regression line: $y = 375.1980842911878 + -0.962657744554297 * t$

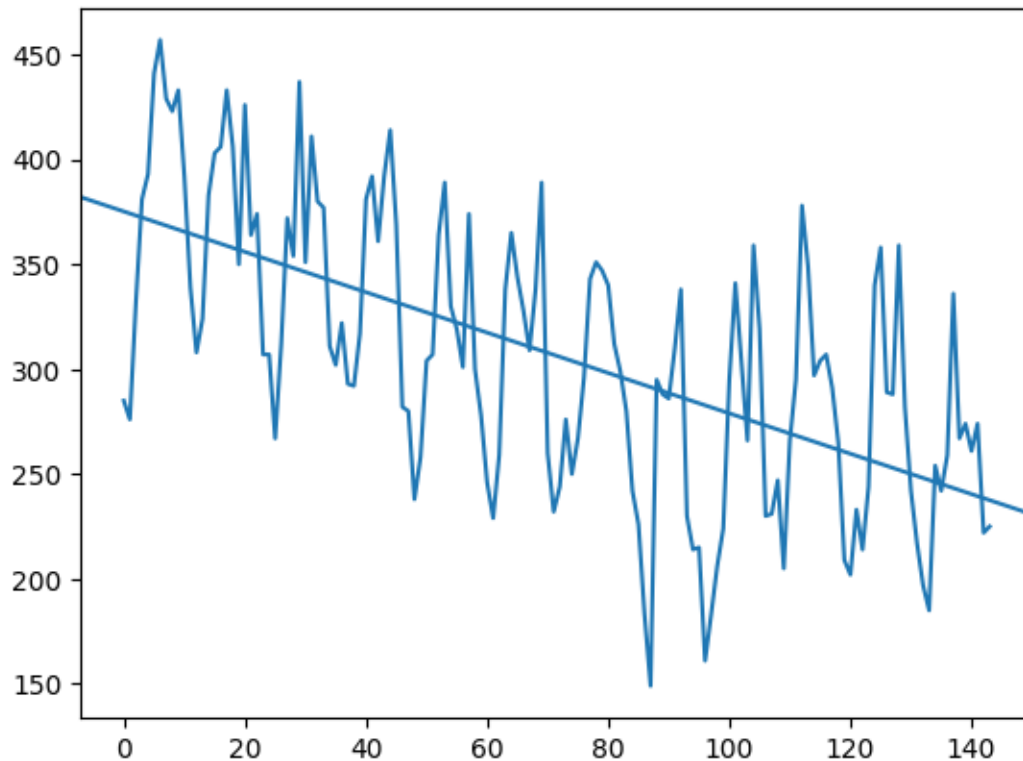
```
[ ]: # Build a linear regression model using sklearn
from sklearn.linear_model import LinearRegression

x = np.arange(len(wounded_data)).reshape(-1, 1)
y = wounded_data['number_of_heavily_wounded'].values

wounded_lrm = LinearRegression().fit(x, y)
b0 = wounded_lrm.intercept_
b1 = wounded_lrm.coef_[0]
print(f'Equation of the regression line: y = {b0} + {b1} * t')
```

Equation of the regression line: $y = 375.19808429118774 + -0.9626577445542965 * t$

```
[ ]: # plot the time series and the regression line
plt.plot(x, y);
plt.axline((0, b0), slope=b1);
```



9.5.5 Exercises

1. What ‘proves’ in the previous model the trend is actually descending?
2. What is the meaning of b_0 in the previous model?
3. What is the meaning of b_1 in the previous model?
4. What is an important shortcoming of the linear regression model?
5. Calculate the MAE
 - Create trainingsdata, i.e. the all data up to and including 2023
 - Create testdata, i.e all data from 2024
 - Calculate the equation of the regression line based on the trainingsdata
 - Using this linear regression model, generate forecasts for the period from January 2024 through December 2024.
 - Calculate the mean absolute error.

```
[ ]: from sklearn.metrics import mean_absolute_error, mean_squared_error

# Build a linear regression model using np.polyfit
train = wounded_data[:'2023-12-01']
test = wounded_data['2024-01-01':]
```

Equation of the regression line: $y = 376.78742310321263 + -0.9964075699310756 * t$

The MAE is 31.61827278118966

9.6 (3) Simple Moving Average (SMA)

9.6.1 Introduction

The available data are given on a monthly basis, and a time window of six months is selected. Suppose the forecast for the next month is then calculated as the average of the values from the previous six months. For example, the average of January through June is used to predict July, the average of February through July is used to predict August, and so on. This rolling procedure gives rise to the term **moving average**.

The choice of the time window (or span) plays a crucial role. If the window is large—say, 12 months—each average is based on many observations, which reduces the influence of extreme values. As a result, the forecast series becomes smoother than the original data.

In contrast, if the window is small—say, three months—each average is based on fewer observations. This makes the forecasts more sensitive to extreme values, resulting in a series that is less smooth and more responsive to short-term fluctuations.

9.6.2 Formula

We basically need a method that takes the last observations into account, but forgets observations after some time. Older observations may no longer be representative of the current situation. Moving averages are such methods.

$$SMA(t) = \frac{1}{m} \sum_{i=k}^t x_i$$

With m the window size (the number of observations taken into account) and $k = t - m + 1$.

9.6.3 Why is this useful?

- It smooths out short-term fluctuations in the data.
- It adapts over time.
- It is easy to understand and implement.
- It works well when the data has no strong trend or seasonality.

9.6.4 Why is this still limited?

- It gives equal weight to all (k) past observations, even though more recent data may be more relevant.
- It does not capture trends (it tends to lag behind increasing or decreasing data).
- It does not account for seasonality.
- The choice of (k) is subjective and can significantly affect the results.

9.6.5 Conclusion

The simple moving average provides a more flexible way to smooth data and make short-term forecasts, but it still has important limitations. More advanced methods are often needed for time series with trends or seasonal patterns.

9.6.6 Example

The Pandas `Series` class has a method `rolling()` for “rolling window” calculations, which is perfectly suitable for what we want to do here.

We call the method `rolling(m)` with the desired time range and then calculate the `mean()`:

```
[ ]: wounded_data['SMA3'] = wounded_data['number_of_heavily_wounded'].rolling(3).
      ↪mean()
wounded_data['SMA5'] = wounded_data['number_of_heavily_wounded'].rolling(5).
      ↪mean()
wounded_data['SMA10'] = wounded_data['number_of_heavily_wounded'].rolling(10).
      ↪mean()
wounded_data.head(15)
```

```
[ ]:      number_of_heavily_wounded      SMA3      SMA5      SMA10
date
2013-01-01      285          NaN          NaN          NaN
2013-02-01      276          NaN          NaN          NaN
2013-03-01      332    297.666667          NaN          NaN
2013-04-01      381    329.666667          NaN          NaN
2013-05-01      393    368.666667    333.4          NaN
2013-06-01      441    405.000000    364.6          NaN
2013-07-01      457    430.333333    400.8          NaN
2013-08-01      429    442.333333    420.2          NaN
2013-09-01      423    436.333333    428.6          NaN
2013-10-01      433    428.333333    436.6    385.0
2013-11-01      392    416.000000    426.8    395.7
2013-12-01      339    388.000000    403.2    402.0
2014-01-01      308    346.333333    379.0    399.6
2014-02-01      324    323.666667    359.2    393.9
2014-03-01      383    338.333333    349.2    392.9
```

You’ll notice that the first data points in the SMA columns are empty (NaN). At those time slots there are still not enough past observations to calculate the mean. The larger the time window, the more empty data points.

If you want to avoid empty cells, you can add the parameter `min_periods=1` to the method call of `rolling()`. In that case the mean will be calculated with any available past observation.

When we compute a simple moving average using:

```
wounded_data['SMA3'] = wounded_data['number_of_heavily_wounded'].rolling(3).mean()
```

the value at time t is calculated using the current observation and the previous ones, e.g. X_{t-2} , X_{t-1} , X_t .

This means the moving average at time t already includes the actual value at time t .

When making predictions, we should only use past information, not the current or future values. Otherwise, we are using information that would not be available in a real forecasting setting (this is called data leakage).

To fix this, we shift the moving average:

```
wounded_data['SMA3'] = wounded_data['number_of_heavily_wounded'].rolling(3).mean().shift(1)
```

What does `.shift(1)` do?

It moves the moving average one step forward in time. The value at time t now depends only on $X_{t-3}, X_{t-2}, X_{t-1}$.

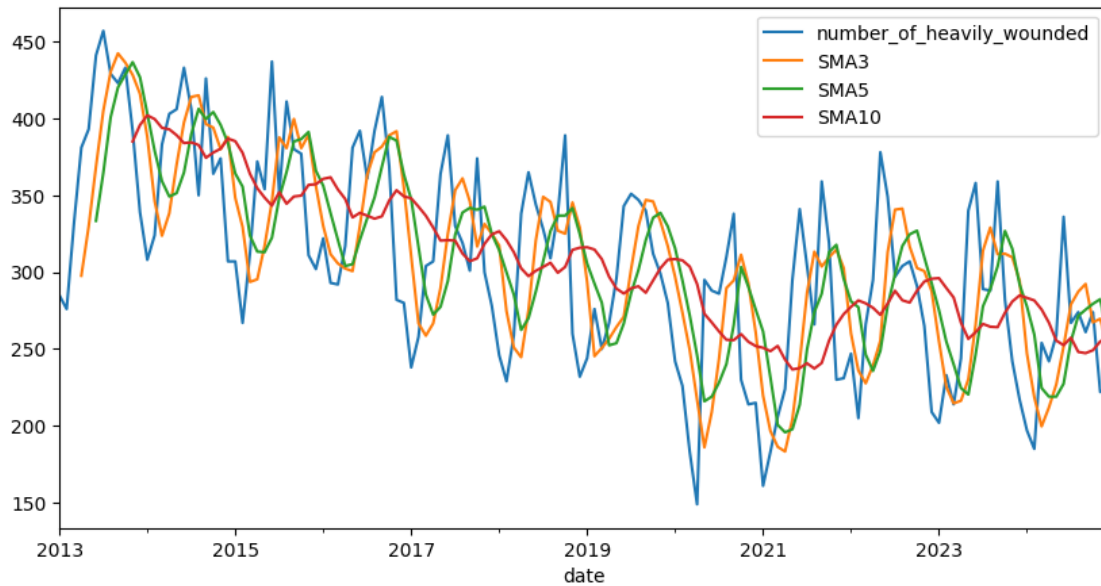
This makes it a valid predictor for time t .

So we replace the previous Python code by the following Python code.

```
[ ]: wounded_data['SMA3'] = wounded_data['number_of_heavily_wounded'].rolling(3).
      ↪mean().shift(1)
wounded_data['SMA5'] = wounded_data['number_of_heavily_wounded'].rolling(5).
      ↪mean().shift(1)
wounded_data['SMA10'] = wounded_data['number_of_heavily_wounded'].rolling(10).
      ↪mean().shift()
wounded_data.head(12)
```

```
[ ]:
      number_of_heavily_wounded      SMA3      SMA5      SMA10
date
2013-01-01      285           NaN      NaN      NaN
2013-02-01      276           NaN      NaN      NaN
2013-03-01      332           NaN      NaN      NaN
2013-04-01      381      297.666667      NaN      NaN
2013-05-01      393      329.666667      NaN      NaN
2013-06-01      441      368.666667      333.4      NaN
2013-07-01      457      405.000000      364.6      NaN
2013-08-01      429      430.333333      400.8      NaN
2013-09-01      423      442.333333      420.2      NaN
2013-10-01      433      436.333333      428.6      NaN
2013-11-01      392      428.333333      436.6      385.0
2013-12-01      339      416.000000      426.8      395.7
```

```
[ ]: wounded_data.plot(
      y=['number_of_heavily_wounded', 'SMA3',      'SMA5',      'SMA10'],
      figsize=(10,5));
```



As you will notice, SMA's with a short window will still follow fluctuations in the observations, while those with a longer window are more “stable”, and resilient to short-term fluctuations.

As mentioned before, there are multiple criticisms to the moving average method. First, it puts equal weight on each value in a typical moving average. But one can argue that if the next month's forecast should be based on the observations of the past 12 months, more weight should be placed on the more recent observations. The second criticism is that the moving averages method requires a lot of data storage.

9.6.7 Exercise

Calculate the MAE for the values of 2024 in case of SMA3, SMA5, SMA10.
Do these models perform better than simple linear regression?

```
[ ]: y_true = wounded_data['number_of_heavily_wounded'][-12:].values
```

9.7 (4) Simple Exponential Smoothing (SES)

We now consider Simple Exponential Smoothing, which is a more advanced model than the simple moving average.

9.7.1 Why is this useful?

- It gives **more weight to recent observations**, making it more responsive to changes.
- It requires **very little data storage** (only the last forecast and observation).
- It is relatively **simple to understand and implement**.
- It is widely used in business, especially when many forecasts need to be updated frequently and automatically.

9.7.2 Why is this better than Simple Moving Average?

- It does not treat all past observations equally.
- It reacts faster to recent changes in the data.
- It avoids the need to store a fixed window of past values.

9.7.3 When should we use it?

Simple Exponential Smoothing is appropriate when: - the data has **no clear trend**, and - there is **no seasonality**.

9.7.4 Limitations

- It does not capture **trends** or **seasonal patterns**.
- The choice of α can strongly influence the results.
- It may lag behind rapid changes if α is too small.

9.7.5 How does it work?

In Simple Exponential Smoothing, each forecast is calculated as a weighted average of past observations, where the weights decrease exponentially over time.

Every exponential model has at least one smoothing constant, which is always a number between 0 and 1. Simple exponential smoothing has a single smoothing constant denoted by α . The second new term is L_t , called the level of level the series at time t . This value is not observable but can only be estimated. Essentially, it is an estimate of where the series would be at time t if there were no random noise. Then the simple exponential smoothing method is defined by the following two equations:

Level:

$$L_t = \alpha Y_t + (1 - \alpha)L_{t-1}$$

Forecast: $F_{t+k} = L_t$

where:

- α is the smoothing parameter (between 0 and 1)
- Y_t is the most recent observation
- L_{t-1} is the previous forecast

Even though you usually don't have to substitute into these equations manually, you should understand what they say.

The first of both equation shows how to update the estimate of the level. It is a weighted average of the current observation, Y_t and the previous level, L_{t-1} , with respective weights α and $(1 - \alpha)$.

The second equation shows how forecasts are made. It says that the k -period-ahead forecast, F_{t+k} , made of Y_{t+k} in period t is the most recently estimated level, L_t . This is the same for any value of $k \geq 1$. The idea is that in simple exponential smoothing, you believe that the series is not really going anywhere. So as soon as you estimate where the series ought to be in period t (if it weren't for random noise), you use this as the forecast for any future period.

The smoothing constant α is analogous to the span in moving averages. By performing some algebra, you can verify that L_t satisfies the following equation, where sum extends back to the first

observation at time = 1.

$$L_t = \alpha Y_t + \alpha(1 - \alpha)Y_{t-1} + \alpha(1 - \alpha)^2 Y_{t-2} + \alpha(1 - \alpha)^3 Y_{t-3} + \dots$$

This equation shows how the exponentially smoothed forecast is a weighted average of previous observations. Furthermore, because $1 - \alpha$ is less than 1, the weights on the Y 's decrease from time t backward.

If α is close to 0, then $1 - \alpha$ is close to 1 and the weights decrease very slowly. In other words, observations from the distant past continue to have a large influence on the next forecast. This means that the graph of the forecasts will be relatively smooth, just as with a large span in the moving averages method. But if α is close to 1, the weights decrease rapidly, and only very recent observations have much influence on the next forecast. In this case forecasts react quickly to sudden changes in the series.

9.7.6 Example

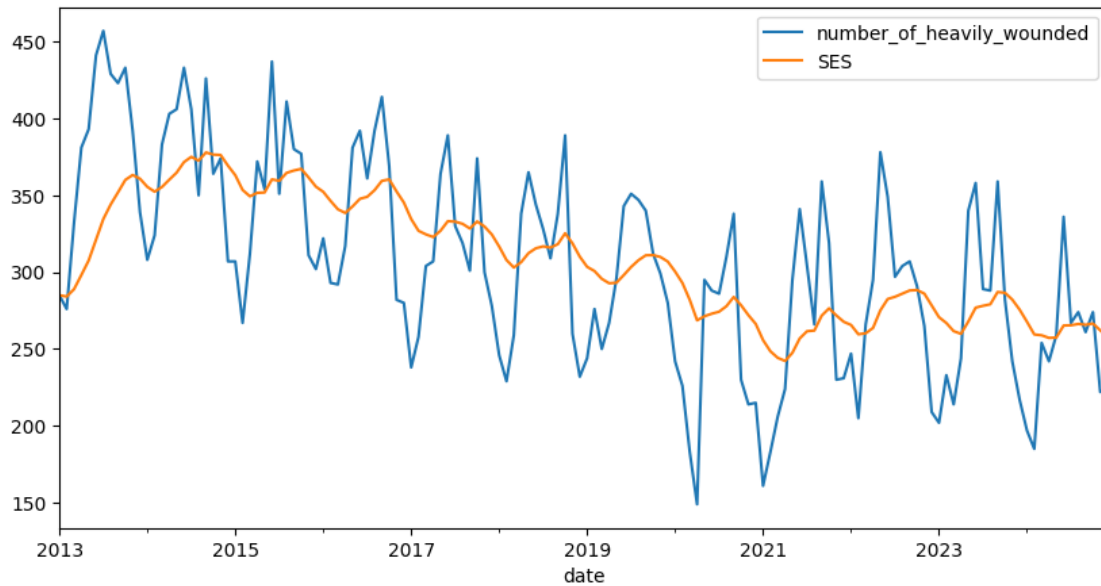
```
[ ]: # We use the library statsmodels!
from statsmodels.tsa.holtwinters import SimpleExpSmoothing

# Train the model
data_ses = SimpleExpSmoothing(wounded_data['number_of_heavily_wounded']).
    ↪fit(smoothing_level=0.1, optimized=False)

# Add fitted values to the dataframe
wounded_data['SES'] = data_ses.level
wounded_data.head(10)
```

```
[ ]:      number_of_heavily_wounded      SMA3      SMA5      SMA10      SES
date
2013-01-01      285      NaN      NaN      NaN      285.000000
2013-02-01      276      NaN      NaN      NaN      284.100000
2013-03-01      332      NaN      NaN      NaN      288.890000
2013-04-01      381      297.666667      NaN      NaN      298.101000
2013-05-01      393      329.666667      NaN      NaN      307.590900
2013-06-01      441      368.666667      333.4      NaN      320.931810
2013-07-01      457      405.000000      364.6      NaN      334.538629
2013-08-01      429      430.333333      400.8      NaN      343.984766
2013-09-01      423      442.333333      420.2      NaN      351.886289
2013-10-01      433      436.333333      428.6      NaN      359.997661
```

```
[ ]: wounded_data.plot(y=['number_of_heavily_wounded', 'SES'], figsize=(10,5));
```



We can retrieve the automatically calculated parameters of the model: use `. params_formatted` to obtain a dataframe containing all parameters.

```
[ ]: data_ses.params_formatted
```

```
[ ]:
      name  param  optimized
smoothing_level  alpha    0.1    False
initial_level    1.0  285.0    False
```

9.7.7 Exercise

Use these parameters to verify that the values in the SES column are correct.

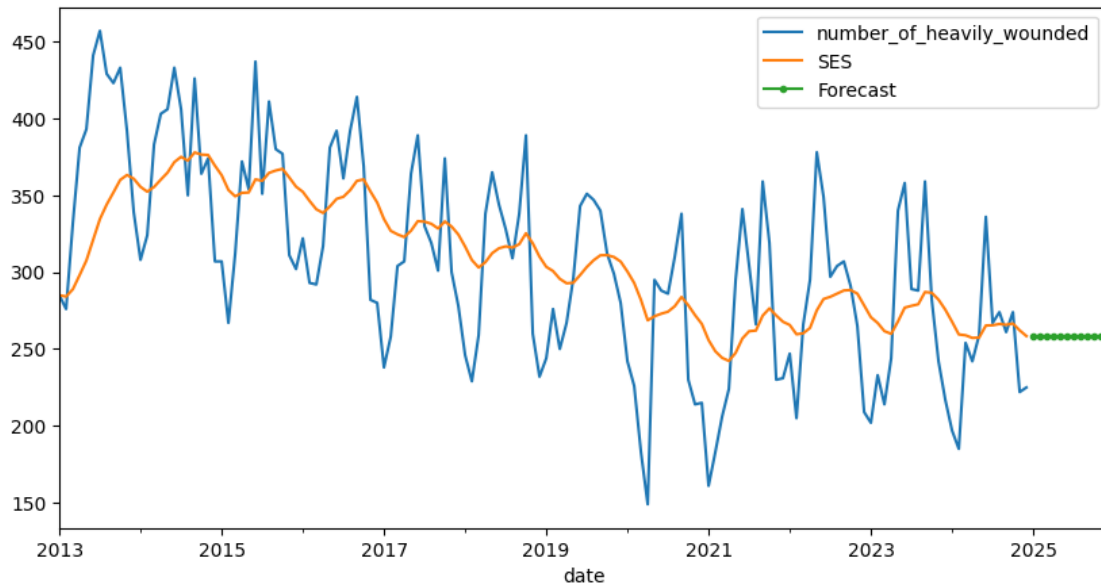
```
[ ]:
```

.forecast

The statsmodels classes allow us to do a prediction based on the model:

```
[ ]: data_ses_fcast = data_ses.forecast(12)

wounded_data.plot(y=['number_of_heavily_wounded', 'SES'], figsize=(10,5))
data_ses_fcast.plot(marker='.', legend=True, label='Forecast');
```



If we look back at the formulae of Single Exponential Smoothing, the forecasted value for any time of $k \geq 1$ is the *level* of the series at time t :

$$L_t = \alpha Y_t + (1 - \alpha) L_{t-1}$$

$$F_{t+k} = L_t$$

If we take a look at the plot, this looks correct, but let's check if this is actually the case using the data coming out of the model.

First we retrieve the last level's L_t of the model

```
[ ]: # data_ses.level[-1] is deprecated. To access a value by position, use `ser.
      ↪iloc[pos]`
data_ses.level.iloc[-1]
```

```
[ ]: np.float64(258.41826050561957)
```

Now we print out the forecasted values:

```
[ ]: data_ses_fcast.values
```

```
[ ]: array([258.41826051, 258.41826051, 258.41826051, 258.41826051,
          258.41826051, 258.41826051, 258.41826051, 258.41826051,
          258.41826051, 258.41826051, 258.41826051, 258.41826051])
```

We can see that the last value of `data_ses.level` is indeed the same as all the forecasted values for $k \geq 1$.

When we use Single Exponential Smoothing, we assume that the underlying time series fluctuates around some constant value. This is what is reflected by the forecasted values (in green), which are always the same.

9.7.8 Exercise

As in the previous paragraphs, calculate the MAE assuming the trainingsdata is all the available data upto and including 2023 and the test data is the data from 2024. Be aware which value to use as prediction for 2024!

```
[ ]: y_true = wounded_data['number_of_heavily_wounded'][-12:].values
```

.fittedvalues

Sometimes we want to compare the actual, observed value to the value that was predicted by the model. In the previous example, we saw that the value L_t at time t is the model's prediction for the actual value at time $t+1$. So, to calculate the difference between the actual, observed value (at time $t+1$) and the predicted value (at time t), we would need to shift the column `data['SES']` one down and make the difference between the columns `data['number_of_heavily_wounded']` and `data['SES']`. This seems quite a burden. Luckily `.fittedvalues` is there to come to the rescue: it contains the value predicted by the model at time t .

```
[ ]: wounded_data['SES_predicted_value'] = data_ses.fittedvalues
wounded_data.tail()
```

```
[ ]:
      number_of_heavily_wounded      SMA3      SMA5      SMA10      SES \
date
2024-08-01      274  287.333333  271.6  248.1  266.296846
2024-09-01      261  292.333333  275.6  247.3  265.767161
2024-10-01      274  267.333333  279.4  249.2  266.590445
2024-11-01      222  269.666667  282.4  254.9  262.131401
2024-12-01      225  252.333333  259.6  257.4  258.418261

      SES_predicted_value
date
2024-08-01      265.440940
2024-09-01      266.296846
2024-10-01      265.767161
2024-11-01      266.590445
2024-12-01      262.131401
```

Finally we can also retrieve the automatically calculated parameters of the model: use `.params_formatted` to obtain a dataframe containing all parameters:

9.8 (5) Double exponential smoothing (Holt's Method)

We now consider a more advanced model that can handle trends in the data. Unlike simple exponential smoothing, this method does not assume that the data fluctuates around a constant level. Instead, it allows the series to increase or decrease over time.

In this approach, we model both the **level** and the **trend** of the series:

Level:

$$L_t = \alpha Y_t + (1 - \alpha)(L_{t-1} + T_{t-1})$$

Trend:

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

Forecast:

$$F_{t+k} = L_t + kT_t$$

where:

- α controls the smoothing of the level (between 0 and 1)
- β controls the smoothing of the trend (between 0 and 1)
- L_t is the level at time t
- T_t is the trend at time t

These equations are not as bad as they look. (And don't forget that the Python does all of the calculations for you.)

The first equation says that the updated level is a weighted average of the current observation and the previous level plus the estimated change.

The second equation says that the updated trend is a weighted average of the difference between two consecutive levels and the previous trend.

The third equation says that the k -period-ahead forecast made in period t is the estimated level plus k times the estimated change per period.

Everything that was said about α for single exponential smoothing applies to both α and β in Holt's model. The new smoothing constant β controls how quickly the method reacts to observed changes in the trend. If β is small, the method reacts slowly. If it is large, the method reacts more quickly.

9.8.1 How does it work (intuitively)?

- The model looks at the most recent observation.
- It adjusts its estimate of the current level.
- It updates its estimate of the trend (is the data going up or down?).
- It combines both to make a forecast.

So instead of predicting a constant value, the model predicts a value that follows the direction of the data.

9.8.2 Why is this useful?

- It captures **trends** in the data (increasing or decreasing patterns).
- It is more flexible than simple exponential smoothing.
- It still requires relatively **little data storage**.
- It produces more accurate forecasts when a trend is present.

9.8.3 Why is this better than Simple Exponential Smoothing?

- It explicitly models the **trend component**.
- It adapts when the data is steadily increasing or decreasing.
- It provides better forecasts for data with a clear direction over time.

9.8.4 When should we use it?

Double exponential smoothing is appropriate when: - the data shows a **trend**, and - there is **no strong seasonality**.

9.8.5 Limitations

- It does not capture **seasonal patterns**.
- The choice of α and β can strongly influence the results.
- It may perform poorly if the trend changes rapidly.

9.8.6 Conclusion

Double exponential smoothing extends simple exponential smoothing by including a trend component. It is a practical and widely used method when data exhibits a clear upward or downward trend, but it remains limited when seasonal patterns are present.

9.8.7 Example

In Python, you can instantiate the class `Holt` and call the `fit()` method. The smoothing parameters α and β can be set with arguments `smoothing_level` and `smoothing_trend`, respectively. If you don't specify these, suitable values will be calculated using some rule of thumb.

```
[ ]: from statsmodels.tsa.api import Holt

data_des = Holt(wounded_data['number_of_heavily_wounded']).fit(smoothing_level=.
    ↪1, smoothing_trend=.2, optimized=False)

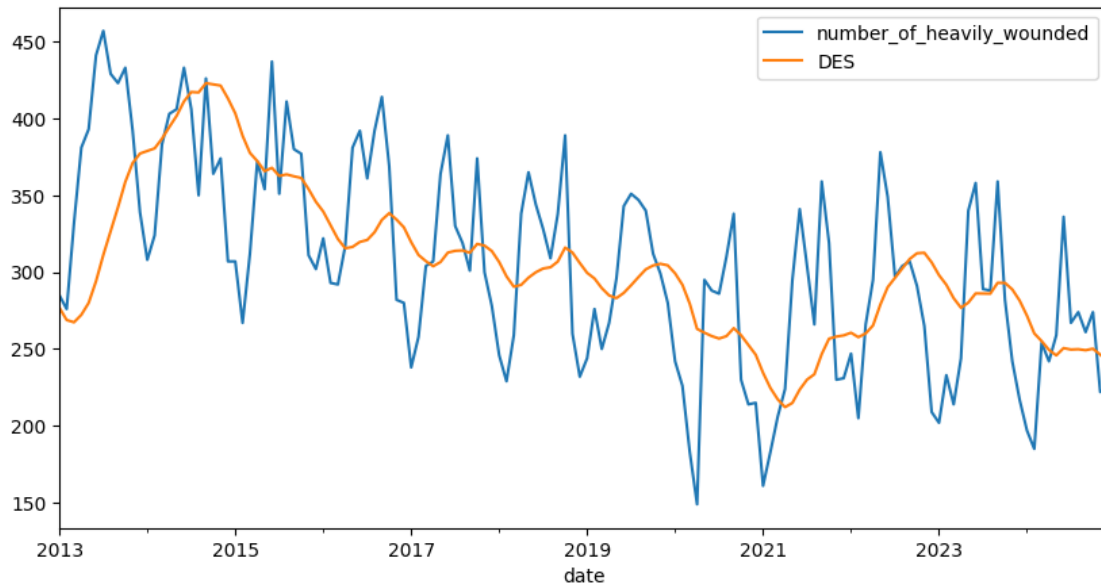
wounded_data['DES'] = data_des.level
wounded_data.head()
```

```
[ ]:
```

	number_of_heavily_wounded	SMA3	SMA5	SMA10	SES \
date					
2013-01-01	285	NaN	NaN	NaN	285.0000
2013-02-01	276	NaN	NaN	NaN	284.1000
2013-03-01	332	NaN	NaN	NaN	288.8900
2013-04-01	381	297.666667	NaN	NaN	298.1010
2013-05-01	393	329.666667	NaN	NaN	307.5909

	SES_predicted_value	DES
date		
2013-01-01	285.000	276.900000
2013-02-01	285.000	268.872000
2013-03-01	284.100	267.389360
2013-04-01	288.890	272.247197
2013-05-01	298.101	279.994306

```
[ ]: wounded_data.plot(y=['number_of_heavily_wounded', 'DES'], figsize=(10,5));
```



We can retrieve the parameters of the model (smoothing parameters, initial estimate of constant term and trend):

```
[ ]: data_des.params_formatted
```

```
[ ]:
```

	name	param	optimized
smoothing_level	alpha	0.1	False
smoothing_trend	beta	0.2	False
initial_level	l.0	285.0	False
initial_trend	b.0	-9.0	False

```
[ ]: data_des.level
```

```
[ ]: date
```

2013-01-01	276.900000
2013-02-01	268.872000
2013-03-01	267.389360
2013-04-01	272.247197
2013-05-01	279.994306
...	
2024-08-01	249.858626
2024-09-01	249.228868
2024-10-01	250.197509
2024-11-01	246.345335
2024-12-01	242.691473

Freq: MS, Length: 144, dtype: float64

```
[ ]: data_des.trend
```

```
[ ]: date
2013-01-01    -8.820000
2013-02-01    -8.661600
2013-03-01    -7.225808
2013-04-01    -4.809079
2013-05-01    -2.297841
...
2024-08-01    -1.937661
2024-09-01    -1.676080
2024-10-01    -1.147136
2024-11-01    -1.688144
2024-12-01    -2.081287
Freq: MS, Length: 144, dtype: float64
```

9.8.8 Exercise

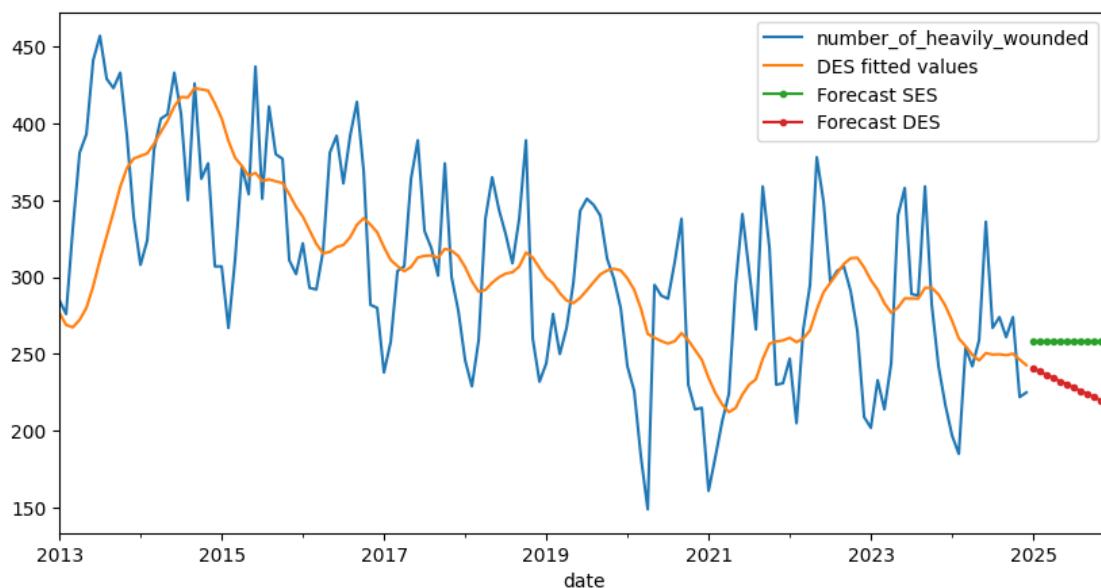
Use these parameters to verify that the values in the DES column are correct.

```
[ ]:
```

To use this model for forecasting:

```
[ ]: data_des_fcast = data_des.forecast(12)

# Plot observations, fitted values and forecast
wounded_data['number_of_heavily_wounded'].plot(legend=True)
wounded_data['DES'].plot(legend=True, label='DES fitted values',
    figsize=(10,5));
data_ses_fcast.plot(marker='.', legend=True, label='Forecast SES');
data_des_fcast.plot(marker='.', legend=True, label='Forecast DES');
```



As we can see, the predictions are now:

```
[ ]: data_des_fcast
[ ]: 2025-01-01    240.610185
      2025-02-01    238.528898
      2025-03-01    236.447610
      2025-04-01    234.366323
      2025-05-01    232.285035
      2025-06-01    230.203748
      2025-07-01    228.122461
      2025-08-01    226.041173
      2025-09-01    223.959886
      2025-10-01    221.878598
      2025-11-01    219.797311
      2025-12-01    217.716023
      Freq: MS, dtype: float64
```

The formula for predicting values, looks like this:

$$F_{t+k} = L_t + kT_t$$

Take a look at the values of the level's L_t and the trend T_t , to understand the predicted values, as are printed here before. * Explain the forecasted value of 2025-01-01 * Explain the forecasted value of 2025-02-01 * ...

```
[ ]: data_des.level.iloc[-1]
[ ]: np.float64(242.6914726599233)
[ ]: data_des.trend.iloc[-1]
[ ]: np.float64(-2.081287440717265)
[ ]: # predictions
      data_des.level.iloc[-1] + data_des.trend.iloc[-1] * 2
[ ]: np.float64(238.52889777848878)
```

Again, we can use `.fittedvalues` to compare the predicted value at time t to the actual, observed value at time t .

```
[ ]: wounded_data['DES_fittedvalues'] = data_des.fittedvalues
      wounded_data.tail()

[ ]:      number_of_heavily_wounded      SMA3      SMA5      SMA10      SES \
      date
      2024-08-01      274      287.333333      271.6      248.1      266.296846
```

2024-09-01	261	292.333333	275.6	247.3	265.767161
2024-10-01	274	267.333333	279.4	249.2	266.590445
2024-11-01	222	269.666667	282.4	254.9	262.131401
2024-12-01	225	252.333333	259.6	257.4	258.418261

	SES_predicted_value	DES	DES_fittedvalues
date			
2024-08-01	265.440940	249.858626	247.176251
2024-09-01	266.296846	249.228868	247.920964
2024-10-01	265.767161	250.197509	247.552788
2024-11-01	266.590445	246.345335	249.050373
2024-12-01	262.131401	242.691473	244.657192

9.8.9 Exercise

As in the previous paragraphs, calculate the MAE assuming the trainingsdata is all the available data upto and including 2023 and the test data is the data from 2024. Be aware which values to use as prediction for 2024!

```
[ ]: y_true = wounded_data['number_of_heavily_wounded'][-12:].values
```

9.9 (6) Triple Exponential Smoothing (Holt - Winters method)

We now consider a model that can handle both trends and seasonality in the data. Unlike the previous methods, this model assumes that the data may follow a repeating pattern over time (for example, monthly or yearly cycles), in addition to increasing or decreasing.

In this approach, the model keeps track of three components: - the **level** (the current value of the series), - the **trend** (the direction of change), - the **seasonal component** (the repeating pattern).

By combining these three elements, the model can produce forecasts that follow both the long-term direction and the recurring seasonal behavior of the data.

How can you tell whether there is seasonality in a time series? The easiest way is to check whether a graph of the time series has a regular pattern of ups and/or downs in particular months or quarters.

In an **additive seasonal model**, an appropriate seasonal index is added to a base forecast.

In a **multiplicative seasonal model**, a base forecast is multiplied by an appropriate seasonal index.

9.9.1 Formulas for Holt-Winters with additive seasonal component

Level:

$$L_t = \alpha(Y_t - S_{t-m}) + (1 - \alpha)(L_{t-1} + T_{t-1})$$

Trend:

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

Seasonal component:

$$S_t = \gamma(Y_t - L_t) + (1 - \gamma)S_{t-m}$$

Forecast for k time units after the last observation:

$$F_{t+k} = L_t + kT_t + S_{t-m+k}$$

In these formulas, the following additional notation is used:

- m is the length (in number of time units) of the seasonal cycle. Common values are 7 for a weekly cycle, 4 for a quarterly one, 12 for a yearly cycle, etc.
- $0 < \gamma < 1$ is the smoothing parameter for the seasonal trend (like α and β). This new smoothing constant controls how quickly the method reacts to observed changes in the seasonality pattern. If γ is small, the method reacts slowly. If it is large, the method reacts more quickly. Set this value in the `fit()` method using argument `smoothing_seasonal`.
- S_t are estimates for the seasonal correction factors, one for each time unit within the seasonal cycle.

To understand how the forecast equation works in the additive Holt-Winters model, suppose you have observed data through June and you want to forecast September, which is three periods ahead. (Here, t refers to June and $t+k=t+3$ refers to September.)

The method starts by taking the current level and adding 3 times the current trend. This gives a baseline forecast for September that represents where the series would be if there were no seasonal effects.

Next, the method adds the seasonal component: it adds the most recent estimate of the seasonal effect for September (typically taken from the previous September, i.e., S_{t-m+3}).

So, the final forecast for September is obtained by: * projecting the level forward using the trend, and * then adding the appropriate seasonal adjustment.

This additive structure is appropriate when the seasonal fluctuations remain roughly constant in size, regardless of the level of the time series.

Note that in the forecast formula of the additive Holt-Winters model, you need to make sure that you use the correct seasonal index. Since the seasonal component repeats in cycles, you must “loop back” through the seasons when forecasting further ahead.

So, continuing from the previous example (forecasting September based on June), if you forecast further, for example 15 months ahead, you “wrap around” the seasons:

- 15 months ahead of June is again September
- So you still use the September seasonal index

9.9.2 Intuitive Explanation

Think of this model as combining three ideas:

1. **Where are we now?** → the current level
2. **Where are we going?** → the trend
3. **What time of the cycle is it?** → the seasonal pattern

At each step: - the model updates its estimate of the current level, - it adjusts the trend (is the data going up or down?), - and it accounts for the position within the seasonal cycle (e.g., winter vs. summer).

For example, if sales are generally increasing over time but are always higher in December, the model will: - follow the upward trend, and
- add a seasonal boost for December.

9.9.3 Why is this useful?

- It captures **trend and seasonality at the same time**.
- It produces more accurate forecasts for data with repeating patterns.
- It adapts over time as new data becomes available.
- It is widely used in practice for business and economic time series.

9.9.4 When should we use it?

Triple exponential smoothing is appropriate when: - the data shows a **trend**, and
- there is a **clear seasonal pattern** (e.g., monthly or yearly cycles).

9.9.5 Limitations

- It is more complex than previous methods.
- It requires choosing several parameters (for level, trend, and seasonality).
- It may perform poorly if the seasonal pattern changes significantly over time.

9.9.6 Conclusion

This model predicts future values by combining the current level, the direction of change, and repeating seasonal patterns.

9.9.7 Example

```
[ ]: from statsmodels.tsa.holtwinters import ExponentialSmoothing

data_tes = ExponentialSmoothing(train, trend='add', seasonal='add',
    ↪seasonal_periods=12, freq='MS').fit()

wounded_data['TES'] = data_tes.level
# Notice there is NaN for 2024 in column TES!
wounded_data.tail(24)
```

```
[ ]:      number_of_heavily_wounded      SMA3      SMA5      SMA10      SES \
date
2023-01-01      202      255.000000      275.2      296.1      270.693526
2023-02-01      233      225.333333      254.8      289.7      266.924173
2023-03-01      214      214.666667      240.0      283.5      261.631756
2023-04-01      244      216.333333      224.6      267.1      259.868580
2023-05-01      340      230.333333      220.4      256.6      267.881722
```

2023-06-01	358	266.000000	246.6	260.9	276.893550
2023-07-01	289	314.000000	277.8	266.3	278.104195
2023-08-01	288	329.000000	289.0	264.5	279.093775
2023-09-01	359	311.666667	303.8	264.2	287.084398
2023-10-01	282	312.000000	326.8	273.6	286.575958
2023-11-01	242	309.666667	315.2	280.9	282.118362
2023-12-01	217	294.333333	292.0	284.9	275.606526
2024-01-01	197	247.000000	277.6	283.3	267.745873
2024-02-01	185	218.666667	259.4	281.6	259.471286
2024-03-01	254	199.666667	224.6	275.7	258.924158
2024-04-01	242	212.000000	219.0	267.1	257.231742
2024-05-01	259	227.000000	219.0	255.5	257.408568
2024-06-01	336	251.666667	227.4	252.5	265.267711
2024-07-01	267	279.000000	255.2	257.3	265.440940
2024-08-01	274	287.333333	271.6	248.1	266.296846
2024-09-01	261	292.333333	275.6	247.3	265.767161
2024-10-01	274	267.333333	279.4	249.2	266.590445
2024-11-01	222	269.666667	282.4	254.9	262.131401
2024-12-01	225	252.333333	259.6	257.4	258.418261

	SES_predicted_value	DES	DES_fittedvalues	TES
date				
2023-01-01	278.326140	298.067357	308.741508	268.757426
2023-02-01	270.693526	291.793749	298.326387	274.263239
2023-03-01	266.924173	283.071626	290.746251	269.935869
2023-04-01	261.631756	276.840284	280.489204	267.011844
2023-05-01	259.868580	280.175270	273.528077	273.917073
2023-06-01	267.881722	286.173251	278.192502	278.219678
2023-07-01	276.893550	286.107970	285.786634	273.638922
2023-08-01	278.104195	286.007058	285.785620	270.322208
2023-09-01	279.093775	293.056095	285.728995	277.594313
2023-10-01	287.084398	293.019107	294.243452	271.530767
2023-11-01	286.575958	288.765436	293.961595	269.112245
2023-12-01	282.118362	281.501823	288.668692	267.112821
2024-01-01	275.606526	271.674534	279.971705	NaN
2024-02-01	267.745873	260.136484	268.484983	NaN
2024-03-01	259.471286	255.149510	255.277233	NaN
2024-04-01	258.924158	249.438242	250.264714	NaN
2024-05-01	257.231742	245.849337	244.388152	NaN
2024-06-01	257.408568	250.582335	241.091483	NaN
2024-07-01	265.267711	249.650387	247.722652	NaN
2024-08-01	265.440940	249.858626	247.176251	NaN
2024-09-01	266.296846	249.228868	247.920964	NaN
2024-10-01	265.767161	250.197509	247.552788	NaN
2024-11-01	266.590445	246.345335	249.050373	NaN
2024-12-01	262.131401	242.691473	244.657192	NaN

Again, we can retrieve the internals of the model. These are the relevant attributes:

- `level` contains all level estimates (L_i) for each time unit in the training data
- `trend` contains all trend estimates (T_i)
- `season` contains all estimates for the seasonal adjustment factors. (S_i)
- `params` (dict output) or `params_formatted` (table output) contains all parameters used to bootstrap the model (including automatically calculated ones).

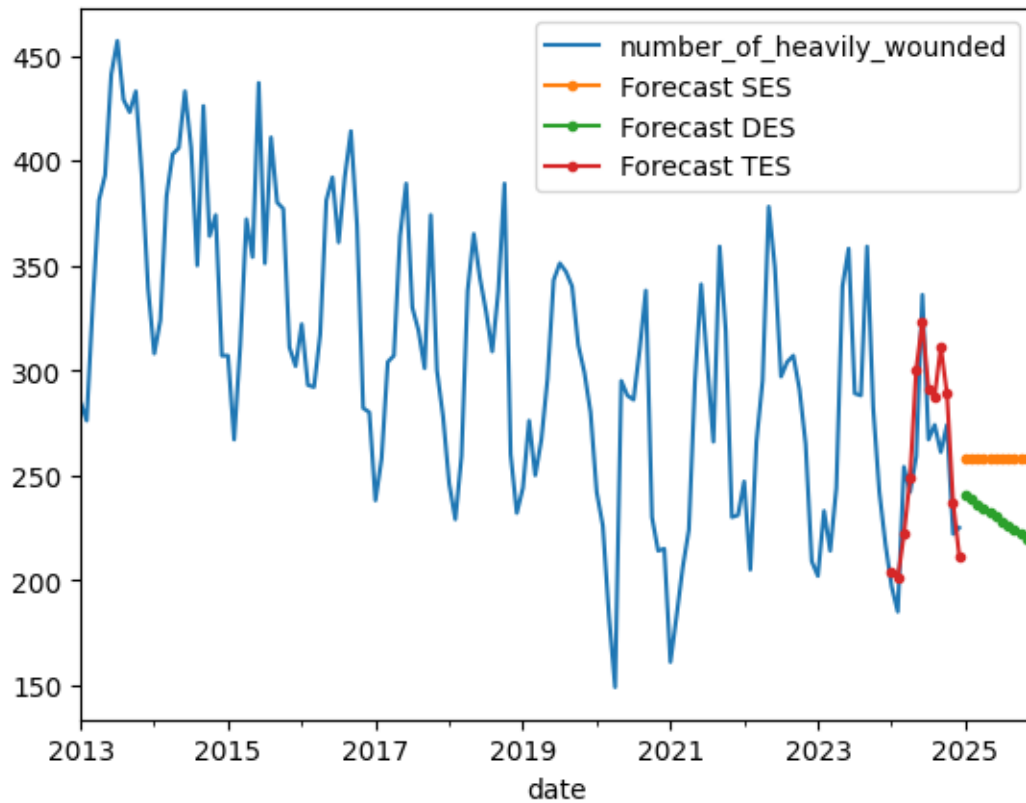
```
[ ]: data_tes.params_formatted
```

```
[ ]:
      name      param optimized
smoothing_level  alpha    0.212327      True
smoothing_trend  beta     0.000000      True
smoothing_seasonal gamma   0.000000      True
initial_level    l.0    373.832397      True
initial_trend    b.0    -0.808378      True
initial_seasons.0 s.0   -62.587319      True
initial_seasons.1 s.1   -64.687045      True
initial_seasons.2 s.2   -42.881435      True
initial_seasons.3 s.3   -15.163413      True
initial_seasons.4 s.4    37.467699      True
initial_seasons.5 s.5    60.820067      True
initial_seasons.6 s.6    29.355497      True
initial_seasons.7 s.7    26.982987      True
initial_seasons.8 s.8    51.429454      True
initial_seasons.9 s.9    29.964373      True
initial_seasons.10 s.10  -21.139083      True
initial_seasons.11 s.11  -45.694391      True
```

To use this model for forecasting:

```
[ ]: # forecast is now for the values of 2024!
data_tes_fcast = data_tes.forecast(12)

# Plot observations, fitted values and forecast
wounded_data['number_of_heavily_wounded'].plot(legend=True)
data_ses_fcast.plot(marker='.', legend=True, label='Forecast SES');
data_des_fcast.plot(marker='.', legend=True, label='Forecast DES');
data_tes_fcast.plot(marker='.', legend=True, label='Forecast TES');
```



As we can see, the predictions are now:

```
[ ]: data_tes_fcast
```

```
[ ]: 2024-01-01    203.717125
      2024-02-01    200.809021
      2024-03-01    221.806252
      2024-04-01    248.715896
      2024-05-01    300.538629
      2024-06-01    323.082620
      2024-07-01    290.809671
      2024-08-01    287.628784
      2024-09-01    311.266872
      2024-10-01    288.993413
      2024-11-01    237.081579
      2024-12-01    211.717893
      Freq: MS, dtype: float64
```

9.9.8 Exercise

As in the previous paragraphs, calculate the MAE assuming the trainingsdata is all the available data upto and including 2023 and the test data is the data from 2024. Be aware which values to

use as prediction for 2024!

```
[ ]: y_true = wounded_data['number_of_heavily_wounded'][-12:].values
```

Forecasting is done using the final values for the level, trend and seasonal adjustment factor estimates. Let's retrieve them from the model:

```
[ ]: wounded_hw.level.iloc[-1]
```

```
[ ]: np.float64(267.11282146039713)
```

```
[ ]: wounded_hw.trend.iloc[-1]
```

```
[ ]: np.float64(-0.8083781352409689)
```

```
[ ]: wounded_hw.season.iloc[-12:]
```

```
[ ]: date
2023-01-01    -62.587319
2023-02-01    -64.687045
2023-03-01    -42.881435
2023-04-01    -15.163413
2023-05-01     37.467699
2023-06-01     60.820067
2023-07-01     29.355497
2023-08-01     26.982987
2023-09-01     51.429454
2023-10-01     29.964373
2023-11-01    -21.139083
2023-12-01    -45.694391
Freq: MS, dtype: float64
```

9.9.9 Exercise

The formula for predicting values, looks like this:

$$F_{t+k} = L_t + kT_t + S_{t-m+k}$$

Take a look at the values of the level's L_t and the trend T_t , to understand the predicted values, as are printed here before.

The model performs forecasts by:

- Taking the last level estimate
- Taking the last trend estimate
- Taking the last estimate for the appropriate seasonal adjustment factor
- Computing the forecast formula given above
- Explain the forecasted value of 2024-01-01

- Explain the forecasted value of 2024-02-01
- ...

```
[ ]: data_tes.forecast(12)
```

```
[ ]: 2024-01-01    203.717125
      2024-02-01    200.809021
      2024-03-01    221.806252
      2024-04-01    248.715896
      2024-05-01    300.538629
      2024-06-01    323.082620
      2024-07-01    290.809671
      2024-08-01    287.628784
      2024-09-01    311.266872
      2024-10-01    288.993413
      2024-11-01    237.081579
      2024-12-01    211.717893
      Freq: MS, dtype: float64
```

```
[ ]:
```

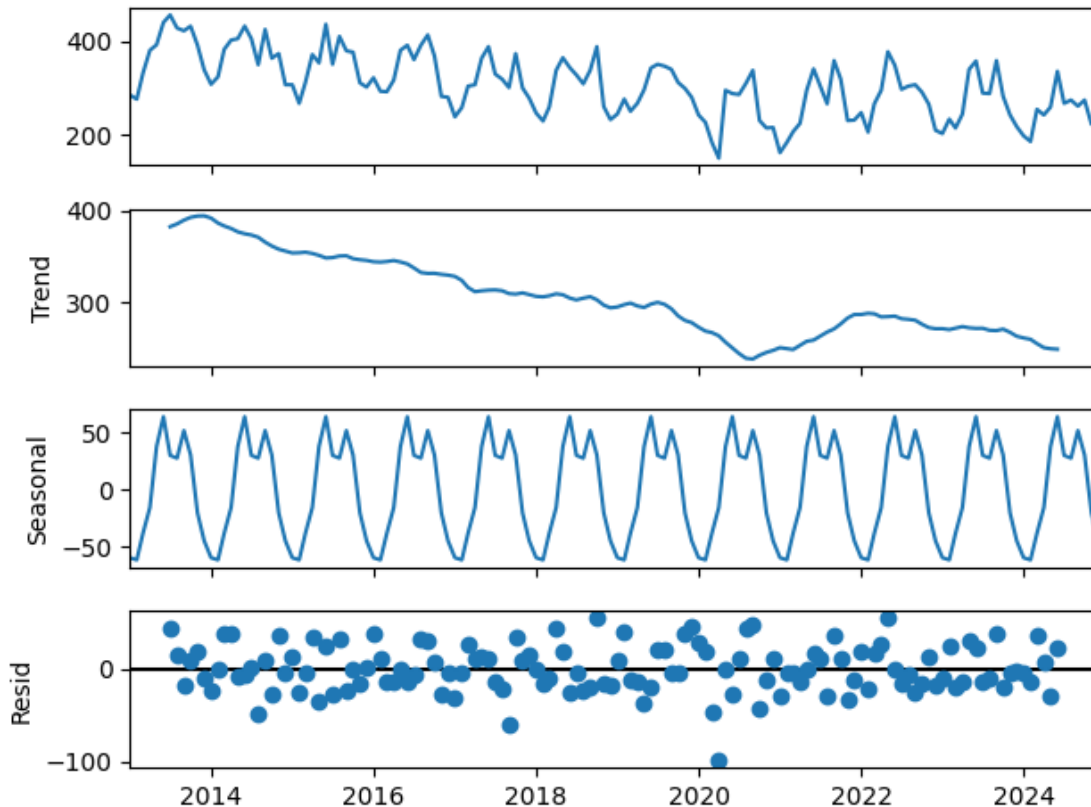
9.9.10 Decomposing a time series

A data series that shows a long term and seasonal trend can be “decomposed”:

```
[ ]: from statsmodels.tsa.seasonal import seasonal_decompose

      # remove other columns (e.g. SES, DES, ...)
      data = wounded_data[['number_of_heavily_wounded']]

      wounded_decomposed = seasonal_decompose(data, model='additive')
      wounded_decomposed.plot();
```



- The top graph shows the observations
- The second graph is the long term trend (which seems to be linear, so the choice for an additive model was appropriate)
- The third graph shows the seasonal trend
- The final graph are the residues

The semicolon at the end of line 4 is not strictly necessary, but if you leave it out, it's possible that the plot is shown twice.

9.10 Sources

Khan, E. (2021-01-05) *Python Code on Holt-Winters Forecasting*. Retrieved 2022-04-15 from <https://medium.com/analytics-vidhya/python-code-on-holt-winters-forecasting-3843808a9873>

Moreno, A.I. (2020-07-08) *Moving Averages with Python: Simple, cumulative and exponential moving averages with Pandas*, retrieved 2022-04-15 from <https://towardsdatascience.com/moving-averages-in-python-16170e20f6c>

Paramita (2020-12-17) *Holt winters Model, Easiest Times series Model. Additive multiplicative trend and seasonality*, retrieved 2022-04-16 from <https://www.youtube.com/watch?v=06cUkdQeLUQ>

Perktold, J., et al. (2022-04-13) *statsmodels manual: Exponential Smoothing*. Retrieved 2022-04-15 from https://www.statsmodels.org/dev/examples/notebooks/generated/exponential_smoothing.html